

四问源码逐行伴读

公式链、程序语句与带答案练习

目录

Q1 逐行伴读：从角度约束到有限区域的直径	2
Q2 逐行伴读：从安全选站到最坏后验半径	26
Q3 逐行伴读：从集合不变量到动作、通信和计时	48
Q4 逐行伴读：从定向观测模型到每一条程序语句	76
附录 带物理行号的原始源码	99

Q1 逐行伴读：从角度约束到有限区域的直径

本册是前面几何课程的源码实验册。先读 `course/01_从坐标到凸包与角楔.md` 的第 5–8 节，以及 `course/02_直径包围圆与鲁棒选址.md` 的第 3.1–3.3 节；然后按本册把公式逐项代回程序。只看函数名称不算过关：随机指到一行时，应能说出“它读了什么、保持什么条件、改变了什么、下一步用在哪里”。

行号对应本仓库原始文件；不向源码插入注释，所以原行号不变。每个非空物理行都有独立的 [L...] 标记；空行只起排版作用，略去。代码片段超过 76 个字符时以省略号截短，完整原行仍在 `source/Q1/`。

0 先把这一问写成一个完整的数学问题

已知。第 i 个检测点为 $s_i = (s_{ix}, s_{iy})$ ，读数为 θ_i ，角度误差半宽为 δ ，程序要求 $0 \leq \delta < 90^\circ$ 。观测条数 $m \geq 1$ 。坐标单位为米；输入角度单位为度，三角函数计算前转为弧度。

未知。同一个源的位置 $x = (x_1, x_2)$ 。不能把不同检测站的读数当成不同源，也不能用生成器中的真值代替待求位置。

模型。令 $a_i = \theta_i - \delta$ 、 $b_i = \theta_i + \delta$ ，则

$$n_i^- = (\sin a_i, -\cos a_i), \quad n_i^+ = (-\sin b_i, \cos b_i), \quad H_i^\pm = \{x : n_i^\pm \cdot x \leq n_i^\pm \cdot s_i\}.$$

角度在这条公式里取弧度。每次观测留下一个前向角楔 $W_i = H_i^- \cap H_i^+$ ，全部观测的共同允许区域为

$$F_{\text{angle}} = \bigcap_{i=1}^m W_i. \quad (\text{Q1.1})$$

当 $\delta = 0$ 时，还须加入 $-u(\theta_i) \cdot (x - s_i) \leq 0$ ，把整条直线限制成向前的射线。浮点代码把楔顶 s_i 也作为闭集合的一部分；在源与站重合处，真实方位角本身没有定义。

要求。先分类交集，再对非空有界交集计算顶点、面积、直径与最远点对。若 $F = \text{conv}\{v_0, \dots, v_{n-1}\}$ ，则

$$D(F) = \max_{j,k} \|v_j - v_k\|, \quad A(F) = \frac{1}{2} \left| \sum_{j=0}^{n-1} \text{cross}(v_j, v_{j+1}) \right|, \quad v_n = v_0. \quad (\text{Q1.2})$$

找到最远点对 p, q 后，程序另做一个检查：令 $c = (p + q)/2$, $r = D/2$ ，检查 $\max_j \|v_j - c\| \leq r$ 。这是“直径圆是否覆盖”的判断；本文件没有求最小包围圆。

适用范围。本文件实际求的是式 (Q1.1)。它没有把半径 1800 米的场地圆盘与此集合相交，也没有以观测距离再裁一次。误差 δ 属于测量模型；EPS 属于浮点比较规则，两者不能互相替代。

从公式到主调用。main 读 JSON $\rightarrow$ solve_localization 整理观测 $\rightarrow$ _wedge 造不等式 $\rightarrow$ intersect_halfplanes 求交并分类 $\rightarrow$ _finite_result 计算指标 $\rightarrow$ polygon_diameter 用凸包和旋转卡壳求最远点对 $\rightarrow$ main 写 JSON。

下面按照文件物理顺序逐行读；第一次可先找到这七个名字，再回头补辅助函数。

1 q1_solver.py

源码：source/Q1/q1_solver.py。共 302 个物理行、250 个非空行。SHA-256 前 16 位：a5d3d1207a41154b；完整值可对原文件运行 sha256sum 核对。

1.1 模块、类型与数值常量

这些行准备程序的工具和约定，不产生定位结果。Point 是类型提示；Python 不会因此自动验证每一个传入对象。几何输入的有限性检查主要在 `_normalise` 与 `_observation` 中完成。

- [L001] `#!/usr/bin/env python3`：Unix 类系统直接执行脚本时，请环境查找 Python 3。通过 `python q1_solver.py` 启动时，这行只是注释。
- [L002] `"""问题一：有界误差 AOA 半平面交与凸多边形直径；只求解，不绘图。"""`：模块说明：本文件解决有界角误差定位的几何计算，不包含绘图；这个字符串也会成为命令行帮助文字。
- [L003] `from __future__ import annotations`：延后处理类型注解，减少导入时求值的依赖；它不改变不等式和数值算法。
- [L005] `import argparse`：导入命令行参数解析器，供文件末尾的 `main` 使用。
- [L006] `from collections import deque`：导入双端队列；半平面交需要既删尾部又删头部。
- [L007] `from dataclasses import dataclass`：导入数据类装饰器，给半平面对象自动生成初始化等方法。
- [L008] `import json`：导入 JSON 读写工具；观测输入和结果输出均用 JSON。
- [L009] `import math`：导入三角函数、平方根、距离、有限性判断及圆周常数。
- [L010] `from pathlib import Path`：导入跨平台路径对象，处理输入文件、输出文件与父目录。
- [L011] `from typing import Iterable`：导入可迭代对象的类型提示，使输入可以是列表或其他可遍历容器。
- [L013] `Point = tuple[float, float]`：用两个浮点数的元组表示二维点或向量；这里坐标和向量共用表示形式，含义由调用位置决定。
- [L014] `EPS = 1.0e-10`：设置一般浮点比较阈值为 10^{-10} ；后续有的地方按坐标放大，有的地方直接用于叉积比较，不能称为统一的严格误差界。
- [L015] `PARALLEL_EPS = 1.0e-12`：设置近似平行判定阈值为 10^{-12} ，用于小行列式和近零系数。
- [L016] `NUMERICAL_ANGLE_GUARD_DEG = 0.0`：保留名为“数值角保护”的常量，数值为零；当前文件没有后续引用，因此不会把 $\pm 1^\circ$ 自动扩大。

1.2 HalfPlane 与 HalfPlane.residual

输入与输出。对象存储 $n = (a, b)$ 和 c ，表示 $n \cdot x \leq c$ ；`residual(point)` 返回 $n \cdot x - c$ 。

不变量。数据类冻结字段赋值，后续算法用新的半平面对象替换旧对象。单位法向量不是数据类自动保证的条件，由 `_normalise` 保证。

数学对应。残差为负表示内侧，为零表示边界，为正表示外侧。 $\|n\| = 1$ 时，残差还等于带符号的垂直距离。

证明边界。`residual` 只算数，不决定容差；实际“算不算在外面”由 `_outside` 判定。

- [L019] `@dataclass(frozen=True)`：把下方类声明为冻结的数据类；正常使用时不能重新给 `normal`、`offset` 字段赋值。
- [L020] `class HalfPlane:`：定义半平面的数据类型。

- [L021] `"""闭半平面 ``normal · X <= offset``. """`: 明确采用“小于等于”的闭半平面约定, 后续法向量符号均按此约定推导。
- [L023] `normal: Point`: 记录法向量 (a,b) ; 它指向不等式数值增大的方向。
- [L024] `offset: float`: 记录右端偏移 c , 不是边界上某个点。
- [L026] `def residual(self, point: Point) -> float::` 定义残差方法: 输入一个点, 输出一个浮点数。
- [L027] `return self.normal[0] * point[0] + self.normal[1] * point[1] - self.offset`: 逐分量计算 $ax + by - c$; 这就是后续所有内外测试的共同代数式。

1.3 `_cross`

输入/输出。输入两个二维向量, 输出一个标量。不变量。不修改输入。数学对应。 $\text{cross}(a, b) = a_x b_y - a_y b_x$ 。证明边界。正负号表示定向面积; 只有把向量正确写成两个点的差, 才能据此判断三个点的转向。

- [L030] `def _cross(a: Point, b: Point) -> float::` 定义二维叉积辅助函数; 前导下划线表示它主要供模块内部使用。
- [L031] `return a[0] * b[1] - a[1] * b[0]`: 计算行列式 $a_x b_y - a_y b_x$; 若坐标单位为米, 结果单位为平方米。

1.4 `_sub`

输入/输出。输入两个点或向量, 返回 $a - b$ 。不变量。返回新元组, 不改原坐标。数学对应。从 b 指向 a 的位移。证明边界。减法次序决定方向, 交换输入会改变叉积符号。

- [L034] `def _sub(a: Point, b: Point) -> Point::` 定义二维减法。
- [L035] `return a[0] - b[0], a[1] - b[1]`: 分别减横、纵坐标, 返回两个数构成的元组。

1.5 `_distance2`

输入/输出。输入两点, 返回距离平方。不变量。返回非负数 (正常有限输入下), 不改变点。数学对应。 $\|a - b\|^2 = (a_x - b_x)^2 + (a_y - b_y)^2$ 。证明边界。比较距离大小可比较平方; 输出单位是平方米, 不能直接当直径打印。

- [L038] `def _distance2(a: Point, b: Point) -> float::` 定义平方距离函数。
- [L039] `dx, dy = a[0] - b[0], a[1] - b[1]`: 先求两坐标差, 供下一行复用。
- [L040] `return dx * dx + dy * dy`: 将差的平方相加; 在卡壳中先比较这个量, 最后只开一次平方。

1.6 `_normalise`

输入。三种约束写法之一: `HalfPlane((a,b),c)`、`((a,b),c)` 或 `(a,b,c)`。

输出与不变量。返回法向长度为 1 的新对象: $(n, c) \mapsto (n/\|n\|, c/\|n\|)$ 。除数为正, 所以不等号方向与可行集合不变。

数学对应。同时除以法向量长度, 才能让同向半平面的 `offset` 可直接比较, 也才能将残差解释为距离。

证明边界。零法向被拒绝, 而不是被解释成“恒真/恒假约束”。某些畸形结构可能由 Python 解包直接报错; 这不是一个接受任意 JSON 的通用校验器。

- [L043] `def _normalise(raw) -> HalfPlane::` 定义统一格式与缩放的函数; `raw` 可以采用三种约定写法。
- [L044] `if isinstance(raw, HalfPlane)::` 先判断输入是否已经是半平面对象。
- [L045] `a, b = raw.normal`: 若是, 从法向量提取 a, b ; 仍会在后面重新归一化。
- [L046] `c = raw.offset`: 从对象取右端 c 。
- [L047] `elif len(raw) == 2::` 否则尝试长度为 2 的形式。

- [L048] `(a, b), c = raw`: 把 `((a,b),c)` 解包成三个系数；若结构不符合，Python 会抛解包异常。
- [L049] `elif len(raw) == 3:`: 再尝试长度为 3 的形式。
- [L050] `a, b, c = raw`: 把 `(a,b,c)` 解包。
- [L051] `else:`: 三种格式都没匹配时走错误分支。
- [L052] `raise ValueError("半平面应写为 HalfPlane、((a,b),c) 或 (a,b,c)")`: 抛出说明合法格式的异常，使错误输入停止传播。
- [L053] `a, b, c = float(a), float(b), float(c)`: 统一转为浮点数；不能转成数的字符串也会在这里失败。
- [L054] `if not all(math.isfinite(v) for v in (a, b, c)):`: 检查三个数均不是无穷或 NaN。
- [L055] `raise ValueError("半平面系数必须为有限数")`: 遇到非有限数时明确拒绝。
- [L056] `length = math.hypot(a, b)`: 用 `hypot` 求 $\sqrt{a^2 + b^2}$ ，比手写平方和更适合一般数值范围。
- [L057] `if length == 0.0:`: 检查法向量长度是否恰为零。
- [L058] `raise ValueError("半平面法向量不能为零")`: 零向量不能给出边界方向，故抛出异常。
- [L059] `return HalfPlane((a / length, b / length), c / length)`: 法向和右端一起除以正长度，返回等价约束；只除法向而不除偏移会移动边界。

1.7 _direction

输入/输出。输入半平面 $n \cdot x \leq c$ ，输出沿边界行进的切向 $d = (-b, a)$ 。

不变量与数学对应。取边界点 x_0 ，有 $\text{cross}(d, x - x_0) = c - n \cdot x$ ，所以可行侧在行进方向左边。

证明边界。这里是边界的方向，不是源的示向读数；向量长度为 1 还依赖输入已归一化。

- [L062] `def _direction(hp: HalfPlane) -> Point:`: 定义半平面到有向边界的转换。
- [L063] `# 沿该方向前进时，可行域位于有向边界左侧。:` 注释明确“可行域在左侧”的统一方向约定。
- [L064] `a, b = hp.normal`: 取出法向量两个分量。
- [L065] `return -b, a`: 将法向量逆时针旋转 90° ，得到 $(-b, a)$ ；该方向与边界平行。

1.8 _same_direction

输入/输出。输入两个半平面，输出布尔值：其边界是否近似同向平行。

不变量与数学对应。叉积接近 0 判平行，点积大于 0 判同向；仅凭叉积为零不能区分同向与反向。

证明边界。同向平行可以合并成更紧的一条；反向平行可能形成条带或矛盾，不能合并。代码用阈值处理“近似平行”，并非精确几何谓词。

- [L068] `def _same_direction(a: HalfPlane, b: HalfPlane) -> bool:`: 定义同向检查。
- [L069] `da, db = _direction(a), _direction(b)`: 分别取得两条有向边界的切向量。
- [L070] `return abs(_cross(da, db)) <= PARALLEL_EPS and da[0] * db[0] + da[1] * db...`: 要求叉积绝对值不超过平行阈值且点积为正；两个条件用 `and` 同时满足。

1.9 _ordered

输入/输出。输入一批半平面，返回已归一化、按边界方向角排序并合并同向约束的列表。

不变量。排序不改变交集；精确同向约束 $n \cdot x \leq c_1, c_2$ 可改为 $n \cdot x \leq \min(c_1, c_2)$ 。

数学对应。方向环从 0 到 2π ，列表头尾也可能属于同一方向，故需再次合并。

证明边界。近似同向由阈值识别，其合并并非严格有理数运算下的集合恒等；默认米级输入采用浮点容差。

- [L073] `def _ordered(halfplanes: Iterable[HalfPlane]) -> list[HalfPlane]`: 定义预处理, 允许输入任意可遍历的半平面集合。
- [L074] `planes = [_normalise(hp) for hp in halfplanes]`: 逐条归一化, 确保比较偏移时各法向量长度相同。
- [L075] `planes.sort(key=lambda hp: math.atan2(_direction(hp)[1], _direction(hp)[0]))`: 以边界切向的 atan2 角为键排序, 再对 2π 取模统一到 $[0, 2\pi)$ 。
- [L076] `merged: list[HalfPlane] = []`: 建立新的合并结果列表。
- [L077] `for hp in planes:`: 按排序后的次序处理各条约束。
- [L078] `if merged and _same_direction(merged[-1], hp):`: 如果已有约束且新约束与末项同向, 进入合并分支; 先判断非空避免 `[-1]` 越界。
- [L079] `if hp.offset < merged[-1].offset:`: 比较偏移: 单位法向相同时, 更小的右端更严格。
- [L080] `merged[-1] = hp`: 新条件更紧则替换末项; 否则维持旧条件, 不追加冗余项。
- [L081] `else:`: 若不与末项同向, 走追加分支。
- [L082] `merged.append(hp)`: 把一个新方向放到结果尾部。
- [L083] `if len(merged) > 1 and _same_direction(merged[0], merged[-1]):`: 处理角度环的接缝: 排序头尾在 $0/2\pi$ 两侧也可能近似同向。
- [L084] `if merged[-1].offset < merged[0].offset:`: 比较头尾哪一条更紧。
- [L085] `merged[0] = merged[-1]`: 尾项较紧则用它替换头项。
- [L086] `merged.pop()`: 删除重复的尾项; 无论第 85 行是否执行, 都只保留一条。
- [L087] `return merged`: 返回处理后的列表, 而不是原输入次序。

1.10 _intersection

输入/输出。输入两条半平面的边界, 返回直线交点或 `None`。

数学对应。联立 $a_x x + a_y y = c_a$ 、 $b_x x + b_y y = c_b$; 行列式 $\Delta = a_x b_y - a_y b_x$ 。若非零, 克拉默法则给

$$x = \frac{c_a b_y - a_y c_b}{\Delta}, \quad y = \frac{a_x c_b - c_a b_x}{\Delta}.$$

不变量。只求边界直线相交, 不验证交点满足其他约束。

证明边界。 `None` 表示没有可稳定计算的唯一交点; 它同时包含近似平行、重合情形, 不能直接推出可行域为空。

- [L090] `def _intersection(a: HalfPlane, b: HalfPlane) -> Point | None`: 定义交点函数, 返回类型允许“点或无结果”。
- [L091] `ax, ay = a.normal`: 取第一条边界法向的两个分量。
- [L092] `bx, by = b.normal`: 取第二条边界法向的两个分量。
- [L093] `determinant = ax * by - ay * bx`: 计算两法向组成的二维行列式。
- [L094] `if abs(determinant) <= PARALLEL_EPS:`: 检查行列式是否过小, 避免用近零数作分母。
- [L095] `return None`: 近似平行时返回 `None`, 交由调用者作后续分类。
- [L096] `return ((a.offset * by - ay * b.offset) / determinant,`: 按克拉默法则计算交点横坐标, 并开始返回二元组。

- [L097] $(ax * b.offset - a.offset * bx) / determinant$: 计算纵坐标并结束二元组；本行与上一行是一条跨行 return。

1.11 _outside

输入/输出。输入已归一化半平面和一个点，返回该点是否超过容差地位于外侧。

数学对应。判据为 $n \cdot x - c > EPS \max(1, |c|, |x_1|, |x_2|)$ 。

不变量。边界及容差内的微小正残差均视作“没有在外面”。

证明边界。输出 False 是数值接受，不能等同于严格验证 $n \cdot x \leq c$ 。坐标很大时容差随之增大。

- [L100] `def _outside(hp: HalfPlane, point: Point) -> bool`: 定义实际使用的点在外侧判据。
- [L101] `scale = max(1.0, abs(hp.offset), abs(point[0]), abs(point[1]))`: 以 1、偏移量、两个坐标的绝对值中最大者作为尺度。
- [L102] `return hp.residual(point) > EPS * scale`: 用残差与尺度化阈值比较；注意严格大于才返回 True。

1.12 _cuts

输入/输出。输入新半平面 hp 及两条旧边界 a,b，返回旧交点是否被新半平面排除。

不变量与数学对应。先求旧交点，再代入新不等式；它正是队列弹出条件的封装。

证明边界。若旧边界近似平行，函数返回 False；这表示没有可用交点触发本次删除，不能证明这两条约束相容。

- [L105] `def _cuts(hp: HalfPlane, a: HalfPlane, b: HalfPlane) -> bool`: 定义“新约束是否切掉旧拐角”的辅助判断。
- [L106] `point = _intersection(a, b)`: 求两条旧边界的候选交点。
- [L107] `return point is not None and _outside(hp, point)`: 只有交点存在且在新约束外才返回真；and 短路可避免把 None 当坐标传入。

1.13 _feasible_point: 二维约束怎样变成一维区间

输入/输出。输入已归一化约束列表。返回一个满足全部约束的候选点，或 None 表示这套容差计算发现矛盾。它不返回最优点，也不计算顶点集合。

循环不变量。每轮开始时，point 满足此前已经处理的所有约束（按 _outside 的容差）。若它也满足新约束，继续使用它；若违反新约束，就沿新边界寻找同时满足旧条件的新点。

为什么能限制在新边界上？精确算术下，设旧点 p 满足全部旧约束而违反新约束。若确实存在新可行点 q ，线段 pq 全在旧可行集内；它从新半平面的外侧走到内侧，必穿过新边界。因此只检查这条边界不会漏掉“存在可行点”这一事实。

公式链。新法向 $n = (n_x, n_y)$ 已单位化，故边界可写为

$$x(t) = bn + t(-n_y, n_x) = \text{base} + t \text{ tangent}.$$

把旧约束 $m \cdot x \leq d$ 代入，得到

$$\underbrace{m \cdot \text{tangent}}_{\text{coefficient}} t \leq \underbrace{d - m \cdot \text{base}}_{\text{bound}}. \quad (\text{Q1.3})$$

系数为正给上界，系数为负给下界，系数为零时检查 $0 \leq \text{bound}$ 。把所有条件合成一个区间 $[\text{lower}, \text{upper}]$ ；区间空则没有可行点。

证明边界。上述推导给精确算法的理由；代码引入近零判别与容差，因此不是任意病态输入的精确线性规划求解器。它最坏会逐轮查看此前所有约束，总工作量可到 $O(m^2)$ 。

- [L110] `def _feasible_point(planes: list[HalfPlane]) -> Point | None`: 定义可行点搜索，输入约束列表，输出点或 None。
- [L111] `"""二维线性约束可行性；只用于区分空集与无界集。"""`: 说明此函数在本程序中用于区分空集与非空无界集，不承担定位最优性。
- [L112] `point = (0.0, 0.0)`: 从原点开始；处理零条约束时，任何点都可行。
- [L113] `for i, hp in enumerate(planes)`: 依次取得约束序号 i 和本轮约束 `hp`。
- [L114] `if not _outside(hp, point)`: 检测当前点是否已经在新半平面内或容差范围内。
- [L115] `continue`: 满足新约束就跳过后续搜索，保留原有可行点。
- [L116] `nx, ny = hp.normal`: 违反新约束后取其单位法向量。
- [L117] `base, tangent = (nx * hp.offset, ny * hp.offset), (-ny, nx)`: 同一行建立边界基点 bn 与切向 $(-n_y, n_x)$ ；因此任意边界点可用单参数表示。
- [L118] `lower, upper = -math.inf, math.inf`: 一开始参数没有上下限制，以正负无穷表示整条直线。
- [L119] `for previous in planes[:i]`: 逐个处理索引小于 i 的旧约束；这些条件必须继续满足。
- [L120] `coefficient = previous.normal[0] * tangent[0] + previous.normal[1] * tang...`: 计算旧法向与新边界切向的点积，即式 (Q1.3) 中参数 t 的系数。
- [L121] `bound = previous.offset - (previous.normal[0] * base[0] + previous.normal...`: 计算旧右端减去旧法向在边界基点上的点积，即式 (Q1.3) 的右端。
- [L122] `if coefficient > PARALLEL_EPS`: 系数显著为正时，不等式除以正数得到上界。
- [L123] `upper = min(upper, bound / coefficient)`: 用旧上界与本条上界的较小者收紧区间。
- [L124] `elif coefficient < -PARALLEL_EPS`: 系数显著为负时，不等式除以负数，方向反转成下界。
- [L125] `lower = max(lower, bound / coefficient)`: 用旧下界与本条下界的较大者收紧区间。
- [L126] `elif bound < -EPS * max(1.0, abs(previous.offset))`: 系数近零时，若右端仍显著为负，就出现 $0 \leq$ 负数的矛盾。
- [L127] `return None`: 报告没有可行参数，返回 None。
- [L128] `if lower > upper + EPS * max(1.0, abs(lower), abs(upper))`: 合并全部旧约束后，检查下界是否超过上界，并允许尺度化容差。
- [L129] `return None`: 若参数区间为空，返回 None。
- [L130] `parameter = min(max(0.0, lower), upper)`: 把 0 截入允许区间：0 在区间内就取 0，否则取离 0 较近的端点。
- [L131] `point = (base[0] + parameter * tangent[0], base[1] + parameter * tangent[1])`: 把所选参数代回边界参数方程，更新二维可行点。
- [L132] `return point`: 所有约束处理后返回当前点；输入为空时会返回最初的原点。

纸笔轨迹：三个约束，答案全部给出

按次序输入 $y \geq 1$ 、 $x \leq 3$ 、 $x + y \geq 5$ 。第一轮原点违反 $y \geq 1$ ，沿 $y = 1$ 取到 $(0, 1)$ 。第二轮已经满足 $x \leq 3$ ，不改点。第三轮违反 $x + y \geq 5$ ，须沿 $x + y = 5$ 找点。

第三条归一化后 $n = (-1, -1)/\sqrt{2}$ ， $b = -5/\sqrt{2}$ ，因此 `base` 为 $(2.5, 2.5)$ ，`tangent` 为 $(1, -1)/\sqrt{2}$ 。旧条件分别给

$$y \geq 1 \Rightarrow t \leq 1.5\sqrt{2}, \quad x \leq 3 \Rightarrow t \leq 0.5\sqrt{2}.$$

最终 $t \in (-\infty, 0.5\sqrt{2}]$, 包含 0, 所以返回 (2.5, 2.5)。直接代回: $2.5 \geq 1$ 、 $2.5 \leq 3$ 、 $2.5 + 2.5 = 5$ 。

追问与答案。为什么不返回 (3, 2)? 它也是可行点, 但本函数只求“任意可行点”, 代码的截断规则选择了 $t = 0$ 。若把前两条改成 $y \leq 1, x \leq 3$, 它们在 $x + y = 5$ 上要求 $t \geq 1.5\sqrt{2}$ 与 $t \leq 0.5\sqrt{2}$, 区间矛盾, 返回 None。

1.14 _is_bounded

输入/输出。输入归一化半平面列表, 返回有界性方向条件的布尔值。

数学对应。非空多面集无界, 当且仅当存在非零退行方向 d 满足所有 $n_i \cdot d \leq 0$ 。在平面上, 这等价于全部法向量落在某个闭半圆中; 按角排序后, 等价于出现至少 π 的环形空隙。因此最大空隙严格小于 π 才排除退行方向。

不变量。对已有可行点 x_0 , 若存在上述 d , 则 $x_0 + td$ 对所有 $t \geq 0$ 仍可行。

证明边界。此函数不检查是否为空。空集也可能有这套法向分布; 主函数在兜底路径先求可行点再调用它。阈值附近会被保守地视作不能证实有界。

- [L135] `def _is_bounded(planes: list[HalfPlane]) -> bool::` 定义有界方向判别。
- [L136] `if len(planes) < 3::` 少于三条非零半平面不能围出非空有界集合。
- [L137] `return False::` 因此直接返回 False; 不表示集合必定非空。
- [L138] `angles = sorted(math.atan2(hp.normal[1], hp.normal[0]) % math.tau for hp ...::` 计算全部法向的 $[0, 2\pi)$ 角并排序; 这里用法向, 和前面的边界方向相差固定 90° , 角间隙相同。
- [L139] `gaps = [angles[i + 1] - angles[i] for i in range(len(angles) - 1)::` 计算相邻法向的角间隔。
- [L140] `gaps.append(angles[0] + math.tau - angles[-1]):` 补上最后一个角跨过 2π 回到第一个角的间隔。
- [L141] `return max(gaps) < math.pi - PARALLEL_EPS::` 要求最大间隔小于 π 再减去平行阈值; 只有没有半圆空隙时才返回真。

即时题与答案。 $x \geq 0, y \geq 0$ 的法向角为 $\pi, 3\pi/2$, 最大环形间隙 $3\pi/2$, 故无界; 正方形四条边的法向每隔 $\pi/2$, 没有退行方向。线段 $x = 0, -1 \leq y \leq 1$ 虽面积为零, 四个法向仍阻断全部非零退行方向, 所以有界。

1.15 _convex_hull

输入/输出。输入一批二维点, 输出它们的凸包顶点, 按逆时针顺序排列; 空、单点、共线点分别返回空列表、一个点、两 endpoints。

不变量与数学对应。先按横坐标、再按纵坐标排序, 分别维护下链和上链。每条链只保留满足正向转弯的拐角。凸包是包含全部点的最小凸集; 本处输入是候选交点或待测直径的点, 并不是检测站坐标。

证明边界。精确算法用叉积 ≤ 0 删除非左转拐角; 当前实现用 $\leq \text{EPS}$, 所以极小的正向凸起也可能被当作近似共线。它没有为任意极端坐标提供精确几何保证。

- [L144] `def _convex_hull(points: Iterable[Point]) -> list[Point]:` 定义 Andrew 单调链凸包函数, 接受任意可遍历点集。
- [L145] `rows = sorted(set((float(x), float(y)) for x, y in points)):` 把各坐标转成浮点, 构成元组; 用集合删除完全相同的点, 再按字典序排序。这里不按距离阈值合并近邻点。
- [L146] `if len(rows) <= 1::` 先处理没有点或只有一个不同点的情况。
- [L147] `return rows::` 这两种情况下排序结果本身就是凸包, 直接返回。

内部函数 chain

输入/输出。输入沿一个方向有序的点列，返回一条凸链。循环不变量。result 存放已读点形成的当前外侧链，每三个连续点的叉积大于 EPS。数学对应。若末尾出现右转或近共线，删掉中间旧点，直到新点能接成合法左转。证明边界。不变量依赖有序输入；把乱序点直接传给这段函数不能得到一般凸包。

- [L149] def chain(sequence):: 在凸包函数内部定义造链过程，上、下链共用它。
- [L150] result: list[Point] = []: 建立当前链的空列表。
- [L151] for point in sequence:: 依次读取有序点；正向调用处理下链，反向调用处理上链。
- [L152] while len(result) >= 2 and _cross(_sub(result[-1], result[-2]), _sub(point, result[-1])) < EPS: 已有至少两个点时，计算“倒数第二点 → 末点 → 新点”的叉积；不超过阈值就继续删除。第二个差是“新点减末点”，与“新点减倒数第二点”给同一个叉积。
- [L153] result.pop(): 删除链的末点，即刚检查的三点中的中间点；新点还未入链。
- [L154] result.append(point): 所有不合格拐角清理完后，把新点追加进链。
- [L155] return result: 返回这一次扫描所得的链。

回到外部 _convex_hull 的最后一行：

- [L157] return chain(rows)[-1] + chain(reversed(rows))[-1]: 正向、反向各造一条链，各去掉最后一个公共端点，再拼接；这样最左、最右端点均只出现一次。

完整执行小例：复用课程五点例

原点为 $A = (0, 0)$, $B = (10, 0)$, $C = (10, 10)$, $D = (6, 5)$, $E = (4, 1)$ 。排序后是 A, E, D, B, C 。下链的数值轨迹如下。

本轮读入	叉积及删除	处理后的 result
A	不足两旧点	$[A]$
E	不足两旧点	$[A, E]$
D	$\text{cross}((4, 1), (2, 4)) = 14$	$[A, E, D]$
B	先得 -26 ，删 D ；再得 -10 ，删 E	$[A, B]$
C	得 100 ，保留	$[A, B, C]$

反向次序 C, B, D, E, A ：加入 D 时叉积为 -40 ，删 B ；加入 E 时叉积为 6 ，保留；加入 A 时先得 -14 删 E ，再得 -10 删 D ，得到 $[C, A]$ 。最后 $[A, B] + [C] = [A, B, C]$ 。

问：为何不能把第 152 行 while 改成一次 if？答：加入 B 时需要连续删 D, E ；只删一次会留下不合法拐角。

问：为什么叉积为零也删点？答：同一直线边上的中间点不是必需拐角，保留两端即可表示同一凸集；如果需求是返回所有边界输入点，就要另改输出约定。

问：两个循环嵌套是不是 $O(n^2)$ ？答：造链中每点至多加入一次、弹出一，累计 $O(n)$ ；函数总成本主要为排序的 $O(n \log n)$ 。

1.16 polygon_diameter

输入/输出。输入有限点集，返回 (最大距离, 最远点对)。空集返回 (None, None)；单点直径为零；两点直接计算距离。

数学对应。对凸多边形，最远点对能取在顶点上。先求凸包再以旋转卡壳寻找候选。第 i 条边 $e = v_{i+1} - v_i$ 的叉积高度为

$$H_i(j) = \text{cross}(e, v_j - v_i). \quad (\text{Q1.4})$$

在同一条边上，最大 $H_i(j)$ 对应最远侧支撑点。找到远侧点后再比较两条点对距离；高度不是直径。

循环不变量。best2 是已检查候选点对的最大距离平方；best_pair 是实现该值的一对点。j 沿逆时针凸包向前移动，并沿用上一条边的搜索位置；i 依次走过每条边。

证明边界。单向推进依赖凸包顺序与凸性。代码在等高时停止移动，输出一个最远点对；它没有枚举和返回全部并列最远点对或全部对踵点。EPS 让非常小的高度增加被当作相等，须承认浮点实现的适用尺度。

- [L160] def polygon_diameter(vertices: Iterable[Point]): 定义直径函数，输入可以是乱序点，因为下一步先求凸包。
- [L161] """"用旋转卡壳返回 ``(直径, 最远点对)``。""": 说明返回两部分：直径数值和一对最远点。
- [L162] points = _convex_hull(vertices): 求有序凸包，去除内部点和不必要的共线点。
- [L163] count = len(points): 记录凸包点数。
- [L164] if count == 0: 检测空集。
- [L165] return None, None: 空集没有本程序定义的有限直径及点对，返回两个 None。
- [L166] if count == 1: 检测只有一点的退化集。
- [L167] return 0.0, (points[0], points[0]): 直径为零，点对的两个元素都用该点。
- [L168] if count == 2: 检测只有两个端点的线段。
- [L169] return math.dist(*points), (points[0], points[1]): 星号把两点展开成 math.dist 的两个参数；直接返回距离与端点对。
- [L170] best2, best_pair, j = 0.0, (points[0], points[0]), 1: 一般多边形分支初始化：最大平方距离为 0、初始点对重合、远侧指针从索引 1 开始。
- [L171] for i in range(count): 依次把每个顶点作为当前边起点。
- [L172] following = (i + 1) % count: 用模运算求下一顶点；最后一条边自动连回索引 0。
- [L173] edge = _sub(points[following], points[i]): 计算当前有向边向量。
- [L174] while (_cross(edge, _sub(points[(j + 1) % count], points[i]))): 开始判断下一远侧顶点的叉积高度；索引也按点数取模，跨尾部后回到开头。
- [L175] > _cross(edge, _sub(points[j], points[i])) + EPS): 将下一高度与当前高度加容差比较；仅显著增加时继续移动。它与上一行组成一个 while 条件。
- [L176] j = (j + 1) % count: 远侧指针前进一步；不从头重置，从而复用凸性的单调顺序。
- [L177] for first in (i, following): 分别以当前边的两个端点作为距离候选的一端。
- [L178] candidate = _distance2(points[first], points[j]): 计算该端点到当前远侧点的距离平方。
- [L179] if candidate > best2: 只有严格变大才更新纪录；距离并列时保留先前找到的点对。
- [L180] best2, best_pair = candidate, (points[first], points[j]): 同一行同时更新最大值和实现它的点对，保证二者一致。
- [L181] return math.sqrt(best2), best_pair: 最后对最大平方距离开方，连同点对返回。

卡壳执行表：把高度比较与距离比较分开

用 $A = (0, 0), B = (4, 0), C = (3, 3), D = (0, 2)$ ，凸包次序就是 A, B, C, D 。

当前边	沿用的 j 怎样走	停下时比较的平方距离	best2
AB	从 B 高度 0 走到 C 高度 12; D 高度 8, 不再走	$AC^2 = 18, BC^2 = 10$	18

当前边	沿用的 j 怎样走	停下时比较的平方距离	best2
BC	从 C 的 0, 经 D 的 10, 到 A 的 12	$BA^2 = 16, CA^2 = 18$	18
CD	从 A 的 6, 到 B 的 10	$CB^2 = 10, DB^2 = 20$	20
DA	B 高度 8, 下一点 C 为 6, 保留 B	$DB^2 = 20, AB^2 = 16$	20

答案为 $\sqrt{20} = 4.47213595499958$, 返回点对可以是 (D, B) 。同一对端点顺序颠倒不改变几何答案。

追问：为何“对边最远的点”未必与起点组成最远点对？答：叉积只量垂直高度；两点距离还包含沿边方向的分量，所以找到远侧点后还要计算距离，并检查当前边两端。

练习及答案。矩形 $(0, 0), (4, 0), (4, 3), (0, 3)$ 的第一条边有两个等高远侧点。代码不会在等高处继续移动；它会保留一个，继续检查后面的边。最终直径为 5，可返回两条对角线之一。不要把“未列出另一条对角线”当作直径算错。

1.17 _finite_result

输入/输出。前提是非空有限形状的候选顶点。返回包含状态、顶点、直径、点对、面积和直径圆覆盖检查的字典。

不变量。顶点先整理成凸包；度量由同一组整理后顶点计算。点、线段的面积按鞋带式自然为零。

数学对应。直径和面积见式 (Q1.2)；覆盖裕量为 $r - \max_j \|v_j - c\|$ 。由于圆盘是凸集，如果全部顶点在圆内，则整个凸包都在圆内。

证明边界。diameter_circle_covers=True 是容差下的检查结果；这个函数没有重新优化圆心。空列表不满足本函数的调用前提，直接传空可能在读取 pair 时失败。

- [L184] def _finite_result(vertices: list[Point]) -> dict:: 定义有限结果包装函数，接受顶点列表。
- [L185] vertices = _convex_hull(vertices): 再次求凸包，确保去重、边界有序和去除共线中间点。
- [L186] if len(vertices) == 2 and _distance2(vertices[0], vertices[1]) <= EPS * EPS:: 若整理后只剩两点，且平方距离不超过 EPS^2 ，把极短线段按一点处理。
- [L187] vertices = vertices[:1]: 保留第一个点，构成单点列表。
- [L188] status = "point" if len(vertices) == 1 else "segment" if len(vertices) == ...: 按点数生成状态：1 为 point, 2 为 segment, 其他正常非空情形为 polygon；嵌套条件表达式不是三次计算。
- [L189] diameter, pair = polygon_diameter(vertices): 调用直径函数取得距离及实现它的点对。
- [L190] area = abs(sum(_cross(vertices[i], vertices[(i + 1) % len(vertices)])): 开始鞋带公式：对每个顶点与下一个顶点取叉积、求和、取绝对值。
- [L191] for i in range(len(vertices))): / 2.0: 遍历全部顶点并除以 2；两行合起来是一条面积赋值。
- [L192] midpoint = ((pair[0][0] + pair[1][0]) / 2.0, (pair[0][1] + pair[1][1]) / ...: 分别平均最远点对的横、纵坐标，得到其连线中点。
- [L193] radius = diameter / 2.0: 半直径作为这一个“直径圆”的半径。
- [L194] farthest = max(math.dist(midpoint, point) for point in vertices): 计算所有顶点到中点的最大距离；这是实际检查圆是否漏点的量。
- [L195] return {"status": status, "vertices_m": [list(p) for p in vertices],: 开始返回字典，存状态和顶点坐标；元组转列表便于 JSON 输出。
- [L196] "diameter_m": diameter, "diameter_pair_m": [list(p) for p in pair],: 存直径数值与最远点对坐标。

- [L197] "area_m2": area, "diameter_circle_covers": farthest <= radius + EPS,: 存面积; 用最大顶点距离不超过半径加容差来生成覆盖布尔值。
- [L198] "diameter_circle_cover_margin_m": radius - farthest}: 存“半径减最远顶点距离”的带符号裕量; 负值通常表示漏点, 极小负值也可能仍在容差内。

带答案反例。三角形 $A = (0, 0), B = (4, 0), C = (2, 3)$ 的最长边为 $AB = 4$ 。程序得到面积 6、直径圆心 $(2, 0)$ 、半径 2; C 离圆心 3, 所以 `diameter_circle_covers` 为 `False`, 覆盖裕量为 -1 。三点的最小包围圆半径其实为 $13/6$, 但那是另一个优化问题, 由 Q2 的相应函数求解。

为什么固定输入的裕量是极小负数, 布尔值却是真? 仓库固定例为约 -2.49×10^{-14} 米, 绝对值小于 10^{-10} 米; 第 197 行允许该浮点差, 因此返回真。它不是一个额外的物理测量误差。

1.18 intersect_halfplanes: 主求交与退化分类

输入/输出。输入半平面集合, 返回分类字典。有限非空结果可为 `polygon`、`segment`、`point`; 其他状态为 `empty`、`unbounded` 或 `uncertain`。

队列不变量。已处理的约束按边界方向有序; 队列存放仍可能作为有效边界的半平面。新约束排除队尾或队首相邻边界的交点时, 相应旧边界从候选队列删除。最后还要检查首尾闭合。

数学对应。要求的是 $P = \bigcap_i \{x : n_i \cdot x \leq c_i\}$, 不是求所有边界交点的凸包。队列先筛边, 再检查候选点满足输入约束; 空集和无界集须另行处理。

证明边界。队列法在精确算术、正确方向和适当退化处理下成立。这里使用数值容差与兜底检查; `uncertain` 明确保留“当前构造不能稳定确认”的分支。不能从有限个可行顶点单独推出已穷尽整个交集, 完整结论还依赖队列算法。代码第 225 行实际复核 `_ordered` 后的全部保留约束; 对于精确同向约束, 它们与原约束等价。

复杂度。排序和常规队列阶段约为 $O(m \log m)$; 第 225 行逐顶点逐约束验证为 $O(mv)$, 兜底可行点搜索可到 $O(m^2)$ 。因此不能把整个实现无条件称为 $O(m \log m)$ 。

- [L201] `def intersect_halfplanes(halfplanes: Iterable[HalfPlane], *, include_circ...:` 定义主求交函数; 星号后的 `include_circle` 必须用关键字传参, 默认真。
- [L202] `"""用按方向排序的双端队列计算半平面交。"""`: 说明主要算法是按方向排序的双端队列。
- [L203] `del include_circle # 仅保留该参数以兼容第二问的调用接口。:` 删除兼容参数。当前实现并不因它为真而加入场地圆, 也不因它为假而跳过直径圆覆盖检查。注释说明保留名称是为了兼容 Q2 调用。
- [L204] `planes = _ordered(halfplanes):` 归一化、排序并合并同向约束。
- [L205] `active: deque[HalfPlane] = deque():` 建立存放半平面的空双端队列。
- [L206] `for hp in planes:` 按方向次序加入新半平面。
- [L207] `while len(active) >= 2 and _cuts(hp, active[-2], active[-1]):` 若队尾两条旧边界交点被新约束排除, 就重复清理尾部; 至少两条才有可检查的交点。
- [L208] `active.pop():` 弹出队尾一条旧边界; 下一轮重新检查新的尾部相邻两条。
- [L209] `while len(active) >= 2 and _cuts(hp, active[0], active[1]):` 对队首做同样检查, 处理从另一端失效的拐角。
- [L210] `active.popleft():` 弹出最前面的旧边界; 这就是使用双端队列而非普通栈的原因。
- [L211] `active.append(hp):` 两端清理完成后加入新边界。
- [L212] `while len(active) >= 3 and _cuts(active[0], active[-2], active[-1]):` 所有约束加入后, 用首条约束检查尾端交点, 完成环形闭合的一侧清理。
- [L213] `active.pop():` 尾端交点不合格则继续弹出尾边。
- [L214] `while len(active) >= 3 and _cuts(active[-1], active[0], active[1]):` 反过来, 用末条约束检查首端交点, 完成另一侧闭合清理。

- [L215] `active.popleft()`: 首端交点不合格则继续弹出首边。
- [L217] `ring = list(active)`: 把队列转换为列表, 便于索引循环相邻元素。
- [L218] `vertices = [_intersection(ring[i], ring[(i + 1) % len(ring)]) for i in ra...]`: 求每对相邻边界的交点, 末条与首条也相邻; 列表里可能有 `None`。队列为空时循环不执行, 因此不会对零取模。
- [L219] `if ring and all(point is not None for point in vertices):`: 要求有边界且全部相邻交点存在, 才尝试构成有限候选图形。
- [L220] `polygon = _convex_hull(vertices)`: 整理相邻交点的凸包, 消除重复或近共线拐角。
- [L221] `# A bounded intersection may legitimately degenerate to one point or a:` 注释提醒: 有界交集可能退化成一个点。
- [L222] `# line segment. Accept those only when the recession cone is bounded;` 注释接着包含线段情形, 并要求结合退行方向的有界性检查。
- [L223] `# otherwise two adjacent half-planes could be mistaken for a point.:` 注释说明理由: 两条非平行边的交点不等于整个半平面交是一个点; 交集可能是无限角域。
- [L224] `finite_shape = len(polygon) >= 3 or (polygon and _is_bounded(planes))`: 三点以上形成有限候选多边形; 只有一点或两点时, 还必须通过有界方向检查才接受为有限形状。
- [L225] `if finite_shape and all(not _outside(hp, point) for hp in planes for poin...)`: 既要通过有限形状条件, 又要确保每个候选顶点都满足预处理后每条约束; `all` 遇到第一项不合格会停止。
- [L226] `return _finite_result(polygon)`: 条件通过, 计算有限形状的各项指标并返回。
- [L228] `witness = _feasible_point(planes)`: 若常规构造未返回结果, 另找一个可行点作分类依据。
- [L229] `if witness is None:`: 检查可行点搜索是否发现矛盾。
- [L230] `return {"status": "empty", "vertices_m": [], "diameter_m": None, ...}`: 无可行点时开始返回空集状态; 空集没有顶点及本程序定义的直径。
- [L231] `"diameter_pair_m": None, "area_m2": None}`: 点对、面积均用 `None`, 避免把“无定义”写成数值零。
- [L232] `if not _is_bounded(planes):`: 有可行点时, 再检查是否存在退行方向, 即是否不能判为有界。
- [L233] `return {"status": "unbounded", "vertices_m": [], "diameter_m": None, ...}`: 若无界, 返回 `unbounded`; 顶点字段留空, 直径不输出有限值。
- [L234] `"diameter_pair_m": None, "area_m2": None, ...`: 点对、面积仍用 `None`。
- [L235] `"feasible_point_m": list(witness)}`: 额外记录一个可行点, 证明返回状态并非仅凭空列表猜测。
- [L236] `return {"status": "uncertain", "vertices_m": [], "diameter_m": None, ...}`: 若有可行点、方向条件看似有界, 却未获得可信有限形状, 则返回 `uncertain`。
- [L237] `"diameter_pair_m": None, "area_m2": None}`: 该分支不伪造面积或点对; 需要进一步检查数值退化或输入情况。

主循环手算: 同向合并、尾部弹出与闭合

依次输入 $y \geq 0, x \leq 5, x \leq 4, x + y \leq 3, y \leq 4, x \geq 0$ 。代码用 (a, b, c) 表示 $ax + by \leq c$, 因此输入元组依次为

$[(0, -1, 0), (1, 0, 5), (1, 0, 4), (1, 1, 3), (0, 1, 4), (-1, 0, 0)]$

预处理把 $x \leq 5, x \leq 4$ 合成 $x \leq 4$; $(1, 1, 3)$ 同除 $\sqrt{2}$ 。边界方向次序为 $0^\circ, 90^\circ, 135^\circ, 180^\circ, 270^\circ$ 。

新边界	被检查的旧尾交点	第 208 行是否弹出	加完后的队列
$y \geq 0$	尚不足两边	否	$[y \geq 0]$

新边界	被检查的旧尾交点	第 208 行是否弹出	加完后的队列
$x \leq 4$	尚不足两边	否	$[y \geq 0, x \leq 4]$
$x + y \leq 3$	$(4, 0)$, 违反 $4 + 0 \leq 3$	弹 $x \leq 4$	$[y \geq 0, x + y \leq 3]$
$y \leq 4$	$(3, 0)$, 满足	否	$[y \geq 0, x + y \leq 3, y \leq 4]$
$x \geq 0$	$(-1, 4)$, 违反	弹 $y \leq 4$	$[y \geq 0, x + y \leq 3, x \geq 0]$

闭合后交点为 $(3, 0), (0, 3), (0, 0)$ 。复核全部保留约束后，输出三角形：面积 $9/2$ ，直径 $3\sqrt{2} \approx 4.2426406871$ 。本例触发两次尾部删除，没有触发头部删除；不能由此断言头部代码没用。

头部弹出与最后闭合：再给一个实际触发例

按方向序输入 $y \geq -4, x - y \leq 4, x + y \leq 5, x \geq 1, x + y \geq -5$ 。在加入 $x \geq 1$ 前，队列为前三条。旧尾交点 $(4.5, 0.5)$ 满足 $x \geq 1$ ，所以尾部不弹；旧首交点 $(0, -4)$ 却违反 $x \geq 1$ ，因此第 210 行真正执行，弹掉 $y \geq -4$ 。

最后加入 $x + y \geq -5$ 后，队列暂为 $[x - y \leq 4, x + y \leq 5, x \geq 1, x + y \geq -5]$ 。闭合阶段，第 212 行用首条 $x - y \leq 4$ 检查末两条交点 $(1, -6)$ ；它的 $x - y = 7 > 4$ ，因此第 213 行弹掉最后的 $x + y \geq -5$ 。最终三角形顶点为 $(1, -3), (4.5, 0.5), (1, 4)$ ，面积 12.25，直径 7。这一个输入同时展示“头部删除”和“闭合后仍须再删”。

退化分类：输出字段应该怎么读

约束	数学答案	程序状态与关键输出
$x \leq 0, x \geq 1$	相加得 $0 \leq -1$ ，空集	empty，直径为 None
$x \geq 0, y \geq 0$	第一象限，含无限射线	unbounded，可行点可为 $(0, 0)$
$x = 0, -1 \leq y \leq 1$	长 2 的竖直线段	segment，面积 0、直径 2
$x = 0, y = 0$	原点	point，面积和直径均为 0

问：为何检查到几个合法顶点还不一定够？答：两条不平行边界交于一点，但两个半平面的交一般是无界角域。必须结合边界结构和退行方向，不能把唯一交点当成全集。

问：为什么 None 不用数字 0 代替？答：0 表示已计算出长度为零；None 表示此分支没有有限直径或相应量未被确认。把两者混同会把无界区域误读成精确定位。

问：新增约束后区域会扩大吗？答：精确模型 $P_{k+1} = P_k \cap H_{k+1} \subseteq P_k$ ，不扩大；顶点数却可能增加。浮点表示与容差须另行看待。

1.19 _observation

输入/输出。接受字典形式的观测，或长为 3 的 $(x, y, \text{bearing})$ 。返回统一的浮点三元组 (x, y, angle) ，角度归到 $[0, 360)$ 。

不变量。点位与示向度仍属于同一次观测；只改变表示形式，不估计误差，不计算真值。

数学对应。方位角每加减 360° 对应同一方向，因此取模不改变角楔。

证明边界。只保证通过的三个数有限。字段缺失、点位不可解包等情况可能直接出现 TypeError 或 KeyError；本函数并未把所有格式错误统一成一个自定义异常。

- [L240] `def _observation(row) -> tuple[float, float, float]`：定义观测解析函数，输出三个浮点数。
- [L241] `if isinstance(row, dict)`：先判断是否为字典输入。
- [L242] `position = row.get("position_m", row.get("position"))`：优先取 `position_m`，缺少该键时取兼容字段 `position`；若键存在但值为 None，不会自动换用备用字段。

- [L243] `if isinstance(position, dict):`: 如果点位本身也是字典, 按下一行读取横纵坐标。
- [L244] `position = (position["x"], position["y"])`: 把 `{"x":..., "y":...}` 转成二元组; 缺少字段会报错。
- [L245] `bearing = row.get("bearing_deg", row.get("svd_deg"))`: 优先用 `bearing_deg`, 不存在时用兼容名 `svd_deg`。
- [L246] `x, y = position`: 把位置解包成 `x, y`。
- [L247] `else:`: 非字典则按序列格式解释。
- [L248] `if len(row) != 3:`: 要求序列恰有三个元素。
- [L249] `raise ValueError("每条观测必须包含 x、y 和示向度")`: 不满足长度要求时抛出明确错误。
- [L250] `x, y, bearing = row`: 按次序读取横坐标、纵坐标、示向角; 次序不可互换。
- [L251] `x, y, bearing = float(x), float(y), float(bearing)`: 将三项统一转成浮点数。
- [L252] `if not all(math.isfinite(v) for v in (x, y, bearing)):`: 拒绝 NaN 或无穷等不能形成正常几何约束的数。
- [L253] `raise ValueError("观测数据必须为有限数")`: 有限性失败时抛出错误。
- [L254] `return x, y, bearing % 360.0`: 返回坐标及对 360 取模后的示向度, 例如 -1 度变为 359 度。

1.20 `_wedge`

输入/输出。输入检测点 s 、报告方向 θ 和误差半宽 δ , 返回 2 个闭半平面; $\delta = 0$ 时返回 3 个。

数学对应。下边界方向 $u(a)$ 左侧条件为 $\text{cross}(u(a), x - s) \geq 0$, 整理成法向 $(\sin a, -\cos a)$; 上边界右侧条件整理成法向 $(-\sin b, \cos b)$, 均写成 $n \cdot x \leq n \cdot s$ 。

不变量。每条边界经过检测点; 对 $0 < \delta < 90^\circ$, 两侧交成前向角楔。零误差额外用前向不等式排除背向射线。

证明边界。`_wedge` 自身没有检查半宽范围, 它依赖调用者 `solve_localization` 的校验。模型只包含角度, 没有最大接收半径和目标场地圆。

- [L257] `def _wedge(position: Point, bearing_deg: float, epsilon_deg: float) -> li...`: 定义一条观测到半平面列表的转换。
- [L258] `x, y = position`: 取站点横、纵坐标。
- [L259] `lower, upper = map(math.radians, (bearing_deg - epsilon_deg, bearing_deg ...)`: 先求角度上下界, 再一起转成弧度; 无需先把上下界各自取模, 三角函数本来具有周期性。
- [L260] `normals = [(math.sin(lower), -math.cos(lower))]`: 建立下边界左侧的法向量。
- [L261] `(-math.sin(upper), math.cos(upper))`: 建立上边界右侧的法向量; 这两行合成含两个元素的列表。
- [L262] `if epsilon_deg == 0.0:`: 检测严格零误差这一特殊情形; 零附近的非零数仍走普通窄角楔模型。
- [L263] `angle = math.radians(bearing_deg)`: 把报告方向转为弧度。
- [L264] `normals.append((-math.cos(angle), -math.sin(angle)))`: 追加 $-u(\theta)$ 法向, 使 $-u(\theta) \cdot (x - s) \leq 0$, 保留向前的半条直线。
- [L265] `return [HalfPlane(normal, normal[0] * x + normal[1] * y) for normal in no...`: 对每个法向计算偏移 $n \cdot s$ 并包装为半平面对象; 这是把经过站点的几何边界写成标准形式。

必会代换：读数是 0° 时，符号应朝哪边

令站点 $(0,0)$ 、读数 0° 、半宽 1° 。两个法向约为 $(-0.0174524, -0.9998477)$ 与 $(-0.0174524, 0.9998477)$ 。两不等式合起来是

$$-\tan 1^\circ x \leq y \leq \tan 1^\circ x.$$

点 $(100,0)$ 可行，点 $(-100,0)$ 不可行； $(100,2)$ 也不可行，因为 $2 > 100 \tan 1^\circ \approx 1.7455$ 。若半宽改成 0，两原约束只给 $y = 0$ ；第 264 行再给 $x \geq 0$ 。

问：报告 359.5° 、误差 1 度跨过零点怎么办？答：直接对 358.5° 和 360.5° 取正弦余弦，周期性自然得到跨零的楔，不用把它误拆成几乎整周的角度区间。

1.21 solve_localization

输入/输出。输入观测可迭代对象、误差半宽与兼容参数；返回几何结果，并附规范化观测与实际采用的半宽。

不变量。所有观测针对同一未知点；每条观测的全部半平面共同参与同一次交集运算。

数学对应。列表推导式实现式 (Q1.1)，而非先求每条射线交点再取平均。

证明边界。函数检查半宽范围与非空观测，但不要求观测一定能围住目标；即使只有一个站，也允许运行并返回无界状态。生成器至少给三站，不是该求解函数的硬性条件。

- [L268] `def solve_localization(observations, epsilon_deg=1.0, *, include_circle=T...`：定义可被其他 Python 程序直接调用的定位接口，默认角误差半宽为 1 度。
- [L269] `epsilon_deg = float(epsilon_deg)`：把半宽统一成浮点数。
- [L270] `if not 0.0 <= epsilon_deg < 90.0:`：检查半宽在左闭右开区间 $[0,90)$ ；NaN 或无穷也不能通过此条件。
- [L271] `raise ValueError("epsilon_deg 必须位于 [0,90) 内")`：不合范围就抛错，避免把不适用的两个半平面公式用于宽角域。
- [L272] `rows = [_observation(row) for row in observations]`：逐条解析观测，得到统一的 $(x,y,angle)$ 列表。
- [L273] `if not rows:`：检查是否至少存在一条观测。
- [L274] `raise ValueError("至少需要一条观测")`：没有观测就报错；不把整个平面当成有效的定位输入。
- [L275] `planes = [hp for x, y, angle in rows for hp in _wedge(x, y, angle, epsi...`：外层遍历各条观测，内层遍历该观测产生的半平面，把所有约束展平成一个列表。
- [L276] `result = intersect_halfplanes(planes, include_circle=include_circle)`：调用主半平面交函数；参数虽继续传递，但第 203 行会将其删除。
- [L277] `result.update(observations=[{"position_m": [x, y], "bearing_deg": angle}]`：开始补充结果字段：将内部三元组转换回标准观测字典。
- [L278] `for x, y, angle in rows],:`：对每条规范化观测生成一个字典，本行结束列表推导式。
- [L279] `error_half_width_deg=epsilon_deg)`：记录真正用于建模的误差半宽，便于读结果时核对前提。
- [L280] `return result`：返回同时包含几何结果与输入摘要的字典。

1.22 main 与命令行入口

输入/输出。 `main(argv=None)` 接受命令行参数列表；None 表示使用进程实际命令行。读取输入 JSON，写结果 JSON，向终端打印简要信息，正常结束返回整数 0。

不变量。输出的 `case_id` 与输入案例对应；求解只读取 `observations` 和可选 `epsilon_deg`，没有读取 `local_truth`。

数学对应。本节只将模型输入、求解结果搬进搬出文件，未增加几何条件。

证明边界。本函数没有捕获文件不存在、JSON 损坏、格式错误等异常；这类错误会让 Python 非零退出。状态 `empty`、`unbounded` 或 `uncertain` 是成功计算后的分类，也可正常返回退出码 0，退出码不代表“定位精度合格”。

- [L283] `def main(argv=None) -> int`: 定义主入口，允许测试时手工传参数列表。
- [L284] `parser = argparse.ArgumentParser(description=__doc__)`: 创建参数解析器，并把模块说明用作帮助文字。
- [L285] `parser.add_argument("--input", type=Path, required=True, help="观测 JSON 文件")`: 增加必填 `--input` 参数，直接转换为 `Path`；没给参数时由解析器报错。
- [L286] `parser.add_argument("--output", type=Path, default=Path("q1_result.json"))`: 增加 `--output` 参数；不写时在当前工作目录输出 `q1_result.json`。
- [L287] `args = parser.parse_args(argv)`: 解析参数，得到有 `input` 和 `output` 属性的对象。
- [L288] `payload = json.loads(args.input.read_text(encoding="utf-8"))`: 按 UTF-8 读取输入文本，再解码成 Python 字典或其他 JSON 对象；此处预期后面可按字段读取。
- [L289] `result = solve_localization(payload["observations"], payload.get("epsilon...: 取观测并求解；输入不写 epsilon_deg 时用默认 1 度。真值字段未参与计算。`
- [L290] `result["case_id"] = payload.get("case_id", args.input.stem)`: 优先沿用输入案例编号；没有该键时使用输入文件去扩展名后的文件名。
- [L291] `args.output.parent.mkdir(parents=True, exist_ok=True)`: 创建输出父目录，允许递归创建且目录已存在时不报错。
- [L292] `args.output.write_text(json.dumps(result, ensure_ascii=False, indent=2) + ...`: 把字典转成缩进两格的 JSON，保留中文字符，追加换行，再以 UTF-8 写入；同名文件会被覆盖。
- [L293] `print(f"状态: {result['status']}")`: 向终端打印分类状态。
- [L294] `print(f"顶点数: {len(result['vertices_m'])}")`: 打印返回顶点个数；无界和空集都可能是 0，必须结合状态读。
- [L295] `print(f"面积: {result['area_m2']:.3f} m² if result['area_m2'] is not None ...`: 面积非 `None` 时保留三位小数打印，否则写“无有限值”；面积 0 会正常打印为 0.000。
- [L296] `print(f"直径: {result['diameter_m']:.3f} m if result['diameter_m'] is not ...`: 直径同样区分数值与 `None`，避免格式化空值报错。
- [L297] `print(f"结果已写入: {args.output}")`: 打印结果路径，便于找到输出文件。
- [L298] `return 0`: 正常结束返回退出码 0。
- [L301] `if __name__ == "__main__":`: 只有作为脚本直接运行才触发入口；被其他模块导入时不自动读文件。
- [L302] `raise SystemExit(main())`: 调用 `main`，把其返回整数交给系统作为进程退出码。

一次真实数据贯穿所有函数

仓库 `examples/q1_seed_20260929_input.json` 含 4 条观测，`local_truth` 只供检查。沿主调用跟踪：4 个读数经 `_wedge` 形成 8 条角度半平面；预处理后队列求交；返回 6 个顶点。结果为 `polygon`，面积约 773.5172573 平方米，直径约 44.4585640 米，直径圆覆盖布尔值为真。这些是此输入的复算结果，不能由此推出任意四站都有相同精度。

从仓库根目录运行：

```
python source/Q1/q1_solver.py \  
--input examples/q1_seed_20260929_input.json \  
--output q1_review_result.json
```

结果写到当前工作目录。比较输出时应看数值和状态，不应要求 JSON 键顺序或最远点对端点次序成为数学结论。

2 q1_generator.py

源码：source/Q1/q1_generator.py。共 43 个物理行、40 个非空行。SHA-256 前 16 位：33e7c5d2c751054d。

2.1 模块与常量

生成器解决的任务是“先知道一个真值，再制造满足角误差界的观测”。求解器解决的任务是“只看观测，反推允许位置集”。两个方向相反；演示时把这一点先说清，就不会误以为求解器偷读真值。

- [L001] `#!/usr/bin/env python3`：指定脚本解释器为 Python 3；手工调用 Python 时它只是注释。
- [L002] `from __future__ import annotations`：启用延后处理类型注解；本文件注解很少，不影响随机算法。
- [L003] `import argparse,json,math,random`：一行导入四个模块：argparse 管参数，json 管格式，math 管几何，random 管随机数。
- [L004] `from pathlib import Path`：导入路径处理工具，用于创建输出位置。
- [L006] `ARENA_R=1800.0`：设置场地尺度 1800 米，供生成源真值时使用；它没有自动传入求解器进行圆域裁剪。

2.2 main

输入。命令行可给随机种子、检测点数及输出文件。默认点数 4；实际至少取 3。

输出。一个 JSON，含案例编号、种子、角误差半宽、观测列表与 `local_truth.source_m`。此外终端打印生成信息。函数没有显式 `return`，正常返回 `None`。

构造不变量。始终先固定同一个源真值；每个检测点的报告均从该真值的方向生成，并在两位小数舍入后再次检查最终环形角误差不超过 1 度。

数学对应。若 U, V 独立均匀分布于 $[0, 1)$ ，则 $\phi = 2\pi U, r = 0.82R\sqrt{V}$ 在半径 $0.82R$ 的圆盘内按面积均匀取点。因为 $\Pr(r \leq t) = t^2/(0.82R)^2$ ，正好与圆盘面积成比例。检测点取在真值周围不同方向，距离抽于 $[500, 1450]$ 米。

证明边界。这是角度几何测试数据生成器，没有模拟未知接收半径、频道或官方动作成本；“误差合法”不能被说成“复刻了全部官方物理接口”。检测点位置不限制在场地圆内；源仅在半径 1476 米内抽样，未覆盖所有半径 1800 米圆域的边缘真值。

- [L008] `def main():`：定义生成器的主函数。
- [L009] `ap=argparse.ArgumentParser(description='问题一随机测试数据生成器')`：创建命令行解析器，说明用途为随机测试数据生成。
- [L010] `ap.add_argument('--seed',type=int,default=None)`：增加整数种子选项；默认 `None` 表示稍后自动选种子。
- [L011] `ap.add_argument('--points',type=int,default=4)`：增加检测点数选项，默认 4。
- [L012] `ap.add_argument('--output',type=Path,default=Path('q1_input.json'))`：增加输出路径选项，默认当前工作目录的 `q1_input.json`。
- [L013] `args=ap.parse_args()`：解析当前进程参数；与求解器不同，这个 `main` 没有 `argv` 形参。

- [L014] `seed=args.seed if args.seed is not None else random.SystemRandom().randra...`: 用户给种子则直接使用; 否则用系统随机源从 1 到 $2^{63} - 1$ 选一个种子。保存种子以后, 同一实现中可以复现实例。
- [L015] `rng=random.Random(seed)`: 创建本地伪随机数发生器, 后续所有抽样使用它, 避免依赖全局随机状态。
- [L016] `n=max(3,args.points)`: 将检测点数下限设为 3; 输入 1 或负数也会改为 3, 而不是抛错。
- [L017] `ang=rng.random()*math.tau`: 抽取源真值的极角, 范围为整周 $[0, 2\pi)$ 。
- [L018] `rr=ARENA_R*math.sqrt(rng.random())*0.82`: 抽取真值半径, 使用平方根使面积均匀, 再乘 0.82 留出部分内区; 若直接用均匀半径, 会偏重圆心附近的面积密度。
- [L019] `source=(rr*math.cos(ang),rr*math.sin(ang))`: 把极坐标转换为源的笛卡尔坐标。
- [L020] `observations=[]`: 建立空观测列表。
- [L021] `base=rng.random()*math.tau`: 抽取检测点总体分布的一个随机基准角。
- [L022] `for k in range(n):`: 依次制造实际数量 n 个检测点与方向报告。
- [L023] `a=base+2*math.pi*k/n+rng.uniform(-0.20,0.20)`: 基准角加等分角, 再加入正负 0.20 弧度的小扰动; 此行扰动单位是弧度, 不是示向误差的 1 度。
- [L024] `d=rng.uniform(500.0,1450.0)`: 抽取站到源的目标距离, 范围 500 至 1450 米。
- [L025] `p=(round(source[0]+d*math.cos(a),6),round(source[1]+d*math.sin(a),6))`: 以源为圆心放置检测点, 并把坐标保留六位小数; 此后真方向按舍入后的站坐标重算。
- [L026] `true=math.degrees(math.atan2(source[1]-p[1],source[0]-p[0]))%360`: 用 $\text{atan2}(\text{源纵差}, \text{源横差})$ 得到从站看源的真方向, 转为度并归到 $[0, 360)$ 。
- [L027] `# The simulator reports two decimals. Rejection after quantisation`: 注释说明要模拟两位小数的报告格式。
- [L028] `# guarantees that the generated final datum still obeys the  $\pm 1^\circ$  bound.`: 注释说明舍入后仍需拒绝超界数据, 从而保证最终存盘读数满足误差界。
- [L029] `while True:`: 进入拒绝采样循环; 直到量化后的读数仍合规才退出。
- [L030] `report=round((true+rng.uniform(-1.0,1.0))%360,2)%360.0`: 先加正负 1 度的均匀扰动, 再归一到整周, 保留两位小数, 再对 360 取模处理舍入成 360.00 的情形。
- [L031] `final_error=(report-true+180.0)%360.0-180.0`: 把“报告减真值”映射到 $[-180, 180)$, 得到跨越 0 度也正确的有符号最短角差。
- [L032] `if abs(final_error)<=1.0:break`: 同一物理行先判断最终误差绝对值是否不大于 1, 再执行 `break`; 不合格就回到第 30 行重新抽扰动。
- [L033] `observations.append({'position_m':[p[0],p[1]],'bearing_deg':report})`: 将本次站位和合规报告放入观测列表; 没有把真方向作为求解器字段。
- [L034] `payload={'case_id':f'Q1-seed-{seed}','seed':seed,'epsilon_deg':1.0,`: 开始构造输出字典: 案例编号中含种子, 同时单独记录种子和 1 度半宽。
- [L035] `'observations':observations,`: 存入已完成的所有观测。
- [L036] `'local_truth':{'source_m':[source[0],source[1]]}`: 将源真值放在专门的 `local_truth` 内, 供学习者核对; 求解器不会读取它。
- [L037] `args.output.parent.mkdir(parents=True,exist_ok=True)`: 创建输出父目录, 允许其已存在。
- [L038] `args.output.write_text(json.dumps(payload,ensure_ascii=False,indent=2),en...`: 把数据用 UTF-8、中文不转义、两格缩进写成 JSON; 已有同名文件将被覆盖。
- [L039] `print(f'随机种子: {seed}')`: 打印种子, 方便下次复现。

- [L040] `print(f'检测点数量: {n}')`: 打印实际使用的检测点数, 可能不同于用户输入的小于 3 的值。
- [L041] `print(f'测试数据已保存: {args.output}')`: 打印输出文件路径。
- [L043] `if __name__ == '__main__': main()`: 同一行包含入口判断与调用: 直接执行文件才运行生成器, 导入时不自动产生数据。

拒绝采样: 为何加了合法噪声还要重抽

舍入前的扰动即使在允许区间内, 最终两位小数报告仍可能越界。程序检查的是最后写入文件的数, 而不只检查刚抽出的噪声。

完整数值例。真方向为 0.004° , 噪声取 -0.9995° , 舍入前归一值为 359.0045° , 两位小数得到 359.00° , 最终差为 -1.004° , 应重抽。若随后噪声为 $+0.496^\circ$, 报告为 0.50° , 最终差为 0.496° , 接受。

问: 直接算 `abs(report-true)` 有什么错? 答: 真方向 359.9° 、报告 0.1° 实际只差 0.2° , 普通减法却给约 359.8° 。环形差 $(\text{差}+180)\%360-180$ 正是为此使用。

问: 种子等于 20260929 会证明算法普遍正确吗? 答: 种子只是复现输入; 一次输出对应一个实例, 不能代替几何证明, 也不能代表所有布站或误差组合。

问: `local_truth` 能删吗? 答: 求解器的 `main` 只读 `observations`、`epsilon_deg` 和案例编号, 因此删掉真值字段不影响这份观测的求解; 生成器与教学核对才会用到它。

固定种子生成并求解的一组命令为:

```
python source/Q1/q1_generator.py --seed 20260929 --points 4 --output q1_input.json
python source/Q1/q1_solver.py --input q1_input.json --output q1_result.json
```

这两个命令都在仓库根目录执行, 输出路径相对于当前工作目录。不要混用 `run.bat` 的工作目录后再猜结果在哪个文件夹。

3 run.bat

源码: `source/Q1/run.bat`。12 个物理行, 全部非空。SHA-256 前 16 位: 652454be48f97e36。

输入/输出。Windows 批处理脚本, 无自定义参数。先在脚本所在目录生成随机输入, 再调用求解器, 最后暂停以供查看。

不变量。两个 Python 命令在同一目录运行; 若生成失败, 不继续用不完整或旧输入求解; 结束前恢复启动脚本之前的工作目录。

数学对应。这是运行顺序控制, 没有新的数学方法。

边界。当前脚本不指定种子, 每次自动生成新实例; 固定种子演示应使用上一节的显式命令。脚本面向 Windows, Linux/macOS 学习者直接调用两个 Python 文件。

- [L001] `@echo off`: 关闭批处理逐条显示命令; 开头 `@` 还让这一行自身不回显。
- [L002] `setlocal`: 把后续环境变量修改限制在当前批处理范围内。
- [L003] `chcp 65001 >nul`: 把控制台代码页改为 UTF-8, 帮助显示中文; `>nul` 丢弃这条命令的普通提示输出。
- [L004] `pushd "%~dp0" || exit /b 1`: 把当前目录切换到此批处理自身所在目录, 先保存原目录; 双引号支持含空格的路径。若切换失败, `||` 后立即以状态 1 退出。
- [L005] `python q1_generator.py --output q1_input.json`: 运行生成器并指定输入文件名, 没有提供 `--seed`, 因此使用自动种子。
- [L006] `if errorlevel 1 goto finish`: 若上一命令退出码至少为 1, 就跳到 `finish`, 避免继续运行求解器。`errorlevel 1` 是“至少为 1”的检查。

- [L007] `python q1_solver.py --input q1_input.json --output q1_result.json`: 生成成功后, 读取刚才的 JSON 并写出定位结果 JSON。
- [L008] `:finish`: 定义结束处理标签, 既可由错误跳转到达, 也可由上一行执行后顺序到达。
- [L009] `set "RC=%ERRORLEVEL%"`: 先把当前退出码保存到 RC; 外层双引号是 `set` 的写法, 不成为变量值的一部分。
- [L010] `popd`: 恢复 `pushd` 保存的原工作目录。
- [L011] `pause`: 等待按键, 让用户来得及看结果; 这一步不计算也不验证定位正确性。
- [L012] `exit /b %RC%`: 以事先保存的状态码退出当前批处理; 避免 `popd` 或 `pause` 改变最后报告的状态。

4 与现有教材逐项核对: 哪些说法必须带上前提

本节只记录教材与代码的对应边界。没有修改算法或假装补入未实现的功能。

教材中学到的标准结论	本代码的证据	答辩时完整说法
目标圆域先验可与角楔取交	求解器 203 行删除 <code>include_circle</code> ; 275 行只造角楔	本实现计算角度约束交集; 场地圆是另一个可用先验, 但当前 Q1 未加入它
最小包围圆求最优覆盖中心	192–198 行只用最远点对中点检查一个圆	返回的是直径圆是否覆盖; 没有在当前 Q1 中求最小包围圆
零角误差应给射线	262–264 行补前向半平面	两条重合角边只给直线, 第三条才去掉背向部分
方向排序队列阶段可达 $O(m \log m)$	225 行全顶点全约束复核; 119 行可行点兜底嵌套	排序队列的复杂度与完整实现的最坏复杂度分开讲
凸包与卡壳有精确几何证明	152、175 等行用了 EPS	理论建立在相应精确条件上, 当前程序用浮点容差落实, 不能把阈值当严密误差认证
全部原约束应得到满足	204 行先规范化合并, 225 行遍历 <code>planes</code>	实际复核的是预处理后的保留约束; 精确同向合并与原约束等价, 近似合并依赖容差
对踵点方法可求直径	174–180 行只保留当前一侧指针并更新一对端点	输出一组最大距离及实现点对, 不声称列出了所有对踵点和全部并列最远点对
三站可以提供定位信息	生成器 16 行至少三站; 求解器 273 行只要求一站	站数本身不保证有界或高精度, 仍要检查布站几何与求交状态

`course/01` 已明确说明场地圆未加入, `course/02` 已区分直径圆和最小包围圆, `course/11` 已说明空、无界、线段的分类。以上主要是对照时必须保留的限定。一个应细化的表述是“最终用全部原约束复核”: 源码实际循环变量是 `_ordered` 输出的 `planes`, 应按表中说法理解。当前未从所核对部分发现足以支持“实际默认求解了场地圆”或“Q1 已求最小包围圆”的代码证据。

5 闭卷验收: 每题都给标准答案

5.1 第一层: 先写清问题, 不背函数名

题 1. 给两站 $s_1 = (0, 0)$, $s_2 = (2, 0)$, 两报告分别为 $45^\circ, 135^\circ$, 误差半宽为 0。写出允许集合与程序状态。

答案。第一站给 $y = x, x \geq 0$; 第二站给 $y = 2 - x, x \leq 2$ 。联立得唯一点 $(1, 1)$, 状态 `point`, 直径与面积均为 0。若遗漏前向条件, 这个特例仍凑巧给同一点, 但不能据此删掉通用实现的前向分支。

题 2。同样两站都报告 0° 、误差为 0，答案是什么？

答案。第一站给 $y = 0, x \geq 0$ ，第二站给 $y = 0, x \geq 2$ 。交为 $y = 0, x \geq 2$ ，是一条无界射线，状态 `unbounded`。即使站数增加，也未必形成有界区。

题 3。把一条约束 $2x \leq 6$ 归一化，会变成什么？哪行负责？

答案。法向长为 2，同时除法向与偏移得到 $x \leq 3$ 。对应 `_normalise` 第 56–59 行；只把法向变成 1、偏移仍留 6 会错误地变为 $x \leq 6$ 。

5.2 第二层：把一程序译成数学，再代数算一次

题 4。 `_outside` 接收约束 $x \leq 3$ 和点 $(4, 2)$ ，返回什么？

答案。残差为 $4 - 3 = 1$ ，尺度为 $\max(1, 3, 4, 2) = 4$ ，阈值为 4×10^{-10} ，故返回真。点 $(3, 2)$ 残差为零，返回假。这里判的是违约束，不是离源远近。

题 5。某轮 `_feasible_point` 得到 $2t \leq 6$ 与 $-t \leq -1$ ，区间如何更新？

答案。第一条给 $t \leq 3$ ，第二条给 $t \geq 1$ ，区间 $[1, 3]$ 。第 130 行 `min(max(0,1),3)` 得 1，再代入边界参数方程。若第二条改为 $-t \leq -4$ ，就要求 $t \geq 4$ ，区间空。

题 6。 `polygon_diameter` 中叉积高度更大，是否立刻能替换 `best2`？

答案。不能。高度比较只更新远侧指针；第 178 行另算平方距离，第 179–180 行才按距离更新最远点对。这是支撑方向与欧氏距离两个不同量。

题 7。 `_finite_result` 中面积零，能否认为定位没有误差？

答案。不能。线段面积也是零，端点仍可相隔任意正长度；应看 `status` 和 `diameter_m`。例如 $(0, -1)$ 到 $(0, 1)$ 的线段面积 0、直径 2。

5.3 第三层：改变一行会怎样

题 8。把 `_wedge` 上边界法向也写成下边界形式，会发生什么？

答案。两条都保留“左侧”，无法得到夹在上下射线之间的窄楔；应从各自叉积不等式推法向，不能只因公式看着对称就复制符号。先用 $\theta = 0$ 的点 $(100, 0)$ 与 $(-100, 0)$ 检查方向最直观。

题 9。把 `if candidate > best2` 改为 `>=`，直径数值必然变吗？

答案。正常情况下最大值不变；并列最远点时会改为保留后遇到的一对，可能改变输出端点。几何答案与确定的输出代表应分开。

题 10。删除生成器第 29–32 行的重抽，只保留“加正负 1 度后保留两位小数”行不行？

答案。不能保证最终报告仍严格在误差界内，因为舍入可能再推远至边界之外。前面给出的 0.004° 真值与 -0.9995° 扰动会形成 -1.004° 最终误差。

题 11。把 `include_circle` 设为假，会改变什么？

答案。在当前 Q1 实现中不会改变几何结果，也不会阻止输出直径圆覆盖字段，因为主函数直接删除这个兼容参数。答辩必须沿实执行代码判断，不能猜名称含义。

题 12。 `run.bat` 返回退出码 0，是否表示直径不超过 40 米？

答案。没有这一含义。0 表示程序正常结束；几何精度与是否能用某清除圆覆盖，要读结果字段并采用正确的几何判据。脚本本未设置“达标则退出 0，否则退出 1”的业务规则。

5.4 两分钟口述答案

本问的输入是同一源在多个已知站点上的示向度，每条示向允许有界角误差。我们先把一条观测写成两个经过站点的半平面，零误差时再补前向限制。全部约束的交给共同允许的位置区域。程序把约束归一化、按方向排序、合并同向冗余边界，用双端队列维护候选边界，最后验证顶点并区分

空集、无界、点、线段和多边形。对有限结果，先用凸包整理顶点，再用旋转卡壳比较最远点对，面积用鞋带公式。最远点对的中点和半直径只定义一个待检查的圆，所以还要检查是否覆盖全部顶点；Q1 没有求最小包围圆，也没有把场地圆先验加入求交。生成器的真值只用来产生和检查观测，求解器不读取它。

5.5 方法选择：回答“为什么这样写”

当前问题	本实现选择	可比较的方法	选择理由与限制
多条带角误差观测怎样共同限制位置	角楔半平面交	用几条中心方向线交会；最小二乘点估计	已知的是误差界而非误差分布，集合交能表达“所有与读数相容的位置”；一个点估计本身不能给出整片允许区域。若需求改成概率估计，须另加噪声模型
输入为一批半平面，没有预给有界窗口	排序加双端队列	从一个大矩形开始逐边裁剪	队列法直接从边界出发并可做无界分类；随意选大矩形会把无界问题截成假有限问题。若本来已有可靠有界先验，逐边裁剪是合理替代
整理乱序候选交点	Andrew 单调链	Graham 扫描；小规模遍历支撑边	先按坐标排序，用同一叉积循环造上下链，容易实现和复核；这不是说其他凸包方法错误
在凸多边形中找最远点对	旋转卡壳	全部顶点两两枚举	凸性和有序性允许远侧指针复用；卡壳扫描为线性，小规模全枚举则更直接，也可作数值核对
无法构成稳定候选多边形时做分类	增量可行点和退行方向检查	完整线性规划库；精确几何库	二维结构允许用一维区间和法向角隙实现轻量兜底；它不是对任意病态输入的精确求解认证
需要知道最远点中点能否直接覆盖	对所有凸包顶点检查距离	直接声称半直径就是最小覆盖半径	逐顶点检查有明确凸性依据；若检查失败，还需另求最小包围圆，不能继续使用半直径作为安全半径

口述规则。先说当前问题要求，再说可用结构，最后说代价与局限。例如：“我现在要保留全部角误差相容位置，所以把读数写成不等式做交；如果只做方向线最小二乘，会得到一个代表点，需要额外构造不确定区域才能回答本问。”避免只背“此方法更优、更稳定”而不给前提。

6 覆盖清单与本次复核范围

- q1_solver.py: 250/250 个非空物理行有独立标记；含类方法、嵌套 chain、容差分支、兼容参数、文件读写与入口。
- q1_generator.py: 40/40 个非空物理行有独立标记；含同一物理行中的多个 import、条件及 break、入口及调用。
- run.bat: 12/12 个物理行有独立标记；含编码、工作目录、错误跳转与退出码。
- 复算了本册三角形求交、空集/无界/线段/点四分支、可行点区间例、四边形直径与直径圆漏点例，以及实际触发队首删除的五约束例。

- 凸包的五点数据沿用前文所标注的 CGAL 入门例；其他小坐标用于跟踪本源码，属于本册教学补充，不冒称外部教材原例。

如何算过关。第 1 天先口述式 (Q1.1) 并手算一个角楔；第 2 天走完凸包、队列与卡壳三个轨迹。之后每次随机抽 10 个非空代码行，先说输入和前提，再说这一行的作用与后续使用。记录答不上来的行号，回到所属函数的五项说明复练。读完本册提供了逐行解释依据；是否能脱稿回答全部行，须用这种抽查实际检验。

Q2 逐行伴读：从安全选站到最坏后验半径

怎样使用这一份伴读

这份材料对应仓库当前的三个原文件：source/Q2/q2_solver.py（538 行）、source/Q2/q2_generator.py（35 行）、source/Q2/run.bat（12 行）。行号按原文件计数，空行不承担计算；每个非空行都有 [L...] 说明。它接在第 3—4 课与“真实复算”章之后阅读：先用下面的模型串起整个程序，再按源码次序逐行追踪。生成器、求解器与启动脚本的行号各自重新从 1 开始。

本题究竟求什么？已知首次检测位置与示向；源位置、源自己的接收半径和下一次示向都未知。我们先选择能保证接收的第二站，再评价该站遇到不利报告时仍会留下多大的定位不确定性。第二站坐标是决策，未来报告是不确定量，报告出现后的清除中心又是一个决策。程序不能先偷看未来报告再选第二站。

1. 把一般公式代入本题常数

设 s_1 为首站， z_1 为首报告； $\delta = 1^\circ$ ；源场地 $\Omega = B(0, 1800)$ ；接收半径 $\rho \in [1000, 1500]$ 。角度差一律取环绕后的最小有向差。令

$$W(s, z) = \{g : |\text{wrap}(\arg(g - s) - z)| \leq \delta\}.$$

程序使用凸的放宽模型，省略正常示向所隐含的 5 m 近场小孔。对圆域再作外切多边形近似。先写连续的放宽模型，才看离散化：

$$\begin{aligned} F_1 &= \Omega \cap B(s_1, 1500) \cap W(s_1, z_1), \\ P(s, z) &= F_1 \cap B(s, 1500) \cap W(s, z), \\ Z(s) &= \{z : P(s, z) \neq \emptyset\}, \\ R(P) &= \min_c \max_{g \in P} \|g - c\|, \\ D(P) &= \max_{g, h \in P} \|g - h\|, \\ Q_R(s) &= \sup_{z \in Z(s)} R(P(s, z)), \\ s^* &\in \arg \min_{s \in \mathcal{S}} Q_R(s). \end{aligned}$$

这里 c 是某一次报告后的包围圆圆心， s 是报告发生前选的测量站。`robust_quality` 近似内层报告最大值；`optimize_fixed_baseline` 在指定基线上搜索站点；`solve_case` 再比较输入给的有限条基线。默认还独立求一次 Q_D 对照。最坏半径、最坏直径、最坏面积可来自三个不同报告。

2. 两个安全域的来历

在以首站为原点、首示向为正 x 轴的局部坐标中，写

$$g = r(\cos \eta, \sin \eta), \quad s = L(\cos \psi, \sin \psi), \quad |\eta| \leq \delta.$$

默认 uniform1000 要求完整扇区内的每个可能源都距第二站不超过 1000:

$$\max_{0 \leq r \leq 1500, |\eta| \leq \delta} \|s - g\| \leq 1000.$$

距离平方为 $L^2 + r^2 - 2Lr \cos(\psi - \eta)$ 。固定 η 后它关于 r 凸，最大值在 $r = 0$ 或 1500；在所搜索的前向安全弧上，最不利夹角为 $|\psi| + \delta$ ，故

$$L \leq 1000, \quad \cos(|\psi| + \delta) \geq \frac{L^2 + 1500^2 - 1000^2}{2L \cdot 1500}.$$

备选 full_information 利用“首站已经接收到”这一事实。同一源的半径必须满足 $\rho \geq \max(1000, r)$ ，因此应检查 $\|s - g\| \leq \max(1000, r)$ 。只需检查半径 1000 的扇区：对 $r \leq 1000$ 是前述凸函数端点；对 $r \geq 1000$ ，所需不等式变成 $L^2 - 2r s \cdot u(\eta) \leq 0$ ，其在 $r = 1000$ 成立就保证所有更大的 r 也成立。因此

$$L \leq 1000, \quad \cos(|\psi| + \delta) \geq L/2000.$$

这个“精确”是对不利用场地截断、也不挖近场孔的完整扇区模型而言；加入更多物理先验后可能存在更多安全站点。默认求解没有调用这个备选域。

3. 一句话区分三种保证

- 安全域检查是对完整扇区求最大距离的解析计算，再用浮点容差比较。
- 给定一个报告，外切多边形包含真实相容集；得到的包围圆覆盖这个计算集合。最终半径还按所有顶点重算并加小余量。
- 对报告和站点只作有限搜索。局部黄金分割未经单峰认证，外层也未穷尽连续站域；因此整个数值结果没有连续全局最优或连续最坏值认证。

文件一：source/Q2/q2_solver.py

A. 文件说明、依赖和常数

[L001] `#!/usr/bin/env python3` 是 Unix 类系统直接运行脚本时寻找 Python 3 的说明；用 `python q2_solver.py` 运行时它只是注释。

[L002–L008] L002 打开模块说明，声明同时计算直径 D、最小包围圆半径 R 和面积 A；L004 指明采用有界误差几何；L005 明确未给误差分布、未用重复测量平均；L006 指明圆用外切多边形；L007–L008 说明对每个已评价报告得到外包可行区及其保守 D/R/A。L003 为空行。

[L009–L011] L009–L010 限定有限报告最大值只是数值近似，并列出来、秒、度的单位；本文件实际不优化秒数。L011 关闭说明字符串。

[L012] 延迟解释类型注解，类型主要帮助阅读，不会自动检查输入。

[L014] 导入 `dataclass` 生成数据类，导入 `asdict` 将嵌套数据类转为字典。

[L015] `argparse` 处理命令行参数。

[L016] 导入 `csv`，但当前文件没有使用它。

[L017] `json` 读写输入、结果。

[L018] `math` 提供三角函数、角度转换、平方根、无穷大等。

[L019] `Path` 表示文件路径，并提供读取、写入、建目录方法。

[L020] `random` 在最小包围圆中打乱点序。

[L021] 导入 `time`，但当前文件没有计时代码使用它。

[L022] `Sequence` 表示可索引点序列；同一行导入的 `Iterable` 当前未用。

[L025] `Point` 是两个浮点数的元组类型别名，不是会自动验证长度的类。

[L026] `DELTA_DEG=1.0` 指示向误差半宽，单位度。

[L027] `R_MIN=1000.0` 是最小接收半径，与清除半径 20 m 不同。

[L028] `R_MAX=1500.0` 是最大接收半径，用于构造接收位置的必要约束。

[L029] `ARENA_R=1800.0` 限制源位置所在场地圆。

[L030] `EPS=1e-9` 是半平面包含判定的小容差；它是数值约定，不是物理误差。

B. 向量运算与坐标转换

函数契约。`add/sub` 输入两个二维向量，返回和/差；`mul` 输入向量和标量，返回缩放向量；`dot/cross` 返回标量；`dist` 返回欧氏距离。它们没有单位检查，调用者必须使用一致坐标和单位。

[L033] 定义 `add(a,b)`，类型提示输入输出均为二维点。

[L034] 分别相加两个分量，返回 (a_x+b_x, a_y+b_y) 。

[L037] 定义 `sub(a,b)`。

[L038] 返回 (a_x-b_x, a_y-b_y) ，例如 `sub(b,a)` 是从 `a` 指向 `b` 的向量。

[L041] 定义 `mul(a,k)`。

[L042] 返回 (ka_x, ka_y) 。

[L045] 定义内积 `dot`。

[L046] 计算 $a_x b_x + a_y b_y$ ，用于长度平方和法向投影。

[L049] 定义二维叉积 `cross`。

[L050] 计算 $a_x b_y - a_y b_x$ ；正负表示有向转向，两倍有向三角形面积也由它给出。

[L053] 定义 `dist`。

[L054] `hypot(dx,dy)` 计算 $\sqrt{dx^2 + dy^2}$ 。

`local_to_world` 契约。输入首站全局坐标、首示向角度和局部坐标 (a, b) ，返回第二站全局坐标。它作刚性旋转和平移，保持距离；角度输入是度。公式为 $s_2 = s_1 + R(z_1)(a, b)^T$ 。

[L057] 声明四个输入，返回一个全局二维点。

[L058] 将度换成弧度，供 `sin/cos` 使用。

[L059] 返回值第一分量为 $s_{1x} + a \cos t - b \sin t$ 。

[L060] 第二分量为 $s_{1y} + a \sin t + b \cos t$ ，并关闭元组表达式。

C. 安全站域的解析计算

`_sector_max_distance_sq` 契约。输入局部站点 $s = (a, b)$ 、非负扇区半径与正常角半宽，输出到完整闭扇区的最大距离平方。它解析枚举角向端点与可能的反向驻点；不使用角度网格。主要调用半宽为 1° ，不要把此函数当作任意多圈扇区接口。

[L063] 定义最大距离平方函数。

[L064] 字符串写明要求最大化的表达式以及 r, η 的范围。

[L065] 拆开站点的两个局部分量。

[L066] `length2` 是 $a^2 + b^2 = L^2$ ，也就是源取扇区顶点 $r = 0$ 时的距离平方。

[L067] 将扇区角半宽换成弧度。

[L068] $\text{atan2}(b,a)$ 求站向量的方位 ψ 。

[L069] 记录站向量在扇区下边方向 $u(-\delta)$ 上的内积。

[L070] 同时记录上边方向 $u(\delta)$ 的内积；这两项是角区间端点候选。

[L071] 注释提醒：角区间内部还可能包含与站向量反向的方向。

[L072] 枚举相邻三种 2π 表示，避免反向角因为跨越角度端点而漏检。

[L073] 构造 $\eta = \psi + \pi + 2k\pi$ ，即反向方向。

[L074] 判断该反向方向是否落在扇区角区间。

[L075] 若落在其中，最小投影为 $-L$ ，将它加入候选。

[L076] 取候选投影中的最小值；距离平方中的交叉项是负号，所以它对应最大距离。

[L077] 在 $r = 0$ 的 L^2 与 $r = \text{radius}$ 的最大距离平方之间取大者。能只查径向端点，依据是距离平方关于 r 的凸性。

safe_full_information 契约。输入局部站点、误差半宽和最小接收半径，返回上述完整扇区模型中利用首站接收事实后的安全判定。这里的 **True** 表示足够安全；不是“定位半径已经小于 20”。

[L080] 声明函数与默认角误差。

[L081] 声明默认 $r_{\min}$ ，返回布尔值。

[L082-L087] L082 说明利用首次接收；L084 定义某可能源距首站为 r ；L085 给出其最小相容接收半径 $\max(r_{\min}, r)$ ；L086 将检查归结到截断在 $r_{\min}$ 的首测扇区；L087 关闭说明。所谓 **exact** 的模型边界已在本伴读开头说明。L083 是空行。

[L088] 求到半径 $r_{\min}$ 扇区的最远距离平方，并与 $r_{\min}^2 + 1e-7$ 比较。这里 $1e-7$ 加在距离平方上，用于吸收舍入误差。

safe_uniform_1000 契约。输入局部站点、误差半宽、最小及最大接收半径，返回完整 $r_{\max}$ 扇区能否全部被站点的 $r_{\min}$ 圆覆盖。它对真实截断首测集合是充分条件；可比利用首次接收事实的域更保守。

[L091] 声明函数与角半宽默认值。

[L092] 声明半径范围与布尔返回值。

[L093] 说明统一距离保证的意义。

[L094] 将最远距离函数的扇区半径设为 $r_{\max}$ ，阈值仍为 $r_{\min}^2 + 1e-7$ 。

safe_angle_limit 契约。输入固定基线 L 、安全域名字、误差半宽及接收半径，返回前向安全弧最大偏角（度）；不具备基本可行条件时返回 **None**，角宽消失时可能返回 0.0。调用者还会排除零角宽。它没有替代后面的实际安全复查。

[L097] 声明基线与默认域 **uniform1000**。

[L098] 声明角误差。

[L099] 声明两种半径，返回值可为浮点数或 **None**。

[L100] 说明返回的是固定基线弧上最大 $|\psi|$ 。

[L101] 检查 $0 < L \leq r_{\min}$ ； $L = 0$ 被排除，因为本函数按非零固定基线求角度。

[L102] 不满足条件就返回 **None**。

[L103] 识别备选安全域。

[L104] 使用完整信息公式中的余弦下界 $L/(2r_{\min})$ 。

[L105] 识别默认统一接收域。

[L106] 使用余弦下界 $(L^2 + r_{\max}^2 - r_{\min}^2)/(2Lr_{\max})$ 。

[L107] 进入域名字既非上述二者的分支。

[L108] 对未知域抛 `ValueError`，不擅自采用一种默认含义。

[L109] 若所需余弦大于 1，则无可行方向。

[L110] 返回 `None`。

[L111] 先防止反余弦输入小于 -1，再取反余弦、转度、扣掉示向误差半宽，最后把负角宽截成 0。返回 0 不应被解释为已有非共线可行站。

D. 用外切多边形和半平面形成相容区域

`circle_outer_polygon` 契约。输入圆心、正常正半径与边数，输出按逆时针排列的正外切多边形顶点；显式要求边数至少 24。数学上外切圆半径 `radius` 是多边形内切圆半径，顶点距圆心为 $r/\cos(\pi/n)$ 。这是外包，不能改成半径 `r` 的内接多边形来取得更漂亮的数值。

[L114] 定义圆外包函数。

[L115] 说明每条边都与原圆相切。

[L116] 检查边数是否不足 24。

[L117] 不足则抛出异常。

[L118] 将圆半径换成顶点半径 `rv`。

[L119] 列表推导中计算第 `k` 个顶点的 `x` 分量，角度取 $(2k + 1)\pi/n$ ；半格错位使边法向恰为 $2k\pi/n$ 。

[L120] 同时计算 `y` 分量，并加圆心平移。

[L121] 枚举 `k=0,...,sides-1`，返回完整顶点表。

`clip_halfplane` 契约。输入按边界次序排列的凸多边形及 $n \cdot x \leq h$ 的法向、偏移，输出裁剪后的顶点表；空输入返回空表。它依次检查旧边的两端，穿越边界时插入交点。输入若是一般非凸形状，输出结构不保证能表达所有分离分支；本题前序构造始终维持凸性。

[L124] `normal` 是 `n`，`offset` 是 `h`。

[L125] 判断顶点表是否为空。

[L126] 空表直接返回空表。

[L127] 创建未清理的输出列表。

[L128] 先令旧边起点 `a` 为最后一个顶点，以便第一轮处理闭合边。

[L129] `fa=n · a-h`：非正表示在允许侧，小正数按 `EPS` 也接受。

[L130] 依次把每个顶点作为本边终点 `b`。

[L131] 计算终点的有符号半平面残差 `fb`。

[L132] 两端的“在允许侧”布尔值不同时，判定边穿越边界。

[L133] 沿边 $a + t(b - a)$ 求交，解 $f_a + t(f_b - f_a) = 0$ 得 $t = f_a/(f_a - f_b)$ 。`EPS` 用于分类而交点仍取原直线，因此极接近边界时属于浮点近似处理。

[L134] 把交点加入输出。

[L135] 若 `b` 在允许侧或容差范围内，则应保留 `b`。

[L136] 追加 `b`。

[L137] 将 `b`、`fb` 更新为下一条边的起点与残差。

[L138] 新建清理后的顶点列表。

[L139] 遍历未清理输出。

[L140] 第一个点直接接收；后续点只有距上一个点大于 $1e-8$ 才保留。

[L141] 追加通过邻接去重检查的点。

[L142] 检查首尾是否因闭合边处理而重复。

[L143] 若首尾距离不超过 $1e-8$ ，删除末尾重复点。

[L144] 返回清理列表；可能是空集、单点、线段或通常的凸多边形。

`clip_disk_outer` 契约。输入旧凸多边形、圆心、半径和正常正边数，输出与圆外切多边形的交。这里没有另做 `sides>=24` 检查，主调用通过 `first_feasible_polygon` 的初始化检查。一般式是逐个相交 $n_k \cdot x \leq n_k \cdot c + r$ 。

[L147] 声明圆盘外包裁剪函数。

[L148] 枚举圆周均匀法向。

[L149] 第 k 个法向角为 $2\pi k/n$ 。

[L150] 构造单位法向 ($\cos, \sin$)。

[L151] 以该圆的一条切线半平面裁剪旧多边形；圆上任意点都满足这条不等式。

[L152] 若交集已为空。

[L153] 提前结束后裁剪，空集再相交仍为空。

[L154] 返回剩余顶点。

`clip_wedge` 契约。输入旧凸多边形、站点、示向及角半宽，输出与前向窄角楔的交。本题半宽为 1° ，两条有向射线半平面可精确表达这个凸角楔；不应把同一表达式直接推广为大于半平面的非凸角域。

[L157] 声明多边形、站点、报告。

[L158] 声明默认角误差和返回类型。

[L159] 计算下边射线角 `bearing-delta`，转弧度。

[L160] 计算上边射线角 `bearing+delta`，转弧度。

[L161] 下边的单位方向记为 `lower`。

[L162] 上边的单位方向记为 `upper`。

[L163] 第一法向为 (`lower_y, lower_x`)，相当于保留下边射线左侧。

[L164] 偏移量取法向与站点的内积，使边界直线通过站点。

[L165] 第二法向为 (`-upper_y, upper_x`)，相当于保留上边射线右侧。

[L166] 同样令边界过站点，返回两次裁剪的交。用 $\sin/\cos$ 构造方向自然兼容 $359^\circ/0^\circ$ 跨界，无需按数值大小截断角区间。

`first_feasible_polygon` 契约。输入首站、首报告、角误差与圆边数，输出 F_1 的凸外包顶点。默认首站原点、首报告 0° ，便于标准例；一般实际输入会覆盖这两个默认值。

[L169] 声明首站与首报告默认值。

[L170] 声明角半宽、180 边默认值及返回类型。

[L171] 说明要计算场地圆、首站最大接收圆、首测角楔的交。

[L173–L175] L173 指明正常方向反馈还意味着距源超过 5 m；L174 说明省略小孔以维持凸性并保守放宽；L175 结束说明字符串。

[L176] 先把全局场地圆 $B(0, 1800)$ 外包成多边形。

[L177] 再与 $B(s_1, 1500)$ 的外包多边形相交。

[L178] 最后与首次示向角楔相交并返回。注意场地始终以全局原点为圆心，不能随首站一起平移掉。

E. 固定区域的面积、最小包围圆与直径

`polygon_area` 契约。输入边界有序的顶点，返回非负面积。至少三点时用鞋带公式；不足三点返回 0。不应传入随机点序来求凸包面积。

[L181] 定义接受点序列的面积函数。

[L182] 对每一条闭合边的两端计算叉积并求和，绝对值消去顺逆时针符号。

[L183] 以取模把最后一条边接回第一点；除以 2 得面积。不足三点直接走条件表达式的 `else 0.0`，避免没有面时执行不必要计算。

`_diameter_circle` 契约。输入两点，返回以两点连线为直径的圆心和半径。它保证覆盖这两点，不能自动保证覆盖别的点。

[L186] 声明返回 `(Point,float)`。

[L187] 两点求和再乘 0.5 得中点。

[L188] 返回中点与两点距离的一半。

`_circumcircle` 契约。输入三点，返回外接圆；若有向面积数值上太小则返回 `None`。一般推导：设圆心为 $a + w$ ，由到 a 、 b 、 c 等距可得 $2u \cdot w = \|u\|^2$ 、 $2v \cdot w = \|v\|^2$ ，解一个二元线性方程组。

[L191] 定义三点外接圆，允许无可用结果。

[L192] 令 $u = b - a, v = c - a$ 。

[L193] `den=2 cross(u,v)` 是公式分母；非共线才有唯一有限外接圆。

[L194] 用 `1e-14` 检查分母是否太接近零。

[L195] 退化或近共线时返回 `None`，避免除以极小量。

[L196] 计算 $\|u\|^2, \|v\|^2$ 。

[L197] 根据二元方程解出圆心相对 a 的 x 分量，再准备加回 a 。

[L198] 同时解出相对 y 分量，完成全局圆心。

[L199] 返回圆心和它到 a 的距离。三点共圆来自上面的等距方程，不是盲猜中心。

`minimum_enclosing_circle` 契约。输入非空有限点集，返回圆心与最终覆盖审计半径。数学依据是平面最小包围圆可由至多三个支撑点确定；这里采用打乱点序后的增量构造。当前程序固定随机种子以复现执行顺序。随机增量算法的平均效率和“固定这个种子对任意输入都有最坏线性时间”是两种不同说法，后者没有保证。实现使用浮点容差，最后按返回圆心重算最大顶点距离并加余量；这是覆盖审计，不能把所有数值细节称为符号精确计算。

[L202] 定义输入任意可索引点序列的 MEC 函数。

[L203] 说明固定随机种子、增量算法及最后覆盖审计。

[L204] 检查是否空点集。

[L205] 空集抛异常；本文件调用前会跳过空后验。

[L206] 复制点表，避免改变调用者的点序。

[L207] 用局部随机发生器和固定种子 20260911 打乱，不改全局随机状态。

[L208] 初始尚无圆，记为 `None`。

[L209] 依次纳入打乱后的点 p ，同时保留其下标 i 。

[L210] 若已有圆且 p 在半径加 `1e-8` 的数值范围内。

[L211] 跳过重建，当前圆已覆盖它。

[L212] 若 p 在圆外，以 p 为中心、半径 0 重新开始；此轮的边界支撑点包含 p 。

[L213] 再依次扫描 p 之前已经处理过的点 q 。

[L214] 若 q 已被当前圆覆盖。

[L215] 无需更新，继续下一个 q 。

[L216] q 在圆外时，先以 p 、 q 为直径构造候选圆。

[L217] 存储有向弦 $q - p$ ，后续以它区分左右。

[L218] 初始化弦左、右两侧的三点外接圆候选为缺失。

[L219] 扫描 q 之前的已处理点 r ，保证新圆也照顾更早的约束。

[L220] 若 r 在当前两点直径圆内或数值边界内。

[L221] 不必由这个 r 产生三点约束。

[L222] 否则计算 p 、 q 、 r 的外接圆 cc 。

[L223] 检查是否共线或近共线而没有外接圆。

[L224] 没有则略过此候选；这是数值退化分支。最终覆盖审计仍会重新看所有输入点。

[L225] $side=cross(pq,r-p)$ 表示 r 在有向弦的哪一侧。

[L226] $signed=cross(pq,cc_center-p)$ 表示该候选圆心相对弦的位置，长度因子相同，足以比较有向偏移。

[L227] r 在左侧时，若尚无左候选或新圆心有向偏移更大，则它给出更紧的左侧边界要求。

[L228] 保存 (有向偏移,圆) 为左候选。

[L229] r 在右侧时，取有向偏移更小的右候选。

[L230] 保存右候选。两侧的极值筛选对应固定弦端点 p 、 q 后的圆心约束，不能任意把最大改成最小。

[L231] 从左右候选提取实际圆，去掉 None。

[L232] 若确有三点候选。

[L233] 在已形成的左右极限候选中取半径较小者。若没有，保留 L216 的直径圆。

[L234] 非空输入至少形成一次圆，用断言表达这个内部不变量。断言不是面向用户的输入校验。

[L235] 固定已求圆心，重新计算它到原输入所有点的最大距离，再加 $1e-7$ 返回。此时即使增量循环的容差跳过了极小偏差，也按此圆心扩大半径。对凸多边形，覆盖所有顶点便覆盖整块多边形，因为圆盘是凸集。

Metrics 契约。这是保存一次给定报告后验几何量的数据容器，不作新的优化。`frozen=True` 禁止正常方式重设字段。

[L238] 用不可变数据类装饰器自动生成构造等常用方法。

[L239] 定义类名 Metrics。

[L240] `diameter_m` 保存该多边形直径，单位米。

[L241] `mec_radius_m` 保存审计后的最小包围圆算法半径，单位米。

[L242] `area_m2` 保存面积，单位平方米。

[L243] `vertex_count` 保存传入裁剪多边形的顶点数，不是另求凸包后的顶点数。

`_hull` 与内层 `chain` 契约。`_hull` 输入有限点序列，返回去重、去数值共线中间点的逆时针凸包；`chain` 输入按某方向排序的点序列，返回单侧凸链。它使用 Andrew 单调链思想。叉积阈值是实现容差，严格几何证明中的零在程序里换成了小正数。

[L246] 定义凸包辅助函数。

[L247] `set` 删除完全相同的点；`sorted` 按 x 、再按 y 的字典序排列。

[L248] 同一行先判断 $\text{len} \leq 2$ ，满足时直接返回；空集、一点、两点不用转向筛选。

[L249] 定义闭包内的单侧链函数 `chain(seq)`。

[L250] 初始化链为空。

[L251] 逐个读取候选 `p`。

[L252] 一行包含条件与循环体：当链至少两点、末边接 `p` 不构成超过容差的左转，就 `pop` 删除最后一点；一直删到保持凸性或只余一点。叉积用两条相接边计算，与用同一基点等价。

[L253] 在满足凸性后把 `p` 追加到链。

[L254] 返回这条链。

[L255] 分别正序、逆序建链；各去末端重复点再拼接为闭合边界的顶点序列，列表中不额外重复首点。

polygon_diameter 契约。输入点序列，输出最大距离及实现它的点对；空输入返回 `(0, None)`。先求凸包，再用旋转卡壳。凸包点序及凸性是单调推进对踵点的前提。

[L257] 定义直径函数。

[L258] 同行先求凸包 `pts`，再记录顶点数 `n`。

[L259] 空凸包返回 0 与无点对。

[L260] 单点直径为 0，点对的两端都是该点。

[L261] 两点直接算距离及点对。

[L262] 同行初始化对踵指针 `j=1`、当前最优距离 `-1` 和缺失点对。

[L263] 依次枚举有向凸包边的起点 `i`。

[L264] 同行计算环绕后的终点 `ni`，再构造边向量。

[L265] 若下一个 `j` 对该边的叉积高度比当前更大，则还未到最远支撑点。固定边长时比较叉积等价于比较垂距，不必除以边长。

[L266] 让 `j` 沿凸包前进一格，取模处理闭环。

[L267] 分别检查当前边的两个端点与支撑点 `j` 的距离。

[L268] 计算这一个候选点对的欧氏距离 `d`。

[L269] 同行完成“若更大，则同时更新最优距离和点对”；分号分开两个赋值，两个都属于 `if` 分支。

[L270] 返回直径及对应顶点对。本实现按浮点高度容差运行，不能把其 $1e-10$ 判定解释成物理角误差。

polygon_metrics 契约。输入非空有序凸多边形顶点，返回一次报告的四项 `Metrics`；允许单点、线段，空输入会在 MEC 中抛错。

[L272] 定义统一评分入口。

[L273] 求直径，把最远点对用 `_` 丢弃，因为本层仅需要标量。

[L274] 求包围圆，把圆心丢弃，只保留半径。

[L275] 构造 `Metrics`：直径转 `float`、放入半径、计算面积、记录原顶点数。

F. 第二接收约束与结果数据类

physical_base 契约。输入首测外包、第二站和边数，输出再加第二站最大接收圆约束的外包。它还没有使用第二报告，也没有重检站点安全。

[L278] 定义第二接收基底。

[L279] 复制首测点表，再与第二站中心、1500 m 半径的圆外包相交；返回可供许多报告重复裁剪的基底。

post_region 契约。输入接收基底、第二站、某个报告及角误差，返回该报告对应的后验外包。空列表表示程序放宽模型不相容。

[L282] 定义报告后验函数。

[L283] 声明误差默认值与返回类型。

[L284] 复制基底并加第二角楔。复制保证测试一个报告不会修改下一报告的起始区域。

RobustQuality 契约。保存一个固定第二站经过有限报告搜索所得的结果，不保存外层站点最优证明。三个“最坏”各自取最大，未必同时出现。

[L287] 普通数据类装饰器，这个类未冻结字段。

[L288] 定义 RobustQuality。

[L289] worst_R_m 是记录列表中最大的包围圆半径。

[L290] worst_D_m 是记录列表中最大的直径。

[L291] worst_A_m2 是记录列表中最大的面积。

[L292] 保存半径最坏的报告角。

[L293] 保存直径最坏的报告角。

[L294] 保存面积最坏的报告角。

[L295] 保存半径最坏那一个报告的完整 Metrics；其中面积不必等于 worst_A_m2。

[L296] feasible_report_count 是后面实际填入的 len(records)：已记录非空报告条数，可能有重复，不能按名字理解为全部可行角度数。

[L297] report_evaluations 实际填入 len(cache)：非空评价的不同归一化角度键数。空报告未缓存，因此它不是总调用次数或全部粗网格数量。

[L298] 记录采用的圆外包边数。

[L299] 记录请求的粗报告步长；若 360 不能整除它，实际步长由 count 调成 360/count。

as_dict 契约。无额外输入，把本对象递归转换成可 JSON 化字典，并附一条明确的认证范围标志。

[L301] 定义成员方法。

[L302] asdict(self) 递归转换，内层 Metrics 也变成字典。

[L303] 加 continuous_report_maximum_certified=False，明确未认证连续报告最大值。

[L304] 返回字典。

G. 固定第二站：先扫报告，再细化候选峰

robust_quality 契约。输入固定第二站、首观测、圆边数及正常正报告步长，输出一个 RobustQuality。本函数本身不检查第二站是否保证接收；外层先筛安全再调用。refine_step_deg 虽在接口中出现，当前函数体没有使用。输出是有限记录中的最大值，不是已认证的 $\sup_z$ 。

先看循环的不变量。base 与报告无关，整个函数不改；cache 保存已经算过的非空后验及指标；coarse 保留一整圈粗报告的顺序，空报告也占位置；records 保存参与最终取最大的报告。四个对象承担不同职责，不能互相替代。

[L307] 固定第二站是第一个位置参数；* 后的参数必须按名字传入，防止首站和精度混位。

[L308] 声明首示向和误差半宽。

[L309] 声明圆边数默认 120、报告粗步长默认 0.5°。

[L310] 声明 refine_step_deg=0.05 和返回类型；此形参目前没有进入计算。

[L311] 说明本函数数值近似最坏 D/R/A。

[L313–L317] L313 写报告可行当且仅当后验非空；L314 解释其中同一位置与该报告的角误差相容；L315–L316 说明为何不能把独立、不相容的源位置和报告拼在一起；L317 结束说明。这里必须限定为本函数的外包且省略近场孔的模型：其非空集合可能比真实物理可行集更大。

[L318] 用首站、首报告、角误差与指定精度构造首测外包。

[L319] 加第二站最大接收圆，得到报告无关的 `base`。

[L320] 建立“归一化报告角 $\rightarrow$ (Metrics,poly)”的缓存字典。

内层 `evaluate` 契约。输入任意等价表示的报告角 z ，返回非空后验的指标和顶点；无可行区返回 `None`。只对非空报告缓存。

[L322] 定义嵌套函数，它可读取外层的 `base/second/delta_deg/cache`。

[L323] 把 z 模 360 后保留 10 位小数作为键，使相差整圈的报告复用；舍入也可能合并极近报告。

[L324] 检查该键是否已经缓存。

[L325] 是则返回已有结果，省去裁剪和几何计算。

[L326] 对归一化报告裁剪 `base`，得到后验。

[L327] 判断是否空集。

[L328] 空集返回 `None`；此分支没有写缓存。

[L329] 非空时先算 Metrics，再与原后验顶点打包。

[L330] 写入缓存。

[L331] 返回结果。

[L333] 初始化用于最终取最大的 (角度,Metrics) 记录表。

[L334] 令 `count=ceil(360/report_step)`，至少一个。常用参数下全圆实际粗步长 $h = 360/\text{count} \leq$ 请求步长。传 0 会除零，接口没有自行验证正值。

[L335] 建立按圆周顺序存放的粗表；元素允许为 `None`。

[L336] 遍历 `count` 个粗报告。

[L337] 取 $z_i = i \cdot 360/\text{count}$ ，不重复 360° 。

[L338] 调用 `evaluate`。

[L339] 检查是否存在非空后验。

[L340] 用 z 与指标组成记录，后验顶点留在缓存即可。

[L341] 将该非空粗报告加入最终记录表。

[L342] 同时加入保序粗表。

[L343] 进入空报告分支。

[L344] 在粗表中保留一个 `None` 占位，避免破坏相邻报告的对应关系。

[L345] 若全部粗报告都没有非空后验。

[L346] 抛 `RuntimeError`。有限粗网格未找到可行报告不等于连续报告集必为空；正常本题设置有足够宽报告区间，但本接口没有另作这种认证。

[L348–L350] 三行注释依次说明：对有竞争力的粗局部峰做有界一维搜索；有限搜索可复现、可比较加密结果；它不是连续报告域区间认证。

[L351] 计算黄金比例 $\gamma = (\sqrt{5} - 1)/2 \approx 0.618$ ，它使缩区间后一个旧内点可复用。

[L352] 依次对半径、直径、面积三个字段寻找候选峰。即便外层只优化 R ，内部仍同时评价三类最坏指标。

[L353] 新建当前字段的候选峰列表，每项存 (字段值,粗网格下标)。

[L354] 遍历粗表及其下标。

[L355] 空报告不具备峰值。

[L356] 跳过它。

[L357] `getattr` 按字段名读取当前报告的这一个指标。

[L358] 用取模找到前一个粗报告， 0° 的前邻居在接近 360° 处。

[L359] 同样找后邻居。

[L360] 前邻居非空时读其指标；为空时用负无穷，让可行边界上的点也能成为候选峰。

[L361] 对后邻居作同样处理。

[L362] 当前值不小于两个邻值时认作一个粗网格局部最大。

[L363] 将其数值和下标加入候选表；这不证明连续区间内只有一个峰。

[L364] 若没有收集到候选峰，走防御性后备路径。

[L365] 在已有记录里找该字段最大的一行，并提取其字段值。

[L366] 再取该记录角度，准备换成粗网格下标；代码重复了一次 `max`，不是不同的数学目标。

[L367] 角度除粗步长、四舍五入、再模 `count`，拼出一个后备候选。

[L368] 候选按值从大到小排序，只细化前 8 个；值相同还会按下标倒序。剩余峰不作局部细化。

[L369] 将候选下标换回粗报告角 `center`。

[L370] 以左右各一个粗步长形成搜索区间。跨 0° 时允许负角或超过 360° ，由 `evaluate` 归一化。

内层 `field_value` 契约。输入角度，返回当前字段的标量；不可行报告返回 $-\infty$ ，便于求最大。它不保证目标单峰或连续。

[L372] 定义供黄金搜索调用的标量函数。

[L373] 复用 `evaluate` 和缓存。

[L374] 非空时取当前字段，否则返回负无穷。

[L376] 左内点 $c = b - \gamma(b - a)$ ；这里变量 `c` 是报告角，不是前文 MEC 圆心。

[L377] 右内点 $d = a + \gamma(b - a)$ 。

[L378] 评价两个内点，保存 `fc,fd`。

[L379] 固定执行 34 轮；停止条件不是 `refine_step_deg`，也不是函数值误差界。

[L380] 本题找最大：若左内点值不小于右内点值，按单峰搜索规则保留左侧区间。

[L381] 同时更新三个变量：右边界变成旧 `d`；新的右内点变成旧 `c`；其函数值直接复用旧 `fc`。Python 先计算全部右侧，再赋值，不会用到“刚改过的 `d`”。

[L382] 在新区间计算新的左内点。

[L383] 仅新算这个点的值。

[L384] 否则右内点的值更大。

[L385] 同时令左边界变旧 `c`，新左内点变旧 `d`，复用旧 `fd`。

[L386] 计算新的右内点。

[L387] 新算其值。每轮只增加一次函数评价，黄金比例的复用性质就在这里。

[L388] 迭代结束后选左端点、左内点、中点、右内点、右端点五个位置。

[L389] 分别评价；若已缓存可复用。

[L390] 只保留非空后验。

[L391] 将归一化角度与指标加入 `records`。中间 34 轮的其余非空评价在 `cache` 中，但未在此逐一加入 `records`。

[L393] 从 `records` 取半径最大的行。

[L394] 独立取直径最大的行。

[L395] 独立取面积最大的行。

[L396] 构造结果，先传半径最大值、直径最大值。

[L397] 再传面积最大值和半径最坏报告角。

[L398] 传直径、面积各自的最坏角。

[L399] 传半径最坏那一行的完整指标、记录数、非空缓存键数、圆边数。

[L400] 最后记录请求的粗步长，返回数据类。

关于黄金分割的结论必须完整。对已证明单峰的区间，比较内点可可靠删除区间。本代码只从粗网格邻值挑局部峰，没有证明每个括区单峰，最多选八个峰，还可能遇到可行性边界与几何支撑点切换。因此它借用了黄金分割的计算步骤作为局部数值搜索，不能借用未经验证的单峰前提来宣称连续最坏值认证。最终最大值还只取 `records`，不是所有 `cache` 中间评价的最大值。

H. 固定基线：在首示向两侧搜索站点

`candidate` 契约。输入基线、偏角（度）和可选横向符号，返回局部坐标。正常外层直接传有符号偏角，`side` 采用默认 1。

[L403] 定义候选点函数；`side` 的类型提示是 `int`，但没有运行时强制。

[L404] 将偏角转弧度。

[L405] 返回 $(L \cos \psi, \text{side } L \sin \psi)$ 。`side=-1` 可以镜像横向坐标；外层并不是同时把负偏角和负 `side` 都传入。

`optimize_fixed_baseline` 契约。输入目标 R/D/A、一个固定基线、首次观测、安全域及正常正搜索步长，返回本次已检查安全候选中目标最小的一项及搜索元数据。候选为空时抛 `ValueError`。它把连续弧转成两侧粗网格，并分别在每侧最好粗点附近作细网格；不会保证找到固定基线上的连续最优。

[L408] 目标和基线可按位置传入；其余参数要求按名传入。

[L409] 声明首站、首报告默认值。

[L410] 默认安全域为 `uniform1000`。

[L411] 默认角误差为 1° 。

[L412] 默认外层粗偏角步长 1° 。

[L413] 默认局部细偏角步长 0.1° 。

[L414] 默认圆外包边数 96。

[L415] 默认内部报告粗步长 1° ，输出字典。外层偏角步长与内层报告步长控制的是不同变量。

[L416] 检查目标字符串是否属于 R、D、A。

[L417] 非法则报错。

[L418] 用解析公式得到该基线的安全偏角上限。

[L419] 上限不存在或非正时，无法安排所需非共线搜索。

[L420] 抛出说明性的异常。

[L421] 根据域名选对应安全检查函数；未知域已经在 L418 中报错。

[L422] 初始化全体已评价候选记录，每项是 (目标分数,偏角,完整内层质量)。

内层 score 契约。输入一个 RobustQuality, 按外层指定的目标取一个标量。三者单位不同, 只能同目标比较。

[L424] 定义评分提取函数。

[L425] 构造 R/D/A 与对应字段的映射, 再按 metric 取值; 没有把三个目标加权相加。

[L427-L429] 三行注释说明为何必须搜首示向两侧: 首站在中心、且首测扇区不被场地截断时才有相应镜像对称; 任意首站的场地截断通常破坏它。

[L430] 先搜负偏角, 再搜正偏角; 浮点 -1.0、1.0 用作符号乘数。

[L431] 建立当前这一侧的粗搜索记录。

[L432] 偏角绝对值从 $\max(0.2, \text{coarse_step})$ 起步, 刻意不从 0 开始。这也意味着极小安全弧可能没有被这个网格取到。

[L433] 在不超安全上限加容差的情况下继续粗扫描。

[L434] 同行把绝对值乘方向得到有符号 psi, 再用 candidate 转局部坐标。

[L435] 用完整最远距离函数复查安全; 解析角上限和实际点检查共同筛选。

[L436] 将通过检查的局部点旋转、平移到全局坐标。

[L437] 开始调用 robust_quality, 传入全局第二站、首观测、角误差与圆精度。

[L438] 传内部报告粗步长。

[L439] 传 report_step/5 给 refine_step_deg; 因该形参未使用, 这一行目前没有改变实际细化轮数或精度。

[L440] 同行打包 (score,psi,q), 加入所有记录及本侧粗记录。values 包含两侧, side_values 只用于本侧选细化中心。

[L441] 增加偏角绝对值。若调用者传非正步长, 可能出现不终止等问题; 正常 solve_case 传的是固定正数。

[L442] 若本侧一个安全粗候选也没评价到, 跳过该侧细化。

[L443] 从本侧粗记录取目标最小者, 只保留它的偏角。

[L444] 细化范围取该点左右一个粗步长, 并裁到安全总弧内; 同一行给 lo 和 hi 赋值。

[L445] 对负侧, 右端不得越过 -0.05° , 避免穿过共线零偏角。

[L446] 对正侧, 左端不得低于 0.05° 。

[L447] 从细化区间左端开始。

[L448] 用细网格扫描到右端加小容差。

[L449] 当前 psi 转局部坐标。

[L450] 再次验证该点安全。

[L451] 通过后转全局坐标。

[L452] 重新评价该站的内层最坏指标, 参数与粗站点评价相同。

[L453] 传报告粗步长。

[L454] 再传当前未生效的 refine_step_deg。

[L455] 将细站候选加入全记录; 这里不做跨粗细网格的站点去重。

[L456] 按 fine_psi_deg 递增 psi。

[L457] 两侧搜索后检查全记录是否为空。

[L458] 空则报错; 这表示算法没评价到安全候选, 不是已证明数学安全域为空。

[L459] 从实际候选中取目标最小的一行, 拆成分数、偏角与质量。

[L460] 根据最优偏角重算局部坐标，便于输出。

[L461] 重算全局第二站。

[L462] 开始返回字典，记录目标与安全域。

[L463] 记录基线和有符号偏角。

[L464] 同一行依次记录局部 a、b，全局第二站坐标和搜索阶段目标值。

[L465] 保存安全角上限与所选站的完整搜索阶段质量字典。

[L466] `candidate_evaluations` 是记录长度，包含可能重复的粗细候选，不能当作不同站点数。

[L467] 显式写 `continuous_global_optimum_claimed=False`。

[L468] 说明使用两侧固定基线一维粗细搜索。

[L469] 再用文字声明有限角采样与局部细化，没有连续全局最优证书。

I. 加密复评、跨基线比较与命令行

`final_quality` 契约。输入已经选好的全局第二站及首观测，返回加密后的内层报告质量字典。它不重新寻找第二站，也不直接保证比粗评分更大或更小：圆外包变紧与报告检查变密是两个不同方向的变化。

[L472] 定义复评函数；默认外包 360 边，首观测默认值仍是标准例。

[L473] 默认报告步长 0.1° ，但实际调用可覆盖。

[L474] 调用 `robust_quality`，传入站点、首观测和圆边数。

[L475] 传报告步长。

[L476] 传未生效的细化步长形参；角误差未在本接口开放，内部采用默认 1° 。

[L477] 转成字典返回，保留未认证标志。

`solve_case` 契约。输入 JSON 解析所得字典和 `quick` 开关，输出两种目标的对照及有限基线搜索表。必须给 `first_bearing_deg`；首站、基线有默认值。代码只读取报告、首站、候选基线、案例编号，不读取 `local_truth`。参数并没有完整 schema 校验，正常流程假定输入格式正确。

[L481] 声明单案例总入口，`quick` 默认关闭。

[L482] 读取首站数组；缺省使用原点。

[L483] 将两个坐标转 `float` 组成元组；长度不足会抛错。

[L484] 必取首报告并转 `float`、模 360，统一角度表示。

[L485] 读取候选基线数组并逐个转 `float`；缺省为 600、700、800、900、1000 m。

[L486] `quick` 搜索用 72 边，正常搜索用 96 边。

[L487] `quick` 内部粗报告步长 2° ，正常 1° 。

[L488] `quick` 外层粗偏角步长 2° ，正常 1° 。

[L489] `quick` 外层细偏角步长 0.25° ，正常 0.1° 。

[L490] 初始化跨基线、跨目标记录。

[L491] 先优化 D，再优化 R；虽然底层支持 A，这个主程序没有以面积为目标另做一次选站。

[L492] 依次枚举给定基线。

[L493] 用异常捕获包裹本次固定基线搜索。

[L494] 传目标、基线、首站和首报告；未传 `domain`，所以用默认 `uniform1000`。

[L495] 传外层粗细偏角步长及搜索圆边数。

[L496] 传内层报告步长并结束调用。

[L497] 若抛 `ValueError`，进入跳过路径。它会捕获该调用内部传播来的任何 `ValueError`，并非只捕获“基线不安全”。

[L498] 跳过本条基线，继续下一条。

[L499] 正常返回时，把这一基线/目标的最佳记录加入总表。

[L500] 同行判断总表为空并抛中文错误。这是没有数值可用候选的运行结果，不是连续域不可行的证明。

[L501] 只取目标 D 的行，按搜索阶段目标值选跨基线最小者。

[L502] 同样在 R 目标行中独立选最小者；不把 D 最好的点直接冒充 R 最好的点。

[L503] 最终复评圆边数 `quick` 为 180、正常为 360。

[L504] 最终复评报告步长 `quick` 为 0.5° 、正常为 0.2° ，覆盖 `final_quality` 的 0.1° 默认值。

[L505] 初始化输出对照表。

[L506] 对 D 与 R 两个入选站分别标记“Q1直径方法”“Q2最小包围圆方法”。这里 Q1 标签指以直径为目标的对照，实际仍在 Q2 安全选站框架里重新搜索。

[L507] 把 JSON 风格的第二站列表转为元组。

[L508] 对这个固定站按最终分辨率重新评价报告，不重新优化站角或基线。

[L509] 记录方法名、原选基线和偏角。

[L510] 记录局部 a、b 与全局第二站；a、b 不是全局坐标。

[L511] 记录复评的三个最坏指标；这些和 `baseline_search` 中粗阶段目标值精度不同。

[L512] 对照表只保留半径最坏报告角；若要直径或面积最坏角，须直接查看 `final_quality` 返回的完整字典。

[L513] 用所返回的记录最大半径与 20 比较，生成 `meets_20m_on_evaluated_reports`。True 不是连续报告保证；False 也不能证明所有真实站位都不可能两次定位达标。严格说这一字段针对被保留记录，并未把缓存中全部中间评价取最大。

[L514] 返回总结果，附案例编号、首站、规范化首报告。

[L515] 明示误差半宽 1° 、默认安全域 `uniform1000`。

[L516] 标记主方法并放入方法对照。

[L517] 开始把各基线搜索结果压缩成目标、基线、偏角等字段。

[L518] 加入各自搜索阶段目标值，列表推导遍历全部正常返回的行。

[L519] 备注说明 D 对照独立选点，内层连续示向最大值来自有限网格和局部细化。主输出精简掉了一些详细计数与认证字段，但这不改变其数值边界。

main 契约。 输入可选命令行参数列表，读取输入文件、调用总求解、写结果并打印两方法摘要，正常返回 0。若求解或读取失败，本函数没有统一捕获，异常交给 Python 结束进程。

[L521] 定义 `main(argv=None)`；None 让解析器使用真实进程参数。

[L522] 创建参数解析器及中文帮助说明。

[L523] 注册 `--input`，转为 `Path`，默认当前工作目录的 `q2_input.json`。

[L524] 注册 `--output`，默认 `q2_result.json`。

[L525] 注册 `--quick` 布尔开关；不出现为 False，出现为 True。

[L526] 解析参数；非法参数由 `argparse` 给出帮助和退出码。

[L527] 以 UTF-8 读取输入文本，再把 JSON 转成 Python 对象。

[L528] 调用 `solve_case`，第二实参对应 `quick`。

[L529] 创建输出的父目录；`parents=True` 可逐级建立，已存在则不报错。

[L530] 将结果缩进为 2 的 JSON 写入 UTF-8；`ensure_ascii=False` 保持中文可读。

[L531] 打印摘要标题。

[L532] 遍历两条方法对照。

[L533] 在一条输出中格式化方法名、基线、偏角、复评 R/D/A。显示小数位只是打印舍入，JSON 内数值保留更多位。

[L534] 再打印全局第二站坐标，保留 3 位小数。

[L535] 打印实际输出路径。

[L536] 正常完成返回退出码 0。

[L538] 同行完成入口判定与退出：直接运行本文件时调用 `main`，并以返回值作为退出码；被其他脚本导入时只定义函数，不立即跑搜索。

文件二：source/Q2/q2_generator.py

生成器的任务与前提

`main` 契约。读取可选随机种子与输出路径，生成一组物理上存在相容源和接收半径的首测数据，写 JSON 并打印摘要。生成器保存真值仅供核验，求解器不读真值。它生成随机训练输入，不是为误差建立用于优化的概率模型。

为何数据相容：首站在半径 450 m 内；源先选在首站 450—1400 m 距离处，必要时投影回半径 1750 m 的圆。首站在这个圆内，向圆作径向投影不会增加源到首站的距离，因此首测距离仍不超过 1400 m；源也在 1800 m 场地内。取 $\rho = \max(1000, \|g - s_1\|) \leq 1400$ 就存在一个允许接收半径。若发生投影，源到首站至少为 $1750 - 450 = 1300$ m；不投影时至少 450 m，所以不会落在 5 m 近场。

[L001] Python 3 的脚本解释器说明。

[L002] 延迟解释类型注解；这个文件未大量使用注解，保留它不改变算法。

[L003] 同行导入参数解析、JSON、数学和随机模块。

[L004] 导入路径类。

[L006] 定义无显式参数的入口函数；它直接读真实命令行。

[L007] 创建生成器参数解析器。

[L008] 注册可选整数种子；不给时为 `None`。

[L009] 注册输出路径，默认 `q2_input.json`。

[L010] 解析参数。

[L011] 有用户种子则采用；否则通过系统随机源选 1 到 $2^{63}-1$ 范围的整数。输出保存该种子，之后可重做同样数据。

[L012] 用该种子建立局部伪随机发生器，后续各次抽样顺序决定结果。

[L013] 注释说明首测相容、真值仅本地检查、求解器不读取。

[L014] 同行抽首站极角 `sa` 和径向距离 `sr`。`math.tau=2 $\pi$` ；半径均匀抽取不等于圆盘面积均匀抽取，生成器没有声称后者。

[L015] 把首站极坐标转为全局直角坐标。

[L016] 同行抽源相对首站的极角 `ga` 及初始距离 `gd` ∈ [450,1400]。

[L017] 用相对位移加首站，得到初始源全局坐标。

[L018] `math.hypot(*g)` 将两个分量展开，求源到全局原点的距离。

[L019] 检查是否超出 1750 m 的生成安全圈。

[L020] 同行先算投影比例 $1750/d$ ，再等比例缩小两个全局分量；不是缩小源相对首站的位移。

[L021] 重新用最终源坐标减首站坐标求真实方位，再转度和模 360，避免投影后仍用旧角 ga 。

[L022] 开始重新抽误差的循环。

[L023] 真实角加一个 $[-1,1]$ 度误差，模 360，四舍五入到两位小数，再模 360。第二次取模把舍入得到的 360.00 变成 0.00。

[L024] 计算报告减真角的环绕差，规范到 $[-180,180)$ ；例如 359.7° 到 0.2° 的误差是 0.5° ，不是 -359.5° 。

[L025] 同行检查舍入后的误差是否仍不超过 1° ；满足则退出。若不满足则回 L023 重抽，修复两位小数舍入可能越界的情况。

[L026] 构造 JSON 字典，写案例编号、实际种子和首站坐标。

[L027] 写首报告与五条候选基线。

[L028] 在 `local_truth` 中保存源真坐标和真方向，方便检验；没有把源半径作为求解器输入。

[L029] 创建输出父目录，已存在允许继续。

[L030] 写带中文、缩进的 UTF-8 JSON。

[L031] 打印实际种子。

[L032] 打印首测坐标，3 位小数只是显示。

[L033] 打印示向，2 位小数与生成报告精度一致。

[L034] 打印保存路径。

[L035] 直接运行时调用 `main`；导入模块时不生成文件。这里没显式 `return 0`，正常 Python 进程依然以成功状态结束。

文件三：source/Q2/run.bat

启动脚本的任务与条件

本文件供 Windows `cmd` 执行，前提是 `python` 指令可用。它先随机生成新输入，再运行正常精度求解；没有指定 `seed`，也没有 `quick`，因此不同启动输入会变、耗时可能比固定短演示长。需要固定种子演示时直接运行本伴读末尾给出的两条命令，不把这个脚本描述为固定种子测试。

[L001] `@echo off` 隐藏后续命令本身；当前这行的 `@` 也不回显。

[L002] `setlocal` 限制环境变量修改的作用域。

[L003] 设置控制台 UTF-8 代码页 65001，并把成功提示重定向到 `nul`。

[L004] `pushd "%~dp0"` 切到批处理自身所在目录且保存原目录；路径加引号兼容空格。`|| exit /b 1` 表示切换失败就以 1 退出。

[L005] 生成 `q2_input.json`，不带种子意味着本次选新种子。

[L006] 若生成器退出码至少为 1，跳转到清理标签，不继续求解。

[L007] 正常精度求解刚生成的输入，写 `q2_result.json`；没有 `--quick`。

[L008] `:finish` 定义跳转标签。成功路径执行到这里，生成失败路径也跳到这里。

[L009] 先保存此前程序的 `ERRORLEVEL` 为 `RC`，避免接下来清理操作覆盖有用退出状态；引号写法避免尾部空格进入变量。

[L010] `popd` 恢复原工作目录。

[L011] pause 等用户按键，便于双击运行时查看结果或错误。

[L012] exit /b %RC% 结束该批处理并返回保存的退出码；不是关闭整个系统。

带答案的逐行练习

下面的整数数据专为手算执行轨迹设置，不冒充外部教材原例。标准方法的教材例题仍见第 3—4 课。此处每一题都要完成“列一般式 → 代数据 → 指出代码 → 说明结论范围”。

练习 1：从局部站位得到全局站位

已知。首站 (10, 20)，首报告 90° ，基线 5，偏角满足 $\cos \psi = 4/5, \sin \psi = 3/5$ ，且 $\psi > 0$ 。求。candidate 和 local_to_world 的返回值。

解。candidate 返回 $(a, b) = (4, 3)$ 。旋转矩阵 $R(90^\circ)$ 把它变成 $(-3, 4)$ ，加首站得第二站 (7, 24)。对应求解器 L403—L405、L057—L060；有限浮点实现的 $\cos(90^\circ)$ 会留下极小近零残差，不改变几何含义。此题只算坐标，没有检验其符合本题安全域。

练习 2：安全是哪个不等式

已知。默认半径与角误差，基线 $L = 1000$ 。求。两种安全弧上限；偏角 45° 在各域是否通过。

解。uniform1000 的余弦下界为 $(10^6 + 1500^2 - 10^6)/(2 \cdot 1000 \cdot 1500) = 0.75$ ，故上限为 $\arccos(0.75) - 1^\circ \approx 40.409622^\circ$ 。full_information 的下界为 0.5，故上限 $60^\circ - 1^\circ = 59^\circ$ 。 45° 不通过默认域，却通过完整扇区的首接收信息域。对应 L103—L111 与 L088/L094。不能因为备选域允许它，就声称默认程序搜过了这个站点。

练习 3：亲手执行一次裁剪

已知。逆时针方形顶点 (0, 0), (2, 0), (2, 2), (0, 2)，约束 $x + y \leq 3$ 。求。输出顶点与面积，解释交点公式。

解。法向 (1, 1)、offset=3。边 (2, 0) → (2, 2) 的残差是 -1 与 1，故 $t = (-1)/(-1 - 1) = 1/2$ ，交点 (2, 1)；边 (2, 2) → (0, 2) 同理交于 (1, 2)。按代码顺序输出

$$[(0, 0), (2, 0), (2, 1), (1, 2), (0, 2)].$$

面积是原方形 4 减去右上直角三角形面积 $1/2$ ，得 3.5。对应 L128—L137 的边扫描以及 L181—L183 的鞋带公式。若把约束改成 $x + y \leq -1$ ，所有顶点在外、无穿越，输出空表；robust_quality 会在 L327—L328 跳过这样的报告。

练习 4：直径圆为什么不是包围圆

已知。三点 $A = (0, 0), B = (4, 0), C = (2, 3)$ 。求。D、面积、直径圆与 MEC；输出的三个值各代表什么。

解。 $AB = 4$ ，另外两边为 $\sqrt{13} < 4$ ，故 $D = 4$ ；面积 $4 \cdot 3/2 = 6$ 。直径圆中心 (2, 0)、半径 2，漏掉距圆心 3 的 C。对三角形，圆心由对称性写 (2, y)；等距给 $4 + y^2 = (3 - y)^2$ ，故 $y = 5/6, R = 13/6 \approx 2.1666667$ 。这个三角形是锐角三角形，其三点外接圆即 MEC。代码审计半径约为 $13/6 + 10^{-7}$ ，不是必须恰等数学分数。

对应 L186—L199 的两种圆、L202—L235 的增量与审计、L272—L275 的合并。面积小、直径小、包围半径小是相关但不同的要求；清除保证直接比较包围半径与 20。

练习 5：外包误差与报告漏峰不能抵消着讲

已知。对每个报告有 $P(z) \subseteq \tilde{P}(z)$ ；仅对有限 Z_h 评价。求。写两个合法的不等式，判断是否已得连续最坏值上界。

解。同报告下 $R(P(z)) \leq R(\tilde{P}(z))$ 。同一个外包评分函数下， $\max_{z \in Z_h} R(\tilde{P}(z)) \leq \sup_{z \in Z} R(\tilde{P}(z))$ （这里假定样本属于所讨论的同一可行报告域）。一个放大空间集合，另一个缩小检查集合，因此没有由此推出“有限结果 $\geq$ 真连续最坏半径”。外包模型还可能多出假可行报告，范围比较更不能省略。对应圆外包 L114—L178、有限扫描 L334—L344 与认证字段 L303。

练习 6：黄金分割求峰顶的一步

已知。教学函数 $f(z) = 4 - (z - 2)^2$ ，区间 $[0, 4]$ 。这是严格单峰函数。求。第一轮内点、函数值和保留区间。

解。 $\gamma = (\sqrt{5} - 1)/2$ ， $c = 4 - 4\gamma \approx 1.527864$ ， $d = 4\gamma \approx 2.472136$ ；两值相等，约为 3.777088。按 L380 的 $fc > fd$ 分支，令右端为旧 d ，新右内点为旧 c ，保留 $[0, 2.472136]$ ；真实峰 2 仍在其中。旧 fc 被复用，下一轮只新算左内点。Q2 用同一算式，但它没有证明每个实际报告括区都严格单峰，所以本题的删区间理论结论不能无条件照搬。

练习 7：看懂三个“最大”和两个计数

已知。最终 records 恰为三行：报告 10° 的 (R,D,A)=(4,7,8)， 20° 为 (3.8,7.4,9)，另一次仍记录 10° 的 (4,7,8)。缓存还包含一个未入 records 的中间报告 15° ，其 (R,D,A)=(4.2,7.5,8.5)。求。按源码会返回什么，哪些字段名容易误读。

解。max(records) 给 worst_R=4、worst_D=7.4、worst_A=9；半径最坏报告 10° ，其完整指标仍是 (4,7,8)。feasible_report_count=3，report_evaluations=3（三个不同非空角度键 10、15、20）。数量偶然相等，集合含义不同：records 有重复、没收 15° ；cache 无重复、收了中间评价。最终 R 也不是全部缓存中间评价的最大 4.2。这是按 L329—L330、L341、L391、L393—L400 构造的执行检查，不是声称真实标准例确实发生了这种漏峰。

练习 8：读取真实复算的 55.8199286 m

已知。真实复算章的首站为 $(-245.094242089, 222.577343884)$ ，首示向 239.09° ；quick 入选 $L = 1000$ 、 $\psi = -33.25^\circ$ 。求。局部与全局站点、输出结论及限制。

解。局部 $(a, b) \approx (836.286156, -548.293230)$ ；旋转平移后全局 $s_2 \approx (-1145.108945, -213.282190)$ 。复评采用 180 边与 0.5° 粗报告步长，再局部细化。记录中的半径最坏报告约 280.977229° ；四点后验 MEC 约 55.8199286 m，所以代码字段 meets_20m_on_evaluated_reports 为 false。这里只能说此站在此次放宽模型和保留报告记录下未满足 20 m 判据，不能据此证明所有真实源、所有连续站位都不可能两次定位。

对应 L503—L513；前章给了四个顶点及固定报告复算代码。若只复算该报告的四点，不需要重做整个外层搜索。

方法选择与评委追问：附完整短答

问：你的模型叫什么，算法叫什么？

答：外层模型是保证二次接收约束下的极小极大选址；内层每个报告的清除中心是欧氏最小包围圆问题。空间集合用半平面裁剪；直径用凸包与旋转卡壳；最小包围圆用随机顺序增量构造；报告用有限网格和黄金分割式局部搜索；站点用有限基线上的两侧粗细角度网格。不能只用“鲁棒算法”一个词替代这些不同层次。

问：为什么不用牛顿法？

答：牛顿法若用于一维光滑最小化，通常需一、二阶导数并迭代 $x_{k+1} = x_k - f'(x_k)/f''(x_k)$ ，还需处理初值、曲率与收敛条件。这里评分由裁剪后验、MEC 和报告取最大组成，支撑点和最坏报告会切换，代码没有可直接使用的光滑导数。现有实现依赖可算函数值，采用无导数搜索。它没有证明牛顿法必然失败，也没有证明当前方法全局最优。若评委说的是解方程的牛顿法，则对应 $x_{k+1} = x_k - F(x_k)/F'(x_k)$ ，必须先说明要求解的方程是什么，不能把名字直接移植过来。

问：哪些是启发式？是否所有几何计算都是启发式？

答：安全域的扇区最大距离有解析推导；凸包、几何裁剪、MEC 各有标准数学问题。启发式或未认证的数值选择主要在只挑最多八个粗峰、局部套用黄金搜索、每侧只细化最佳粗站、只比较输入有限基线等位置。其作用是减少计算，但可能漏掉更坏报告或更好站。浮点容差是实现近似，另有圆外包近似；它们与“启发式选哪几个候选”也要区分。

问：第二站能接收，为什么还加第二个 1500 m 圆？

答：安全筛选是“对首测相容真源保证接收”的充分条件；后验构造则按两次接收的必要条件逐项写约束。默认域下该约束往往冗余，但保留它使 `robust_quality` 对一般固定站也表达两次接收模型，且对外包多边形有再次裁剪作用。它不能替代外层安全保证，因为某些源能接收不等于所有可能源都能接收。

问：为什么报告非空就可行？源的接收半径必须是同一个。

答：在忽略近场孔的连续放宽模型里，同一点同时满足两次距离不超过 1500 和两个角楔时，可取 $\rho = \max(1000, \|g - s_1\|, \|g - s_2\|) \leq 1500$ ，这个同一半径与两次接收相容。实际代码又把圆外包，非空可能来自放宽误差；若候选位于近场，普通角报告的物理语义也要另处理。所以应说“程序放宽模型的相容报告”，不能把源码说明的 `exactly` 无限制扩大到真实物理系统。

问：为什么不直接优化面积，两个目标结果一样是不是重复计算？

答：半径直接表达能否用一处清除位置覆盖全部候选，面积不能控制细长区域最远端。D 与 R 是不同函数，故对照必须各自选站；某一输入偶然得到同一点，不证明两目标一般等价。代码 L491—L502 确实做了两次独立搜索。

问：程序每个常数是什么性质？

答：1°、1000/1500/1800 m 和最后 20 m 来自题设；EPS、1e-8、1e-7 是浮点容差或覆盖余量；72/96/180/360 是多边形分辨率；2°/1°/0.25°/0.1° 与 0.5°/0.2° 是不同层的采样参数；8 是最多细化的候选峰数；34 是黄金循环次数；20260911 是 MEC 打乱点序的复现种子。它们的来源和可否调节不同。

对照源码后发现的命名与表述边界

这一节记录当前实现应怎样解释，不修改算法、不虚构已经修复。

1. `safe_full_information` 的 `exact` 有模型范围。对完整扇区及首次接收信息可作解析等价推导；未利用场地截断和正常报告的近场排除，不能称为所有物理先验下的最大安全域。
2. `robust_quality` 注释的 `feasible exactly` 有放宽范围。外包多边形和省略的近场孔可能引入真实模型以外的报告。
3. `refine_step_deg` 未使用。改它不会改变搜索；真实控制量是粗括区、最多 8 峰和每峰 34 轮。
4. 两个报告计数字段的名称宽于实现。一个计含重复的保留记录，一个计不同非空缓存键；都不是全部连续可行报告数，缓存键数也不包括空报告调用。
5. 最终最大值只遍历 `records`。粗报告与末尾五点被保留，其余黄金中间评价仅在 `cache`。`meets_20m_on_evaluated_reports` 应按该实际取最大范围解释。
6. 默认参数不等于实际运行参数。`final_quality` 默认 0.1°，`solve_case` 正常显式传 0.2°，`quick` 传 0.5°；默认域也是 `uniform1000`，未用备选 `full_information`。
7. 搜索最佳不等于连续最佳。外层未必取到安全弧端点，排除了零偏角及附近部分；只在每侧最佳粗点附近细化，只比较给定基线。返回的 `false` 认证标志必须与口述一致。
8. 覆盖审计与精确最小性不同。MEC 末尾按给定圆心重算最远顶点距离，保障计算顶点的浮点覆盖；含小余量的结果并不等于符号精确最小半径。凸包、卡壳、裁剪也使用数值容差。

9. 生成器真值不是优化先验。生成器用随机方式造题；求解器不读取 seed 或 local_truth，也没有据此假设误差均匀分布。
10. 固定站复评没有重选站。粗搜索选出的两个站各自加密，未按加密评分重新排序所有粗候选。这是控制计算量的流程，不是更高精度全域最优认证。

七天冲刺中怎样练这份材料

第 3 天用约 90 分钟走完模型总图、练习 1—6 与求解器 A—G；当天再用 45 分钟读 H—I，手指源码口述每一个循环。第 6 天用约 45 分钟做练习 7—8、生成器与批处理，再闭卷回答上面的七个追问。每次抽查都按“这一行读哪些变量、改变什么、为什么允许这样做、保证到哪一步”四问回答。

可复现短演示在仓库根目录运行：

```
python3 source/Q2/q2_generator.py --seed 20260929 \  
--output examples/q2_seed_20260929_input.json  
python3 source/Q2/q2_solver.py \  
--input examples/q2_seed_20260929_input.json \  
--output examples/q2_seed_20260929_quick_check.json --quick
```

Windows 将 python3 换为可用的 python，并把多行命令合成一行；上面反斜杠续行是 Bash 写法。第二条仍要搜索多条基线、多个报告，不能把它称为只算四点的瞬时演示。想迅速讲一个具体结果，使用真实复算章的 post_region → minimum_enclosing_circle 四点片段。

Q3 逐行伴读：从集合不变量到动作、通信和计时

读前约定：本章究竟解释什么

本章对应仓库 source/Q3/ 中的五个文件。它按原文件物理行号解释每个非空行；一段包含多行时会依次说明各行作用。空行仅用于排版，不改变程序行为。q3_official_solver.py 的第 1–502 行与 q3_local_solver.py 逐行相同，只讲一次；正式客户端的增补部分单独讲。行号针对本仓库这一版，修改源码后必须重新核对。

先把程序要完成的任务写成模型。存在 10–16 个静止全向源，每源占用一个不同频道，频道编号为 1–20。真实位置 x_i 属于 $\Omega = B(0, 1800)$ ，接收半径 $\rho_i \in [1000, 1500]$ ；策略不知道 x_i, ρ_i ，只得到动作反馈。一次 `measure(p,i)` 可返回无信号、距离不超过 5 m 的近距离提示，或者带有至多 1° 物理误差的示向。清除半径为 20 m。以下结论依赖这些条件，不能直接移用到 Q4。

三条必须先记住的公式链

第一条：定位是维护集合。对存在且尚未清除的频道，维护凸外包集合 P_i ，要求

$$x_i \in P_i. \quad (\text{Q3-I})$$

正观测给出角楔与距离上界；负观测允许去掉以测点为心的 1000 m 开圆盘。代码用容易计算的凸外包集合保留真值：

$$P_i^+ = P_i \cap \{\text{示向两侧半平面}\} \cap \{u^T(x - p) \leq R\}, \quad R = \min(1500, \max_{v \in V(P_i)} \|v - p\|),$$

$$P_i^+ = \text{conv}(P_i \setminus B^\circ(p, 1000)) \quad (\text{无信号时}).$$

负观测的凸包会把一部分已排除点重新包回来，但不会因此排掉真值。程序用容差修正这些理想集合运算。容差是工程防护，代码没有实现严格的区间算术证明。

第二条：清除是验证整个集合。给定外包圆 $P_i \subseteq B(c_i, r_i)$ ，若 $r_i \leq 19.8$ ，则所有满足

$$\|q - c_i\| \leq 19.8 - r_i$$

的站位都有 $\|q - x_i\| \leq \|q - c_i\| + \|c_i - x_i\| \leq 19.8 < 20$ 。这才是 `nearest_certified_clear` 的依据；“中心看起来很接近”不是依据。

第三条：全清除要同时解决两件事。局部定位解决“已发现的频道怎样清除”；覆盖证书解决“有没有尚未发现的频道”。在所有源数至多 16 的前提下，已经发现 16 个不同频道也能结束搜索。FOUND、CLEARED、UNKNOWN、ABSENT 正是这两层逻辑的状态。

结论级别用语

- 几何结论：在已列模型前提和精确运算下成立，例如外包集合和三角不等式。
- 数值实现：浮点近似，带有限容差和复核，例如圆交点、包围圆半径。
- 启发式：用于减少时间，没有全局最优保证，例如插入评分、侧移方向。
- 运行防线：一旦假设、协议或进展被破坏就抛异常；抛异常不等于数学证明。

文件一：source/Q3/q3_local_solver.py（502 行）

1. 基本向量运算与常数

这些函数的输入输出都是二维实数坐标，长度单位为米，内部角度为弧度。它们没有状态。unit 的零向量分支是工程约定；其余是向量定义本身。

行号	逐行作用及理解要点
[L001-L005]	L1 模块说明声明“保守二维几何、只依赖标准库”。 L2 推迟类型注解求值；L3 导入数学函数；L4 导入伪随机数，供包围圆打乱顺序；L5 导入 Iterable，表示凸包可接受可迭代点集。导入不会读取源真值。
[L007-L010]	L7 将二维浮点元组命名为 Point，只是类型提示。L8 定义一周弧度 2π ；L9 定义几何容差 10^{-7} ；L10 将 1.0051° 转为弧度，等于物理误差 1° 、两位小数舍入裕量 0.005° 和额外 0.0001° 之和。后续所有角楔采用这个更宽的角。
[L012]	add(a,b) 输入两向量，输出逐分量和；同时服务坐标平移。
[L013]	sub(a,b) 输出 $a - b$ ，把两点变成位移向量。
[L014]	mul(a,s) 输出标量乘法 sa ；s 可以为负或为分数。
[L015]	dot(a,b) 输出 $a_x b_x + a_y b_y$ ，用于投影、距离平方和半平面判断。
[L016]	cross(a,b) 输出二维有向面积 $a_x b_y - a_y b_x$ ；其正负区分左转和右转。
[L017]	norm(a) 输出 $\sqrt{a_x^2 + a_y^2}$ ；*a 将元组拆给 hypot。
[L018]	dist(a,b) 输出两点欧氏距离，等于 norm(sub(a,b))。
[L019-L021]	L19 定义单位方向函数；L20 算模长；L21 若模长超过 EPS 则逐分量除以它，否则返回固定方向 (1,0)，避免除零。零向量没有数学上唯一的单位方向。
[L022-L023]	输入测点 p 和多边形顶点；输出最大顶点距离，空表返回 0。对凸多边形，任一点是顶点凸组合，范数的凸性保证最大距离可由顶点取得。空表返回 0 只是辅助函数约定，不能当“空定位集可安全清除”。

2. hull：点集到凸包

契约。输入有限点集；输出按逆时针排列的凸包极点，去掉重复点和边上冗余共线点。扫描中 lo、hi 各自保持凸链；结论是单调链凸包算法。它不求源的位置，只在负观测后把剩余集合凸化。

行号	逐行作用及理解要点
[L025-L028]	L25 定义函数；L26 set 去重再字典序排序；L27 不超过两点时直接返回；L28 分别建立下凸链和上凸链两个独立列表。

行号	逐行作用及理解要点
[L029-L031]	L29 按从左到右遍历。L30 只要最后两个已存点与新点形成非左转，就删除末点，再继续检查； ≤ 0 也删共线中间点。L31 新点入下链。循环不变量是链上每个连续三点严格左转。
[L032-L035]	L32 反序遍历；L33 用相同叉积规则维护上链；L34 新点入上链；L35 两链各去掉重复端点再拼接，得到一圈边界，不在末尾重复首点。

3. clip: 半平面裁剪

契约。输入按边界顺序排列的凸多边形、非零法向量 n 、阈值 b ；输出 $P \cap \{n^T x \leq b\}$ 的顶点外近似。遍历时 a 是上一顶点， $f_a = n \cdot a - b$ ； c 是当前顶点。这是 Sutherland-Hodgman 式逐边裁剪。真实集合保持性依赖约束方向正确。

交点公式来自 $x(t) = a + t(c - a)$ ，代入边界得 $f_a + t(f_c - f_a) = 0$ ，因此 $t = f_a / (f_a - f_c)$ 。

行号	逐行作用及理解要点
[L037-L042]	L37 定义函数；L38 说明保留 $n^T x \leq b$ ；L39 空集交任何集合仍为空。L40 增大阈值 $\text{EPS} * \max(1, \text{norm}(n))$ ，略向外放宽边界；当 $\ n\ \geq 1$ 时对应法向距离 EPS ，较短法向量时放宽更多。L41 建输出表；L42 以上一顶点为末顶点开始，从而首轮处理闭合边，并计算其带符号剩余量。
[L043-L047]	L43 遍历当前顶点；L44 算其剩余量。L45 比较两端是否位于半平面内；只有内外不同才穿过边界。L46 求线段交点参数，L47 加入交点。这里不是求两条无限直线的所有交点。
[L048-L050]	L48 当前顶点在内则保留；L49 把当前点与剩余量作为下一轮的上一点；L50 只有至少两点时才进入去重。
[L051-L056]	L51 先保留首点；L52 遍历余点；L53 仅保留离前一保留点超过 10^{-9} 的点；L54 若末点与首点几乎相同就删末点；L55 更新输出；L56 返回。这里清理的是相邻重复点，不重新求凸包。

4. arena_polygon: 为什么必须外接

契约。输入合法边数 $\text{sides} \geq 3$ ，输出半径 1800 圆域的外接正多边形，默认 48 边。目标不变量是 $\Omega \subseteq P$ 。程序未显式检查非法边数，正常调用只用默认值。

行号	逐行作用及理解要点
[L058-L061]	L58 定义默认 48 边；L59 注释强调外接而非内接；L60 令顶点半径为 $1800 / \cos(\pi/n)$ ，其边心距恰为 1800；L61 每隔 $2\pi/n$ 生成一顶点，并整体偏转 π/n 。若误用顶点半径 1800，会切掉真实圆域靠近圆弧的部分，破坏初始不变量。

5. bearing_update: 两条角边加一条轴向界

契约。输入已有外包多边形、测点、已接收到的角度；输出角楔与距离约束的三角形外包。前提是源未清除、观测有效、源距不超过 1500。它没有精确裁剪圆弧；将圆盘放宽为轴向半平面，所以保持凸多边形数据结构。

设 $u = (\cos \theta, \sin \theta)$ 、 $u_- = u(\theta - \delta)$ 、 $u_+ = u(\theta + \delta)$ 。角楔是 $\text{cross}(u_-, x - p) \geq 0$ 和 $\text{cross}(u_+, x - p) \leq 0$ 。把叉积写成法向量点积，就得到 L73 的两个法向量。

行号	逐行作用及理解要点
[L063-L067]	L63 定义函数；L64-L67 文档说明结果是 $P \cap$ 角扇形 $\cap B(p, 1500)$ 的外包，并说明当前多边形可提供更紧距离上界。文档的“四行”是一段字符串，不是四个执行步骤。
[L068-L072]	L68 取物理上界 1500 与当前最大顶点距离加容差的较小值；L69 度转弧度；L70、L71 生成下、上边界方向；L72 写出两条叉积不等式，给下一行的符号提供依据。
[L073-L076]	L73 逐个取下边界的右法向量与上边界的左法向量；L74 使用 $n^T x \leq n^T p$ 裁剪。L75 生成中心方向 u ；L76 再裁掉 $u^T(x - p) > R$ 的点并返回。用径向向上界推出轴向上界合法，但反推不成立，所以这是放宽。

6. exclude_disk_hull: 怎样合法利用无信号

契约。输入凸外包集合 P 、负观测测点 c 、允许排除的半径；输出剩余区域的凸外包。理想几何下，极点只可能来自原多边形的保留顶点及边与圆的交点。前提是负反馈确实排除了该开圆盘；Q3 无信号可用 1000，失败清除可用 20，Q4 单个负反馈不满足此前提。

沿边 $x(t) = a + td$ 解 $\|x(t) - c\|^2 = r^2$ ，得到 $At^2 + Bt + C = 0$ ，其中 $A = d^T d$ 、 $B = 2(a - c)^T d$ 、 $C = \|a - c\|^2 - r^2$ 。输出的凸包可能再次包含圆盘中的点，所以负观测历史需要保存。

行号	逐行作用及理解要点
[L078-L082]	L78 定义函数；L79 定义凸化后的安全放宽；L80 说明候选极点；L81 说明缩小排除半径留数值裕量；L82 结束文档字符串。
[L083-L085]	L83 空集直接返回。L84 半径减 10^{-5} 并截到非负，再算平方；少排一点更保守。L85 收集离圆心不小于 $r - 1e-6$ 的旧顶点，多保留一点也偏向保守。
[L086-L090]	L86 枚举每条边起点；L87 循环下标取终点，并分别算边向量和圆心到起点向量；L88 算二次项；L89 极短边跳过以免除近零；L90 算一次项和常数项。
[L091-L095]	L91 判别式；L92 负值说明按当前浮点计算无实交点；L93 开方前再次用零下界；L94 遍历两个根；L95 只保留落在边线段参数范围的根，并给端点小裕量。
[L096-L098]	L96 将带裕量的参数截回 $[0, 1]$ ；L97 生成交点加入候选；L98 对候选求凸包。即使同一交点被两边加入，hull 会去重。圆盘完全挖在多边形内部时，旧顶点全保留，凸包可能完全不缩小，这是凸化的代价。

7. inside_polygon: 核查点属于哪个退化集合

契约。输入点和逆时针凸多边形，输出含容差的成员判断。空、多点、单点、线段分别处理；不修改状态。此函数可供审计，主策略不靠它偷看真值。tol 对线段投影参数和法向距离有不同用法，不应宣称统一绝对距离容差。

行号	逐行作用及理解要点
[L100-L104]	L100 定义默认容差 10^{-4} ；L101 空集不含点；L102 单点按距离判断；L103 两点进入线段分支；L104 解包端点并算边向量。
[L105-L107]	L105 算点投影在线段上的参数，分母用 10^{-30} 防零；L106 同时检验投影处于稍放宽的 $[0, 1]$ 、离支撑直线足够近。L107 三点以上要求对每条有向边均处于左侧或容差内；这依赖逆时针顶点顺序。

8. 包围圆的三个小工具

契约。_diameter(a,b) 输出以两点为直径端点的圆；_circum(a,b,c) 输出三点外接圆，近共线返回 None；_in(circle,p) 输出容差成员判断。均无状态。外接圆公式来自圆心到三点等距，平移到 a 附近有助于减小大坐标相减的损失。

行号	逐行作用及理解要点
[L109-L110]	L109 定义两点圆；L110 先取中点，再返回 (圆心, 两点距离的一半)。一行中含赋值和返回两个动作。
[L111-L114]	L111 定义三点圆；L112 解释先平移；L113 算 $u = b - a, v = c - a, D = 2 \text{cross}(u, v)$ ；L114 若分母近零则不造一个巨大、不稳定的圆。
[L115-L118]	L115 算 $\ u\ ^2, \ v\ ^2$ ；L116 解二元线性方程得到相对圆心再加回 a；L117 返回圆心和到 a 的距离。L118 定义圆内判断，半径放宽 10^{-9} 。

9. mec: 随机增量只是求圆的次序，证书还要复核

契约。输入非空有限点集；输出中心 c 和覆盖全部输入点的复核半径 r。理想算法求最小包围圆；浮点程序用于清除的半径是最后重新计算的最大点距加 10^{-6} 。安全判断依赖这个复核量，不依赖随机性或“最优圆”标签。若中心不是数值意义最优，也只会影响效率。

为何出现三层循环。新点在旧圆内就不改变解；若在外，新的解可选以此点为边界点。第二个不在圆内的点也成为强制边界点。第三层在直线两侧寻找支持圆，而非随便用最近三个点的外接圆。

行号	逐行作用及理解要点
[L120-L123]	L120 定义输出 (Point,float)；L121-L123 文档说明随机增量与全量半径复核，随机顺序每次固定。不能把这段文档读成“无需数值误差分析”。
[L124-L128]	L124 空点集报错；L125 复制点表，用独立种子 271828 打乱，不改变调用者列表或全局随机状态；L126 初始无圆；L127 顺序加入点；L128 新点已被覆盖则继续下一点。
[L129-L132]	L129 否则以新点为零半径圆；L130 遍历此前所有点；L131 当前点 q 已在圆内则略过；L132 否则先建以 p,q 为直径的圆，两者作为后续强制边界。

行号	逐行作用及理解要点
[L133-L137]	L133 注释说明两侧极端支持圆；L134 算 pq 并初始化左、右候选；L135 遍历比 q 更早的点；L136 在直径圆内的点不需形成额外支持圆；L137 对外点算所在线侧及三点外接圆。
[L138-L141]	L138 近共线没有可靠三点圆则跳过；L139 用圆心到 pq 的有向面积作为排序量；L140 左侧保存有向量最大的支持圆；L141 右侧保存有向量最小的支持圆。两处不是仅按半径贪心。
[L142-L146]	L142 取存在的左右候选圆；L143 若存在，选两侧候选中半径较小者，否则保持直径圆；L144 取最终中心；L145 强调后续不用内部半径认证；L146 遍历原全部点重算最大距离再加 10^{-6} 。即使某中间浮点分支漏了点，最终半径仍把它算入。

10. nearest_certified_clear: 安全站位的最近点

契约。输入当前站位、已认证外包圆、清除安全半径（默认 19.8）；要求 $r \leq \text{clear}_r$ ，输出圆盘 $B(c, \text{clear}_r - r)$ 中离当前位置最近的点。此圆盘是所有合法清除站位的一个充分子集，不一定是完整可清除域。

行号	逐行作用及理解要点
[L148-L151]	L148 定义默认安全半径；L149 写出目标圆盘；L150 剩余半径 $\text{slack} = \text{clear}_r - r$ ；L151 若为负，前提不成立就抛错。
[L152-L154]	L152 算当前位置到圆心距离；L153 已处于安全圆盘内则无需移动；L154 否则沿圆心指向当前位置的射线投影到半径 slack 的边界。该分支 $d > \text{slack} \geq 0$ ，因此不会除零。

11. 圆弧覆盖工具：将连续几何转成角区间

`arc_covered_by_disk`

契约。给定参考圆周 $x(\theta) = c + R(\cos \theta, \sin \theta)$ ($R > 0$) 和覆盖圆盘 $B(s, r)$ ，返回被覆盖的角区间，统一在 $[0, 2\pi]$ 。由余弦定理得

$$\cos(\theta - a) \geq \frac{d^2 + R^2 - r^2}{2dR}, \quad d = \|s - c\|, \quad a = \arg(s - c).$$

相切产生的零长度弧被当为空；这会使证书更难通过，而不会凭孤立相切点宣称覆盖了一段弧。

行号	逐行作用及理解要点
[L156-L160]	L156 定义函数；L157 算圆心距；L158 近同心时按 $r \geq R$ 返回整圈或空；L159 覆盖盘包含整个参考圆则返回整圈；L160 两圆外离/外切，或覆盖盘在参考圆内侧且至多相切，则没有正长度被覆盖弧。
[L161-L163]	L161 算覆盖盘方向并化到 $[0, 2\pi)$ ；L162 把余弦值截到 $[-1, 1]$ 防止舍入后 acos 越界，再得半弧宽；L163 得中心角左右端点。

行号	逐行作用及理解要点
[L164–L166]	L164 若跨过 0 则拆成两段；L165 若跨过 2π 同样拆段；L166 否则返回单段。返回列表是为了统一处理跨周情况。

merge_intervals

契约。输入端点有序的闭区间，输出按左端排序、合并相交或极近区间的列表；遍历中 out 是已处理输入的并集近似。

行号	逐行作用及理解要点
[L168–L173]	L168 定义；L169 建结果；L170 按左端排序遍历；L171 无已有段或与上一段分离超过 10^{-12} 则新增 [l,u]；L172 否则将上一段右端延长到两者较大值；L173 返回。列表端点可修改，区别于不可变元组。

interval_subset

契约。输入若干目标区间和覆盖区间，输出所有目标是否被覆盖。x 是当前目标已连续覆盖到的位置，不能跨过未覆盖缺口。容差可能桥接极小角度缝隙，外层还用 1 m 半径余量降低风险。

行号	逐行作用及理解要点
[L175–L179]	L175 定义；L176 先合并覆盖段；L177 每个目标分别检查；L178 从目标左端开始；L179 顺序遍历覆盖段。
[L180–L185]	L180 整段在已覆盖位置左边则跳过；L181 下一段左端超过当前位置且不在容差内，存在缺口，结束此目标扫描；L182 否则把已覆盖位置推进；L183 到目标右端即可停止；L184 尚未到右端则返回假；L185 全部目标成功才返回真。

12. covers_arena: 不是网格试验的连续证书

契约。输入已经完成全频道未知源扫描的站位集，输出半径 999 m 圆盘并集是否通过圆域覆盖检查。物理保证接收半径至少 1000，留出 1 m 余量。先证场地外边界全被覆盖，再检查每个传感圆周位于场地内的弧都被其他圆盘覆盖；后一步用于排除内部漏洞。

为何只看外圆边界不够？一圈传感盘可以覆盖整个场地外边界，却留下中心空洞。若有限圆盘并集在场地内部仍有一个开空洞，其边界必含某个传感圆的暴露弧；检测这些弧才能排除漏洞。这里是基于圆弧计算的几何证书，不是抽样若干网格点。

行号	逐行作用及理解要点
[L187–L191]	L187 默认圆盘半径 999；L188–L191 说明连续圆盘并覆盖检查、两类边界和去重。只有 sites 代表有效全扫描时，几何覆盖才能转化为“未知频道不存在”。
[L192–L197]	L192 建唯一站位列表；L193 遍历站位；L194 与已有点均相距超过 10^{-5} 才加入，避免重复圆的退化；L195 建场地边界覆盖弧；L196 累加每站覆盖场地圆周的弧；L197 若整圈不被区间并覆盖立即失败。

行号	逐行作用及理解要点
[L198-L201]	L198 逐个检查传感圆；L199 计算它落在场地内的圆弧；L200 没有内侧弧就无需检查；L201 准备其他盘对它的覆盖弧。
[L202-L205]	L202 遍历其他站；L203 必须排除自己，否则自己全覆盖自己会使证书失去意义；L204 若任一场地内弧未被别人覆盖则失败；L205 两类检查都通过才返回真。

13. ring_sites: 九站方案的外圈八站从哪来

契约。输入环上站数 $m \geq 6$ 、相位和方向，输出 m 个外圈站。注释提到“原点加 m 站”，但本函数不返回原点；原点在 `run` 中单独扫描。默认参数 $m=7$ 也不是正式策略使用值，`Config.ring_m=8` 才是主线。

外圈最难覆盖的场地边界点处在相邻站角平分线上，距离平方是 $1800^2 + a^2 - 3600a \cos(\pi/m)$ 。令该值为 995^2 ，取关于 a 的较小根，再向外加 1 m 得 L210。 $m=8$ 时 $a \approx 945.973419$ m；实际运行还用 `covers_arena` 检查完整圆域，而不是只信边界距离公式。

行号	逐行作用及理解要点
[L207-L210]	L207 定义环参数；L208 文档概括整体方案；L209 排除不足六点的调用（该范围内还涉及平方根可行性）；L210 解上述二次方程求环半径。995 是构造裕量，与覆盖检查的 999、物理 1000 不同。
[L211-L212]	L211 顺时针取负角步长，否则正角步长；L212 输出 m 个均匀角点，既决定站位也给出初始访问顺序。

14. 动作接口、配置与频道状态

Actions、Config 与 Track

`Actions` 只规定策略向外能发什么动作，不提供真值查询。`Config` 是不变的策略参数；`Track` 是每个频道独立、可更新的定位状态。`UNKNOWN` 表示尚未发现；`FOUND` 表示已发现且未清除；`CLEARED` 表示清除成功；`ABSENT` 表示已有全局证据判定该频道不存在源。

行号	逐行作用及理解要点
[L215-L219]	L215-L219 是源码中间的说明字符串，不是新的执行入口：强调策略只消费公开动作反馈。L217 所写 <code>geometry.py</code> 属于拼合前组织说明；本文件的几何函数已经放在前 212 行，没有实际导入这个文件，也没有导入 Q1 或历史结果。
[L220-L222]	L220 导入数据类和默认值工厂；L221 再次导入 <code>math</code> ，重复导入通常取缓存模块；L222 导入回调类型和结构化接口 <code>Protocol</code> 。
[L224-L226]	L224 定义动作协议；L225 规定 <code>measure(位置,频道)</code> 返回字典；L226 规定 <code>clear</code> 同样返回字典。行末 ... 是占位体，正式功能由模拟器或 HTTP 客户端实现。
[L228-L235]	L228 令配置数据类冻结，实例字段不能正常赋值修改；L229 定义类；L230-L235 文档区分默认主线与本地对照模式，并说明未声称执行 2-opt，角误差另含裕量。

行号	逐行作用及理解要点
[L236-L241]	L236-L237 概括主线；L238 外圈八站，连同原点共九站；L239 侧移量为当前半径的 0.10 倍；L240 插入评分阈值为 600 m；L241 开启无信号排除。这些参数分别服务覆盖、局部定位、调度、集合更新，不能说都是“算法最优参数”。
[L242-L244]	L242 在搜索站顺便重测已发现源；L243 完成某源服务后也顺便重测其他源；L244 默认模式 <code>integrated</code> 在搜索中插入服务。 <code>opportunistic_clear</code> 的实际调用是 <code>_sweep_found</code> ，并非无条件清除其他源。
[L246-L250]	L246-L248 标记以下选项是默认关闭的本地研究功能，并说明保留平坦字段名便于数据类工具；L249 不在服务点额外全扫描；L250 若打开额外扫描，距离规则默认阈值 500 m。
[L251-L254]	L251 默认额外扫描策略名 <code>distance</code> ；L252 不删减未来覆盖站；L253 不旋转环；L254 非认证冒险清除阈值为 0，即关闭。解释正式算法时不能把这四种研究策略当成已经启用。
[L256-L260]	L256 定义可变数据类；L257 类名；L258 初始未知；L259 每次新建 <code>Track</code> 都用工厂创建独立场地外包多边形；L260 每个频道独立创建负观测列表。 <code>default_factory</code> 避免 20 个频道共用同一个可变列表。
[L261-L265]	L261 初始圆心原点；L262 初始半径无穷大，阻止未定位就清除；L263 尚无正测向站位；L264 初始角占位 0；L265 正示向计数为 0。半径和角的初值都不代表真实观测。

15. Strategy.__init__ 与 _audit

契约。构造器输入动作接口、配置、可选审计回调；输出策略对象。初始 20 个频道未知，机器人原点、接收频道 1，尚无覆盖证书。`_audit(i)` 记录规模并把该频道状态交给可选观察者。正式 `run_case` 默认不传审计；本地审计即使看真值，也应只做断言，不把真值反馈给策略决策。

行号	逐行作用及理解要点
[L267-L271]	L267 定义策略类；L268 构造器默认使用冻结 <code>Config()</code> ；L269 保存接口、配置、回调；L270 对 1-20 各建独立 <code>Track</code> ；L271 设置位置、当前接收频道、累计虚拟时间。
[L272-L275]	L272 初始化已扫描站、待扫描站、覆盖完成标志和证书原因；L273 初始化清除数、动作数、覆盖检查数；L274 初始化最大顶点数与单次服务最大迭代数；L275 环计划尚未建立。这些诊断计数不参与真值估计。
[L277-L279]	L277 定义审计辅助；L278 取频道并更新见过的最大顶点数；L279 若给了回调就传入 (频道,Track,当前位置)。不返回定位结果，也不替代包围圆认证。

16. _observe: 一个反馈怎样变成状态和集合

契约。输入频道和计划测点，先真实调用 `measure`，再根据接受且有效的响应更新状态。返回反馈类型。`FOUND` 频道始终应保持真值包含性与有效包围圆；若集合变空则立刻中止，不能偷偷重置大集合并继续报成功。

三角外包的顶点在以测点为原点、示向为横轴时是 $(0, 0), (R, R \tan \delta), (R, -R \tan \delta)$ 。其外接圆心在 $(h, 0)$ ，由等距条件

$$h^2 = (R - h)^2 + R^2 \tan^2 \delta \implies h = \frac{R}{2 \cos^2 \delta}.$$

这给出独立于随机包围圆过程的备援中心。实际半径仍对 `t.poly` 全量重算。首次正观测 $R \leq 1500$ ，理想半径不超过约 750.230847 m。

行号	逐行作用及理解要点
[L281-L286]	L281 定义观察操作；L282 调动作接口；L283 未被接受就中止，不能拿拒绝响应更新模型；L284 更新策略所记录的位置、接收频道和动作数；L285 以接口返回的累计时间为准；L286 取频道对象与反馈类型。
[L287-L290]	L287 无信号分支；L288 保存排除半径 1000 的负观测，即使此时频道尚未知也保存；L289 只有已发现且开启负观测利用时即时裁剪；L290 用凸化排除更新定位集。未知频道不会仅因一次无信号就改成 <code>ABSENT</code> 。
[L291-L295]	L291 空集合说明模型/观测/数值不一致，中止；L292 更新包围圆；L293 做审计。L294 若返回近距离提示，L295 直接在当前点认证清除，因为已知距离至多 5 m，小于 20 m。
[L296-L300]	L296 正示向分支；L297 一行依次置 <code>FOUND</code> 、记本次测点、角度转浮点、正示向计数加一；L298 在裁剪前求距离上界，供独立三角圆使用；L299 做示向更新；L300 若配置允许则利用历史负反馈。
[L301-L304]	L301 逐条重新排除历史开圆盘：此前凸化可能把已排除区域又包入，故重用历史有意义；L302 空集合中止；L303 算候选包围圆；L304 注释说明三角包围证明。
[L305-L309]	L305 度转弧度并计算三角圆轴向距离 h ；L306 将中心从局部坐标变回全局坐标；L307 对当前全部顶点复核以此为心的半径再加裕量；L308 仅当比 <code>MEC</code> 候选更小时替换中心和半径；L309 审计。二者都是外包证书，取较小半径不损失安全性。
[L310-L311]	L310 未知反馈类型中止；L311 返回本次反馈类型，让 <code>_serve</code> 检查无信号和收缩情况。

17. _clear: 成功、失败和“结果未知”不能混为一谈

契约。输入频道、目标站位和是否已有清除证书；调用接口后只在明确成功时改为 `CLEARED`。已认证动作如果失败会抛异常；研究性未认证动作失败才用 20 m 排除继续定位。此处处理的是明确响应，网络结果不确定由正式客户端另行处理。

行号	逐行作用及理解要点
[L313-L318]	L313 默认认为调用者已认证；L314 发清除；L315 未接受就中止；L316 更新位置、动作数、虚拟时间；这里不改接收频道，因为清除动作不等于接收机换台。
[L319-L321]	L317 检查明确成功；L318 改状态并清除数加一。L319 成功返回真；L320 既不成功也不是明确“范围内无目标”则报协议类型错误；L321 若此前有证书却失败，必须中止并暴露不变量失败。不能把认证失败算作成功或换一个随机点掩盖。
[L322-L326]	L322 未认证失败可推出目标不在 20 m 内，记录负观测；L323 凸化排除；L324 排空则中止；L325 重算包围圆并审计；L326 返回假。默认 <code>speculative_r=0</code> 下不进入这条研究失败恢复路径。

18. `_coverage` 与 `_certify_and_prune`：怎样证明搜索结束

契约。`_coverage(points)` 返回几何覆盖检查并计数。`_certify_and_prune` 更新搜索完成标志、频道不存在状态、未来扫描计划。允许两种证书：已有 16 个不同频道；已扫描站保证覆盖整个场地。`prune` 分支仅在研究配置下删未来站，且须重新证明剩余站加历史站仍覆盖。

行号	逐行作用及理解要点
[L328-L330]	L328 定义包装函数；L329 覆盖检查数加一；L330 返回圆弧证书结果。计数只用于运行记录。
[L332-L337]	L332 定义搜索认证；L333 已认证就不重复处理；L334 FOUND 与 CLEARED 共达 16 时，利用源数上界；L335 标完成、清空待扫描站、记原因 <code>cardinality</code> ；L336 遍历频道；L337 所有未知频道改为不存在。此依据是“已找到上限个不同源”，不是“猜测本局有几个源”。
[L338-L343]	L338 上界证书完成即返回；L339 否则验证历史全扫描站；L340 通过则标完成、清空计划、记原因 <code>disk_union</code> ；L341-L342 将未知改不存在；L343 返回。几何覆盖与每站扫遍当时未知频道共同保证不会漏掉仍未知的源。
[L344-L348]	L344 只有开启剪枝且已有计划才继续；L345-L346 注释强调默认关闭，且删点前需剩余联合覆盖；L347 初始化下标；L348 逐个考虑待扫描站。
[L349-L351]	L349 构造去掉第 k 站的候选未来列表；L350 若历史站加该列表仍覆盖，就实际删除第 k 站，下一轮继续检查移到同一下标的新元素；L351 否则下标前移。不能删除后仍加一下标，否则会漏检紧邻候选。

19. `_scan` 与 `_sweep_found`：全扫描和顺便重测不同

契约。`_scan(p)` 对当前所有未知频道执行检测，并登记一处全扫描位置；`_sweep_found(p)` 只选择有保证接收、且与上次测向站有足够间隔的已发现频道补测。两者都发真实动作、消耗时间；后者不能登记成全扫描。

行号	逐行作用及理解要点
[L353-L358]	L353 定义全扫描; L354 收集此刻未知频道; L355 注释减少第一步换台; L356 排序键 (是否非当前频道, 编号), 若当前频道在未知列表则优先它; L357 每频道调用观察; L358 完成该列表后登记站位。已发现频道无需被全扫描重复搜索, 若某源仍未知, 它一直在此列表中。
[L359-L361]	L359 若允许顺便测向, L360 重测已发现源; L361 检查搜索完成或剪枝。因为补测或近距离清除也可能让已发现总数达到 16, 所以更新后再检查有意义。
[L363-L367]	L363 定义顺便重测; L364 注释提醒不在同一频道同一位置重复; L365 遍历各频道; L366 不是已发现、已可清除、或尚无上次正测点则跳过; L367 与上次正测点距离不足 25 m 也跳过, 避免信息改善太小却付测量费用。25 m 是启发式阈值。
[L368-L369]	L368 若所有可行顶点距当前站都至多 999, 则真源也保证在接收范围内; L369 才观察。仅检查圆心距离不够, 因为真源可能位于外包集远端。

20. _make_plan: 固定环主线与研究旋转

契约。建立待扫描环站顺序, 输出通过对象字段体现。默认固定相位 0、逆时针, 不求 TSP 最优路。若研究配置 rotate=True, 只搜索有限相位/方向, 以经验评分选择; 仍没有全局最优证明。

行号	逐行作用及理解要点
[L371-L376]	L371 定义; L372 若搜索已结束; L373 清空计划并标就绪; L374 返回; L375 默认相位 0、逆时针; L376 收集已发现源状态, 为可选旋转评分准备。
[L377-L382]	L377 只有开旋转且已有发现才搜索; L378-L379 明示关闭的研究分支和启发式性质; L380 建候选表; L381 对每个已发现源; L382 考虑其示向及偏转半个站位角两种相位。
[L383-L387]	L383 将示向转弧度并加偏移; L384 枚举顺逆时针; L385 生成该方案环站; L386 评分归零; L387 对每个已发现源计算贡献。
[L388-L391]	L388 找离该源估计中心最近的环站下标; L389 评分包含较晚访问的惩罚 $0.12 \cdot k \cdot \text{首站半径}$ 和该站到估计中心距离; L390 保存 (评分, 相位, 方向); L391 取最小元组, 丢弃评分只保留几何方案。0.12 为经验系数。
[L392-L394]	L392 使用最终相位/方向生成待扫站; L393 标计划就绪; L394 尝试搜索认证或可选删站。默认方案此时并未单凭计划声明搜索结束, 只有实际扫描历史才能认证。

21. _tracking_point: 为什么朝中心走, 又为什么侧移

契约。输入当前频道的外包圆、可选下一覆盖站; 输出下一个服务点。若已可清除返回最近认证点; 否则在估计中心两测试两个小偏移, 用全顶点距离验证接收保证, 再以移动距离选择。偏移与方向选择属启发式, 接收条件属几何充分条件。

若 q 离原中心最多 $0.1r$, 则旧集到 q 的距离上界为 $1.1r$ 。结合三角圆可得理想收缩

$$r^+ \leq \frac{1.1}{2 \cos^2 \delta} r \approx 0.550169288r.$$

它依赖确有新正示向；近距离反馈直接清除。代码用 0.9 的宽松阈值检查实际进展，不能反过来说“理论收缩系数就是 0.9”。

行号	逐行作用及理解要点
[L396-L401]	L396 定义；L397 读中心半径；L398 半径至多 19.8 直接返回最近认证清除点；L399 算当前位置朝圆心的单位方向及其垂直向量；L400 侧移量取 <code>offset*r</code> ；L401 偏移为正时生成左右两个点，否则只取圆心。
[L402-L404]	L402 注释要求整个可行集都在接收域内；L403 用最大顶点距离至多 999 过滤候选；L404 若都不合格则回退圆心。默认首次半径小于 751 且后续收缩，所以圆心仍在保证接收范围内；这不是对任意改坏配置、任意状态的独立证明。
[L405-L407]	L405 没有下一搜索站时；L406 选从当前位置出发最近的合法候选；L407 否则选“当前移动距离 + 0.15 × 到下一站距离”最小者，轻度偏向后续路线。0.15 没有全局最优依据。

22. `_serve`：一条源的定位—清除闭环

契约。输入一个 `FOUND` 频道及可选下一搜索站；循环测向直到可认证清除。结束时该频道应已清除，否则抛错。单次服务测量计数和全局计数分别记录。正常默认状态下理论几何收缩支持有限步达到阈值；18 步只是程序保护上限。

行号	逐行作用及理解要点
[L409-L413]	L409 定义；L410 取频道引用并清零本次测量步数；L411 只要仍已发现未清除就循环；L412 达 18 次仍未完成则中止；L413 已有认证半径时进入清除分支。
[L414-L418]	L414 求最近认证点；L415 发认证清除后跳出服务循环。L416 若研究性冒险阈值启用且半径够小；L417 注释其风险性质；L418 在中心尝试未认证清除，成功就退出，失败则 <code>_clear</code> 更新排除集后继续。默认不会走 L416-L418 的动作。
[L419-L424]	L419 选跟踪点；L420 保存旧半径；L421 测量后步数加一；L422 如果无信号；L423 注释这违反已验证接收保证；L424 抛错停止，不临时改成随机搜索。
[L425-L427]	L425 正示向后的半径若大于 $0.9 \cdot \text{oldr} + 0.001$ ；L426 抛出收缩不变量失败。近反馈导致清除，不要求再检半径。L427 退出服务后更新观察到的最大局部测量次数。
[L428-L430]	L428 若开启服务后的顺便重测；L429 在当前位置扫描符合条件的已发现频道；L430 更新全局覆盖/数量证书。这里的 <code>_sweep_found</code> 可能收到 <code>near</code> 并清除，但不会无证书地批量清除。
[L431-L434]	L431 仅在研究额外扫描开启、搜索未完成且少于 16 个已清除时；L432 注释默认关闭；L433 用辅助函数评估；L434 值得才在服务终点做全扫描。

23. _worth_extra_scan: 把启发式的量纲讲清

契约。返回额外全扫描是否值得的布尔值，仅研究选项可达。距离策略直接判断是否离历史站足够远；另一策略比较“可能节省的移动时间”和“检测成本上界”。它只在已有计划上估算，不是最优控制解。

行号	逐行作用及理解要点
[L436-L441]	L436 定义；L437 文档注明研究用途；L438 默认距离策略；L439 当前位置到最近历史站的距离达到 <code>scan_spacing</code> 才值得，无历史站用无穷大；L440 其他策略若无未来站便不需要节省路线；L441 复制计划并置下标 0，避免估算时改真实计划。
[L442-L446]	L442 循环检查假设计划；L443 先删掉候选站得到剩余表；L444 若加上当前位置这个假设扫描站后仍覆盖，则从假设列表删站；L445 否则前移下标；L446 一个未来站也删不了就返回假。
[L447-L449]	L447 定义局部路径长函数，输入站序列输出米；L448 把“当前位置+所有站”和“所有站”错位配对，累加当前位置→首站→次站的开放路线，不含回程；L449 原路线与假设路线长度差除以速度 5，转为可能节省的秒数。
[L450-L452]	L450 统计未知频道数 q ；L451 注释说明是代理比较；L452 要求节省秒数大于 $6 \cdot q + 5$ ：每频道测量 5 s 加至多 1 s 换台，再加 5 s 余量。不要把此式当完整未来成本函数。

24. _choose: 插入代价是评分，不是定位证书

契约。输入下一搜索站或 `None`；返回要先服务的频道，或 `None` 表示继续搜索。仅从 `FOUND` 中选，不访问真值。默认模式使用

$$\Delta_i = \|p - c_i\| + \|c_i - s\| - \|p - s\|, \quad J_i = \Delta_i + r_i,$$

取 J_i 最小且不超过 600 m 的任务。 Δ_i 是“若途经中心”的插入距离；实际服务会移动、测量、换台，所以 J_i 是启发式代理。系数 1.0 与阈值 600 均无全局最优声明。

行号	逐行作用及理解要点
[L454-L459]	L454 定义可返回 <code>None</code> ；L455 收集已发现频道；L456 没任务直接返回空；L457 无下一站或顺序模式时；L458 选“到中心距离+半径”最小的任务；L459 延迟模式在仍有搜索站时不插任务。分支顺序保证搜索站耗尽后，延迟模式也会开始清除已发现源。
[L460-L463]	L460 建评分表；L461 逐个任务；L462 取中心与半径；L463 用三角形绕路差算插入距离，理论上非负。这个点是估计中心，不是真源坐标。
[L464-L468]	L464 注释说明服务未必结束于中心；L465 加半径作不确定性惩罚，单位仍是米；L466 保存评分和频道；L467 取最小元组，评分相同时小频道优先；L468 阈值内返回频道，否则返回 <code>None</code> 。即使一直不满足阈值，搜索站耗尽后也会由 L457-L458 服务，不会永远饿死任务。

25. run 与 run_case：主调用链

契约。run() 从原点开始执行一局，返回统计字典，不能仅根据循环退出次数宣布成功。停止依据是清除 16 个源，或没有已发现任务也没有未知频道；未知消除来自前述证书。run_case(api,audit) 建立默认策略并运行，不修改配置。

调用关系可以顺着一条实际路线读：run → _scan → _observe → bearing_update/exclude_disk_hull → mec；随后 run → _choose → _serve → _tracking_point → _observe/_clear。所有动作最终都到传入 api，因此同一策略能配本地模拟器或正式客户端。

行号	逐行作用及理解要点
[L470-L474]	L470 主过程；L471 先扫描原点，再根据状态建立环计划；L472 清零主循环次数；L473 无限循环由证书或异常结束；L474 计次。
[L475-L479]	L475 循环超 1000 或动作超 5000 时中止，防程序失控；L476 已清除 16；L477 注释说明利用源数上界；L478 结束；L479 重新收集仍待服务的已发现频道。
[L480-L483]	L480 收集未知频道；L481 两类列表都空则已完成；L482 取未来路线首站，无站则 None；L483 交调度器选择是否先服务。
[L484-L487]	L484 有被选频道；L485 服务该频道，保留下一搜索站；L486 否则若还有搜索站；L487 从计划弹出该站再全扫描。只有实际完成 _scan，它才加入 scans。
[L488-L492]	L488 没有待服务选择、也无站、但尚有未知频道时；L489 注释强调不能凭已扫次数宣布不存在；L490 重做认证；L491 若没有证书就中止；L492 其他情况结束。这里“计划验证”不能替代实际已扫站认证。
[L493-L495]	L493 返回明确成功清除数和累计虚拟时间；L494 每源耗时除以已清除数，零清除返回 None；L495 输出动作数、全扫描站数、是否以圆盘并证书结束、证书原因。若原因是 cardinality，coverage_certified 为假不意味着任务失败，只是没有用 disk_union 这一证书。
[L496-L498]	L496 记录覆盖检查次数与最大多边形顶点数；L497 最大局部测量次数；L498 将频道号转字符串生成末态字典。返回字典不含隐蔽源坐标。
[L501-L502]	L501 定义供两种客户端共用的入口，允许可选审计；L502 新建默认 Config() 和 Strategy，立刻调用 run 并返回结果。本地 solver 文件没有 __main__ 启动块，通常由模拟器导入。

文件二：source/Q3/q3_official_solver.py（712 行）

1. 公共前缀的准确映射

[L001-L502] 与文件一 q3_local_solver.py 的 L001-L502 逐行相同，全部解释沿用文件一，不重复复制。L503-L504 是空行。下面逐段解释新增正式客户端，行号仍采用正式文件原行号。

2. 先理解通信状态机

一次动作可能有三类结果：服务器明确接受并返回有效结果；服务器明确拒绝；请求可能已执行但客户端没有拿到可信结果。第三类不能直接“再发一个新动作”，否则一次移动、检测或清除可能被执行两遍。

客户端采用 `pending` 记录未决请求；一次逻辑动作生成一个 `request_id`，重试复用同一编号和同一请求体。幂等成立还依赖服务器按该编号去重并返回一致结果的协议约定，不是客户端单靠 `UUID` 就能强迫任意服务器做到。未决动作不清楚前，`action` 禁止生成下一个编号；结束动作 `/exit` 同样要遵守这一规则。

本节包含两种时间：`time.monotonic()` 用于现实运行截止与网络超时；接口返回的 `virtual_time_s` 用于赛题动作成本。二者不能互换。

3. 导入与错误类型

行号	逐行作用及理解要点
[L505-L508]	L505 一次导入命令行、日期、HTTP、JSON、环境变量、套接字、终端、时钟、Unicode 分类和 <code>UUID</code> 模块；L506 导入数据类转字典工具 <code>asdict</code> ，此增补部分实际未使用；L507 导入路径工具；L508 导入 URL 拆分。依赖仍均为标准库。
[L510-L512]	L510 定义明确协议错误 <code>ProtocolError</code> ；L511 定义预算不足 <code>BudgetExceeded</code> ；L512 定义结果无法确定 <code>ResultUnknown</code> 。三类均继承 <code>RuntimeError</code> ，空类体 <code>pass</code> 不添加行为，区别在调用方能按类型处理。

4. `OfficialClient.__init__`

契约。输入回环 HTTP 地址、队号、尚不存在的日志路径及超时/重试参数；输出未入场的客户端对象。只连接本机官方接口，不从网络查询目标真值。初始化日志失败会直接报错，避免无记录启动。

行号	逐行作用及理解要点
[L514-L518]	L514 定义客户端类；L515 构造器默认单次超时 5 s、最多重试 6 次；L516 拆 URL；L517 要求 HTTP、有主机、路径空或 /、无查询和片段；L518 不符合则报格式错误。不是所有合法网络 URL 都是合法官方接口地址。
[L519-L522]	L519 仅允许 127.0.0.1、localhost、::1；L520 其他主机拒绝。L521 队号非空、UTF-8 字节数至多 64，且无控制/格式字符；L522 失败则报错。字节长度与中文字符数量并不相同。
[L523-L527]	L523 保存主机、端口（未给则 80）、队号；L524 保存超时重试参数并置连接空；L525 生成 <code>UUID</code> 的 12 位十六进制前缀、动作序号 0、未设截止；L526 初始化虚拟预算 360000 s、入退出标志和重试数；L527 初始化位置原点、接收频道 1、无未决动作。
[L528-L531]	L528 记录是否尝试退出和退出响应；L529 初始化日志失败标志；L530 建日志父目录，再以 'x' 独占创建文件，已存在就报错以免覆盖旧记录；L531 写程序名、脱敏队号和 UTC 时间元信息。

5. 日志、连接和数值验证小函数

`_log(obj)` 契约：尝试写一条 JSON 行并刷新；失败保留标志、打印到错误流，不立即掩盖正在处理的动作结果。`flush` 不是向 GitHub 备份，也不等同文件系统掉电持久性保证。

行号	逐行作用及理解要点
[L532-L537]	L532 定义；L533 开始捕获日志异常；L534 紧凑序列化、保留中文、拒绝 NaN/Infinity，加换行后写出并刷新；L535 捕获异常；L536 标日志不完整并记错误表示；L537 立即向标准错误打印。日志失败后动作仍可能已经成功，所以不能假装该动作未发生。

`_disconnect()` 契约：无输入，尽力关闭旧连接并总把连接字段清空。`_connection(timeout)` 契约：取得或创建 HTTP 连接，更新底层超时并返回；可抛连接异常，交重试层处理。

行号	逐行作用及理解要点
[L538-L542]	L538 定义断开；L539 只有已有连接才关；L540 尝试关闭；L541 忽略关闭时 <code>OSError</code> ；L542 无论关闭如何都清空连接引用，后续必须重新建立。
[L543-L547]	L543 定义取连接；L544 尚无连接时；L545 建 <code>HTTPConnection</code> 并真正连接；L546 有套接字则设 <code>TCP_NODELAY</code> 减少小包等待；L547 保存已连接对象。
[L548-L550]	L548 若指定新超时则改连接对象属性；L549 还改现有 <code>socket</code> 超时，否则仅改属性未必改变已有套接字；L550 返回连接。

`_safe_payload(payload)` 契约：复制请求字典，把队号替换成占位符用于日志，不更改真实请求。`_finite(value)` 契约：静态函数，判断值是有限 `int/float` 且不是布尔值；`bool` 在 Python 是 `int` 子类，须单独排除。

行号	逐行作用及理解要点
[L551-L552]	L551 定义脱敏；L552 复制、替换 <code>robot_id</code> 、返回三步在同一行执行；原请求队号仍送官方接口。
[L553-L557]	L553 标为不依赖 <code>self</code> 的静态方法；L554 定义；L555 排除布尔和非 <code>int/float</code> ；L556 转 <code>float</code> 后检有限；L557 溢出或类型/值错误则返回假。字符串 <code>'1.0'</code> 也被拒绝，不悄悄接受类型漂移。

`_cutoff(path)` 契约：从总截止减去不同保留时间，返回新动作最迟启动判断点或 `None`。`_attempt_timeout(path)` 契约：根据剩余现实时间返回本次超时；不足时抛预算异常，不发新动作。

行号	逐行作用及理解要点
[L558-L560]	L558 定义；L559 尚未入场得截止前返回空；L560 普通动作保留 8 s，退出保留 0.25 s。此保留用于收尾，不是赛题虚拟费用。
[L561-L566]	L561 定义；L562 取路径截止；L563 无截止用默认超时；L564 用单调时钟算剩余秒数；L565 已耗尽则抛错；L566 取不超过默认超时和剩余时间的值，并设置 0.05 s 最小超时。若剩余不足 0.05，它会略超过动作截止，但仍留上述收尾裕量。

6. _validate_success: 有 JSON 不等于有可信动作结果

契约。输入接口路径、HTTP 状态、解析后的对象；成功返回 `None`，明确拒绝抛 `ProtocolError`，自相矛盾或结构异常抛 `ResultUnknown`。函数不修改状态。有效结果必须类型正确、时间不倒退、对应接口字段合法。

行号	逐行作用及理解要点
[L567-L571]	L567 定义；L568 顶层必须是字典；L569 <code>accepted</code> 必须真正为 <code>bool</code> ；L570 检两个公共时间字段；L571 每个必须为有限、非负数。缺字段通过 <code>.get</code> 得 <code>None</code> 后失败。
[L572-L576]	L572 非 200 时；L573 明确 <code>accepted=False</code> 归为拒绝；L574 非 200 却接受归为结果不可信；L575 200 下也必须真接受；L576 虚拟时间若较上次倒退超过 10^{-5} 则拒绝信任。未盲信 HTTP 成功码。
[L577-L581]	L577 入场响应分支；L578 遍历三个预算字段；L579 都要求有限非负；L580 读取剩余现实秒数；L581 要求不超过 1200 且是整数值。该校验与接口约定相关，不是一般 HTTP 必须规则。
[L582-L585]	L582 检测响应分支；L583 取类型；L584 只允许三种测量反馈；L585 示向时要求角度是有限数且在 $[0, 360)$ 。近或无信号无需角度字段。
[L586-L588]	L586 清除只允许成功或范围无目标；L587 否则未知；L588 退出原因须为 <code>user_exit</code> 。这些判断将协议异常与几何失败区分开。

7. _request: 同一动作如何安全重试

契约。输入路径、含固定 `request_id` 的载荷和可选已编码请求体；返回可信接受响应。第一次发送前记录 `pending`，循环中不生成新 ID；成功或明确拒绝清空未决，结果不明时保留。它是私有传输函数，公共调用经 `action` 保证不会在已有未决动作时调用它发新动作。

“最多重试 6 次”对应 `range(7)`：首次加六次重试；计数器 `retry_count` 只统计额外尝试，不含首次。

行号	逐行作用及理解要点
[L589-L593]	L589 定义；L590 如未给编码体则一次性用 UTF-8 JSON 编码，禁止 NaN；L591 无未决记录时；L592 保存路径、载荷副本、编码体；L593 写脱敏请求日志。此后每次重发用同一个 <code>body</code> 。
[L594-L598]	L594 保存末次异常及“曾有结果不明”的标志；L595 首次加重试循环；L596 记本次起始单调时刻；L597 开始尝试；L598 算预算内超时并取连接。
[L599-L601]	L599 发 POST 和 JSON 内容类型；L600 取响应、状态、读完整响应体；L601 UTF-8 解码并解析 JSON，遇 NaN/Infinity 等常量时用生成器 <code>throw</code> 抛 <code>ValueError</code> ，不接受非标准数值。
[L602-L605]	L602 捕获无效 JSON/编码；L603 记录前 1000 字节的容错解码原文、ID、次数、状态；L604 抛结果未知并保留原异常链；L605 合法 JSON 则记录响应体与本次耗时，尚未把它认定为动作成功。

行号	逐行作用及理解要点
[L606-L611]	L606 对限流或暂时服务器错误特殊处理； L607-L609 检查它明确拒绝且两个公共时间字段为有限数；L610 无这些可信特征则结果未知；L611 若明确拒绝则用带 HTTP 前缀的 OSErrors 标记可安全重试。这类临时错误不误算作已接受。
[L612-L616]	L612 调严格响应校验；L613 捕获明确协议拒绝； L614 清空 pending；L615 将拒绝继续抛给上层。 L616 通过校验后更新已知虚拟时间、清空 pending、返回结果三个动作连续执行。
[L617-L621]	L617 预算不足分支；L618 若之前没有任何结果不确定；L619 清空 pending；L620 记“发送前取消”； L621 抛回预算异常。若曾有不确定动作，就不清空 pending，不能把它当从未发过。
[L622-L625]	L622 明确拒绝直接向上抛，不再次当网络错误； L623 捕获传输、超时、HTTP 协议或未知响应异常；L624 保存异常，并累计 unknown：只有明确临时 HTTP 拒绝可保持已知未执行；普通网络错误保守视为可能已执行。L625 断旧连接并记重试日志。
[L626-L631]	L626 已达重试末次就结束循环；L627 否则额外重试数加一；L628 指数退避 $0.1 \cdot 2^{\text{attempt}}$ 秒，封顶 1.5，并取截止；L629 等待不超过剩余时间；L630 已无正等待时间就结束；L631 睡眠该退避时间。网络重试等待不是虚拟移动或检测费用。
[L632-L633]	L632 曾有不确定结果则保留原 pending 并抛 ResultUnknown，错误说明只能沿用原 ID；L633 若每次均可知未执行，则清空 pending，抛连接失败。 两类结束区别决定上层是否能安全发 /exit。

8. action：只有通过参数、预算、未决检查才生成 ID

契约。输入合法动作路径和所需位置/频道；验证后分配新 ID，调用 _request。只有接受结果确认后才更新客户端本地位置/接收频道。这些更新是记账，不会自行使服务器机器人移动。

虚拟动作上界为 $\|p_{\text{new}} - p\|/5 + 5 \text{ s}$ ；检测换台再加 1 s。清除失败虽只费 3 s，预算检查使用 5 s 上界，因此偏保守。

行号	逐行作用及理解要点
[L634-L638]	L634 定义；L635 路径限四个接口；L636 若已有未知结果，禁止产生新请求；L637 测量/清除必须有坐标频道；L638 入场/退出不得带这些参数。不能用新 ID 掩盖未知旧动作。
[L639-L643]	L639 初始化规范坐标空；L640 如给坐标；L641 必须恰二维、有限、每坐标绝对值至多两百万；L642 转为浮点元组；L643 频道必须真正的 int、非 bool，范围 1-20。这里的大坐标界是接口防错，不是限制机器人只能在 1800 m 圆内走。
[L644-L648]	L644 动作类路径先验算虚拟费用；L645 路程/5 加 5；L646 测量且接收频道不同再加 1；L647 可能超过服务器预算减 0.001 则不发；L648 再验现实时间。预算失败发生在分配新动作编号之前。

行号	逐行作用及理解要点
[L649-L654]	L649 序号加一；L650 构造默认场地、队号、随机前缀-序号 的载荷；L651 如有坐标；L652 以 {x,y} 对象加入；L653 如有频道；L654 加入频道字段。
[L655-L658]	L655 发请求并等待可信结果；L656 检测/清除确认后更新位置；L657 只有检测才更新接收机频道；L658 返回服务器结果。出现异常时这些成功后的更新不会执行。

9. 等待接口、入场、出场与关闭

`wait_until_open(wait_s)`：只探测 TCP 端口能否连接，不入场、不证明服务器已能正确处理动作。`enter()`：发入场，依据响应建立现实和虚拟预算。`measure/clear`：适配 Actions 接口。`exit()`：确认退出后缓存响应，重复调用不重发；未决状态仍由 `action` 阻止。`can_exit()`：判断是否允许第一次安全收尾。`close()`：关连接日志，不代替 `/exit`。

行号	逐行作用及理解要点
[L659-L663]	L659 默认等待 300 s；L660 算等待截止；L661 在窗口内循环；L662 尝试；L663 用 0.5 s TCP 建连探测，成功即在上下文退出时关探测连接并返回。这不是计分动作。
[L664-L665]	L664 连接错误则等 0.25 s 再试；L665 超出窗口报超时。该循环用于等待用户启动模拟器。
[L666-L668]	L666 定义入场；L667 先记发送前单调时刻，发 <code>/enter</code> ，取剩余时长；L668 以发送前时刻加剩余秒数作为较保守截止，接收服务器虚拟上限，标入场并返回响应。网络往返时间因此未被额外赠给客户端。
[L669-L670]	L669 将 <code>measure(p,ch)</code> 转为 <code>action('/measure',p,ch)</code> ；L670 同样适配清除。这两个一行函数让同一 <code>Strategy</code> 无需知道 HTTP 细节。
[L671-L675]	L671 定义退出；L672 若确认已退出就返回缓存；L673 在发前标已尝试；L674 发退出、确认后标已退出并缓存返回；L675 允许退出须已入场、未确认退出、未尝试退出、无 <code>pending</code> ，且仍有现实时间。这避免上层反复另造退出动作编号。
[L676-L681]	L676 定义资源关闭；L677 断连接；L678 尝试关日志；L679 捕获关闭失败；L680 记录日志不完整和错误；L681 向标准错误报告。关闭日志失败会影响最后进程退出码。

10. 正式 main：完成策略、确认退出、保存记录都要看

契约。解析命令行、取得队号、运行一场，返回 0 表示程序流程完成且日志未报失败，返回 1 表示未完成或日志不完整。这不是成绩排名，也不代替服务器成绩报告。队号由参数、环境变量、交互输入依次选择，不硬编码在源文件。

行号	逐行作用及理解要点
[L683-L688]	L683 定义主函数；L684 建命令行解析器；L685 可选队号；L686 默认本机 2026 端口；L687 可选端口等待秒数；L688 解析参数。

行号	逐行作用及理解要点
[L689-L692]	L689 依次尝试命令行队号、CUMCM_ROBOT_ID、交互输入并去首尾空白；L690 生成精确到微秒的时间戳；L691 生成日志路径并构造客户端；L692 初始未完成，记真实计时器，尚无汇总。客户端构造在后面 try 之前，初始化失败直接终止。
[L693-L697]	L693 进入受保护运行段；L694 打等待提示、等接口、入场，随后重新起运行计时；L695 打启动提示；L696 运行默认策略，完成后确认退出；L697 向策略汇总增加本机耗时及网络重试次数。此实际耗时口径从入场后开始，含退出过程。
[L698-L700]	L698 把汇总写到同名 .summary.json；L699 在动作日志再写汇总、策略完成和退出确认标志，并置成功；L700 若任何日志操作已报失败则抛错，下面异常处理会将成功标志重新置假。
[L701-L703]	L701 无清除源时均值显示 N/A，否则三位小数；L702 打清除数、虚拟时间和每源均值；L703 打日志位置。显示不是证书本身，证书信息在汇总字典。
[L704-L709]	L704 捕获普通异常与用户中断；L705 标不成功；L706 尽力记 fatal 并立即显示错误；L707 仅满足 can_exit 才收尾；L708 尝试退出；L709 如退出也失败，记录异常以及未决动作路径/编号，未复制完整敏感载荷。
[L710-L712]	L710 无论运行是否成功都关资源；L711 只有完成且日志无失败返回 0，否则 1；L712 直接执行本文件时调用主函数，以 SystemExit 将返回值变成进程退出码；被导入时不自动入场。

文件三：source/Q3/q3_local_simulator.py（120 行）

1. 模拟器不是策略，也不是官方成绩

模拟器保存真实源表，负责根据动作产生反馈、计费；求解器只调用它的 measure/clear。Python 对象本身并不实施信息权限隔离，但本策略代码没有读取 sources 或真实坐标。测试框架可以用真值评估全清除，不能把这些真值作为策略输入。

本地总虚拟时间按

$$T = \frac{D}{5} + 5N_{\text{measure}} + N_{\text{switch}} + 5N_{\text{success clear}} + 3N_{\text{failed clear}}$$

累加。换频道只在检测动作中计费；清除动作不修改接收频道。runtime_s 则是电脑运行这些计算花的现实时间，不是这个 T 。

2. 导入与 Source 数据类

行号	逐行作用及理解要点
[L001-L006]	L1 是 Unix 启动器提示, Windows 由 python 调用时只是注释; L2 推迟注解; L3 导入数据类; L4 导入命令行、日期、JSON、数学、随机数和计时; L5 路径工具; L6 导入同目录求解器并命名为 solver, 这才连到前述 run_case。
[L008-L010]	L8 把源数据类冻结; L9 定义 Source; L10 同一行声明频道、位置、接收半径、误差函数相位四字段。phase 仅控制本地模拟误差, 不是策略能读取的真实示向或朝向。

3. Simulator.__init__、_move 与 _error

构造器契约。输入源表和局种子; 输出原点、频道 1、未清除任何源的场景, 假设频道唯一。_move(p) 契约。累加直线距离及距离/5 秒, 修改当前位置, 不检测任何频道。_error(s,p) 契约。以源和位置生成确定的、被夹在 $[-1, 1]$ 度内的误差, 不是每次重新独立抽噪声。

行号	逐行作用及理解要点
[L012-L015]	L12 定义模拟器; L13 构造器; L14 把源按频道建字典、清除集合为空、机器人原点、接收频道 1; L15 初始化时间、距离、检测/换台/失败清除计数及种子。若输入重复频道, 字典会覆盖前项; 正常 make_case 保证无重复。
[L016-L017]	L16 定义移动; L17 依次算当前位置到目标距离、累加距离、累加距离/5 的秒数、更新位置。即使距离为零也合法, 动作固定费由调用者另加。
[L018-L019]	L18 定义误差; L19 对位置和相位代入两个光滑三角函数, 以 0.62 与 0.38 加权, 再夹到 $[-1, 1]$ 。这些频率和系数是测试设计, 不来自定位理论, 也不代表官方误差分布。

4. measure 与 clear

measure 契约。输入站位和频道, 按移动、检测、可能换台计费, 返回与策略接口兼容的字典。先判断源是否存在、已清除、在接收范围外, 再判断 near, 再计算示向。

clear 契约。输入站位频道, 移动后仅按真值距离判断成功; 成功 5 s、失败 3 s。这里用真值是模拟环境的职责, 不能移入求解器。

行号	逐行作用及理解要点
[L020-L024]	L20 定义检测; L21 移动后加 5 s、检测计数加一; L22 频道不同则加 1 s、换台计数加一; L23 更新接收频道、查源、建接受响应和当前时间; L24 没源、已清除或超其真实接收半径任一成立即无信号。
[L025-L029]	L25 在接收条件下且真距至多 5 m 返回 near; L26 其余有信号情况; L27 算真方位角; L28 加本地误差、模 360、四舍五入到两位并输出 direction; L29 返回字典。舍入可能把极接近 360 的数变成 360.0, 本地策略三角函数可处理; 正式客户端则按协议要求小于 360, 这两者校验并非完全同一实现。

行号	逐行作用及理解要点
[L030-L034]	L30 定义清除; L31 移动、取源、判断存在且未清除且真距至多 20; L32 成功加入已清除集合并加 5 s; L33 否则加 3 s并计失败; L34 返回接受状态、最新时间和清除结果。整个分支不改 self.channel, 说明清除不让接收机换台。

5. make_case: 如何复现同一局

契约。输入整数局种子, 输出 10-16 个不同频道的源。位置按圆域面积均匀生成; 半径在 1000-1500 均匀生成; 误差相位均匀生成。固定种子保证同一 Python 随机算法环境下可复现, 不保证官方场景相同。

半径为什么用 $1800\sqrt{U}$? 圆内半径不超过 r 的面积占比是 $r^2/1800^2$; 令其等于均匀变量 U 得该公式。直接用 $1800U$ 会在单位面积意义下过密地抽到中心。

行号	逐行作用及理解要点
[L036-L039]	L36 定义; L37 创建独立 RNG、抽整数源数、从 20 频道无放回抽样、建输出表; L38 逐频道生成; L39 抽均匀角、按面积抽半径、转笛卡尔坐标。该行三个表达式都是场景生成, 不是求解。
[L040-L041]	L40 抽接收半径及误差相位, 构造冻结 Source 加入表; L41 返回表。抽样频道唯一, 满足模拟器字典和 Q3 每频道单源前提。

6. _average: 两个“平均每源耗时”不一样

契约。输入局记录列表, 输出均值字典; 空列表返回 None。若第 k 局时间 T_k 、清除数 $n_k > 0$, 逐局等权平均为 $\frac{1}{K} \sum_k T_k/n_k$, 按清除源合并的平均为 $\sum_k T_k / \sum_k n_k$ 。各局源数不同时一般不相等。失败局统计反映部分运行成本, 不能装成成功完成任务的平均。

行号	逐行作用及理解要点
[L043-L047]	L43 定义; L44 空输入返回空; L45 统计局数、总源数、总已清除数, 其中 sources 后续未使用; L46 取非空每源均值列表, 零清除局不进入该均值; L47 开始返回字典。
[L048-L053]	L48 局数; L49 每局平均真实源数; L50 每局平均清除数; L51 每局平均虚拟时间; L52 对有效局的每源均值再等权平均, 空则 None; L53 所有局总时间除以总清除数, 零分母则 None。L52 和 L53 不是重复字段。
[L054-L058]	L54 平均移动距离; L55 总距离除以 $5*n$ 得每局移动秒; L56 检测数乘 5 后按局平均; L57 换台数对应每次 1 s; L58 成功清除数乘 5, 加失败数乘 3, 再按局平均。四种时间相加应等于 L51。
[L059-L062]	L59、L60、L61 分别给平均检测、换台、失败清除次数; L62 结束字典并返回。平均次数可以非整数, 因为跨局平均。

7. _print_average

契约。输入标题与统计字典, 只打印, 不修改模型或文件。浮点格式只是显示精度, 不改变保存的数值。

行号	逐行作用及理解要点
[L064-L069]	L64 定义；L65 无统计就不打印；L66 标题；L67 平均源数两位小数；L68 平均清除数；L69 平均每局总虚拟时间。
[L070-L073]	L70 若有逐局等权每源均值则打印；L71 若有合并加权均值则打印；L72 平均距离；L73 按移动/检测/换台/清除顺序显示四种平均时间。
[L074-L076]	L74 平均检测次数；L75 平均换台次数；L76 平均失败清除次数，保留三位小数，便于看到较低失败频率。

8. 本地 main

契约。输入命令行中的局数、主种子、输出路径；依次生成局、运行策略、用真值核验、保存包含失败局的明细与汇总。正常用非负局数；代码未额外验证负局数，使用时不要依赖非法参数的行为。主种子先生成各局种子，`make_case` 再按局种子生场景，因此两层种子不能混写。

行号	逐行作用及理解要点
[L078-L083]	L78 定义；L79 建解析器；L80 接受整数局数、种子；L81 接受输出路径并解析；L82 无局数参数时交互读取，空输入默认 1000；L83 有种子就用，否则系统随机生成主种子。
[L084-L088]	L84 用主种子建 RNG，初始化局记录、现实计时、全清除计数；L85 逐局循环；L86 抽局种子、生成真值源、建模拟器；L87 捕获求解异常；L88 运行策略，用模拟器实际已清除集合与真值源数比较，再标无异常。这里的成功核验不能只信求解器自报 <code>cleared</code> 。
[L089-L095]	L89 捕获异常以允许继续下一局；L90 置空策略结果、失败、错误字符串；L91 将布尔全清除转 0/1 累加；L92 开始记录序号、局种子、真实源数、是否全清；L93 记真实已清除数、虚拟时间和按已清除源计算的均值；L94 记距离与动作计数；L95 记策略汇总和异常。失败时也保留已消耗时间与计数。
[L096]	按约每 5% 进度、且最多间隔 100 局打印； <code>max(1,...)</code> 防取模零。 <code>flush=True</code> 让长测试中进度及时可见。
[L098-L102]	L98 选出全清除局；L99 分别计算全局和成功局统计；L100 开始汇总；L101 记问题号、局数、主种子；L102 记全清除局数和比例，零局时比例为 0。
[L103-L109]	L103 保存全部局均值；L104 保存成功局均值；L105 为兼容读取保留成功局平均时间；L106 成功局等权每源均值；L107 成功局合并每源均值；L108 记电脑实际耗时和所有局明细；L109 结束字典。零成功时三个兼容均值都是 <code>None</code> 。
[L110-L111]	L110 采用指定路径或在 <code>results/</code> 下按本地时间命名 JSON；L111 建父目录，再 UTF-8 写格式化 JSON。默认文件名精确到秒，在同秒重跑可能覆盖，需保留不同 <code>--output</code> 以作固定实验档案。
[L113-L118]	L113 打空行；L114 打全清除局数及比例；L115 打全部局平均；L116 若存在失败再额外打印成功局平均，方便区分；L117 现实耗时；L118 输出文件路径。

行号	逐行作用及理解要点
[L120]	直接运行此文件才调用 <code>main</code> ；被导入做单步演示时不会自动启动批量测试。这里没有显式非零退出码汇总失败局，判断测试通过与否要查看 JSON 的全清除与错误字段。

文件四：source/Q3/run_local.bat（9 行）

契约。在 Windows 批处理环境运行本地模拟器，转交用户参数，最终保留 Python 进程退出码。前提是命令 `python` 可用且当前源码同目录；该脚本不启动正式接口。

行号	逐行作用及理解要点
[L001–L004]	L1 关闭命令回显，前缀 <code>@</code> 也隐藏这一行；L2 将环境变量改动限定在本批处理；L3 切到 UTF-8 代码页并隐藏正常提示；L4 切到脚本所在目录， <code>%~dp0</code> 是脚本驱动器与路径，失败就以代码 1 退出。使用 <code>pushd</code> 还能记录原目录供恢复。
[L005–L009]	L5 启动 <code>q3_local_simulator.py</code> ， <code>%*</code> 原样转交全部命令行参数；L6 立即保存 Python 退出码；L7 恢复原目录；L8 暂停等待按键便于看结果；L9 用已保存的码退出，避免 <code>popd/pause</code> 改写错误码。

文件五：source/Q3/run_official.bat（9 行）

契约。同样的 Windows 启动与退出码保护，但启动正式客户端。前提是队号和官方模拟器准备好；网络动作、入场退出逻辑在 Python 内处理。

行号	逐行作用及理解要点
[L001–L004]	L1 关闭回显；L2 隔离环境；L3 UTF-8 输出；L4 进入脚本目录，失败立即返回 1。每步与本地脚本同义。
[L005–L009]	L5 将所有用户参数传给 <code>q3_official_solver.py</code> ；L6 保存退出码；L7 恢复目录；L8 暂停；L9 返回 Python 原码。正式客户端 0/1 的定义见正式 <code>main</code> ；它不同于本地批量测试未显式设置失败退出码的行为。

带答案训练：用具体数据把公式搬回代码

练习 1：一次半平面裁剪

题目。给逆时针正方形 $P = [(0, 0), (2, 0), (2, 2), (0, 2)]$ ，裁剪条件 $x \leq 1$ 。先忽略容差，逐边写 `clip` 输出；再说明代码会怎样改变边界。

答案。从末点 (0,2) 到 (0,0)，两端都在内，加入 (0,0)；从 (0,0) 到 (2,0) 内到外， $f_a = -1, f_c = 1, t = 1/2$ ，加入 (1,0)；从 (2,0) 到 (2,2) 都在外，不加；从 (2,2) 到 (0,2) 外到内，加入 (1,2) 和 (0,2)。结果是四顶点长方形。实际 L40 把 b 增为 $1 + 10^{-7}$ ，两个交点的横坐标相应略大，属于外放宽。对应求解器 L37–L56。

练习 2：有无信号为何可能不缩小凸包

题目。 $P = [-2, 2]^2$ ，在 origin 获得可靠排除单位开圆盘的负观测。exclude_disk_hull 的理想输出是什么？是否说明负观测没有用？

答案。正方形四顶点均在圆外，剩余区域仍含四顶点，其凸包就是原正方形。负观测有信息，信息是“中心圆盘不能含真值”；单个凸多边形不能精确表达中间的洞，凸化丢失这部分信息。程序仍在 negatives 保存观测，之后新的角楔可能切走四角，使再次排除有效。对应 L78–L98、L288、L301。

练习 3：包围圆为何要审计半径

题目。输入点 (0,0),(4,0),(0,3)，求最小包围圆。代码为何返回的半径略大？如果某次数值中心算得不最优，清除还能否有依据？

答案。三角形直角在 origin，斜边中点 $c = (2, 1.5)$ ，半径 2.5。L146 复核最大点距并加 10^{-6} ，本版本返回 2.500001。只要复核半径确实覆盖全部顶点，凸性保证覆盖整个多边形，清除充分条件仍可用；中心不最优可能导致多走或多测，不会仅因“不最优”就丢失包含关系。浮点误差仍需裕量与运行核查，不能把它说成任意精度的严格区间证书。

练习 4：不用猜，算出三角包围圆

题目。忽略浮点裕量，取示向 0° 、 $R = 100$ 、半角 δ 。写外包三角形顶点，并从等距条件推半径。

答案。顶点为 $(0, 0)$, $(100, 100 \tan \delta)$, $(100, -100 \tan \delta)$ 。对称性给圆心 $(h, 0)$ 。到 origin 与右上顶点等距： $h^2 = (100 - h)^2 + 10000 \tan^2 \delta$ ，化简 $200h = 10000 \sec^2 \delta$ ，得 $h = 50 \sec^2 \delta$ ；半径也为 h 。对 $R = 1500$, $\delta = 1.0051^\circ$ ， $h \approx 750.230847$ 。对应 L305–L308；不是“把半径直接除二”而忽略角宽。

练习 5：七次方向更新与 18 步保护是什么关系

题目。首次包围半径取 750.230847，默认侧移率 0.1，忽略数值裕量。按收缩上界估算再做多少次方向观测后半径至多 19.8？代码为什么还有 18 步？

答案。 $q = 1.1 / (2 \cos^2 \delta) \approx 0.550169288$ ，需要 $750.230847 q^k \leq 19.8$ ，故 $k \geq \log(19.8/750.230847) / \log q \approx 6.08$ ，取 $k = 7$ 。这是模型前提与理想几何下的上界估算；near 会更早结束。L412 的 18 是容错进展上限，L425 的 0.9 是宽松运行检测线，二者都不是最优测量次数证明。

练习 6：最近认证清除点

题目。当前点 $p = (0, 0)$ ，外包圆心 $c = (30, 0)$ ，半径 $r = 10$ 。求默认清除站位，并证明对所有可行源安全。

答案。slack=19.8-10=9.8，距离圆心 30，大于 slack，所以 $q = c + \frac{9.8}{30}(p - c) = (20.2, 0)$ 。任意真源在 $B(c, 10)$ ， $\|q - x\| \leq 9.8 + 10 = 19.8 < 20$ 。比直接走到中心少走 9.8 m，省 1.96 s 移动时间。对应 L148–L154、L414。

练习 7：覆盖与清除不能互相替代

题目。已清除 12 个源，但仍有未知频道，历史扫描站尚未通过覆盖证书。能否宣布完成？若已发现和已清除频道合计 16 呢？

答案。前一种不能：模型允许至多 16、至少 10，清除 12 不说明本局恰好 12。应继续搜索或通过连续覆盖证书排除未知。后一种可结束搜索，因为已发现 16 个不同频道达到源数上界；但仍为 FOUND 的源必须继续清除。对应 L334–L343 与 run L476–L481。coverage_done=True 不等于 cleared 已达真实源数。

练习 8：代入插入评分并决定下一步

题目。机器人 $p = (0, 0)$ ，下一站 $s = (100, 0)$ 。A 频道中心 $(40, 30)$ 、半径 20；B 中心 $(50, 0)$ 、半径 80。默认 integrated 模式选谁？评分是否就是真实耗时？

答案。A 的插入距离 $50 + \sqrt{60^2 + 30^2} - 100 \approx 17.082039$ ，评分 37.082039 m；B 的插入距离 $50 + 50 - 100 = 0$ ，评分 80 m。A 较小且小于 600，选 A。评分不是耗时：还没设计具体测量、换台，实际服务终点也未必在中心。半径惩罚是代理项。对应 L454–L468。

练习 9：真实与虚拟时间单步计算

题目。初始原点、接收频道 1。先在 $(30, 40)$ 检测频道 1，再原地检测频道 2，再原地成功清除频道 2，最后原地检测频道 2。求总虚拟时间、移动距离、检测次数、换台次数。假定成功清除条件确已满足。

答案。移动 50 m 费 10 s；三次检测费 15 s；仅第二次检测从频道 1 换到 2，费 1 s；成功清除费 5 s，总 31 s。各次动作结束时间依次为 15、21、26、31 s；移动 50 m、检测 3 次、换台 1 次。最后检测被清除频道返回无信号仍收 5 s。清除不额外换台。电脑只运行了 0.01 s 或 2 s，都不会改变这些虚拟费用。

练习 10：同样叫平均，为什么结果不同

题目。两局都全清除：第一局 10 源、1000 s；第二局 16 源、3200 s。求两种每源均值。

答案。逐局等权为 $(1000/10 + 3200/16)/2 = 150$ s/源；合并加权为 $(1000 + 3200)/(10 + 16) = 4200/26 \approx 161.538462$ s/源。必须连同口径报数，不能选择较小的 150 却称“全部源合并平均”。对应模拟器 L52–L53。

练习 11：已发送请求超时后为什么不能换 ID

题目。清除请求 ID 为 abc-7。服务器可能已经清除并计费，但响应途中丢失。应如何重试？超过重试次数后能否直接发一个新 ID 的 /exit？

答案。重发相同路径、载荷、请求体及 abc-7，依赖服务器幂等协议返回原结果。不能发 abc-8 重新清除，因为会被当新动作。超过重试上限且结果仍不明，L632 保留 pending；L636 禁新动作，can_exit 因 pending 非空返回假，因此不能新造 /exit。程序会保留不完整记录并失败退出。关闭 TCP 连接不会证明服务器未执行旧动作。

练习 12：区分可重试拒绝、明确拒绝和未知结果

题目。分别处理：① HTTP 503，JSON 明确 accepted=false 且公共字段有效；② HTTP 400，字段有效且明确拒绝；③ HTTP 200 却返回不是 JSON 的内容。

答案。① L606–L611 作为可知未执行的暂时错误，可同 ID 退避重试；② 经校验是 ProtocolError，清 pending 后向上报拒绝，不按暂时网络错误重试；③ L601–L604 是 ResultUnknown，同 ID 尝试恢复可信结果，最终未知就保留 pending。仅看状态码或仅看 JSON 能否解析都不够。

练习 13：指出默认程序未启用的“方法”

题目。有人汇报：“我们的正式 Q3 每次都优化旋转环、剪掉冗余站、途中按覆盖收益加扫描、进行试探性清除，并用 2-opt 优化路线。”对照代码纠正。

答案。rotate=False、prune=False、incidental=False、speculative_r=0.0，前四项只保留为本地研究选项。没有 2-opt 实现，L233 已明确。默认是原点加八个固定环站、共享正观测、已发现源的局部跟踪、插入评分和证书清除。不能把“源码里有分支”说成“正式入口执行了分支”。

练习 14：异常为什么也是答辩内容

题目。下列异常分别说明哪条前提受到质疑：empty outer set、lost signal at a guaranteed reception point、CERTIFIED clear failed、coverage certificate missing?

答案。依次对应：多次反馈与几何外包不能同时成立或数值出了问题；全体可行点在最小接收域内却无信号，说明 Q3 接收模型或当前外包不可信；全体可行点被 19.8 m 安全条件包含却清除失败，说明模型/状态/接口与证书冲突；搜索计划用完但没有排除未知源的全局证书。程序选择中止并记录，不能把这些异常当已完成测试。每种异常都需要保留触发前状态和动作记录，定位具体原因。

七天训练中的使用方法与完成标准

第一轮先读本章开头的三条公式链，再按课程学习一般方法；第二轮在课程做完本题练习后读对应函数；第三轮让同伴随机指定行号。不要在第一天从导入语句背到文件末尾。

每个被抽到的代码段，都应能依次回答：输入是什么、现在维护什么不变量、这一行采用哪条公式或协议条件、更新了什么状态、输出是什么、条件不成立时怎样处理。回答中明确说“几何充分条件”“浮点复核”“启发式”或“运行保护”，不要只说“为了鲁棒”。

本章自检。不用运行程序，应能独立完成上面 14 题；打开代码后，应能把三条公式链定位到 bearing_update/_observe、nearest_certified_clear/_clear、covers_arena/_certify_and_prune；最后能口述 run → 动作接口 → 本地计时/正式 HTTP。达到这些标准后，再练随机逐行问答。本章提供全部非空行的解释，不声称读一次即可掌握所有程序行为。

Q4 逐行伴读：从定向观测模型到每一条程序语句

0 读这一章时，始终抓住哪三个问题

本章对应仓库中的 `source/Q4/`。行号按当前源码的物理行计数，空行不执行任何操作；所有非空行都在下面标出。`q4_official_solver.py` 的第 1—208 行与 `q4_local_solver.py` 完全相同，统一在第 2 节解释；正式版只额外解释第 211—391 行。第 209—210 行为空行。

每读一个函数，先回答：它知道什么？它要作出什么决定？凭什么允许这个决定？随后才读 Python 语句。Q4 的三个决定分别是：到哪里才能发现源；怎样缩小源的位置集合；什么时候能够安全清除。调度函数还决定先做哪件事。HTTP 客户端负责保证“程序以为已做完的动作”确实得到有效响应。

0.1 统一符号、输入与目标

设真实源位置为 $g \in \mathbb{R}^2$ ，有效接收半径为 $R \in [1000, 1500]$ ，源在 $\|g\| \leq 1800$ 内。一个频道最多有一个源，总源数为 10—16，频道为 1—20。定向源有未知单位向量 n ；在检测位置 p 能接收到它的条件是

$$\|p - g\| \leq R, \quad n^T(p - g) \geq 0. \quad (\text{Q4-1})$$

全向源只有距离条件。定向边界属于可接收范围。对已经收到的信号：距离不超过 5 m 时返回 `near`；否则返回带误差的示向。清除成功只要求存在未清除源且距离不超过 20 m，不要求位于源的辐射半平面。局内源的位置、方向、半径保持不变。

求解器不能读取 g, R, n 。它得到 `no_signal`、`near` 或 `direction`，并维护外包络 P ：真实源如果存在，应一直属于 P 。目标是在时间预算内找到并清除全部源；本策略给出可验证的发现与清除条件，没有总耗时全局最优的证明。

0.2 四个容易混淆的量

程序名	数学含义	不能误说成什么
<code>poly</code>	真实源位置集合的凸多边形外包络 P	不是源的概率分布，也不是每个顶点都真实可实现
<code>center, radius</code>	包围 P 的圆心 c 和经最大距离复核的半径 r	不是已经知道源恰好在圆心
<code>anchor</code> <code>last</code>	最近一次得到 <code>direction</code> 的位置 a 该频道最近一次检测位置，包括无信号点	不是最近一次检测位置 不保证位于定向源的可接收半平面
<code>U</code>	相对正锚点的源距上界，并配合角楔给出三角形外包络的尺度	不要直接把 <code>poly</code> 当作精确圆盘 $B(a, U)$

计算半角为 $\delta = 1.0051^\circ$ 。其中物理误差界为 1° ，两位小数输出的舍入保护为 0.005° ，余下 0.0001° 为小裕量。后三位保护不改变题设物理误差。

1 先把必须会讲的公式串起来

1.1 一次正观测如何变成三个半平面

令示向为 θ , $u = (\cos \theta, \sin \theta)$, u_- 、 u_+ 是将 u 分别旋转 $-\delta, +\delta$ 后的向量。候选位移 $z = g - p$ 满足

$$\text{cross}(u_-, z) \geq 0, \quad \text{cross}(u_+, z) \leq 0, \quad u^\top z \leq U. \quad (\text{Q4-2})$$

把前三个不等式写成 $m^\top g \leq b$, 即可逐次调用 clip。前两条是角楔两侧，第三条是轴向截断。第三条来自 $u^\top z \leq \|z\| \leq U$, 所以是安全的外近似；它没有把圆弧精确画进多边形。局部坐标 $z = xu + yv$ 、 $v = (-u_y, u_x)$ 下，外包络可写为

$$T(U) = \{(x, y) : 0 \leq x \leq U, |y| \leq x \tan \delta\}. \quad (\text{Q4-3})$$

1.2 为什么 25 站能覆盖未知方向

站点为原点、半径 995 m 的 8 个等角站 I_k 、半径 1838 m 的 16 个等角站 E_j 。外圈正十六边形内切半径

$$1838 \cos(\pi/16) = 1802.683 \dots > 1800$$

覆盖整个目标圆域。把内圈分为 $\triangle(O, I_k, I_{k+1})$ ；每个环带五边形 $I_k, E_{2k}, E_{2k+1}, E_{2k+2}, I_{k+1}$ 再分为

$$\triangle(I_k, E_{2k}, E_{2k+1}), \quad \triangle(I_k, E_{2k+1}, I_{k+1}), \quad \triangle(I_{k+1}, E_{2k+1}, E_{2k+2}).$$

下标循环取模，合计 32 个三角形。边长只有 995、 $2(995) \sin(\pi/8)$ 、 $1838 - 995$ 、 $2(1838) \sin(\pi/16)$ 以及

$$\sqrt{995^2 + 1838^2 - 2(995)(1838) \cos(\pi/8)} = 994.519 \dots$$

这五类，均不超过 995 m。任意源 g 所在三角形的三个顶点 p_1, p_2, p_3 都在 1000 m 内。由凸组合 $g = \sum_j \alpha_j p_j$ 、 $\alpha_j \geq 0$ 、 $\sum_j \alpha_j = 1$, 有

$$\sum_j \alpha_j n^\top (p_j - g) = 0.$$

三个点积不可能全部严格小于零，故至少一个近站位于可接收半平面。条件是静止源、半平面辐射、无额外漏检、完成全部未知频道扫描；结论是必能发现。站数最少、路线最短没有在这里证明。1838 m 超出的是目标圆域半径，题目没有把机器狗也限制在该圆内。

1.3 两个无信号为什么才有资格裁剪

在正锚点 a 的坐标系中取

$$q_\pm = a + \lambda U u \pm \eta U v, \quad \lambda = 0.5, \quad \eta = 0.05. \quad (\text{Q4-4})$$

先核对两个前提：

1. 保证距离。对 $g \in a + T(U)$, 有 $\|q_{\pm} - g\| \leq \kappa U$, 其中

$$\kappa = \sqrt{\frac{1}{4} + (\eta + \tan \delta)^2} = 0.50454158 \dots$$

由于 $U \leq 1500$, 距离不超过 756.813 m, 严格小于 1000 m, 因此无信号不能由超距解释。

2. 横跨角楔。 $\eta \geq \lambda \tan \delta$, 本例 $0.05 > 0.008773$; 所以线段 $[q_-, q_+]$ 横跨全部可能源射线。

若两点都无信号, 全向源已被排除; 对定向源, $n^T(q_{\pm} - g) < 0$ 。反设源的轴向坐标 $x = u^T(g - a) \geq \lambda U$, 则 $[a, g]$ 与 $[q_-, q_+]$ 交于

$$r = a + \lambda U u + \frac{\lambda U}{x} y v.$$

从 r 是两个负点的凸组合得 $n^T(r - g) < 0$; 从 $r - g = (1 - \lambda U/x)(a - g)$, 以及正锚点满足 $n^T(a - g) \geq 0$, 却得该点积非负, 矛盾。因此

$$x < \lambda U, \quad P^+ = P \cap \{g : u^T g \leq u^T a + \lambda U\}. \quad (Q4-5)$$

新的距离界为 $U^+ \leq \lambda U / \cos \delta$ 。代码保留闭半平面并用很小的外扩容差。普通单次无信号没有以上矛盾链, `_observe` 因此不直接删候选点。

1.4 从定位到安全清除

圆 $B(c, r)$ 包住 P , 若 $r \leq 19.8$, 选择满足 $\|q - c\| \leq 19.8 - r$ 的清除点, 则对所有 $g \in P$,

$$\|q - g\| \leq \|q - c\| + \|c - g\| \leq 19.8 < 20. \quad (Q4-6)$$

`nearest_clear` 求的正是当前点到圆盘 $B(c, 19.8 - r)$ 的欧氏投影。取圆心也安全, 但可能多走一段。这里用的清除证据是整个候选集合被覆盖; 猜测圆心接近真值并不构成证据。

在理想实数计算下, 成对探测的正分支至多按 κ 缩小尺度, 双负分支至多按 $\lambda / \cos \delta$ 缩小尺度。三角形 $T(U)$ 可被半径 $U / (2 \cos^2 \delta)$ 的圆覆盖; 由 $U_0 \leq 1500$ 得 6 轮后此半径上界约 12.38 m。程序设置 9 轮异常上限, 超过就报错。EPS、末端半径重算和 0.2 m 清除余量提高数值稳健性, 但这里没有带定向舍入的形式化浮点证明。

2 source/Q4/q4_local_solver.py: 策略核心的全部非空行

2.1 模块、常量与二维运算

- [L001] `#!/usr/bin/env python3`: 在支持 shebang 的系统中用环境里的 Python 3 执行; Python 将它视作注释。
- [L002] 延后求值类型注解, 减少定义顺序对注解的约束; 不改变算法计算顺序。
- [L003] 导入 `dataclass` 自动生成数据类初始化方法; `field` 用来为各频道单独创建可变多边形。
- [L004] 导入三角函数/距离工具与随机工具; 此文件的随机性只用于 MEC 点次序, 且固定种子。
- [L006] `Point` 是二元浮点元组类型别名, 便于解释输入; 它不是带运行时验证的新类。
- [L007] `TAU=2*pi`, 一整圈弧度。
- [L008] `EPS=1e-7` 是几何外扩尺度; 不能把它当作题目测量误差。
- [L009] 将计算半角 1.0051° 转成弧度, 理由见式 (Q4-2) 前的误差说明。
- [L010] `CLEAR_CERT=19.8` 是程序的认证清除半径, 小于实际 20 m 阈值。
- [L011] `LAMBDA=0.5` 控制成对站点沿示向轴前进多少个 U 。
- [L012] `ETA=0.05` 控制每一站离轴线的横向距离; 两站间距是 $2\eta U$ 。
- [L013] `REUSE_MIN_BASELINE=25` 是同频道机会复测的最小位移门槛, 不是误差独立性假设。

以下九个纯函数不修改传入元组或列表。输入坐标均须为有限实数；作为调用约定，本策略传入二维点。

行号与函数	输入 → 输出	每个表达式的作用与应会解释的条件
[L015] add(a,b)	两向量 → 向量	分别相加横、纵坐标，返回二元组 $a + b$
[L016] sub(a,b)	两点 → 位移向量	分别作差，得到从 b 指向 a 的向量
[L017] mul(a,s)	向量、标量 → 向量	两坐标各乘 s ；负 s 会翻转方向
[L018] dot(a,b)	两向量 → 标量	返回 $a_x b_x + a_y b_y$ ；用于半平面和投影
[L019] cross(a,b)	两向量 → 标量	返回有向面积 $a_x b_y - a_y b_x$ ；符号判断左右
[L020] norm(a)	向量 → 非负数	*a 展开坐标，hypot 算向量长度
[L021] dist(a,b)	两点 → 非负数	对坐标差使用 hypot，算欧氏距离
[L022] maxdist(p,poly)	点、顶点序列 → 非负数	对每个顶点计算距离并取最大；空序列返回 0。空集默认值仅便于工具使用，不能证明空集含真值

对凸多边形及其内部，maxdist 检查顶点就够：若 $x = \sum \alpha_j v_j$ ，则 $\|x - p\| \leq \sum \alpha_j \|v_j - p\| \leq \max_j \|v_j - p\|$ 。

2.2 clip：凸多边形与一个半平面求交

输入：按边界顺序排列的凸多边形 poly，法向 n，阈值 b。输出：交集顶点列表。前提：边界有序；本程序调用法向非零。循环不变量：每处理一条旧边，输出已经处理部分中应保留的顶点和穿越交点。结论：在容差外扩后的半平面中裁剪，不是重新计算凸包。

- [L024] 定义函数，参数 b 表示 $n^T x \leq b$ 的右端常数。
- [L025] 空多边形与任何集合相交仍为空，立即返回。
- [L026] $b += \text{EPS} * \max(1, \text{norm}(n))$ 向外放宽边界；out=[] 建输出表；a=poly[-1] 把末顶点当首条边的起点；fa=dot(n,a)-b 算它的有符号约束余量。四个语句缺一不可。
- [L027] 顺序取当前终点 c，因此第一条处理的是末点到首点的闭合边。
- [L028] fc 是终点的余量；不大于零表示在保留侧。
- [L029] 两端的“在内”真假不同才穿过边界；!= 在这里比较两个布尔值。
- [L030] 由 $(1-t)f_a + tf_c = 0$ 得 $t = f_a / (f_a - f_c)$ ；再把 $a + t(c - a)$ 加入输出。跨越时分母非零。
- [L031] 终点在内就追加；内到内只加终点，外到内先交点再终点，内到外只交点，外到外都不加。
- [L032] 把本边终点及余量作为下一边起点及余量。
- [L033] 建立去重后的列表。
- [L034] 顺序遍历初步输出。
- [L035] 首点总保留；后续点只有与上一已保留点相距超过 10^{-9} 才保留，避免连续重复交点。
- [L036] 如果首末点几乎相同，删除末点；闭合由循环隐含表达，不在列表中重复首点。
- [L037] 返回清理后的列表。它可能为空、一个点、线段或面；调用者必须处理空集。

2.3 arena_polygon 与 direction_update：安全初始化、加入新约束

arena_polygon 契约：输入边数 sides，默认 64，要求整数且至少 3；返回外切于半径 1800 圆的正多边形，保持“整个合法目标圆域都包含在初始集合”这一条件。

- [L039] 定义初始候选区域函数。
- [L040] 外接圆半径取 $1800 / \cos(\pi/s)$ ，使边到原点的距离恰为 1800；若误取顶点半径 1800 会内接并删掉合法位置。
- [L041] 对每个 k 生成旋转过半个中心角的顶点；横、纵坐标分别用余弦、正弦，列表推导返回全部顶点。

direction_update 契约：输入旧外包络、正观测位置、角度（度）、有效的源距上界；返回三个半平面裁剪结果。前提是示向误差在保护界内、U 不低估真实源距。每次裁剪维持真值属于外包络；输出有可能因矛盾输入为空。

- [L043] 定义更新函数，**theta_deg** 明确以度输入。
- [L044] 先将角度转弧度；分别生成下、上边界方向 lo 、 hi 。
- [L045] 用法向 $(lo_y, -lo_x)$ 与 $(-hi_y, hi_x)$ 各裁一次；右端为 **dot(n,p)**，即边界都通过观测点。两法向的符号对应式 (Q4-2)。
- [L046] 构造中央方向 u ；以 **dot(u,p)+U** 加轴向上界。注意这是直线截断，未精确裁圆弧。
- [L047] 返回更新结果，不直接写入频道状态；状态写入由 **_positive** 统一完成。

2.4 两点圆、三点圆和 mec

_diameter_circle 契约：两点 $\rightarrow$ 中点和半距离，允许重合点；结论是以两点连线为直径的圆，不能据此断言任意第三点在圆内。

- [L049] 定义两点支撑圆函数。
- [L050] 先用加法与乘 $1/2$ 算中点；再返回圆心、两点距离的一半。

_circum 契约：三点 $\rightarrow$ 外心和半径，近共线时返回 **None**。输入点有限；非共线条件由叉积检查。它求过三点的圆，而不是直接判定这就是该三点最小包围圆。

- [L052] 定义三点外接圆函数。
- [L053] 令 $u=b-a$ 、 $v=c-a$ ，并取 $d=2*\text{cross}(u,v)$ 作为二元线性方程组分母。
- [L054] 分母绝对值极小则视为退化，返回 **None**；后续代码不做除法。
- [L055] uu 、 vv 是两个边向量的长度平方；解 $2u^T w = \|u\|^2$ ， $2v^T w = \|v\|^2$ 得相对外心 w ，加回 a 得 o 。
- [L056] 返回外心及到 a 的距离。

mec 契约：非空有限点集 $\rightarrow$ 圆心及包住全部输入点的半径。其几何依据是平面最小包围圆至多由 2 或 3 个边界点支撑。外层维护已处理点的包围圆；遇到外点时把它作为新边界点重建。固定种子打乱改善通常运行次序，不等于随机抽样丢点。最后一行重新检查全部点，从而把返回圆的“覆盖”与中间构造分开核验。

- [L058] 定义包围圆函数。
- [L059] 空集说明定位约束或数值出了问题；显式报错，不能返回零半径假装定位完成。
- [L060] 复制点集而不改变调用者列表；用独立、固定种子 271828 洗牌；初始圆为空。
- [L061] 枚举第 i 个点 p 。
- [L062] 已有圆包含 p （加 10^{-9} 比较容差）则这一轮无需重建。
- [L063] 否则以新点为圆心、零为半径重新开始；此时 p 成为必要边界点。
- [L064] 逐个回看 p 前面的点 q 。
- [L065] 当前圆已包含 q 就跳过。
- [L066] 以 p,q 为直径建圆；保存方向 pq ；左右侧候选支撑圆清空。这里仍可能漏掉更早的点。
- [L067] 遍历 q 之前的 r ，检查固定 p,q 的支撑圆还缺哪些点。
- [L068] r 已被当前两点圆覆盖则不产生新的外接圆候选。
- [L069] 求三点 p,q,r 的外接圆。
- [L070] 退化三点没有可用有限外接圆，略过该候选；此语句不能单独拿来保证整个算法正确，依据还包括支撑点与扫描次序。
- [L071] **side** 判断 r 在有向 pq 哪侧； v 判断外心相对该直线的有向位置。
- [L072] 左侧取 v 更大的外接圆候选；记录的是 (有向量,圆)。
- [L073] 右侧取 v 更小的候选；它与上行合起来求能容纳两侧约束的支撑圆。
- [L074] 从非空的左右候选中提取圆本身。
- [L075] 有三点候选时选半径较小者；没有候选就保留两点圆。
- [L076] 取最终圆心；对原始全部点重算最远距离，再加 10^{-6} 作为输出半径。即使中间半径有舍入误差，这一步仍按返回中心重新包住所有顶点；并未把中心改成最远点中点。

2.5 nearest_clear 与 coverage_route

nearest_clear 契约：当前点 p 、候选包围圆 (c,r) ，前提 $r \leq 19.8$ ；返回安全清除域中离 p 最近的点。不修改位置；实际移动由 API 执行。

- [L078] 定义投影函数。
- [L079] 计算安全中心偏移余量 $slack=19.8-r$ 。
- [L080] 余量为负说明证书未满足，抛错，不能强行去中心清除。
- [L081] 算当前点与包围圆心的距离。
- [L082] 当前点已经在安全圆盘内，就原地返回。
- [L083] 否则沿圆心指向当前点的射线，取距圆心 $slack$ 的边界点；这一分支 $d>slack \geq 0$ ，故不会除以零。

coverage_route 契约：无输入 $\rightarrow$ 25 个有序站点。保证发现的证明依赖站点集合；返回顺序只是本程序选择的访问顺序。

- [L085] 定义路线生成函数。
- [L086] 生成 8 个半径 995、间隔 $\pi/4$ 的内圈点。
- [L087] 生成 16 个半径 1838、间隔 $\pi/8$ 的外圈点。
- [L088] 外圈先访问编号 14、15，再访问 0—13，形成一圈的连续编号顺序；这没有证明是全局最短路线。
- [L089] 拼接原点、8 内点和排序后的 16 外点。返回总数 25。

2.6 Track：一个频道的状态是什么意思

数据类契约：每个实例对应一个频道的独立记录，默认未知。FOUND 代表已经发现尚未清除，CLEARED 代表接口确认清除，ABSENT 代表在题设源数上限或完整覆盖证据下排除了存在。状态不是概率。

- [L091] @dataclass 为下方类生成初始化、显示等方法。
- [L092] 声明频道记录类。
- [L093] 初始状态 UNKNOWN；在发现前既不能称存在也不能称不存在。
- [L094] default_factory=arena_polygon 为每个实例新建候选列表；若共享同一个可变列表，修改一个频道可能污染其他频道。
- [L095] 默认圆心放原点，仅为初始化占位。
- [L096] 默认半径无穷，防止尚未定位就满足清除阈值。
- [L097] 尚无正示向，锚点为 None。
- [L098] 尚未检测，最近点为 None。
- [L099] 示向初值 0° 为占位值，只有正锚点存在时才使用。
- [L100] $U=1500$ 采用题设最大接收半径；首次正观测可据此限制源距。
- [L101] 正示向计数初始 0；此字段只在 `_positive` 增加，不等于全部检测次数。

2.7 策略初始化、正更新与两个 API 包装

Strategy.__init__ 契约：输入提供 `measure(p,ch)`、`clear(p,ch)` 的 API；初始化单局策略状态。前提 API 起点为原点、初始频道为 1，且时间字段以秒返回。策略自身不读取模拟器的真实源。

- [L103] 声明策略类。
- [L104] 定义初始化方法。
- [L105] 保存 API；为 1—20 建独立 Track 字典；当前点设原点，当前检测频道设 1。
- [L106] 清零虚拟时间、动作数、清除数；生成覆盖路线；覆盖索引指向尚未扫描的第 0 站。
- [L107] 初始覆盖未结束；记录遇到过的最大单源成对探测轮数为 0。

_update_mec 契约：输入非空频道候选集；原地更新 `center,radius`，返回 None。它不改变 `poly` 与状态。

- [L108] 对 `t.poly` 调用 MEC，并一次性将二元返回值拆给圆心、半径。

_positive 契约：已接收到频道 i 在 p 的示向；更新候选位置、正锚点和包围圆。前提读数可信、源尚未清除；维持真值位于多边形。没有直接清除。

- [L109] 定义正观测处理。
- [L110] 取该频道记录；用 $\min(1500, \text{maxdist}(p, \text{旧}P) + \text{EPS})$ 结合物理上界与原候选集距离上界；再用新角楔裁剪，暂存在 `new`。
- [L111] 新集合为空时报错，禁止继续拿空集合构造清除证书。
- [L112] 顺次标为 `FOUND`、替换多边形、把本点设为正锚点、保存浮点示向、进一步缩小 `U`、正示向计数加一、重算包围圆。这一行有七个动作，要逐个说清。

`_clear` 契约：输入频道、清除位置和是否有几何证书；调用 API 并将有效响应同步到状态。返回是否清除成功。默认认证失败视为模型、几何或接口的严重矛盾，立即抛错。

- [L113] 定义清除包装，默认 `certified=True`。
- [L114] 请求 API 在 `p` 清除频道 `i`；移动时间由 API 计入。
- [L115] 响应未接受就报错，不把被拒绝动作计入成功状态。
- [L116] 同步当前位置与累计虚拟时间，动作数加一；不在返回时间上额外加移动或 5 s。
- [L117] 成功时标记频道 `CLEARED`、清除数加一、返回 `True`。
- [L118] 若失败类型不是明确的 `no_target_in_range`，属于未知协议结果，报错。
- [L119] 明确未命中且此前声称有证书，也报错；不允许静默继续。
- [L120] 只有显式非认证请求的明确未命中才返回 `False`。当前主要调用路径使用认证清除。

`_observe` 契约：检测一个频道并返回结果种类；统一更新位置、当前检测频道、累计时间、动作数、该频道最近测点。正示向缩小集合；`near` 立即清除；普通无信号只记录，不剪集合。

- [L121] 定义检测包装。
- [L122] 发出检测动作。
- [L123] 拒绝响应报错。
- [L124] 同步位置、检测频道、虚拟时间，动作数加一，令该频道 `last=p`。这里 `last` 对三种结果均更新。
- [L125] 取响应类别。
- [L126] `direction` 把读数转浮点并交给 `_positive`；所以这一路会改 `anchor`。
- [L127] `near` 先确认已发现，再在同一点认证清除；清除包装还会追加一个动作和同步清除后的时间。
- [L128] 除合法的 `no_signal` 外，其他未识别结果报错；合法无信号没有 `poly=...` 更新。
- [L129] 返回原检测类别。即使 `near` 已清除，它仍返回字符串 `near`。

2.8 何时能够宣布一个频道不存在

`_known_count` 契约：无输入 → 已证实存在的不同频道数；计算 `FOUND` 与 `CLEARED`，不把 `ABSENT` 计入。

- [L130] 对各状态的布尔判断求和，布尔真按 1 计数。

`_finish_unknown_if_possible` 契约：仅当达到题设最多 16 个源，或 25 站扫描完毕，才把所有未知频道改为不存在。前提覆盖站点按 `_scan_site` 规则确实扫描未知频道；此函数不发动作、不增加时间。

- [L131] 定义终止证据检查。
- [L132] 已知存在数达到 16 时，使用总数上限。
- [L133] 遍历所有频道。
- [L134] 仅把仍未知的频道变为 `ABSENT`；已发现频道仍待清除。
- [L135] 标记覆盖任务结束。
- [L136] 否则检查已扫描站数是否达到路线长度。
- [L137] 遍历所有频道。
- [L138] 将完整扫描后仍未知的频道标为不存在，依据第 1.2 节覆盖证明。
- [L139] 标记覆盖任务结束；这不意味着 `FOUND` 频道已经清除。

2.9 机会复测与整站扫描

`_reuse` 契约：最多复测 `limit` 个已发现未定位好的频道；无返回值，可能更新多个频道或就地清除。保证候选集合距当前位置不超过 999 m，只排除了“太远”，没有消除定向盲区。25 m 门槛和优先大半径都属于效率选择。

- [L140] 默认最多复用 2 个频道。

- [L141] 初始化候选评分列表。
- [L142] 遍历频道及对应记录。
- [L143] 必须已发现、半径还大于 19.8，且当前位置到所有候选顶点的最大距离不超过 999；三项同时成立才继续。
- [L144] 没有最近测点，或与最近测点距离不足 25，则跳过。等于 25 时允许；比较的是 last，不是 anchor。
- [L145] 保存 (半径,频道)，后续大半径优先。
- [L146] 倒序排序并截取前 limit 个；同半径时较大频道号先被选。
- [L147] 在当前位置检测，不主动移动；换频道与检测仍耗时。无信号仍不裁剪。

_scan_site 契约： 对指定覆盖站的未知频道逐个检测，并完成一个覆盖站计数。扫描中已知源数达到 16 可提前结束，因为剩余频道已由上限排除。

- [L148] 定义整站扫描。
- [L149] 取当前未知频道快照；已发现和已清除频道不重复做发现扫描。
- [L150] 排序键先让当前频道排前，再按频道号排；能减少一次首个换台，但没有解全局换台最优化。
- [L151] 遍历未知频道快照。
- [L152] 每次都在同一个 p 检测；第一动作负责从当前点移动到站点，以后不重复移动。
- [L153] 已知源数满 16，跳出频道循环。
- [L154] 在本站最多复用两个已发现频道；覆盖索引加一；随后检查能否把未知标不存在。这行三个动作依次执行。

2.10 成对探测与双负裁剪

_dual_negative 契约： 输入同一轮的旧正锚点、轴向、距离上界；前提两个按式 (Q4-4) 构造的点均返回无信号，并满足距离与横跨条件。输出为原地缩小候选集、U、包围圆。保留旧正锚点，不将负点当锚点。

- [L155] 定义双负更新。
- [L156] 用 $u^T x \leq u^T a + \lambda U$ 裁剪；这是式 (Q4-5) 的结论。
- [L157] 空结果违反保留真值的不变量，立即报错。
- [L158] 新 U 取理论界 $\lambda U / \cos \delta$ 与多边形最远距离界的较小值；再重算包围圆。这里没有给 anchor 重新赋值。

_pair_round 契约： 对某个 FOUND 频道做一轮，可能只检测一次，也可能检测两次。前提正锚点有效；每轮固定从旧锚点构造两个点，首个正结果出现就结束本轮。

- [L159] 定义一轮探测。
- [L160] 取频道记录引用 t，不是复制记录。
- [L161] 没有正锚点无法用正锚点矛盾证明，报错。
- [L162] 保存本轮旧 U；示向转弧度；造单位轴向 u 与左法向 v。
- [L163] 计算轴向前进 λU 、横向偏移 $\pm \eta U$ 的两个点，分别存为 q1,q2。
- [L164] 若第二点离当前点更近就交换访问顺序；集合本身未变，证明不受影响。
- [L165] 先检测第一点，取得字符串结果。
- [L166] 如果源已经清除，或结果并非无信号，结束本轮。特别是首点 direction 时，新锚点已写入，不能继续拿旧第二点凑一轮双负。
- [L167] 只有首点无信号且仍为已发现状态，才检测第二点。
- [L168] 第二点正示向或已清除时同样结束。
- [L169] 两次都是无信号才做双负更新。此时 _observe 从未改变 anchor，所以这里引用的 t.anchor 仍是本轮旧正锚点；u,U 也仍是旧值。

2.11 服务一个源、插入任务与主循环

_service 契约： 输入已发现频道，重复成对探测，直至得到安全清除证书并清除；最多允许 9 轮，否则异常。返回 None，同时可能做两个机会复测。

- [L170] 定义单源服务。

- [L171] 本次服务轮数从 0 开始。
- [L172] 只要该频道仍为 FOUND 就继续。
- [L173] 每轮重新读取当前记录，使用最新多边形和圆。
- [L174] 先检查半径证书；已有证书时无需再探测。
- [L175] 用最近安全清除点发认证清除，然后跳出循环。
- [L176] 已做满 9 轮但仍没有证书时报错，不会强行清除。
- [L177] 执行一轮并给轮数加一。一次轮数可能对应 1 或 2 个检测动作，不能混成同一计数。
- [L178] 更新历史最大轮数；在最终当前位置复用两个频道；检查是否能结束覆盖。三个动作都有实际作用。

_choose_insert 契约：输入下一覆盖站，或覆盖结束时的 None；输出一个待服务频道或 None。评分是距离量纲的启发式代理：绕行距离加不确定半径；不是精确服务时间，也不是 TSP 最优解。

- [L179] 定义选插入任务。
- [L180] 创建候选列表。
- [L181] 遍历频道。
- [L182] 不是 FOUND 就不需要服务，跳过。
- [L183] 没有下站时，用当前点到候选中心距离加半径评分。
- [L184] 有下站时，用 $\|p - c\| + \|c - s\| - \|p - s\| + r$ ：前 3 项是途经中心的额外路程，最后一项惩罚定位不确定性。
- [L185] 保存 (分数,频道)；同分时频道号较小者胜。
- [L186] 没有候选返回 None。
- [L187] 取最小分数：覆盖已经结束则直接选；否则仅当分数不超过 600 才插入。600 是策略阈值，米量纲；不是证明出来的最优常数。

run 契约：从初始化状态驱动到已发现源全清、未知源均有排除证据，返回统计字典；异常时抛错。持续保持动作时间来自 API，状态改变来自有效响应或几何/源数证据。

- [L188] 定义策略入口。
- [L189] 尚未扫描任何站时先扫描路线原点，不需要额外的显式移动接口。
- [L190] 主循环保护计数清零。
- [L191] 限制最多 2000 次主循环，避免状态错误导致死循环。
- [L192] 每轮计数加一。
- [L193] 先刷新未知频道是否已可排除。
- [L194] 收集还需服务的已发现频道。
- [L195] 收集仍未知频道。
- [L196] 两种集合都空才正常结束；此时状态只能是已清除或不存在。
- [L197] 路线索引有效且覆盖未结束时取下一站，否则设 None。
- [L198] 根据当前局面选插入频道。
- [L199] 有插入任务就服务，然后立刻开始下一主循环；continue 防止同轮继续使用过期的 found/unknown 快照。
- [L200] 没有插入任务但有下站，就扫描下站并进入下一轮。
- [L201] 没下站但仍有已发现频道，就按“距中心+半径”选最近代理任务服务；这也是兜底服务分支。
- [L202] 没有其他可行动作时再刷新未知状态。
- [L203] Python 的 while...else：循环因达到上限而退出、未执行 break 才报错。它不对应某一个 if。
- [L204] 返回清除数量、累计虚拟时间；每源平均为时间除已清数量，零源时用 None 避免除零。
- [L205] 返回动作数、已访问覆盖站数、覆盖结束标记、最高成对轮数。
- [L206] 返回每个频道的最终状态，键转字符串适合 JSON；本行同时闭合字典表达式。

run_case 契约：API → 单局汇总。每次调用都建立新策略，因此不会沿用上一局轨迹。

- [L208] 一行中先构造 Strategy(api)，再调用 run() 并把返回值传给调用者。

3 source/Q4/q4_official_solver.py: 相同策略怎样接上正式 HTTP

3.1 公共前缀、异常种类与客户端初始状态

- [L001-L008] 与第 2 节本地求解器同号行完全相同：模块、类型与常量。此处按同号引用，不另造第二套解释。
- [L009-L208] 与第 2 节本地求解器同号行完全相同：向量、裁剪、包围圆、轨迹、策略与入口。逐行解释以第 2 节为准。此大范围只是字节相同前缀的映射声明，不代替新增代码的逐行解释。

先辨别三层事实：“请求已发送”不等于“动作已执行”；“响应没有收到”不等于“动作未执行”；只有有效响应才能安全同步位置和时间。`request_id` 用来标识同一个动作，重试应保留同一个标识。服务端是否按相同标识去重属于协议前提，客户端不能单靠自己的变量创造服务端幂等性。

- [L211] 导入命令行、日期、HTTP、JSON、环境变量、套接字、标准错误输出、计时、Unicode 字符类别、随机 UUID 工具。`time` 在这里处理现实墙钟预算和重试等待，不代替题目虚拟时间。
- [L212] 导入文件路径对象。
- [L213] 导入 URL 解析器。
- [L215] 定义 `ProtocolError`，表示有效证据表明协议拒绝；类体 `pass` 表示无需额外实现。
- [L216] 定义 `BudgetExceeded`，表示现实或虚拟预算不足。
- [L217] 定义 `ResultUnknown`，表示动作是否执行、或其结果仍无法确定；它不能按普通失败处理。
- [L219] 声明正式客户端类。

`OfficialClient.__init__` 契约：输入本机 HTTP 地址、队号、日志路径及超时/重试配置；建立客户端元数据并创建独占日志。尚未连接、尚未入场、尚未发动作。队号必须非空且合法；日志路径必须可创建。

- [L220] 默认单次超时 5 s，默认额外重试 6 次；所以一个请求最多尝试 7 次。
- [L221] 拆分 URL 为协议、主机、端口、路径、查询、片段。
- [L222] 只允许 HTTP、有主机、路径为空或 /、无查询和片段；不把复杂 URL 当作接口根地址。
- [L223] 主机只允许三个回环表示；这限制客户端只连接本机官方模拟器。
- [L224] 队号不能为空，UTF-8 字节长度不超过 64，不能有 Unicode 控制或格式字符。限制的是编码字节数，不是汉字数量。
- [L225] 保存主机、端口（未写时为 80）、队号、超时、重试；连接对象尚为空。
- [L226] 用 UUID 前 12 个十六进制字符作本次运行前缀；序号从 0 起；现实截止时间尚未获得；虚拟时间从 0 起；先放默认虚拟上限 360000 s。
- [L227] 入场/退出标记都假；累计重试数 0；客户端位置原点、检测频道 1；没有待确认请求 `pending=None`。
- [L228] 尚未尝试退出，尚无退出响应。将“尝试过”和“确认成功”分开记录。
- [L229] 初始化日志失败标记和原因。
- [L230] 创建日志父目录；以 `x` 模式独占创建文件。若同名日志已存在则失败，不覆盖已有证据。
- [L231] 写元数据：程序 Q4、遮蔽的队号占位符、UTC 时间。真实队号仅用于请求，日志按此行遮蔽。

3.2 日志、连接与时间预算

`_log` 契约：接收可 JSON 序列化对象，尽力逐行写入并立即刷新。写入失败记录故障并向标准错误输出，不在这里打断动作；主流程在最后检查日志完整性。

- [L232] 定义写日志函数。
- [L233] 生成紧凑 JSON，保留中文，禁止 NaN；追加换行形成 JSONL，再 `flush()` 刷新 Python 缓冲区。刷新不等于操作系统强制落盘 `fsync`。
- [L234] 捕获本次日志写入/序列化异常。
- [L235] 标记日志不完整，存 `repr(exc)`，向标准错误打印并刷新，方便现场发现。

`_disconnect` 契约：关闭并丢弃当前连接，允许没有连接；不更改 `pending`，因此断开网络不会冒充动作结果已确认。

- [L236] 定义断连函数。
- [L237] 仅在连接对象存在时尝试关闭。

- [L238] 调用连接的 `close()`。
- [L239] 关闭过程中的操作系统异常忽略，后面仍清理引用。
- [L240] 将连接设为 `None`，下一次请求会重建。

`_connection` 契约：返回一个已建立或复用的 HTTP 连接；可按剩余预算更新连接与套接字超时。连接失败由调用者的重试逻辑处理。

- [L241] 定义连接获取函数，可显式传超时。
- [L242] 没有可复用对象时才新建。
- [L243] 用主机端口和选定超时创建 `HTTPConnection`，再调用 `connect()` 建 TCP 连接。
- [L244] 存在套接字时设置 `TCP_NODELAY`，减少小请求可能受 Nagle 算法影响的延迟；不改变动作语义。
- [L245] 保存连接对象供后续复用。
- [L246] 显式提供新超时，更新连接对象的 `timeout` 属性。
- [L247] 连接已经有套接字时同步更新套接字超时，否则仅改属性不足以约束已有 socket。
- [L248] 返回连接对象。

`_finite` 契约：任意值 $\rightarrow$ 布尔值；只允许非布尔的有限 `int/float`。用于协议字段验证，不将字符串 "12" 自动视为数值。

- [L249] `@staticmethod` 表示该方法不需要实例 `self`。
- [L250] 定义有限数检查。
- [L251] Python 中 `bool` 是 `int` 的子类，故先排除布尔，再要求 `int` 或 `float`。
- [L252] 转为浮点后用 `isfinite` 排除无穷和 NaN。
- [L253] 转换溢出或类型/数值异常都返回假，不使验证器自身崩溃。

`_cutoff` 契约：接口路径 $\rightarrow$ 本请求应停止发送的现实时间点，或尚无截止时间时返回 `None`。
`_attempt_timeout` 契约：路径 $\rightarrow$ 本次等待上限，预算耗尽则抛异常。均使用单调时钟，避免系统时钟校正干扰。

- [L254] 定义路径对应的发送截止函数。
- [L255] 入场前尚无服务器预算，则返回 `None`。
- [L256] `/exit` 只预留 0.25 s，其余路径预留 8 s；为退出动作留下现实余量。
- [L257] 定义单次尝试超时计算。
- [L258] 取得上述路径截止点。
- [L259] 未设截止时间时用配置超时。
- [L260] 用截止点减当前单调时钟得到剩余秒数。
- [L261] 已不剩时间就停止发送新动作并抛预算异常。
- [L262] 上限不超过配置超时，并设 0.05 s 最小值。极端剩余不足 0.05 s 时返回值会略大于“剩余”，所以这里是工程保护，不是严格硬实时证明。

3.3 `_validate_success`：响应为什么可以被信任

契约：输入路径、HTTP 状态码、已解析 JSON；通过则无返回值，失败按证据抛 `ProtocolError` 或 `ResultUnknown`。只验证已编写字段，不假定收到 JSON 就全都可信。虚拟时间必须不倒退；它不会自己更新客户端时间。

- [L263] 定义成功响应验证器。
- [L264] 顶层必须是字典，不接受数组或单个数。
- [L265] `accepted` 必须是 JSON 布尔映射的 Python `bool`。
- [L266] 遍历现实时间戳、累计虚拟时间两个共同字段。
- [L267] 两者均须有限且非负，否则结果未知。
- [L268] 单独处理非 200 状态。
- [L269] 非 200 且明确 `accepted=False`，视为明确拒绝，抛协议异常。
- [L270] 非 200 却说接受，两个证据矛盾，结果未知。
- [L271] 状态 200 但没有接受，也作为明确拒绝。
- [L272] 新虚拟时间若比已知值小超过 10^{-5} ，判为倒退；这不是把时间负增量接受下来。

- [L273] `/enter` 还需专门检查预算字段。
- [L274] 枚举最大虚拟时长、最大现实时长、剩余现实时长。
- [L275] 三个值都须有限非负。
- [L276] 剩余现实时长转浮点。
- [L277] 要求不超过 1200 且是整数；这是当前客户端对协议的明确假设。
- [L278] 检测响应走另一类检查。
- [L279] 读取检测结果类别。
- [L280] 只允许 `no_signal`、`near`、`direction`。
- [L281] 只有示向类别要求 `svd_deg` 为有限数且在 $[0, 360)$ ；无信号无需硬凑角度。
- [L282] 清除只允许成功或明确范围内无目标两个结果。
- [L283] 主动退出要求 `exit_reason='user_exit'`，否则结果未知。

3.4 `_request`: 同一动作的重试与 `pending`

输入：路径、包含 `request_id` 的请求字典，可选已编码 `body`。输出：通过验证的响应字典。关键不变量：结果尚不确定时保留同一请求的路径、负载和字节；内部重试不生成新标识。前提：它通常只被 `action` 调用，`action` 保证开始时没有别的待确认动作。

`pending` 不是“动作还在物理执行”的可靠检测器，而是“客户端尚未确定这一请求结果”的记录。即使连 TCP 都没接通，保守分支也可能先记为未知；这样可能提早停止，但不会把可能已经执行的动作换新编号再做一次。

- [L284] 定义底层请求函数。
- [L285] 没有传入可用 `body` 时，将负载编码成 UTF-8 紧凑 JSON，禁止 NaN；同一调用后续重试一直复用这份 `body`。
- [L286] 仅在尚无待确认记录时创建记录。
- [L287] 保存路径、负载副本、编码字节；负载副本避免调用者之后修改同一字典影响记录。
- [L288] 另复制安全日志负载，将真实队号换成占位符，再写请求日志；发给接口的负载未被替换。
- [L289] `last` 记录最近错误；`unknown=False` 表示至今还没有经历可能已经执行却无法确认的情况。
- [L290] 从 0 到 `retries`，包含首次尝试和所有额外重试。
- [L291] 记录本次开始时间，用于日志实际耗时。
- [L292] 开始受异常处理保护的单次尝试。
- [L293] 先检查现实预算并获取连接，然后发送 POST，指定 JSON 内容类型与 UTF-8；三步按顺序发生。
- [L294] 等响应、读状态码、完整读取 `body`。读取失败时动作可能已经执行，所以不能简单标“未执行”。
- [L295] 以 UTF-8 解码并解析 JSON；`parse_constant` 的生成器抛异常写法专门拒绝 NaN/Infinity 这样的非标准数值常量。
- [L296] 捕获无效 JSON 或 UTF-8 编码错误。
- [L297] 写无效响应日志，最多保存前 1000 字节的可替换解码文本，附请求号、尝试号和 HTTP 状态。
- [L298] 转成 `ResultUnknown`，保留原异常因果；无效响应不说明服务端没执行。
- [L299] 记录有效解析出的响应、请求号、尝试次数、现实耗时与 HTTP 状态，随后仍需验证字段。
- [L300] 对 429、500、502、503、504 走可重试状态分支。
- [L301] `common_ok` 首先要求对象是字典且明确拒绝。
- [L302] 再要求现实时间戳是有限数。
- [L303] 再要求虚拟时间是有限数；本分支只检查这两字段“有限”，没有沿用成功验证器的“非负”检查，不能口述成所有分支验证完全相同。
- [L304] 不满足共同条件时，状态不可信，标为结果未知。
- [L305] 条件满足时抛带 HTTP 前缀的 `OSError`，交给统一重试分支；该特定前缀用于表示已知拒绝的重试。
- [L306] 其余响应调用完整成功验证器。
- [L307] 若验证器给出明确协议拒绝，清掉 `pending` 再继续抛出；此时没有悬而未决的动作。
- [L308] 完整验证通过，更新累计虚拟时间、清 `pending`、返回响应。位置和频道的更新仍在外层 `action`。

- [L309] 单独捕获预算耗尽。
- [L310] 如果之前没有结果未知的尝试，允许清 pending。
- [L311] 清 pending 并记录因预算而在本次发送前终止；如果早先某次可能执行，不能走这一分支。
- [L312] 将预算异常继续传给主程序。
- [L313] 协议明确拒绝不再按网络错误重试，直接抛出。
- [L314] 捕获网络、超时、HTTP 库以及结果未知异常。
- [L315] 保存最近异常；累积 unknown。ResultUnknown 或普通连接类故障都令其为真；只有特意构造的 OSError('HTTP ...') 可保留已知拒绝状态。一旦真，不会被后来一次明确拒绝重置为假。
- [L316] 断开旧连接；记录本次重试原因和耗时，pending 仍保留。
- [L317] 已到最后一次尝试就结束重试循环。
- [L318] 额外重试计数加一；指数退避为 $\min(0.1 \cdot 2^{\text{attempt}}, 1.5)$ 秒；取当前路径截止点。
- [L319] 已有截止点时，将等待时长压到可用余额内。
- [L320] 没有正等待时间就停止循环。
- [L321] 只在剩余时间允许时做短暂现实等待；此等待不是题目虚拟动作耗时。
- [L322] 重试耗尽且曾有未知结果，抛 ResultUnknown 并保留 pending；程序没有自动把这一请求换号重发的通路。
- [L323] 所有失败都已知未接受时清 pending，抛连接错误，说明相同请求多次尝试仍无有效响应。

3.5 action：合法输入与动作前预算

契约：输入四种合法路径及适当位置/频道，输出可信响应；为一个新动作分配唯一运行内请求号。只有响应通过才更新客户端位置/检测频道。位置验证界限是接口输入范围，和目标圆域 1800 m 是两种条件。

- [L324] 定义通用动作入口。
- [L325] 路径不在入场、检测、清除、退出四个集合内则拒绝。
- [L326] 上个结果未知时拒绝生成新请求号；这也会阻止新退出请求，见 can_exit。
- [L327] 检测或清除必须同时提供位置和频道。
- [L328] 入场与退出不能带多余位置频道。
- [L329] 初始化规范化位置为空。
- [L330] 只有提供位置时才验证坐标。
- [L331] 要求长度 2、各坐标为有限数、绝对值不超过 2×10^6 ；绝不是机器狗被限定在半径 1800 圆内。
- [L332] 转成两个浮点数的元组，用作统一内部格式。
- [L333] 频道要非布尔整数且为 1—20；不把 True 作为频道 1。
- [L334] 只有检测和清除需要预计虚拟动作开销。
- [L335] 移动时间为距离除 5；操作先统一按不超过 5 s 算。清除失败实际为 3 s，这里用 5 s 是保守上界。
- [L336] 检测若换频道，再加 1 s；清除不额外换检测频道。
- [L337] 累计时间加开销上界超过最大虚拟时长减 0.001 时拒绝发送。
- [L338] 检查现实发送预算；序号加一；用场地、队号、前缀加序号创建请求。只有进入新 action 才增加序号，内部重试不加。
- [L339] 若有位置，将元组写成接口所需 {'x':..., 'y':...}。
- [L340] 若有频道，将频道加入请求。
- [L341] 调底层 _request；异常时下面的位置更新不执行。
- [L342] 检测或清除成功收到有效响应后，客户端当前位置同步到目标位置。
- [L343] 检测才更新当前检测频道；清除不改变它。
- [L344] 返回响应供策略层处理类别与频道状态。

3.6 等待、入场、退出、关闭

wait_until_open 契约：在 wait_s 现实秒内等本机 TCP 端口可连；连通不等于比赛入场，仍需调用 enter。

- [L345] 定义等待方法，默认 300 s。
- [L346] 用单调时钟算等待截止。
- [L347] 截止前持续检查。
- [L348] 开始一次连接尝试。

- [L349] 尝试建立最长 0.5 s 的 TCP 连接；成功立即返回，with 负责关闭这个探测连接。
- [L350] 连接失败等 0.25 s 再试。
- [L351] 总等待到期抛超时异常。

enter 契约：向服务端请求入场；成功后写入服务器给的预算和入场标记。截止从请求开始计，包含请求耗时，因此偏保守。

- [L352] 定义入场方法。
- [L353] 先记请求开始单调时钟；发入场动作；取剩余现实预算。
- [L354] 再次检查剩余预算类型与范围；主要严格验证已在 _validate_success 完成。
- [L355] 设现实截止为请求开始加剩余秒；更新最大虚拟预算；标记已入场；返回响应。

measure/clear 契约：为策略暴露和本地模拟器一样的方法签名；返回通用动作结果，不额外计时。

- [L356] measure(p,ch) 转调 /measure。
- [L357] clear(p,ch) 转调 /clear。

exit 契约：首次请求退出；已确认退出后再次调用返回缓存响应。仅确认后才置 exited=True。

- [L358] 定义退出方法。
- [L359] 已成功退出则直接返回上次响应，避免重复请求。
- [L360] 先标记曾尝试退出；发动作；响应确认后再标记成功、保存响应并返回。若中间异常，仍会留下“尝试过但未确认”的事实。

can_exit 契约：判断发生异常后是否允许补发一个新的退出动作。它不代表当前任务已成功。

- [L361] 必须已入场、未确认退出、未尝试退出、没有 pending，且尚有退出时间。任何一项失败都不补发；尤其不能跨过结果未知动作直接发新 request_id。

close 契约：释放连接与日志；不执行退出协议，所以释放资源与比赛正常退出必须区分。

- [L362] 定义资源关闭方法。
- [L363] 断开 HTTP 连接。
- [L364] 尝试关闭日志。
- [L365] 捕获关闭阶段系统异常。
- [L366] 标记日志失败并存原因、输出错误；不能声称日志完整保存。

3.7 正式 main：成功、失败和文件产物

契约：从命令行/环境/交互取得队号与接口配置，运行单局正式策略、退出并写汇总；返回进程退出码 0 或 1。只有策略完整、退出确认、日志未失败才返回 0。建立客户端前的参数或文件创建异常不在后面 try 内，会直接让程序异常退出。

- [L368] 定义主函数。
- [L369] 建立命令行解析器及中文说明。
- [L370] 添加可选 --robot-id；队号不写死源码。
- [L371] 添加默认本机端口 2026 的 URL 与默认 300 s 等待，随后解析命令行。
- [L372] 队号优先级为命令行、CUMCM_ROBOT_ID 环境变量、现场输入；另建带微秒的时间戳用于避免日志名冲突。
- [L373] 构造 Q4 日志路径与客户端；初始 ok=False；记现实计时起点。
- [L374] 开始主运行异常保护。
- [L375] 提示等待；等端口开放；入场；入场后重置实际程序计时起点，所以报告耗时不含等待端口时间。
- [L376] 提示开始；运行共享策略；然后请求退出。run_case 返回不代表已退出，后一个调用不可漏。
- [L377] 向策略汇总加入实际程序时长与重试总数。
- [L378] 写 .summary.json；再写包含策略完整和退出确认字段的日志条目；暂设 ok=True。一行里的三个动作依次发生。
- [L379] 如果任一日志步骤失败，主动抛异常，不能返回完整交付的成功状态。
- [L380] 没有可计算均值时显示 N/A，否则格式化为三位小数。

- [L381] 打印清除数、虚拟总时间与每源均值；不是以现实运行时长替代成绩时间。
- [L382] 打印原始日志文件位置。
- [L383] 捕获普通异常与键盘中断。
- [L384] 明确置失败标志。
- [L385] 写致命错误日志并向标准错误提示未完成。
- [L386] 只在 `can_exit()` 全条件满足时尝试补发退出。
- [L387] 尝试调用正常退出方法。
- [L388] 补退出也失败时记录原因及 pending 的路径、请求号；不把完整负载和队号直接塞进错误日志。
- [L389] `finally` 无论成败都释放资源。
- [L390] 仅 `ok` 且日志未失败返回 0，否则返回 1；批处理文件将保留此退出码。
- [L391] 文件直接运行时通过 `SystemExit(main())` 让返回整数成为操作系统可见的退出码；被导入时不自动执行。

4 source/Q4/q4_local_simulator.py: 真实世界在本地如何被模拟

4.1 模块、源数据与模拟器状态

本文件知道源的真值，用来回答求解器的动作。求解器只调用 `measure/clear`，并不读取 `sources`。分清两个角色，才能回答“算法是不是偷看了答案”。本地随机分布与误差函数是本程序的测试设置，不能把本地全清除率直接写成官方成绩。

- [L001] 指定可用 Python 3 直接执行的 shebang。
- [L002] 延后求值类型注解。
- [L003] 导入数据类装饰器。
- [L004] 导入命令行、日期、JSON、数学、随机及现实计时工具。
- [L005] 导入路径操作。
- [L006] 导入同目录 `q4_local_solver`，以 `solver` 名称调用；这就是本地入口为何应从 Q4 目录运行。
- [L008] `frozen=True` 使正常赋值接口下源记录不可修改；避免仿真中不小心移动源。
- [L009] 声明源真值数据类。
- [L010] 一行声明六个必填字段：频道、二维位置、接收半径、是否定向、朝向弧度、误差相位。`phase` 只用于本地误差函数，并不是题目可观测量。
- [L012] 声明模拟器类。

`Simulator.__init__` 契约：一组各频道唯一的源 → 初始模拟器。输入若有重复频道，字典构造会只保留最后一个，故唯一频道是生成器负责的前提。输出对象保持真值固定，累计时间和计数随动作单调增加。

- [L013] 定义初始化。
- [L014] 按频道建真值字典；已清频道集合为空；当前位置原点；当前检测频道 1；虚拟时间 0。
- [L015] 清零移动路程、检测次数、换频道次数、失败清除次数。

4.2 移动、定向可见性与固定地点误差

`_move` 契约：目标点 → 原地更新距离、时间与位置；前提速度恒为 5 m/s，不考虑加速转弯。没有返回值。

- [L016] 定义移动更新。
- [L017] 先算旧位置至新点的距离；累计路程加距离；虚拟时间加距离除 5；最后更新位置。先更新位置再算距离会把距离误算为零。

`_visible` 契约：真值源、检测点 → 角度可见布尔值；只检角度，不检接收半径。全向源恒真；定向按式 (Q4-1)。

- [L018] 定义可见性函数。
- [L019] 全向源直接返回真。
- [L020] 从朝向角构造单位向量；返回 $(p - g)^T d \geq -10^{-10}$ 。很小负容差把边界略向外扩；不要把向量写成 $g - p$ ，写反会把可接收半平面翻转。

`_error` 契约：固定源、固定位置 → 确定的有界角误差（度）。它没有每次抽随机数；同地同源误差相同。正弦与余弦构成光滑测试函数，并非声称物理噪声必然如此。

- [L021] 定义误差函数。
- [L022] 用位置缩放、源相位构成正弦和余弦，权重 0.61、0.39 总和为 1；理论范围已在 $[-1, 1]$ ，外层 min/max 再截到这一区间。

4.3 `measure` 与 `clear`：动作规则和返回值

`measure` 契约：位置与频道 → 接受且包含累计虚拟时间的检测响应。顺序是移动、5 s 检测、必要时 1 s 换台、判信号。生成器与求解器负责提供合法参数；本地模拟器没有实现正式客户端的全部输入验证和预算拒绝。

- [L023] 定义检测接口，与策略预期签名相同。
- [L024] 移动到请求点；增加 5 s 检测时间；检测计数加一。
- [L025] 如果请求频道不同，增加 1 s 并计一次换台；判断时仍使用旧频道。
- [L026] 同步新频道；查找源；先建含 `accepted=True` 和累计虚拟时间的响应。
- [L027] 源不存在、已清除、超有效半径、角度不可见任一成立，返回 `no_signal`。这就是无信号不能直接等同超距的程序证据。
- [L028] 通过上一行接收条件后，距离不超过 5 m 才返回 `near`。本程序中“很近但处于定向背面”仍走上一行的无信号；近距离没有跳过可见性。
- [L029] 其余有信号且不近的情况进入示向分支。
- [L030] `atan2` 求从检测点指向源的真方位，转度；加固定误差后模 360；保留两位小数，写为 `direction` 与 `svd_deg`。
- [L031] 返回完整响应。

一个实现边界：第 30 行先模 360 再四舍五入，极接近 360° 的读数有可能舍入为 360.0。本地策略只转弧度， 0° 与 360° 的几何效果相同；正式客户端却按协议拒绝 360，要求 $[0, 360)$ 。所以不能说两套响应验证逐字一致，也不能把这个边界当作题设保证。

`clear` 契约：位置与频道 → 成功或范围内无目标；移动后检查存在、未清除、距离不超过 20。定向可见性不参与清除判定；清除不改变当前检测频道。

- [L032] 定义清除接口。
- [L033] 先移动；取源；三个条件合取为 `ok`。只要一个为假，就不会成功。
- [L034] 成功时加入已清集合并增加 5 s。
- [L035] 失败时增加 3 s 并计一次失败清除。
- [L036] 返回接受标记、累计时间，以及按 `ok` 选择的结果字符串。

4.4 `make_case`：一个随机测试实例如何生成

契约：整数种子 → 可复现源列表。随机分布是测试设计：源数 10—16、位置对面积均匀、半径均匀、朝向均匀；至少一个定向源与至少一个全向源。返回值供模拟器使用，求解器看不到。

- [L038] 定义单局生成函数。
- [L039] 建独立随机数发生器；等概率抽源数；无放回抽不同频道；抽 1 到 $n-1$ 个定向源；从选中的频道抽定向集合；初始化结果列表。一行共六项准备。
- [L040] 遍历已选频道。
- [L041] 角度在 $[0, 2\pi)$ 均匀；半径取 $1800\sqrt{V}$ ，因为均匀面积满足 $\Pr(r \leq t) = t^2/1800^2$ ；然后转直角坐标。若用 $1800V$ ，会使中心附近过密。
- [L042] 建源记录：半径在 1000—1500 均匀抽取，定向标志来自集合，朝向与误差相位各抽一整圈。
- [L043] 返回源列表。

4.5 `_average`：统计中的分母不能换

契约：每局结果字典列表 → 统计字典，空列表返回 `None`。前提各字典含所访问字段。这里同时报告全部局和成功局，避免只挑成功样本却不说明。

- [L045] 定义统计函数。
- [L046] 空列表不能求平均，返回 `None`。
- [L047] 算局数、真值源总数、已清总数。当前代码中 `sources` 计算后未再使用；答辩应如实称冗余统计变量，不能为它编造后续作用。
- [L048] 收集有定义的各局“时间/已清源数”；零清除局的 `None` 不进入这个分母。
- [L049] 开始返回统计字典。
- [L050] 局数等于输入列表长度。
- [L051] 每局真值源数的算术平均。
- [L052] 每局定向源数平均。
- [L053] 每局已清数平均。
- [L054] 每局总虚拟时间平均。
- [L055] 逐局等权每源耗时：先各局除源数，再对有定义的局取平均。
- [L056] 合并源数加权每源耗时：全部局时间之和除全部已清数；存在零清除失败局时，它的耗时仍在分子中。它通常不等于上一行。
- [L057] 每局移动路程平均。
- [L058] 总移动路程除以速度 5 和局数，得到每局平均移动时间。
- [L059] 检测次数乘 5 再除局数，得到平均检测时间。
- [L060] 换台次数每次 1 s，除局数即平均换台时间。
- [L061] 成功清除 5 s/次、失败 3 s/次，加总后除局数，得到平均清除动作时间。
- [L062] 每局检测次数平均，不是仅正示向次数。
- [L063] 每局换台次数平均。
- [L064] 每局失败清除数平均。
- [L065] 闭合返回字典。正常有效记录中第 58—61 行四类时间之和应等于第 54 行。

`_print_average` 契约：标题与上述统计字典 → 控制台文字，没有统计返回值；空统计安静返回。显示格式的小数截取不会改变 JSON 内原数值。

- [L067] 定义格式化输出函数。
- [L068] 没有统计就不打印。
- [L069] 先打印区分全部局/成功局的标题。
- [L070] 打印平均源数，两位小数。
- [L071] 打印平均定向源数。
- [L072] 打印平均清除数。
- [L073] 打印平均总虚拟时间及秒单位。
- [L074] 逐局等权均值有定义才打印；不要把缺值输出成 0。
- [L075] 合并加权均值有定义才打印。
- [L076] 打印移动距离及米单位。
- [L077] 按移动、检测、换台、清除的固定顺序打印四类时间，以便与总时间相加核对。
- [L078] 打印检测次数平均。
- [L079] 打印换台次数平均。
- [L080] 打印失败清除次数，三位小数便于看很小均值。

4.6 本地 main：固定种子、逐局异常与结果文件

契约：命令行/交互配置 → 多局本地测试 JSON 和控制台统计。每局异常保存在该局记录，不中止后续全部局。`base_seed` 决定一列每局种子；复现单局需使用记录的 `seed` 调 `make_case`，而不是混用两者。

- [L082] 定义批量测试入口。
- [L083] 建命令行参数解析器。
- [L084] 定义 `--loops`、`--seed`、`--output`，均可省略；输出参数解析成 Path；然后取参数。
- [L085] 没给局数就交互询问；空输入默认为 1000。此处没有显式验证负局数，实际使用应传正整数。
- [L086] 没给种子则从系统随机源抽基种子；建立可复现伪随机发生器；建结果表、全清数、实际计时起点。
- [L087] 按局数迭代。
- [L088] 为此局抽独立种子；生成真值；建立独立模拟器，避免跨局状态遗留。

- [L089] 尝试调用 `solver.run_case`；是否全清以模拟器真实已清数与真实源数比较；无异常时 `err=None`。不盲信求解器自报数。
- [L090] 单局异常则保存空策略结果、失败标记和异常文本，后续仍继续。
- [L091] 将布尔成功加到计数；开始追加本局字典，记录编号、种子、源数与定向数。
- [L092] 记录全清标志、真已清数、虚拟时间、按已清数定义的单源均值；零清除时均值为 `None`。
- [L093] 记录路程、检测、换台、失败清除、策略返回值及异常。这里的数据来自模拟器计数，便于审计策略。
- [L094] 计算进度打印间隔：至少 1，最多 50，通常约总局数的 1/20；遇间隔就打印并刷新。
- [L096] 筛出全清局，形成成功子样本。
- [L097] 分别算全部局与成功局统计。
- [L098] 开始构建总汇总字典。
- [L099] 写问题号 4、总局数、基种子。
- [L100] 写全清数与全清率；零局时率为 0 防止除零，但零局不提供成功证据。
- [L101] 保留全部局统计。
- [L102] 保留仅成功局统计。
- [L103] 兼容字段 `mean_virtual_time_s` 取成功局平均总时间；字段名字本身没有写成功局，读报告时必须查这里。
- [L104] `mean_average_time_s` 取成功局的逐局等权单源均值。
- [L105] `pooled_average_time_s` 取成功局合并加权单源均值。
- [L106] 写本机实际运行时间，并保留全部逐局记录，便于复算和定位失败。
- [L107] 闭合汇总字典。
- [L108] 优先用指定输出路径，否则生成带时间戳的 `results/q4_local_...json`；创建父目录。
- [L109] 用中文保留、缩进两空格、UTF-8 写 JSON。它没有使用独占模式，同名指定输出会被覆盖；演练宜使用明确文件名。
- [L111] 打印空行，让进度与最终统计分开。
- [L112] 打印全清局数和百分比。
- [L113] 打印全部局统计。
- [L114] 只要有失败局，再额外打印成功局统计，显示样本口径差异。
- [L115] 打印本机现实运行时长。
- [L116] 打印结果文件位置。
- [L118] 直接执行文件时调用 `main()`；被导入时只定义类和函数，不自动批量测试。

5 两个 Windows 启动文件和依赖声明

5.1 source/Q4/run_local.bat

契约：接收任意命令行附加参数并原样交给本地模拟器，确保当前目录是脚本所在目录；结束时保留 Python 退出码，暂停窗口便于看结果。

- [L001] `@echo off` 关闭命令回显；开头的 `@` 连这一条本身也不回显。
- [L002] `setlocal` 把本次脚本环境变量改动限制在局部作用域。
- [L003] 将控制台编码页设为 UTF-8；`>nul` 隐藏切换成功提示。
- [L004] `pushd "%~dp0"` 保存旧目录并进入此 bat 文件所在盘符/目录；失败则 `exit /b 1`，不在错误目录继续运行。
- [L005] 用 PATH 里的 `python` 执行 `q4_local_simulator.py`；`%*` 将用户传入的全部参数转交，例如 `--loops 3 --seed 2026`。
- [L006] 立即将 `%ERRORLEVEL%` 保存为 `RC`；后续 `popd/pause` 可能改变错误码，不能等到最后才取。
- [L007] 恢复原工作目录。
- [L008] 等待按键，防止双击启动的窗口立即消失。
- [L009] 以保存的 Python 返回码退出本脚本。

5.2 source/Q4/run_official.bat

契约：与本地启动器相同的目录、参数和退出码约定；执行目标换为正式客户端。实际正式测试还要求本机官方接口可用以及合法队号。

- [L001] 关闭命令回显。
- [L002] 建局部环境作用域。
- [L003] 使用 UTF-8 控制台并隐藏编码页提示。
- [L004] 保存旧目录并进入 bat 所在目录；失败以代码 1 退出。
- [L005] 执行 q4_official_solver.py 并转交所有参数，可能包括 --robot-id、--url、--wait。
- [L006] 保存 Python 的退出码。
- [L007] 恢复调用前目录。
- [L008] 暂停让人阅读输出。
- [L009] 用保存的退出码返回。正式 main 的 0/1 因此能被外部脚本观察。

5.3 source/requirements.txt

- [L001] 注释声明需要 Python 3.10 或以上；如 Point|None 等类型语法与此版本要求相配。
- [L002] 注释声明程序只用标准库，无第三方包需要安装。注释行不会让 pip 自动升级 Python；应自行检查 python --version。

6 有确定数据的手算训练与标准答案

以下是依照本题程序设计的应用练习，数据为本章自编，不冒充外部教材原例。先遮住答案算一遍，再按“输入—所用条件—代入公式—得到结论—指出源码行”复述；每题都给完整答案。

题 1 “很近”为什么仍可能无信号

已知。定向源 $g = (100, 0)$ ，有效半径 1000，朝向 $n = (-1, 0)$ 。考察检测点 $A = (0, 0)$ 、 $B = (101, 0)$ 、 $C = (-1001, 0)$ ，尚未清除。要求。对每点判定能否接收、返回哪类结果；再判断在 B 执行清除能否成功。不要求计算具体示向误差。

标准答案。在 A ，距离 100，点积 $n^T(A - g) = 100 \geq 0$ ，因此接收且距离大于 5，返回 direction。在 B ，距离虽只有 1，但点积为 -1 ，处于背向，返回 no_signal。在 C ，点积为 1101，角度可见，但距离 1101 超过有效半径，仍返回 no_signal。所以两次相同字符串可能来自不同原因。在 B 清除则距离 1 小于 20、源存在且未清，成功；清除没有角度条件。

源码对应。模拟器 L018–L020 角度判定，L027 先判断接收失败，L028 再判断近距离，L033 清除只用距离与存在性。不能交换 L027、L028 的语义。

题 2 手工执行一次 clip

已知。顺时针或逆时针有序矩形顶点为 $(-1, -1)$ 、 $(3, -1)$ 、 $(3, 2)$ 、 $(-1, 2)$ ，保留半平面 $x \leq 1$ 。先忽略 EPS。要求。列出每条边的两端余量、是否产生交点、最终顶点；再说明实际代码的边界差异。

标准答案。 $n = (1, 0)$ 、 $b = 1$ ，余量 $f(x, y) = x - 1$ 。

正在处理的有向边	起点/终点余量	输出动作
$(-1, 2) \rightarrow (-1, -1)$	$-2, -2$	都在内，加终点 $(-1, -1)$
$(-1, -1) \rightarrow (3, -1)$	$-2, 2$	内到 外， $t = (-2)/(-4) = 1/2$ ，加 $(1, -1)$
$(3, -1) \rightarrow (3, 2)$	$2, 2$	都在外，不加

$(3, 2) \rightarrow (-1, 2)$

$2, -2$

外到内, $t = 1/2$, 先加 $(1, 2)$,
再加 $(-1, 2)$

返回四点 $(-1, -1), (1, -1), (1, 2), (-1, 2)$ 。实际 L026 将 b 改为 $1 + 10^{-7}$, 两个交点的横坐标也变为 $1 + 10^{-7}$ 。这个差异是保守外扩, 不是另一种几何算法。

源码对应。L026 预设闭合首边, L029 判断跨越, L030 插值, L031 保留内侧终点, L035-L036 去重。

题 3 从最小包围圆推到最近清除点

已知 A。点集 $(0, 0), (4, 0), (0, 3)$ 。要求 A。求理论最小包围圆并说明为什么不是任选最远点中点猜答案。

标准答案 A。三角形在 0 点为直角, 最远两点为 $(4, 0), (0, 3)$, 距离 5, 因此任何包围圆半径至少为 $5/2$ 。以它们的中点 $(2, 1.5)$ 为圆心时, 第三点距离也为 $\sqrt{4 + 2.25} = 2.5$, 所以达到了下界, 圆最小。程序 mec 返回半径 2.500001, 多出的 10^{-6} 来自 L076。只有验证第三点也被覆盖后, 才能在本例把直径圆当最小包围圆; 一般锐角三角形可能需要三点支撑。

已知 B。另一个频道的包围圆心 $c = (0, 0)$, 半径 $r = 12$, 当前位置 $p = (30, 0)$ 。要求 B。求最近认证清除点、需走距离、最坏源距上界。

标准答案 B。允许清除中心偏移为 $s = 19.8 - 12 = 7.8$ 。当前点不在安全圆盘内, 投影为 $q = c + s(p - c)/\|p - c\| = (7.8, 0)$ 。移动 22.2 m; 最坏源距 $\|q - c\| + r = 7.8 + 12 = 19.8 < 20$ 。直接走到圆心会多走 7.8 m。对应 L079-L083。

题 4 把双负公式完整代入一次

已知。正锚点 $a = (0, 0)$, 示向 0° , $U = 1200$, 参数与程序相同。已有多边形在 $T(1200)$ 内。要求。求两个探测点、横跨条件、距离保证、双负后的半平面与新 U 上界。假定多边形最大距离一项没有给出更小上界。

标准答案。 $u = (1, 0), v = (0, 1)$, 故 $q_{\pm} = (600, \pm 60)$ 。探测线处每侧角楔半宽为 $600 \tan(1.0051^\circ) = 10.52646 \dots < 60$, 横跨成立。任一候选点至探测点的距离不超过

$$1200 \sqrt{0.25 + (0.05 + \tan 1.0051^\circ)^2} = 605.44990 \dots < 1000.$$

双负不可能由超距造成; 配合正锚点, 保留 $x \leq 600$ 。径向上界为

$$U^+ \leq \frac{600}{\cos 1.0051^\circ} = 600.09233157 \dots$$

注意 U 是距离上界, 故不是简单赋值 600; $x \leq 600$ 是轴向条件。旧 anchor 保持 $(0, 0)$ 。last 则变为最后一个负点, 顺序由离当前位置远近决定。对应 L155-L169。

题 5 一个负点足够吗: 必须会给出反例

已知。 $a = (0, 0)$, 真实源 $g = (800, 0)$, $U = 1500$, 有效半径 1000, $n = (-0.01, -\sqrt{1 - 0.01^2})$ 。候选点 $q_+ = (750, 75), q_- = (750, -75)$ 。要求。验证正锚点、两个测点接收情况, 并判断见到首个无信号能否保留 $x \leq 750$ 。

标准答案。 $n^T(a - g) = 8 > 0$, 锚点角度可见且距源 800, 确为可接收位置。两个探测点距源均为 $\sqrt{50^2 + 75^2} \approx 90.139 < 1000$; $n^T(q_+ - g) = 0.5 - 75\sqrt{0.9999} \approx -74.496 < 0$, 所以它无信号; 另一个点积约 $75.496 > 0$, 有信号。若只见首点无信号就裁 $x \leq 750$, 会删除真实源 $x = 800$ 。因此必须再测另一点; 它转正就更新锚点并结束本轮, 不能执行双负更新。对应 L166-L169。

题 6 25 站证明中，哪一个数真正控制外圈覆盖

已知。外圈有 16 个等角点，原半径 1838；拟将半径改为 1800，保持全部其他步骤。要求。判断是否还能原样引用“外圈多边形包住目标圆域”的证明；指出准确计算。

标准答案。原内切半径为 $1838 \cos(\pi/16) = 1802.683345 \dots > 1800$ ，所以原证明成立。改后内切半径为 $1800 \cos(\pi/16) = 1765.413505 \dots < 1800$ ，在每条边中垂线方向会有合法源位置落在多边形外，因此原三角剖分证明不再覆盖整个目标圆域。这个反证针对的是“原证明仍可直接使用”，不是单凭这一项就断言任何修改方案必然漏检。若要改站点，需重新证明所有连续位置与方向的覆盖。对应 coverage_route L086-L089。

题 7 手算一次在线插入评分

已知。当前点 $p = (0, 0)$ 、下一站 $s = (1000, 0)$ ；已发现任务 A 的中心 $(500, 100)$ 、半径 30，任务 B 的中心 $(100, 500)$ 、半径 10。要求。算两个分数，决定是否插入以及插入谁；说明分数是否等于真实服务时间。

标准答案。A 分数为

$$2\sqrt{500^2 + 100^2} - 1000 + 30 = 49.80390 \dots$$

B 分数为

$$\sqrt{100^2 + 500^2} + \sqrt{900^2 + 500^2} - 1000 + 10 = 549.46496 \dots$$

两者均未超过 600，选择较小的 A。分数单位是米；不包含未来到底需要几次测向、换台、清除点最终在哪里。因此它是选择顺序的代理评分，不能称“已算出真实最小服务时间”。对应 L183-L187。

题 8 累计时间如何从 0 变成 32 s

已知。初始位置原点、当前频道 1；频道 3 有源 $(100, 0)$ 、半径 1000、朝向正西，误差相位 0。依次执行 `measure((0,0),3)`、`measure((80,0),3)`、`clear((80,0),3)`。要求。给出每步累计时间、结果类别，最后路程和三类动作计数；指出策略应怎样读时间。

标准答案。

动作	移动时间	检测/清除时间	换台时间	累计时间	结果
原点检测频道 3	0	5	1	6	direction, 本地示向 0.39°
(80,0) 检测频道 3	$80/5 = 16$	5	0	27	direction, 本地示向 0.60°
原地清除频道 3	0	5	0	32	success, 距离恰为 20

总路程 80 m，检测 2 次，换台 1 次，成功清除 1 次，失败 0 次。策略 `actions=3`，虚拟时间直接依次赋成返回的 6、27、32；不能再给 32 加检测或清除时间。模拟器 L017、L024-L026、L033-L036；策略 L116、L124。

题 9 “已经清除了 10 个” 能不能直接结束

情况 A。清除了 10 个，其他 10 个仍 UNKNOWN，覆盖只完成 5 站。情况 B。16 个不同频道已为 FOUND 或 CLEARED，还有未知频道。情况 C。25 站都完成，剩余未知频道从未接收过。要求。逐一判断允许怎样改状态、还有什么工作。

标准答案。A 不能结束；题设至少 10 不等于恰好 10，仍可能有第 11—16 个源，应继续覆盖。B 可以把未知全部改为 ABSENT，因为已达到最大源数；但所有 FOUND 仍需服务清除。C 在覆盖证明前提和完整扫描成立时可把未知改为 ABSENT；已有 FOUND 仍需清除。只有既无 FOUND 又无 UNKNOWN，L196 才 break。因此 coverage_done=True 与“本局已全部清除”不是同一个字段。

题 10 HTTP 超时后为什么不能换一个请求号再试

已知。客户端已入场，请求号 abc-7 请求清除。服务器可能执行了，但响应读取超时。所有内部重试仍无有效响应；现实预算还充足。要求。说明请求号、pending、位置、退出资格各是什么；再比较“每次都是有效的 503 且 accepted=False”的情况。

标准答案。内部重试一直发送同一个 abc-7 和相同 body；unknown=True。重试耗尽后抛 ResultUnknown，pending 保留原请求；由于外层 action 没有拿到有效返回，客户端位置不作推测更新。新的 action 会在 L326 拒绝；can_exit 因 pending 非空为假，主流程不会发新退出请求。这个选择保留了“不知道执行没执行”的真实状态。

若每次都得到结构可信且明确拒绝的 503，并且此前没有别的未知尝试，则只累积已知拒绝，最终 pending 清空、抛 ConnectionError。是否能补退出还要同时满足已入场、未尝试退出、未确认退出和时间充足。不能只看 pending 一个条件。对应正式 L286–L323、L326、L341–L343、L361。

题 11 两种“平均每源时间”为什么不相等

已知。两局都全清：第一局 10 源、100 s；第二局 16 源、400 s。要求。求逐局等权均值和合并加权均值；再增加一局清除 0 源、耗时 50 s 的失败局，求全部局统计中的这两个量。

标准答案。两局的单源耗时为 10 和 25 s。逐局等权均值 $(10 + 25)/2 = 17.5$ s/源；合并均值 $(100 + 400)/(10 + 16) = 19.230769 \dots$ s/源。加入失败局后，它的单局均值为 None，L048 会略过，因此“全部局的逐局有定义均值”仍为 17.5；而合并均值为 $550/26 = 21.153846 \dots$ 。成功局统计仍只用前两局。必须报告口径，否则不同数值都叫平均值会产生误解。对应模拟器 L048、L055–L056、L096–L105。

题 12 复用时间究竟比较哪个测点

已知。正锚点为 $A = (0, 0)$ ；随后在 $B = (100, 0)$ 检测无信号；现在在 $C = (110, 0)$ ，该频道仍 FOUND，候选半径 30，所有候选点距 C 不超过 999。要求。判断是否被 _reuse 选中，以及此时 anchor, last 各是谁。

标准答案。anchor=A，因为无信号不产生正锚点；last=B，因为每次检测都会更新最近点。 $\|C - B\| = 10 < 25$ ，故 L144 跳过。不能拿 $\|C - A\| = 110$ 去声称满足复测基线。即使超过 25 m，也只能称允许一次机会复测，不能保证误差独立或示向交角足够。

7 最后用这 12 个口头问题自测

每题都先说完整句，再打开源码定位。答案不是要求逐字背诵，但不能丢掉条件。

1. Q4 比 Q3 多了哪个未知量？定向源的辐射方向；导致无信号可能是背向。正观测仍给位置角楔，普通负观测不再直接当距离排除。定位：模拟器 L020、L027，策略 L128。
2. 25 站是怎样证明的？外圈多边形包住目标圆域；剖分三角形所有边小于 1000；任意源所在三角形的近顶点凸包包含源，所以任意朝向至少接收一个顶点。必须同时有距离和方向两部分。定位：策略 L085–L089。
3. 外圈 1838 超出目标圆合法不合法？目标被限制在 1800 圆内；机器狗动作坐标使用接口的大范围坐标条件，两者不是同一个约束。定位：正式 L331。
4. 为什么正锚点一定保留？双负矛盾证明需要 $n^T(a - g) \geq 0$ 。负点没有这个条件，不能把最近检测点直接当正锚点。定位：L112、L124、L169。
5. 两点都无信号就能剪吗？必须先证明两个点都在最小接收半径内、连线横跨角楔、正锚点仍有效，且没有额外漏检。任意两个负点不够。定位：L162–L169。
6. U 为何除一次余弦？轴向界 $x \leq \lambda U$ 与角楔合起来只推出径向界 $\|g - a\| \leq \lambda U / \cos \delta$ 。定位：L158。

7. 成对方法为何会停止？正分支和双负分支都缩小尺度；尺度给包围圆半径上界，最后小于 19.8。程序 9 轮限额是异常保护；理论界与浮点工程实现需分开说。定位：L174–L178。
8. 圆心可能不是真值，怎么还敢清？清除证书对全部候选点成立；清除点在 $B(c, 19.8 - r)$ 内，三角不等式保证到真值小于 20。定位：L078–L083、L175。
9. 哪些部分是启发式？访问顺序、按半径复用、25 m 门槛、插入分数与 600 阈值；它们决定效率，没有最优性证明。发现与清除的几何条件不能和这些混为一谈。定位：L088、L143–L147、L183–L187。
10. 为什么不能重复在同一点测向取平均？本地误差函数同源同地固定；重复不会提供独立噪声样本。策略采用位移与集合交约束。定位：模拟器 L022，策略 L144。
11. 正式重试怎样避免重复动作？同一请求号与 body 重试，并依赖接口按请求号处理重复请求；结果未知保留 pending，禁止新动作。定位：正式 L285–L323、L326。
12. 本地 1000 局全清是否证明最快？它只是指定生成分布、种子、误差模型下的实验；全清的几何证明另列，最快需要与其他策略同案例比较或最优性证明。定位：模拟器 L038–L043、L099–L106。

8 行号覆盖核验与学习完成标准

本章对应的源码未被修改。当前文件中，本地策略有 195 个非空物理行；正式版新增部分有 178 个非空行；模拟器有 109 个非空行；两份 bat 各 9 个；依赖文件 2 个，合计 502 个非重复非空行。正式版前 208 个物理行与本地策略全文相同。空行负责排版，不执行计算；注释和 shebang 已在对应条目中解释。

本章的完成标准不是“看过 502 条”。应实际完成三次不同的复述：

- 第一遍讲模型：不看代码，说出 (Q4-1) 到 (Q4-6) 各自的已知条件、未知量、结论，并手算题 1、4、5。
- 第二遍讲状态：打开 `_observe` → `_positive/_pair_round` → `_dual_negative` → `_service`，逐句说清 `poly/anchor/last/U/status` 哪里改变、哪里保持；手算题 8、9、12。
- 第三遍讲实现：对 `clip`、`mec`、HTTP pending 和统计分母随机抽行，解释变量、分支与异常；做题 2、3、10、11。

复述若卡住，应回到对应公式与完整例题补条件，再读语句。不能用“这里调用某算法”代替说明为什么此处输入满足算法前提，也不能用“最终仿真成功”代替不变量与清除证书。

附录 带物理行号的原始源码

前面每个 [Lxxx] 对应这里的原始物理行。空行也列出；Q3/Q4 正式版与本地版相同的前缀只印一次。长行在纸面折行时仍属于同一物理行。

source/Q1/q1_solver.py

```
0001 |#!/usr/bin/env python3
0002 |"""问题一：有界误差 AOA 半平面交与凸多边形直径；只求解，不绘图。"""
0003 |from __future__ import annotations
0004 |
0005 |import argparse
0006 |from collections import deque
0007 |from dataclasses import dataclass
0008 |import json
0009 |import math
0010 |from pathlib import Path
0011 |from typing import Iterable
0012 |
0013 |Point = tuple[float, float]
0014 |EPS = 1.0e-10
0015 |PARALLEL_EPS = 1.0e-12
0016 |NUMERICAL_ANGLE_GUARD_DEG = 0.0
0017 |
0018 |
0019 |@dataclass(frozen=True)
0020 |class HalfPlane:
0021 |    """闭半平面 ``normal · X <= offset``。"""
0022 |
0023 |    normal: Point
0024 |    offset: float
0025 |
0026 |    def residual(self, point: Point) -> float:
0027 |        return self.normal[0] * point[0] + self.normal[1] * point[1] - self.offset
0028 |
0029 |
0030 |def _cross(a: Point, b: Point) -> float:
0031 |    return a[0] * b[1] - a[1] * b[0]
0032 |
0033 |
0034 |def _sub(a: Point, b: Point) -> Point:
0035 |    return a[0] - b[0], a[1] - b[1]
0036 |
0037 |
0038 |def _distance2(a: Point, b: Point) -> float:
0039 |    dx, dy = a[0] - b[0], a[1] - b[1]
0040 |    return dx * dx + dy * dy
0041 |
0042 |
0043 |def _normalise(raw) -> HalfPlane:
0044 |    if isinstance(raw, HalfPlane):
0045 |        a, b = raw.normal
0046 |        c = raw.offset
0047 |        elif len(raw) == 2:
0048 |            (a, b), c = raw
0049 |        elif len(raw) == 3:
0050 |            a, b, c = raw
0051 |        else:
0052 |            raise ValueError("半平面应写为 HalfPlane、((a,b),c) 或 (a,b,c)")
0053 |    a, b, c = float(a), float(b), float(c)
0054 |    if not all(math.isfinite(v) for v in (a, b, c)):
0055 |        raise ValueError("半平面系数必须为有限数")
```

```

0056 | length = math.hypot(a, b)
0057 | if length == 0.0:
0058 |     raise ValueError("半平面法向量不能为零")
0059 | return HalfPlane((a / length, b / length), c / length)
0060 |
0061 |
0062 | def _direction(hp: HalfPlane) -> Point:
0063 |     # 沿该方向前进时，可行域位于有向边界左侧。
0064 |     a, b = hp.normal
0065 |     return -b, a
0066 |
0067 |
0068 | def _same_direction(a: HalfPlane, b: HalfPlane) -> bool:
0069 |     da, db = _direction(a), _direction(b)
0070 |     return abs(_cross(da, db)) <= PARALLEL_EPS and da[0] * db[0] + da[1] * db[1] > 0
0071 |
0072 |
0073 | def _ordered(halfplanes: Iterable[HalfPlane]) -> list[HalfPlane]:
0074 |     planes = [_normalise(hp) for hp in halfplanes]
0075 |     planes.sort(key=lambda hp: math.atan2(_direction(hp)[1], _direction(hp)[0]) % math.tau)
0076 |     merged: list[HalfPlane] = []
0077 |     for hp in planes:
0078 |         if merged and _same_direction(merged[-1], hp):
0079 |             if hp.offset < merged[-1].offset:
0080 |                 merged[-1] = hp
0081 |             else:
0082 |                 merged.append(hp)
0083 |     if len(merged) > 1 and _same_direction(merged[0], merged[-1]):
0084 |         if merged[-1].offset < merged[0].offset:
0085 |             merged[0] = merged[-1]
0086 |         merged.pop()
0087 |     return merged
0088 |
0089 |
0090 | def _intersection(a: HalfPlane, b: HalfPlane) -> Point | None:
0091 |     ax, ay = a.normal
0092 |     bx, by = b.normal
0093 |     determinant = ax * by - ay * bx
0094 |     if abs(determinant) <= PARALLEL_EPS:
0095 |         return None
0096 |     return ((a.offset * by - ay * b.offset) / determinant,
0097 |            (ax * b.offset - a.offset * bx) / determinant)
0098 |
0099 |
0100 | def _outside(hp: HalfPlane, point: Point) -> bool:
0101 |     scale = max(1.0, abs(hp.offset), abs(point[0]), abs(point[1]))
0102 |     return hp.residual(point) > EPS * scale
0103 |
0104 |
0105 | def _cuts(hp: HalfPlane, a: HalfPlane, b: HalfPlane) -> bool:
0106 |     point = _intersection(a, b)
0107 |     return point is not None and _outside(hp, point)
0108 |
0109 |
0110 | def _feasible_point(planes: list[HalfPlane]) -> Point | None:
0111 |     """二维线性约束可行性；只用于区分空集与无界集。"""
0112 |     point = (0.0, 0.0)
0113 |     for i, hp in enumerate(planes):
0114 |         if not _outside(hp, point):
0115 |             continue
0116 |         nx, ny = hp.normal
0117 |         base, tangent = (nx * hp.offset, ny * hp.offset), (-ny, nx)
0118 |         lower, upper = -math.inf, math.inf
0119 |         for previous in planes[:i]:
0120 |             coefficient = previous.normal[0] * tangent[0] + previous.normal[1] * tangent[1]
0121 |             bound = previous.offset - (previous.normal[0] * base[0] + previous.normal[1] * base[1])
0122 |             if coefficient > PARALLEL_EPS:
0123 |                 upper = min(upper, bound / coefficient)
0124 |             elif coefficient < -PARALLEL_EPS:
0125 |                 lower = max(lower, bound / coefficient)
0126 |             elif bound < -EPS * max(1.0, abs(previous.offset)):
0127 |                 return None
0128 |         if lower > upper + EPS * max(1.0, abs(lower), abs(upper)):
0129 |             return None
0130 |         parameter = min(max(0.0, lower), upper)
0131 |         point = (base[0] + parameter * tangent[0], base[1] + parameter * tangent[1])
0132 |     return point
0133 |
0134 |

```

```

0135 | def _is_bounded(planes: list[HalfPlane]) -> bool:
0136 |     if len(planes) < 3:
0137 |         return False
0138 |     angles = sorted(math.atan2(hp.normal[1], hp.normal[0]) % math.tau for hp in planes)
0139 |     gaps = [angles[i + 1] - angles[i] for i in range(len(angles) - 1)]
0140 |     gaps.append(angles[0] + math.tau - angles[-1])
0141 |     return max(gaps) < math.pi - PARALLEL_EPS
0142 |
0143 |
0144 | def _convex_hull(points: Iterable[Point]) -> list[Point]:
0145 |     rows = sorted(set((float(x), float(y)) for x, y in points))
0146 |     if len(rows) <= 1:
0147 |         return rows
0148 |
0149 |     def chain(sequence):
0150 |         result: list[Point] = []
0151 |         for point in sequence:
0152 |             while len(result) >= 2 and _cross(_sub(result[-1], result[-2]), _sub(point, result[-1])) <= EPS:
0153 |                 result.pop()
0154 |             result.append(point)
0155 |         return result
0156 |
0157 |     return chain(rows)[-1:] + chain(reversed(rows))[-1:]
0158 |
0159 |
0160 | def polygon_diameter(vertices: Iterable[Point]):
0161 |     """用旋转卡壳返回 `` (直径, 最远点对) ``。"""
0162 |     points = _convex_hull(vertices)
0163 |     count = len(points)
0164 |     if count == 0:
0165 |         return None, None
0166 |     if count == 1:
0167 |         return 0.0, (points[0], points[0])
0168 |     if count == 2:
0169 |         return math.dist(*points), (points[0], points[1])
0170 |     best2, best_pair, j = 0.0, (points[0], points[0]), 1
0171 |     for i in range(count):
0172 |         following = (i + 1) % count
0173 |         edge = _sub(points[following], points[i])
0174 |         while (_cross(edge, _sub(points[(j + 1) % count], points[i]))
0175 |               > _cross(edge, _sub(points[j], points[i])) + EPS):
0176 |             j = (j + 1) % count
0177 |         for first in (i, following):
0178 |             candidate = _distance2(points[first], points[j])
0179 |             if candidate > best2:
0180 |                 best2, best_pair = candidate, (points[first], points[j])
0181 |     return math.sqrt(best2), best_pair
0182 |
0183 |
0184 | def _finite_result(vertices: list[Point]) -> dict:
0185 |     vertices = _convex_hull(vertices)
0186 |     if len(vertices) == 2 and _distance2(vertices[0], vertices[1]) <= EPS * EPS:
0187 |         vertices = vertices[:1]
0188 |     status = "point" if len(vertices) == 1 else "segment" if len(vertices) == 2 else "polygon"
0189 |     diameter, pair = polygon_diameter(vertices)
0190 |     area = abs(sum(_cross(vertices[i], vertices[(i + 1) % len(vertices)])
0191 |                   for i in range(len(vertices)))) / 2.0
0192 |     midpoint = ((pair[0][0] + pair[1][0]) / 2.0, (pair[0][1] + pair[1][1]) / 2.0)
0193 |     radius = diameter / 2.0
0194 |     farthest = max(math.dist(midpoint, point) for point in vertices)
0195 |     return {"status": status, "vertices_m": [list(p) for p in vertices],
0196 |            "diameter_m": diameter, "diameter_pair_m": [list(p) for p in pair],
0197 |            "area_m2": area, "diameter_circle_covers": farthest <= radius + EPS,
0198 |            "diameter_circle_cover_margin_m": radius - farthest}
0199 |
0200 |
0201 | def intersect_halfplanes(halfplanes: Iterable[HalfPlane], *, include_circle=True) -> dict:
0202 |     """用按方向排序的双端队列计算半平面交。"""
0203 |     del include_circle # 仅保留该参数以兼容第二问的调用接口。
0204 |     planes = _ordered(halfplanes)
0205 |     active: deque[HalfPlane] = deque()
0206 |     for hp in planes:
0207 |         while len(active) >= 2 and _cuts(hp, active[-2], active[-1]):
0208 |             active.pop()
0209 |         while len(active) >= 2 and _cuts(hp, active[0], active[1]):
0210 |             active.popleft()
0211 |         active.append(hp)
0212 |     while len(active) >= 3 and _cuts(active[0], active[-2], active[-1]):
0213 |         active.pop()

```

```

0214 | while len(active) >= 3 and _cuts(active[-1], active[0], active[1]):
0215 |     active.popleft()
0216 |
0217 | ring = list(active)
0218 | vertices = [_intersection(ring[i], ring[(i + 1) % len(ring)]) for i in range(len(ring))]
0219 | if ring and all(point is not None for point in vertices):
0220 |     polygon = _convex_hull(vertices)
0221 |     # A bounded intersection may legitimately degenerate to one point or a
0222 |     # line segment. Accept those only when the recession cone is bounded;
0223 |     # otherwise two adjacent half-planes could be mistaken for a point.
0224 |     finite_shape = len(polygon) >= 3 or (polygon and _is_bounded(planes))
0225 |     if finite_shape and all(not _outside(hp, point) for hp in planes for point in polygon):
0226 |         return _finite_result(polygon)
0227 |
0228 | witness = _feasible_point(planes)
0229 | if witness is None:
0230 |     return {"status": "empty", "vertices_m": [], "diameter_m": None,
0231 |            "diameter_pair_m": None, "area_m2": None}
0232 | if not _is_bounded(planes):
0233 |     return {"status": "unbounded", "vertices_m": [], "diameter_m": None,
0234 |            "diameter_pair_m": None, "area_m2": None,
0235 |            "feasible_point_m": list(witness)}
0236 | return {"status": "uncertain", "vertices_m": [], "diameter_m": None,
0237 |        "diameter_pair_m": None, "area_m2": None}
0238 |
0239 |
0240 | def _observation(row) -> tuple[float, float, float]:
0241 |     if isinstance(row, dict):
0242 |         position = row.get("position_m", row.get("position"))
0243 |         if isinstance(position, dict):
0244 |             position = (position["x"], position["y"])
0245 |         bearing = row.get("bearing_deg", row.get("svd_deg"))
0246 |         x, y = position
0247 |     else:
0248 |         if len(row) != 3:
0249 |             raise ValueError("每条观测必须包含 x、y 和示向度")
0250 |         x, y, bearing = row
0251 |     x, y, bearing = float(x), float(y), float(bearing)
0252 |     if not all(math.isfinite(v) for v in (x, y, bearing)):
0253 |         raise ValueError("观测数据必须为有限数")
0254 |     return x, y, bearing % 360.0
0255 |
0256 |
0257 | def _wedge(position: Point, bearing_deg: float, epsilon_deg: float) -> list[HalfPlane]:
0258 |     x, y = position
0259 |     lower, upper = map(math.radians, (bearing_deg - epsilon_deg, bearing_deg + epsilon_deg))
0260 |     normals = [(math.sin(lower), -math.cos(lower)),
0261 |               (-math.sin(upper), math.cos(upper))]
0262 |     if epsilon_deg == 0.0:
0263 |         angle = math.radians(bearing_deg)
0264 |         normals.append((-math.cos(angle), -math.sin(angle)))
0265 |     return [HalfPlane(normal, normal[0] * x + normal[1] * y) for normal in normals]
0266 |
0267 |
0268 | def solve_localization(observations, epsilon_deg=1.0, *, include_circle=True) -> dict:
0269 |     epsilon_deg = float(epsilon_deg)
0270 |     if not 0.0 <= epsilon_deg < 90.0:
0271 |         raise ValueError("epsilon_deg 必须位于 [0,90) 内")
0272 |     rows = [_observation(row) for row in observations]
0273 |     if not rows:
0274 |         raise ValueError("至少需要一条观测")
0275 |     planes = [hp for x, y, angle in rows for hp in _wedge((x, y), angle, epsilon_deg)]
0276 |     result = intersect_halfplanes(planes, include_circle=include_circle)
0277 |     result.update(observations=[{"position_m": [x, y], "bearing_deg": angle}
0278 |                                for x, y, angle in rows],
0279 |                  error_half_width_deg=epsilon_deg)
0280 |     return result
0281 |
0282 |
0283 | def main(argv=None) -> int:
0284 |     parser = argparse.ArgumentParser(description=__doc__)
0285 |     parser.add_argument("--input", type=Path, required=True, help="观测 JSON 文件")
0286 |     parser.add_argument("--output", type=Path, default=Path("q1_result.json"))
0287 |     args = parser.parse_args(argv)
0288 |     payload = json.loads(args.input.read_text(encoding="utf-8"))
0289 |     result = solve_localization(payload["observations"], payload.get("epsilon_deg", 1.0))
0290 |     result["case_id"] = payload.get("case_id", args.input.stem)
0291 |     args.output.parent.mkdir(parents=True, exist_ok=True)
0292 |     args.output.write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")

```

```

0293 | print(f"状态: {result['status']}")
0294 | print(f"顶点数: {len(result['vertices_m'])}")
0295 | print(f"面积: {result['area_m2']:.3f} m²" if result["area_m2"] is not None else "面积: 无有限值")
0296 | print(f"直径: {result['diameter_m']:.3f} m" if result["diameter_m"] is not None else "直径: 无有限值")
0297 | print(f"结果已写入: {args.output}")
0298 | return 0
0299 |
0300 |
0301 | if __name__ == "__main__":
0302 |     raise SystemExit(main())

```

source/Q1/q1_generator.py

```

0001 | #!/usr/bin/env python3
0002 | from __future__ import annotations
0003 | import argparse,json,math,random
0004 | from pathlib import Path
0005 |
0006 | ARENA_R=1800.0
0007 |
0008 | def main():
0009 |     ap=argparse.ArgumentParser(description='问题一随机测试数据生成器')
0010 |     ap.add_argument('--seed',type=int,default=None)
0011 |     ap.add_argument('--points',type=int,default=4)
0012 |     ap.add_argument('--output',type=Path,default=Path('q1_input.json'))
0013 |     args=ap.parse_args()
0014 |     seed=args.seed if args.seed is not None else random.SystemRandom().randrange(1,2**63)
0015 |     rng=random.Random(seed)
0016 |     n=max(3,args.points)
0017 |     ang=rng.random()*math.tau
0018 |     rr=ARENA_R*math.sqrt(rng.random())*0.82
0019 |     source=(rr*math.cos(ang),rr*math.sin(ang))
0020 |     observations=[]
0021 |     base=rng.random()*math.tau
0022 |     for k in range(n):
0023 |         a=base+2*math.pi*k/n+rng.uniform(-0.20,0.20)
0024 |         d=rng.uniform(500.0,1450.0)
0025 |         p=(round(source[0]+d*math.cos(a),6),round(source[1]+d*math.sin(a),6))
0026 |         true=math.degrees(math.atan2(source[1]-p[1],source[0]-p[0]))%360
0027 |         # The simulator reports two decimals. Rejection after quantisation
0028 |         # guarantees that the generated final datum still obeys the  $\pm 1^\circ$  bound.
0029 |         while True:
0030 |             report=round((true+rng.uniform(-1.0,1.0))%360,2)%360.0
0031 |             final_error=(report-true+180.0)%360.0-180.0
0032 |             if abs(final_error)<=1.0:break
0033 |             observations.append({'position_m':[p[0],p[1]],'bearing_deg':report})
0034 |         payload={'case_id':f'Q1-seed-{seed}','seed':seed,'epsilon_deg':1.0,
0035 |                 'observations':observations,
0036 |                 'local_truth':{'source_m':[source[0],source[1]]}}
0037 |         args.output.parent.mkdir(parents=True,exist_ok=True)
0038 |         args.output.write_text(json.dumps(payload,ensure_ascii=False,indent=2),encoding='utf-8')
0039 |         print(f"随机种子: {seed}")
0040 |         print(f"检测点数量: {n}")
0041 |         print(f"测试数据已保存: {args.output}")
0042 |
0043 | if __name__ == '__main__':main()

```

source/Q1/run.bat

```

0001 | @echo off
0002 | setlocal
0003 | chcp 65001 >nul
0004 | pushd "%~dp0" || exit /b 1
0005 | python q1_generator.py --output q1_input.json
0006 | if errorlevel 1 goto finish
0007 | python q1_solver.py --input q1_input.json --output q1_result.json
0008 | :finish
0009 | set "RC=%ERRORLEVEL%"
0010 | popd
0011 | pause
0012 | exit /b %RC%

```

source/Q2/q2_solver.py

```
0001 |#!/usr/bin/env python3
0002 |"""Q2 robust second-station design with D, MEC radius R, and area A.
0003 |
0004 |The solver uses only bounded-error geometry. It does not assign a probability
0005 |distribution to bearing errors and does not average repeated measurements.
0006 |All disk boundaries are represented by circumscribed tangent polygons, so the
0007 |reported feasible polygons and their D/R/A values are conservative outer
0008 |approximations for each evaluated report. The maximum over finitely sampled
0009 |reports is a reproducible numerical approximation, not a certified continuous
0010 |worst-case bound. Units are metres, seconds, and degrees.
0011 |"""
0012 |from __future__ import annotations
0013 |
0014 |from dataclasses import asdict, dataclass
0015 |import argparse
0016 |import csv
0017 |import json
0018 |import math
0019 |from pathlib import Path
0020 |import random
0021 |import time
0022 |from typing import Iterable, Sequence
0023 |
0024 |
0025 |Point = tuple[float, float]
0026 |DELTA_DEG = 1.0
0027 |R_MIN = 1000.0
0028 |R_MAX = 1500.0
0029 |ARENA_R = 1800.0
0030 |EPS = 1e-9
0031 |
0032 |
0033 |def add(a: Point, b: Point) -> Point:
0034 |    return a[0] + b[0], a[1] + b[1]
0035 |
0036 |
0037 |def sub(a: Point, b: Point) -> Point:
0038 |    return a[0] - b[0], a[1] - b[1]
0039 |
0040 |
0041 |def mul(a: Point, k: float) -> Point:
0042 |    return a[0] * k, a[1] * k
0043 |
0044 |
0045 |def dot(a: Point, b: Point) -> float:
0046 |    return a[0] * b[0] + a[1] * b[1]
0047 |
0048 |
0049 |def cross(a: Point, b: Point) -> float:
0050 |    return a[0] * b[1] - a[1] * b[0]
0051 |
0052 |
0053 |def dist(a: Point, b: Point) -> float:
0054 |    return math.hypot(a[0] - b[0], a[1] - b[1])
0055 |
0056 |
0057 |def local_to_world(first: Point, bearing_deg: float, a: float, b: float) -> Point:
0058 |    t = math.radians(bearing_deg)
0059 |    return (first[0] + a * math.cos(t) - b * math.sin(t),
0060 |            first[1] + a * math.sin(t) + b * math.cos(t))
0061 |
0062 |
0063 |def _sector_max_distance_sq(s: Point, radius: float, delta_deg: float) -> float:
0064 |    """max ||s-r*u(eta)||^2 for 0<=r<=radius, |eta|<=delta."""
0065 |    a, b = s
0066 |    length2 = a * a + b * b
0067 |    delta = math.radians(delta_deg)
0068 |    psi = math.atan2(b, a)
0069 |    candidates = [a * math.cos(-delta) + b * math.sin(-delta),
0070 |                  a * math.cos(delta) + b * math.sin(delta)]
0071 |    # An interior antipodal direction can be the minimum dot product.
0072 |    for k in (-1, 0, 1):
0073 |        eta = psi + math.pi + 2 * math.pi * k
0074 |        if -delta <= eta <= delta:
0075 |            candidates.append(-math.sqrt(length2))
0076 |    min_projection = min(candidates)
```

```

0077 | return max(length2, length2 + radius * radius - 2 * radius * min_projection)
0078 |
0079 |
0080 | def safe_full_information(s: Point, delta_deg: float = DELTA_DEG,
0081 |     rmin: float = R_MIN) -> bool:
0082 |     """Exact robust domain using the fact that station 1 already received.
0083 |
0084 |     For a possible source at first-station distance r, the smallest compatible
0085 |     receiving radius is max(rmin,r). The condition reduces to covering the
0086 |     first-bearing sector truncated at rmin by rmin-disks.
0087 |     """
0088 |     return _sector_max_distance_sq(s, rmin, delta_deg) <= rmin * rmin + 1e-7
0089 |
0090 |
0091 | def safe_uniform_1000(s: Point, delta_deg: float = DELTA_DEG,
0092 |     rmin: float = R_MIN, rmax: float = R_MAX) -> bool:
0093 |     """Conservative domain max_{g in F1} ||s-g|| <= rmin."""
0094 |     return _sector_max_distance_sq(s, rmax, delta_deg) <= rmin * rmin + 1e-7
0095 |
0096 |
0097 | def safe_angle_limit(baseline: float, domain: str = "uniform1000",
0098 |     delta_deg: float = DELTA_DEG,
0099 |     rmin: float = R_MIN, rmax: float = R_MAX) -> float | None:
0100 |     """Maximum |psi| in degrees on a fixed-baseline safe arc."""
0101 |     if not 0 < baseline <= rmin:
0102 |         return None
0103 |     if domain == "full_information":
0104 |         ratio = baseline / (2 * rmin)
0105 |     elif domain == "uniform1000":
0106 |         ratio = (baseline * baseline + rmax * rmax - rmin * rmin) / (2 * baseline * rmax)
0107 |     else:
0108 |         raise ValueError("unknown safety domain")
0109 |     if ratio > 1:
0110 |         return None
0111 |     return max(0.0, math.degrees(math.acos(max(-1.0, ratio))) - delta_deg)
0112 |
0113 |
0114 | def circle_outer_polygon(center: Point, radius: float, sides: int) -> list[Point]:
0115 |     """Regular circumscribed polygon whose sides are tangent to the disk."""
0116 |     if sides < 24:
0117 |         raise ValueError("circle approximation needs at least 24 sides")
0118 |     rv = radius / math.cos(math.pi / sides)
0119 |     return [add(center, (rv * math.cos((2 * k + 1) * math.pi / sides),
0120 |         rv * math.sin((2 * k + 1) * math.pi / sides)))
0121 |         for k in range(sides)]
0122 |
0123 |
0124 | def clip_halfplane(poly: list[Point], normal: Point, offset: float) -> list[Point]:
0125 |     if not poly:
0126 |         return []
0127 |     out: list[Point] = []
0128 |     a = poly[-1]
0129 |     fa = dot(normal, a) - offset
0130 |     for b in poly:
0131 |         fb = dot(normal, b) - offset
0132 |         if (fa <= EPS) != (fb <= EPS):
0133 |             t = fa / (fa - fb)
0134 |             out.append(add(a, mul(sub(b, a), t)))
0135 |         if fb <= EPS:
0136 |             out.append(b)
0137 |         a, fa = b, fb
0138 |     clean: list[Point] = []
0139 |     for p in out:
0140 |         if not clean or dist(p, clean[-1]) > 1e-8:
0141 |             clean.append(p)
0142 |     if len(clean) > 1 and dist(clean[0], clean[-1]) <= 1e-8:
0143 |         clean.pop()
0144 |     return clean
0145 |
0146 |
0147 | def clip_disk_outer(poly: list[Point], center: Point, radius: float, sides: int) -> list[Point]:
0148 |     for k in range(sides):
0149 |         angle = 2 * math.pi * k / sides
0150 |         n = math.cos(angle), math.sin(angle)
0151 |         poly = clip_halfplane(poly, n, dot(n, center) + radius)
0152 |     if not poly:
0153 |         break
0154 |     return poly
0155 |

```

```

0156 |
0157 | def clip_wedge(poly: list[Point], station: Point, bearing_deg: float,
0158 |             delta_deg: float = DELTA_DEG) -> list[Point]:
0159 |     lo = math.radians(bearing_deg - delta_deg)
0160 |     hi = math.radians(bearing_deg + delta_deg)
0161 |     lower = math.cos(lo), math.sin(lo)
0162 |     upper = math.cos(hi), math.sin(hi)
0163 |     poly = clip_halfplane(poly, (lower[1], -lower[0]),
0164 |                          dot((lower[1], -lower[0]), station))
0165 |     return clip_halfplane(poly, (-upper[1], upper[0]),
0166 |                          dot((-upper[1], upper[0]), station))
0167 |
0168 |
0169 | def first_feasible_polygon(first: Point = (0.0, 0.0), bearing_deg: float = 0.0,
0170 |                          delta_deg: float = DELTA_DEG, sides: int = 180) -> list[Point]:
0171 |     """Outer approximation of F1 = Omega intersect B(S1,1500) intersect W1.
0172 |
0173 |     The direction result also implies distance >5 m. Omitting that tiny inner
0174 |     hole keeps the set convex and is conservative for worst-case uncertainty.
0175 |     """
0176 |     poly = circle_outer_polygon((0.0, 0.0), ARENA_R, sides)
0177 |     poly = clip_disk_outer(poly, first, R_MAX, sides)
0178 |     return clip_wedge(poly, first, bearing_deg, delta_deg)
0179 |
0180 |
0181 | def polygon_area(poly: Sequence[Point]) -> float:
0182 |     return abs(sum(cross(poly[i], poly[(i + 1) % len(poly)])
0183 |                   for i in range(len(poly)))) / 2 if len(poly) >= 3 else 0.0
0184 |
0185 |
0186 | def _diameter_circle(a: Point, b: Point) -> tuple[Point, float]:
0187 |     c = mul(add(a, b), 0.5)
0188 |     return c, dist(a, b) / 2
0189 |
0190 |
0191 | def _circumcircle(a: Point, b: Point, c: Point) -> tuple[Point, float] | None:
0192 |     u, v = sub(b, a), sub(c, a)
0193 |     den = 2 * cross(u, v)
0194 |     if abs(den) < 1e-14:
0195 |         return None
0196 |     uu, vv = dot(u, u), dot(v, v)
0197 |     center = add(a, ((v[1] * uu - u[1] * vv) / den,
0198 |                     (u[0] * vv - v[0] * uu) / den))
0199 |     return center, dist(center, a)
0200 |
0201 |
0202 | def minimum_enclosing_circle(points: Sequence[Point]) -> tuple[Point, float]:
0203 |     """Deterministic-seed incremental MEC with a final covering audit."""
0204 |     if not points:
0205 |         raise ValueError("empty point set")
0206 |     pts = list(points)
0207 |     random.Random(20260911).shuffle(pts)
0208 |     circle: tuple[Point, float] | None = None
0209 |     for i, p in enumerate(pts):
0210 |         if circle is not None and dist(circle[0], p) <= circle[1] + 1e-8:
0211 |             continue
0212 |         circle = p, 0.0
0213 |         for j, q in enumerate(pts[i:]):
0214 |             if dist(circle[0], q) <= circle[1] + 1e-8:
0215 |                 continue
0216 |             circle = _diameter_circle(p, q)
0217 |             pq = sub(q, p)
0218 |             left = right = None
0219 |             for r in pts[j:]:
0220 |                 if dist(circle[0], r) <= circle[1] + 1e-8:
0221 |                     continue
0222 |                 cc = _circumcircle(p, q, r)
0223 |                 if cc is None:
0224 |                     continue
0225 |                 side = cross(pq, sub(r, p))
0226 |                 signed = cross(pq, sub(cc[0], p))
0227 |                 if side > 0 and (left is None or signed > left[0]):
0228 |                     left = signed, cc
0229 |                 if side < 0 and (right is None or signed < right[0]):
0230 |                     right = signed, cc
0231 |             options = [item[1] for item in (left, right) if item is not None]
0232 |             if options:
0233 |                 circle = min(options, key=lambda item: item[1])
0234 |     assert circle is not None

```

```

0235 | return circle[0], max(dist(circle[0], p) for p in points) + 1e-7
0236 |
0237 |
0238 | @dataclass(frozen=True)
0239 | class Metrics:
0240 |     diameter_m: float
0241 |     mec_radius_m: float
0242 |     area_m2: float
0243 |     vertex_count: int
0244 |
0245 |
0246 | def _hull(points: Sequence[Point]) -> list[Point]:
0247 |     pts=sorted(set(points))
0248 |     if len(pts)<=2: return list(pts)
0249 |     def chain(seq):
0250 |         out=[]
0251 |         for p in seq:
0252 |             while len(out)>=2 and cross(sub(out[-1],out[-2]),sub(p,out[-1]))<=1e-10: out.pop()
0253 |             out.append(p)
0254 |         return out
0255 |     return chain(pts)[-1]+chain(reversed(pts))[:-1]
0256 |
0257 | def polygon_diameter(points: Sequence[Point]):
0258 |     pts=_hull(points); n=len(pts)
0259 |     if n==0: return 0.0, None
0260 |     if n==1: return 0.0, (pts[0], pts[0])
0261 |     if n==2: return dist(pts[0], pts[1]), (pts[0], pts[1])
0262 |     j=1; best=-1.0; pair=None
0263 |     for i in range(n):
0264 |         ni=(i+1)%n; edge=sub(pts[ni], pts[i])
0265 |         while cross(edge, sub(pts[(j+1)%n], pts[i]))>cross(edge, sub(pts[j], pts[i]))+1e-10:
0266 |             j=(j+1)%n
0267 |         for a in (i, ni):
0268 |             d=dist(pts[a], pts[j])
0269 |             if d>best: best=d; pair=(pts[a], pts[j])
0270 |     return best, pair
0271 |
0272 | def polygon_metrics(poly: list[Point]) -> Metrics:
0273 |     diameter, _ = polygon_diameter(poly)
0274 |     _, radius = minimum_enclosing_circle(poly)
0275 |     return Metrics(float(diameter), radius, polygon_area(poly), len(poly))
0276 |
0277 |
0278 | def physical_base(first_poly: list[Point], second: Point, sides: int) -> list[Point]:
0279 |     return clip_disk_outer(list(first_poly), second, R_MAX, sides)
0280 |
0281 |
0282 | def post_region(base: list[Point], second: Point, reported_second_deg: float,
0283 |                 delta_deg: float = DELTA_DEG) -> list[Point]:
0284 |     return clip_wedge(list(base), second, reported_second_deg, delta_deg)
0285 |
0286 |
0287 | @dataclass
0288 | class RobustQuality:
0289 |     worst_R_m: float
0290 |     worst_D_m: float
0291 |     worst_A_m2: float
0292 |     report_at_worst_R_deg: float
0293 |     report_at_worst_D_deg: float
0294 |     report_at_worst_A_deg: float
0295 |     metrics_at_worst_R: Metrics
0296 |     feasible_report_count: int
0297 |     report_evaluations: int
0298 |     disk_sides: int
0299 |     report_step_deg: float
0300 |
0301 |     def as_dict(self) -> dict:
0302 |         out = asdict(self)
0303 |         out["continuous_report_maximum_certified"] = False
0304 |         return out
0305 |
0306 |
0307 | def robust_quality(second: Point, *, first: Point = (0.0, 0.0),
0308 |                   first_bearing_deg: float = 0.0, delta_deg: float = DELTA_DEG,
0309 |                   disk_sides: int = 120, report_step_deg: float = 0.5,
0310 |                   refine_step_deg: float = 0.05) -> RobustQuality:
0311 |     """Numerically approximate worst D/R/A over feasible second reports.
0312 |
0313 |     A report z is feasible exactly when P2(z) is nonempty: any point in P2 has

```

```

0314 | a true bearing within delta of z, so a compatible source/error pair exists.
0315 | Thus this report-domain maximization avoids combining an impossible source
0316 | location with an unrelated second reading.
0317 | """
0318 | first_poly = first_feasible_polygon(first, first_bearing_deg, delta_deg, disk_sides)
0319 | base = physical_base(first_poly, second, disk_sides)
0320 | cache: dict[float, tuple[Metrics, list[Point]]] = {}
0321 |
0322 | def evaluate(z: float) -> tuple[Metrics, list[Point]] | None:
0323 |     key = round(z % 360.0, 10)
0324 |     if key in cache:
0325 |         return cache[key]
0326 |     poly = post_region(base, second, key, delta_deg)
0327 |     if not poly:
0328 |         return None
0329 |     result = polygon_metrics(poly), poly
0330 |     cache[key] = result
0331 |     return result
0332 |
0333 | records: list[tuple[float, Metrics]] = []
0334 | count = max(1, int(math.ceil(360.0 / report_step_deg)))
0335 | coarse: list[tuple[float, Metrics] | None] = []
0336 | for i in range(count):
0337 |     z = i * 360.0 / count
0338 |     value = evaluate(z)
0339 |     if value is not None:
0340 |         row = (z, value[0])
0341 |         records.append(row)
0342 |         coarse.append(row)
0343 |     else:
0344 |         coarse.append(None)
0345 | if not records:
0346 |     raise RuntimeError("no feasible second report")
0347 |
0348 | # Refine competitive coarse local maxima by bounded scalar searches. The
0349 | # finite search is reproducible and convergence-checkable, but is not a
0350 | # formal interval certificate for the continuous report domain.
0351 | golden = (math.sqrt(5.0) - 1.0) / 2.0
0352 | for field in ("mec_radius_m", "diameter_m", "area_m2"):
0353 |     candidates: list[tuple[float, int]] = []
0354 |     for i, row in enumerate(coarse):
0355 |         if row is None:
0356 |             continue
0357 |         value = getattr(row[1], field)
0358 |         before = coarse[(i - 1) % count]
0359 |         after = coarse[(i + 1) % count]
0360 |         vb = getattr(before[1], field) if before is not None else -math.inf
0361 |         va = getattr(after[1], field) if after is not None else -math.inf
0362 |         if value >= vb and value >= va:
0363 |             candidates.append((value, i))
0364 |     if not candidates:
0365 |         candidates = [(getattr(max(records, key=lambda row: getattr(row[1], field))[1], field),
0366 |                             int(round(max(records, key=lambda row: getattr(row[1], field))[0]
0367 |                                     / (360.0 / count))) % count)]
0368 |     for _, i in sorted(candidates, reverse=True)[:8]:
0369 |         center = i * 360.0 / count
0370 |         left, right = center - 360.0 / count, center + 360.0 / count
0371 |
0372 |     def field_value(z: float) -> float:
0373 |         item = evaluate(z)
0374 |         return getattr(item[0], field) if item is not None else -math.inf
0375 |
0376 |     c = right - golden * (right - left)
0377 |     d = left + golden * (right - left)
0378 |     fc, fd = field_value(c), field_value(d)
0379 |     for _ in range(34):
0380 |         if fc >= fd:
0381 |             right, d, fd = d, c, fc
0382 |             c = right - golden * (right - left)
0383 |             fc = field_value(c)
0384 |         else:
0385 |             left, c, fc = c, d, fd
0386 |             d = left + golden * (right - left)
0387 |             fd = field_value(d)
0388 |     for z in (left, c, (left + right) / 2, d, right):
0389 |         value = evaluate(z)
0390 |         if value is not None:
0391 |             records.append((z % 360.0, value[0]))
0392 |

```

```

0393 | row_R = max(records, key=lambda row: row[1].mec_radius_m)
0394 | row_D = max(records, key=lambda row: row[1].diameter_m)
0395 | row_A = max(records, key=lambda row: row[1].area_m2)
0396 | return RobustQuality(row_R[1].mec_radius_m, row_D[1].diameter_m,
0397 |                     row_A[1].area_m2, row_R[0] % 360.0,
0398 |                     row_D[0] % 360.0, row_A[0] % 360.0,
0399 |                     row_R[1], len(records), len(cache), disk_sides,
0400 |                     report_step_deg)
0401 |
0402 |
0403 | def candidate(baseline: float, psi_deg: float, side: int = 1) -> Point:
0404 |     psi = math.radians(psi_deg)
0405 |     return baseline * math.cos(psi), side * baseline * math.sin(psi)
0406 |
0407 |
0408 | def optimize_fixed_baseline(metric: str, baseline: float, *,
0409 |                             first: Point=(0.0,0.0), first_bearing_deg: float=0.0,
0410 |                             domain: str = "uniform1000",
0411 |                             delta_deg: float = DELTA_DEG,
0412 |                             coarse_psi_deg: float = 1.0,
0413 |                             fine_psi_deg: float = 0.1,
0414 |                             disk_sides: int = 96,
0415 |                             report_step_deg: float = 1.0) -> dict:
0416 |     if metric not in {"R", "D", "A"}:
0417 |         raise ValueError("metric must be R, D, or A")
0418 |     limit = safe_angle_limit(baseline, domain, delta_deg)
0419 |     if limit is None or limit <= 0:
0420 |         raise ValueError("fixed baseline has no noncollinear safe candidate")
0421 |     safety = safe_uniform_1000 if domain == "uniform1000" else safe_full_information
0422 |     values: list[tuple[float, float, RobustQuality]] = []
0423 |
0424 |     def score(q: RobustQuality) -> float:
0425 |         return {"R": q.worst_R_m, "D": q.worst_D_m, "A": q.worst_A_m2}[metric]
0426 |
0427 |     # Search both sides of the first bearing. They are equivalent only in the
0428 |     # centred, untruncated standard geometry; the arena boundary generally
0429 |     # breaks that symmetry for an arbitrary first station.
0430 |     for side in (-1.0, 1.0):
0431 |         side_values: list[tuple[float, float, RobustQuality]] = []
0432 |         magnitude = max(0.2, coarse_psi_deg)
0433 |         while magnitude <= limit + 1e-12:
0434 |             psi = side * magnitude; local_point = candidate(baseline, psi)
0435 |             if safety(local_point, delta_deg):
0436 |                 point = local_to_world(first, first_bearing_deg, local_point[0], local_point[1])
0437 |                 q = robust_quality(point, first=first, first_bearing_deg=first_bearing_deg, delta_deg=delta_deg, disk_sides=disk_sides,
0438 |                                   report_step_deg=report_step_deg,
0439 |                                   refine_step_deg=report_step_deg/5)
0440 |                 row = (score(q), psi, q); values.append(row); side_values.append(row)
0441 |                 magnitude += coarse_psi_deg
0442 |             if not side_values: continue
0443 |             _, side_best_psi, _ = min(side_values, key=lambda row: row[0])
0444 |             lo = max(-limit, side_best_psi - coarse_psi_deg); hi = min(limit, side_best_psi + coarse_psi_deg)
0445 |             if side < 0: hi = min(hi, -0.05)
0446 |             else: lo = max(lo, 0.05)
0447 |             psi = lo
0448 |             while psi <= hi + 1e-12:
0449 |                 local_point = candidate(baseline, psi)
0450 |                 if safety(local_point, delta_deg):
0451 |                     point = local_to_world(first, first_bearing_deg, local_point[0], local_point[1])
0452 |                     q = robust_quality(point, first=first, first_bearing_deg=first_bearing_deg, delta_deg=delta_deg, disk_sides=disk_sides,
0453 |                                       report_step_deg=report_step_deg,
0454 |                                       refine_step_deg=report_step_deg/5)
0455 |                     values.append((score(q), psi, q))
0456 |                 psi += fine_psi_deg
0457 |             if not values:
0458 |                 raise ValueError("no evaluated safe candidate")
0459 |     best_score, best_psi, best_q = min(values, key=lambda row: row[0])
0460 |     local_point = candidate(baseline, best_psi)
0461 |     point = local_to_world(first, first_bearing_deg, local_point[0], local_point[1])
0462 |     return {"objective": metric, "safety_domain": domain,
0463 |           "baseline_m": baseline, "psi_deg": best_psi,
0464 |           "a_m": local_point[0], "b_m": local_point[1], "second_station_m": [point[0], point[1]], "objective_value": best_score,
0465 |           "safe_angle_limit_deg": limit, "quality_search": best_q.as_dict(),
0466 |           "candidate_evaluations": len(values),
0467 |           "continuous_global_optimum_claimed": False,
0468 |           "search_method": "two-sided fixed-baseline one-dimensional coarse-to-fine search",
0469 |           "numerical_scope": "finite angular sampling with local refinement; no continuous global optimum certificate"}
0470 |
0471 |

```

```

0472 | def final_quality(second: Point, *, first: Point=(0.0,0.0), first_bearing_deg: float=0.0, disk_sides: int = 360,
0473 |         report_step_deg: float = 0.1) -> dict:
0474 |     q = robust_quality(second, first=first, first_bearing_deg=first_bearing_deg, disk_sides=disk_sides,
0475 |         report_step_deg=report_step_deg,
0476 |         refine_step_deg=report_step_deg / 5)
0477 |     return q.as_dict()
0478 |
0479 |
0480 |
0481 | def solve_case(payload: dict, quick: bool=False) -> dict:
0482 |     first_raw=payload.get("first_station_m",[0.0,0.0])
0483 |     first=(float(first_raw[0]),float(first_raw[1]))
0484 |     bearing=float(payload["first_bearing_deg"])%360.0
0485 |     baselines=[float(x) for x in payload.get("baselines_m",[600,700,800,900,1000])]
0486 |     search_sides=72 if quick else 96
0487 |     report_step=2.0 if quick else 1.0
0488 |     coarse=2.0 if quick else 1.0
0489 |     fine=0.25 if quick else 0.1
0490 |     all_rows=[]
0491 |     for metric in ("D","R"):
0492 |         for L in baselines:
0493 |             try:
0494 |                 row=optimize_fixed_baseline(metric,L,first=first,first_bearing_deg=bearing,
0495 |                     coarse_psi_deg=coarse,fine_psi_deg=fine,disk_sides=search_sides,
0496 |                     report_step_deg=report_step)
0497 |             except ValueError:
0498 |                 continue
0499 |             all_rows.append(row)
0500 |     if not all_rows:raise RuntimeError("没有可用的安全第二检测点")
0501 |     best_D=min((r for r in all_rows if r["objective"]=="D"),key=lambda r:r["objective_value"])
0502 |     best_R=min((r for r in all_rows if r["objective"]=="R"),key=lambda r:r["objective_value"])
0503 |     final_sides=180 if quick else 360
0504 |     final_report=.5 if quick else .2
0505 |     comparison=[]
0506 |     for label,row in ((("Q1直径方法",best_D),("Q2最小包围圆方法",best_R)):
0507 |         p=tuple(row["second_station_m"])
0508 |         q=final_quality(p,first=first,first_bearing_deg=bearing,disk_sides=final_sides,report_step_deg=final_report)
0509 |         comparison.append({"method":label,"baseline_m":row["baseline_m"],"psi_deg":row["psi_deg"],
0510 |             "a_m":row["a_m"],"b_m":row["b_m"],"second_station_m":row["second_station_m"],
0511 |             "worst_R_m":q["worst_R_m"],"worst_D_m":q["worst_D_m"],"worst_A_m2":q["worst_A_m2"],
0512 |             "report_at_worst_R_deg":q["report_at_worst_R_deg"],
0513 |             "meets_20m_on_evaluated_reports":q["worst_R_m"]<=20.0})
0514 |     return {"case_id":payload.get("case_id","Q2"),"first_station_m":list(first),"first_bearing_deg":bearing,
0515 |         "error_half_width_deg":DELTA_DEG,"safety_domain":"uniform1000",
0516 |         "main_method":"Q2最小包围圆方法",comparison:comparison,
0517 |         "baseline_search":[{"objective":r["objective"],"baseline_m":r["baseline_m"],"psi_deg":r["psi_deg"],
0518 |             "objective_value":r["objective_value"]} for r in all_rows],
0519 |         "note":"Q1对照会独立按最坏直径重新选点；内层连续示向度最坏值由有限网格和局部细化数值近似。"}
0520 |
0521 | def main(argv=None):
0522 |     ap=argparse.ArgumentParser(description="问题二：安全第二检测点与Q1直径方法对比")
0523 |     ap.add_argument("--input",type=Path,default=Path("q2_input.json"))
0524 |     ap.add_argument("--output",type=Path,default=Path("q2_result.json"))
0525 |     ap.add_argument("--quick",action="store_true",help="缩短本地试跑时间")
0526 |     args=ap.parse_args(argv)
0527 |     payload=json.loads(args.input.read_text(encoding="utf-8"))
0528 |     result=solve_case(payload,args.quick)
0529 |     args.output.parent.mkdir(parents=True,exist_ok=True)
0530 |     args.output.write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding="utf-8")
0531 |     print("===== 问题二结果 =====")
0532 |     for row in result["comparison"]:
0533 |         print(f"[row['method']]: L={row['baseline_m']:.1f} m,  $\psi$ ={row['psi_deg']:.2f}°, 最坏R={row['worst_R_m']:.3f} m, 最坏D={row['worst_D_m']:.3f} m, 面积={row['worst_A_m2']:.3f} m²")
0534 |         print(f" 第二检测点: ({row['second_station_m'][0]:.3f}, {row['second_station_m'][1]:.3f})")
0535 |         print(f"结果已保存: {args.output}")
0536 |     return 0
0537 |
0538 | if __name__=="__main__":raise SystemExit(main())

```

source/Q2/q2_generator.py

```

0001 | #!/usr/bin/env python3
0002 | from __future__ import annotations
0003 | import argparse,json,math,random
0004 | from pathlib import Path
0005 |

```

```

0006 | def main():
0007 |     ap=argparse.ArgumentParser(description='问题二随机首测数据生成器')
0008 |     ap.add_argument('--seed',type=int,default=None)
0009 |     ap.add_argument('--output',type=Path,default=Path('q2_input.json'))
0010 |     args=ap.parse_args()
0011 |     seed=args.seed if args.seed is not None else random.SystemRandom().randrange(1,2**63)
0012 |     rng=random.Random(seed)
0013 |     # 生成一组物理相容的首测，真值仅供本地核验，求解器不会读取。
0014 |     sa=rng.random()*math.tau; sr=rng.uniform(0,450)
0015 |     s=(sr*math.cos(sa),sr*math.sin(sa))
0016 |     ga=rng.random()*math.tau; gd=rng.uniform(450,1400)
0017 |     g=(s[0]+gd*math.cos(ga),s[1]+gd*math.sin(ga))
0018 |     d=math.hypot(*g)
0019 |     if d>1750:
0020 |         scale=1750/d;g=(g[0]*scale,g[1]*scale)
0021 |     true=math.degrees(math.atan2(g[1]-s[1],g[0]-s[0]))%360
0022 |     while True:
0023 |         report=round((true+rng.uniform(-1,1))%360,2)%360.0
0024 |         final_error=(report-true+180.0)%360.0-180.0
0025 |         if abs(final_error)<=1.0:break
0026 |     payload={'case_id':f'Q2-seed-{seed}','seed':seed,'first_station_m':[s[0],s[1]],
0027 |             'first_bearing_deg':report,'baselines_m':[600,700,800,900,1000],
0028 |             'local_truth':{'source_m':[g[0],g[1]],'true_bearing_deg':true}}
0029 |     args.output.parent.mkdir(parents=True,exist_ok=True)
0030 |     args.output.write_text(json.dumps(payload,ensure_ascii=False,indent=2),encoding='utf-8')
0031 |     print(f'随机种子: {seed}')
0032 |     print(f'首测点: ({s[0]:.3f}, {s[1]:.3f})')
0033 |     print(f'示向度: {report:.2f}°')
0034 |     print(f'测试数据已保存: {args.output}')
0035 | if __name__=='__main__':main()

```

source/Q2/run.bat

```

0001 | @echo off
0002 | setlocal
0003 | chcp 65001 >nul
0004 | pushd "%~dp0" || exit /b 1
0005 | python q2_generator.py --output q2_input.json
0006 | if errorlevel 1 goto finish
0007 | python q2_solver.py --input q2_input.json --output q2_result.json
0008 | :finish
0009 | set "RC=%ERRORLEVEL%"
0010 | popd
0011 | pause
0012 | exit /b %RC%

```

source/Q3/q3_local_solver.py

```

0001 | """Conservative 2D geometry. Standard library only; meters/radians internally."""
0002 | from __future__ import annotations
0003 | import math
0004 | import random
0005 | from typing import Iterable
0006 |
0007 | Point = tuple[float, float]
0008 | TAU = 2.0 * math.pi
0009 | EPS = 1e-7
0010 | DELTA = math.radians(1.0051) # 1° physical bound + 0.005° output rounding + 0.0001° guard
0011 |
0012 | def add(a: Point, b: Point) -> Point: return (a[0]+b[0], a[1]+b[1])
0013 | def sub(a: Point, b: Point) -> Point: return (a[0]-b[0], a[1]-b[1])
0014 | def mul(a: Point, s: float) -> Point: return (a[0]*s, a[1]*s)
0015 | def dot(a: Point, b: Point) -> float: return a[0]*b[0]+a[1]*b[1]
0016 | def cross(a: Point, b: Point) -> float: return a[0]*b[1]-a[1]*b[0]
0017 | def norm(a: Point) -> float: return math.hypot(*a)
0018 | def dist(a: Point, b: Point) -> float: return math.hypot(a[0]-b[0], a[1]-b[1])
0019 | def unit(a: Point) -> Point:
0020 |     n=norm(a)
0021 |     return (a[0]/n,a[1]/n) if n>EPS else (1.0,0.0)
0022 | def maxdist(p: Point, poly: list[Point]) -> float:
0023 |     return max((dist(p,v) for v in poly), default=0.0)
0024 |
0025 | def hull(points: Iterable[Point]) -> list[Point]:
0026 |     pts=sorted(set(points))

```

```

0027 | if len(pts)<=2: return pts
0028 | lo=[]; hi=[]
0029 | for p in pts:
0030 |     while len(lo)>=2 and cross(sub(lo[-1],lo[-2]),sub(p,lo[-1]))<=0: lo.pop()
0031 |     lo.append(p)
0032 | for p in reversed(pts):
0033 |     while len(hi)>=2 and cross(sub(hi[-1],hi[-2]),sub(p,hi[-1]))<=0: hi.pop()
0034 |     hi.append(p)
0035 | return lo[-1]+hi[-1]
0036 |
0037 | def clip(poly: list[Point], n: Point, b: float) -> list[Point]:
0038 |     """Intersect polygon with n.x <= b. Expand by EPS in physical units."""
0039 |     if not poly: return []
0040 |     b+=EPS*max(1.0,norm(n))
0041 |     out=[]
0042 |     a=poly[-1]; fa=dot(n,a)-b
0043 |     for c in poly:
0044 |         fc=dot(n,c)-b
0045 |         if (fa<=0)!= (fc<=0):
0046 |             t=fa/(fa-fc)
0047 |             out.append(add(a,mul(sub(c,a),t)))
0048 |         if fc<=0: out.append(c)
0049 |         a,fa=c,fc
0050 |     if len(out)>1:
0051 |         clean=[out[0]]
0052 |         for p in out[1:]:
0053 |             if dist(p,clean[-1])>1e-9: clean.append(p)
0054 |             if len(clean)>1 and dist(clean[0],clean[-1])<1e-9: clean.pop()
0055 |         out=clean
0056 |     return out
0057 |
0058 | def arena_polygon(sides: int=48) -> list[Point]:
0059 |     # CIRCUMscribed regular polygon: never use an inscribed polygon here.
0060 |     r=1800.0/math.cos(math.pi/sides)
0061 |     return [(r*math.cos(TAU*k/sides+math.pi/sides),r*math.sin(TAU*k/sides+math.pi/sides)) for k in range(sides)]
0062 |
0063 | def bearing_update(poly: list[Point], p: Point, theta_deg: float) -> list[Point]:
0064 |     """Outer approximation of P intersect bearing sector intersect B(p,1500).
0065 |     Three supporting halfplanes bound the sector by a triangle. R may be
0066 |     tightened by the CURRENT polygon, so the contraction proof still applies.
0067 |     """
0068 |     R=min(1500.0,maxdist(p,poly)+EPS)
0069 |     a=math.radians(theta_deg)
0070 |     lower=(math.cos(a-DELTA),math.sin(a-DELTA))
0071 |     upper=(math.cos(a+DELTA),math.sin(a+DELTA))
0072 |     # cross(lower,x-p)>=0; cross(upper,x-p)<=0
0073 |     for n in ((lower[1],-lower[0]),(-upper[1],upper[0])):
0074 |         poly=clip(poly,n,dot(n,p))
0075 |     u=(math.cos(a),math.sin(a))
0076 |     return clip(poly,u,dot(u,p)+R)
0077 |
0078 | def exclude_disk_hull(poly: list[Point], c: Point, radius: float) -> list[Point]:
0079 |     """conv(P minus open disk), a SAFE convex outer relaxation.
0080 |     Its extreme points are surviving polygon vertices and edge/circle crossings.
0081 |     Radius is slightly reduced for conservative floating-point behavior.
0082 |     """
0083 |     if not poly: return []
0084 |     r=max(0.0,radius-1e-5); r2=r*r
0085 |     candidates=[p for p in poly if dist(p,c)>=r-1e-6]
0086 |     for k,a in enumerate(poly):
0087 |         b=poly[(k+1)%len(poly)]; d=sub(b,a); f=sub(a,c)
0088 |         A=dot(d,d)
0089 |         if A<1e-20: continue
0090 |         B=2*dot(f,d); C=dot(f,f)-r2
0091 |         disc=B*B-4*A*C
0092 |         if disc<0: continue
0093 |         root=math.sqrt(max(0.0,disc))
0094 |         for t in ((-B-root)/(2*A),(-B+root)/(2*A)):
0095 |             if -1e-9<=t<=1+1e-9:
0096 |                 t=max(0.,min(1.,t))
0097 |                 candidates.append(add(a,mul(d,t)))
0098 |     return hull(candidates)
0099 |
0100 | def inside_polygon(p: Point, poly: list[Point], tol: float=1e-4) -> bool:
0101 |     if not poly: return False
0102 |     if len(poly)==1: return dist(p,poly[0])<=tol
0103 |     if len(poly)==2:
0104 |         a,b=poly; d=sub(b,a)
0105 |         t=dot(sub(p,a),d)/max(dot(d,d),1e-30)

```

```

0106 |     return -tol<=t<=1+tol and abs(cross(d,sub(p,a)))<=tol*max(1,norm(d))
0107 |     return all((cross(sub(poly[(k+1)%len(poly)],v),sub(p,v))>=-tol*max(1,dist(v,poly[(k+1)%len(poly)])) for k,v in enumerate(poly))
0108 |
0109 | def _diameter(a,b):
0110 |     c=mul(add(a,b),0.5); return (c,dist(a,b)*0.5)
0111 | def _circum(a,b,c):
0112 |     # Translate to reduce cancellation.
0113 |     u=sub(b,a); v=sub(c,a); D=2*cross(u,v)
0114 |     if abs(D)<1e-18: return None
0115 |     uu=dot(u,u); vv=dot(v,v)
0116 |     o=add(a,((v[1]*uu-u[1]*vv)/D,(u[0]*vv-v[0]*uu)/D))
0117 |     return (o,dist(o,a))
0118 | def _in(circle,p): return dist(circle[0],p)<=circle[1]+1e-9
0119 |
0120 | def mec(points: list[Point]) -> tuple[Point,float]:
0121 |     """Randomized incremental smallest enclosing circle + full radius audit.
0122 |     Randomness is fixed per call and has no influence on the strategy's safety.
0123 |     """
0124 |     if not points: raise ValueError('empty feasible outer set')
0125 |     pts=list(points); random.Random(271828).shuffle(pts)
0126 |     circle=None
0127 |     for i,p in enumerate(pts):
0128 |         if circle is not None and _in(circle,p): continue
0129 |         circle=(p,0.0)
0130 |         for j,q in enumerate(pts[i:]):
0131 |             if _in(circle,q): continue
0132 |             circle=_diameter(p,q)
0133 |             # Two forced boundary points; choose the extremal circle on each side.
0134 |             pq=sub(q,p); left=None; right=None
0135 |             for r in pts[j:]:
0136 |                 if _in(circle,r): continue
0137 |                 side=cross(pq,sub(r,p)); cc=_circum(p,q,r)
0138 |                 if cc is None: continue
0139 |                 v=cross(pq,sub(cc[0],p))
0140 |                 if side>0 and (left is None or v>left[0]): left=(v,cc)
0141 |                 if side<0 and (right is None or v<right[0]): right=(v,cc)
0142 |                 options=[z[1] for z in (left,right) if z is not None]
0143 |                 if options: circle=min(options,key=lambda z:z[1])
0144 |             c=circle[0]
0145 |             # This audited radius is always used for certification, not an internal radius.
0146 |             return c,maxdist(c,points)+1e-6
0147 |
0148 | def nearest_certified_clear(p: Point, c: Point, r: float, clear_r: float=19.8) -> Point:
0149 |     """Nearest point in B(c,clear_r-r), a certified subset of valid clear sites."""
0150 |     slack=clear_r-r
0151 |     if slack<0: raise ValueError('not certified clearable')
0152 |     d=dist(p,c)
0153 |     if d<=slack: return p
0154 |     return add(c,mul(sub(p,c),slack/d))
0155 |
0156 | def arc_covered_by_disk(c: Point, R: float, s: Point, r: float) -> list[tuple[float,float]]:
0157 |     d=dist(c,s)
0158 |     if d<1e-10: return [(0.,TAU)] if r>=R else []
0159 |     if d+R<=r: return [(0.,TAU)]
0160 |     if d>=R+r or R>=d+r: return []
0161 |     a=math.atan2(s[1]-c[1],s[0]-c[0])%TAU
0162 |     h=math.acos(max(-1.,min(1.,(d*d+R*R-r*r)/(2*d*R))))
0163 |     l=a-h; u=a+h
0164 |     if l<0: return [(0.,u),(l+TAU,TAU)]
0165 |     if u>TAU: return [(l,TAU),(0.,u-TAU)]
0166 |     return [(l,u)]
0167 |
0168 | def merge_intervals(intervals):
0169 |     out=[]
0170 |     for l,u in sorted(intervals):
0171 |         if not out or l>out[-1][1]+1e-12: out.append([l,u])
0172 |         else: out[-1][1]=max(out[-1][1],u)
0173 |     return out
0174 |
0175 | def interval_subset(targets, cover) -> bool:
0176 |     cov=merge_intervals(cover)
0177 |     for a,b in targets:
0178 |         x=a
0179 |         for l,u in cov:
0180 |             if u<x: continue
0181 |             if l>x+1e-10: break
0182 |             x=max(x,u)
0183 |             if x>=b-1e-10: break
0184 |         if x<b-1e-10: return False

```

```

0185 | return True
0186 |
0187 | def covers_arena(sites: list[Point], radius: float=999.0) -> bool:
0188 |     """Continuous disk-union coverage certificate, not a grid test.
0189 |     Check arena boundary AND every exposed sensor-circle arc inside the arena.
0190 |     Deduplicate equal circles first. A 1m radius guard handles roundoff.
0191 |     """
0192 |     pts=[]
0193 |     for p in sites:
0194 |         if all(dist(p,q)>1e-5 for q in pts): pts.append(p)
0195 |     boundary=[]
0196 |     for p in pts: boundary+=arc_covered_by_disk((0.,0.),1800.,p,radius)
0197 |     if not interval_subset([(0.,TAU)],boundary): return False
0198 |     for i,p in enumerate(pts):
0199 |         interior=arc_covered_by_disk(p,radius,(0.,0.),1800.)
0200 |         if not interior: continue
0201 |         others=[]
0202 |         for j,q in enumerate(pts):
0203 |             if i!=j: others+=arc_covered_by_disk(p,radius,q,radius)
0204 |         if not interval_subset(interior,others): return False
0205 |     return True
0206 |
0207 | def ring_sites(m: int=7, phi: float=0., clockwise: bool=False) -> list[Point]:
0208 |     """Origin plus m outer sites; derive m>=6 using R=1800, r=995."""
0209 |     if m<6: raise ValueError('m must be at least 6')
0210 |     a=1800*math.cos(math.pi/m)-math.sqrt(995**2-(1800*math.sin(math.pi/m))**2)+1.
0211 |     sign=-1 if clockwise else 1
0212 |     return [(a*math.cos(phi+sign*TAU*k/m),a*math.sin(phi+sign*TAU*k/m)) for k in range(m)]
0213 |
0214 |
0215 | """Q3 online strategy. Consumes only public action responses, never source truth.
0216 |
0217 | Self-contained geometry is implemented in geometry.py; no Q1 reference solver
0218 | or historical result file is imported or loaded by the online strategy.
0219 | """
0220 | from dataclasses import dataclass, field
0221 | import math
0222 | from typing import Callable, Protocol
0223 |
0224 | class Actions(Protocol):
0225 |     def measure(self, position: Point, channel: int) -> dict: ...
0226 |     def clear(self, position: Point, channel: int) -> dict: ...
0227 |
0228 | @dataclass(frozen=True)
0229 | class Config:
0230 |     """Frozen mainline defaults, with explicit local-comparison options below.
0231 |
0232 |     The submitted entry uses Config() unchanged. Sequential and deferred are
0233 |     retained only as transparent local comparison modes; no 2-opt is claimed.
0234 |     The physical angle error is 1°; DELTA adds a conservative output/numeric guard.
0235 |     """
0236 |     # Mainline: origin + eight ring sites, MEC certification, small lateral
0237 |     # moves, shared observations, and insertion into the remaining search route.
0238 |     ring_m: int=8
0239 |     offset: float=0.10
0240 |     insert_m: float=600.0
0241 |     negative: bool=True
0242 |     opportunistic: bool=True
0243 |     opportunistic_clear: bool=True
0244 |     mode: str='integrated' # Optional local comparisons: sequential, deferred.
0245 |
0246 |     # LOCAL RESEARCH ONLY, disabled in the frozen policy. Keep field names flat
0247 |     # for dataclasses.replace/asdict and benchmark compatibility. scan_* matters
0248 |     # only when incidental is True; speculative_r=0 means no speculative clear.
0249 |     incidental: bool=False
0250 |     scan_spacing: float=500.0
0251 |     scan_policy: str='distance'
0252 |     prune: bool=False
0253 |     rotate: bool=False
0254 |     speculative_r: float=0.0
0255 |
0256 | @dataclass
0257 | class Track:
0258 |     status: str='UNKNOWN'
0259 |     poly: list[Point]=field(default_factory=arena_polygon)
0260 |     negatives: list[tuple[Point,float]]=field(default_factory=list)
0261 |     center: Point=(0.,0.)
0262 |     radius: float=math.inf
0263 |     last: Point|None=None

```

```

0264 | bearing: float=0.0
0265 | measurements: int=0
0266 |
0267 | class Strategy:
0268 |     def __init__(self, api: Actions, config: Config=Config(), audit: Callable|None=None):
0269 |         self.api=api; self.cfg=config; self.audit=audit
0270 |         self.tracks=[i:Track() for i in range(1,21)]
0271 |         self.p=(0.,0.); self.channel=1; self.virtual_time=0.
0272 |         self.scans=[]; self.pending=[]; self.coverage_done=False; self.search_reason=None
0273 |         self.cleared=0; self.actions=0; self.certificate_checks=0
0274 |         self.max_vertices=0; self.max_local_steps=0
0275 |         self._plan_ready=False
0276 |
0277 |     def _audit(self, i):
0278 |         t=self.tracks[i]; self.max_vertices=max(self.max_vertices,len(t.poly))
0279 |         if self.audit is not None: self.audit(i,t,self.p)
0280 |
0281 |     def _observe(self, i: int, p: Point) -> str:
0282 |         response=self.api.measure(p,i)
0283 |         if not response.get('accepted'): raise RuntimeError('measure rejected')
0284 |         self.p=p; self.channel=i; self.actions+=1
0285 |         self.virtual_time=float(response['virtual_time_s'])
0286 |         t=self.tracks[i]; result=response['measure_result']
0287 |         if result=='no_signal':
0288 |             t.negatives.append((p,1000.))
0289 |             if t.status=='FOUND' and self.cfg.negative:
0290 |                 t.poly=exclude_disk_hull(t.poly,p,1000.)
0291 |                 if not t.poly: raise RuntimeError('inconsistent no_signal; Q3 assumptions violated')
0292 |                 t.center,t.radius=mec(t.poly)
0293 |                 self._audit(i)
0294 |             elif result=='near':
0295 |                 self._clear(i,p,certified=True)
0296 |             elif result=='direction':
0297 |                 t.status='FOUND'; t.last=p; t.bearing=float(response['svd_deg']); t.measurements+=1
0298 |                 range_bound=min(1500.0,maxdist(p,t.poly)+EPS)
0299 |                 t.poly=bearing_update(t.poly,p,t.bearing)
0300 |                 if self.cfg.negative:
0301 |                     for c,r in t.negatives: t.poly=exclude_disk_hull(t.poly,c,r)
0302 |                 if not t.poly: raise RuntimeError('empty outer set: inconsistent measurements/numerics')
0303 |                 t.center,t.radius=mec(t.poly)
0304 |                 # Independent triangular enclosure: sec(delta)^2 / 2 contraction.
0305 |                 a=math.radians(t.bearing); h=range_bound/(2*math.cos(DELTA)**2)
0306 |                 tc=add(p,(h*math.cos(a),h*math.sin(a)))
0307 |                 tr=maxdist(tc,t.poly)+1e-6
0308 |                 if tr<t.radius: t.center,t.radius=tc,tr
0309 |                 self._audit(i)
0310 |             else: raise RuntimeError('unknown measure_result')
0311 |         return result
0312 |
0313 |     def _clear(self,i:int,p:Point,certified:bool=True) -> bool:
0314 |         response=self.api.clear(p,i)
0315 |         if not response.get('accepted'): raise RuntimeError('clear rejected')
0316 |         self.p=p; self.actions+=1; self.virtual_time=float(response['virtual_time_s'])
0317 |         if response.get('clear_result')=='success':
0318 |             self.tracks[i].status='CLEARED'; self.cleared+=1
0319 |             return True
0320 |         if response.get('clear_result')!='no_target_in_range': raise RuntimeError('unknown clear_result')
0321 |         if certified: raise RuntimeError('CERTIFIED clear failed: stop rather than silently claim success')
0322 |         t=self.tracks[i]; t.negatives.append((p,20.))
0323 |         t.poly=exclude_disk_hull(t.poly,p,20.)
0324 |         if not t.poly: raise RuntimeError('inconsistent failed clear')
0325 |         t.center,t.radius=mec(t.poly); self._audit(i)
0326 |         return False
0327 |
0328 |     def _coverage(self,points) -> bool:
0329 |         self.certificate_checks+=1
0330 |         return covers_arena(points)
0331 |
0332 |     def _certify_and_prune(self):
0333 |         if self.coverage_done: return
0334 |         if sum(t.status in ('FOUND','CLEARED') for t in self.tracks.values())==16:
0335 |             self.coverage_done=True; self.pending=[]; self.search_reason='cardinality'
0336 |             for t in self.tracks.values():
0337 |                 if t.status=='UNKNOWN': t.status='ABSENT'
0338 |             return
0339 |         if self._coverage(self.scans):
0340 |             self.coverage_done=True; self.pending=[]; self.search_reason='disk_union'
0341 |             for t in self.tracks.values():
0342 |                 if t.status=='UNKNOWN': t.status='ABSENT'

```

```

0343 |     return
0344 | if self.cfg.prune and self._plan_ready:
0345 |     # Disabled local research branch; the frozen ring is not pruned.
0346 |     # Remove a future station ONLY if the remaining union still covers Omega.
0347 |     k=0
0348 |     while k<len(self.pending):
0349 |         future=self.pending[k]+self.pending[k+1:]
0350 |         if self._coverage(self.scans+future): self.pending.pop(k)
0351 |         else: k+=1
0352 |
0353 | def _scan(self,p:Point):
0354 |     unknown=[i for i,t in self.tracks.items() if t.status=='UNKNOWN']
0355 |     # Keep the actual current receiver channel first when possible.
0356 |     unknown.sort(key=lambda i:(i!=self.channel,i))
0357 |     for i in unknown: self._observe(i,p)
0358 |     self.scans.append(p)
0359 |     if self.cfg.opportunistic:
0360 |         self._sweep_found(p)
0361 |     self._certify_and_prune()
0362 |
0363 | def _sweep_found(self,p:Point):
0364 |     # Never repeat the same channel/location; count actual new measurements.
0365 |     for i,t in self.tracks.items():
0366 |         if t.status!='FOUND' or t.radius<=19.8 or t.last is None: continue
0367 |         if dist(t.last,p)<25.: continue
0368 |         if maxdist(p,t.poly)<=999.:
0369 |             self._observe(i,p)
0370 |
0371 | def _make_plan(self):
0372 |     if self.coverage_done:
0373 |         self.pending=[]; self._plan_ready=True
0374 |         return
0375 |     phi=0.; clockwise=False
0376 |     found=[t for t in self.tracks.values() if t.status=='FOUND']
0377 |     if self.cfg.rotate and found:
0378 |         # Disabled local research branch; no rotation in the frozen policy.
0379 |         # Fixed finite candidate set, score is a heuristic (not a minimax theorem).
0380 |         options=[]
0381 |         for t in found:
0382 |             for off in (0.,math.pi/self.cfg.ring_m):
0383 |                 a=math.radians(t.bearing)+off
0384 |                 for cw in (False,True):
0385 |                     sites=ring_sites(self.cfg.ring_m,a,cw)
0386 |                     score=0.
0387 |                     for tr in found:
0388 |                         k=min(range(len(sites)),key=lambda j:dist(sites[j],tr.center))
0389 |                         score+=0.12*k*norm(sites[0])+dist(sites[k],tr.center)
0390 |                     options.append((score,a,cw))
0391 |         _,phi,clockwise=min(options)
0392 |     self.pending=ring_sites(self.cfg.ring_m,phi,clockwise)
0393 |     self._plan_ready=True
0394 |     self._certify_and_prune()
0395 |
0396 | def _tracking_point(self,t:Track,next_site:Point|None) -> Point:
0397 |     c,r=t.center,t.radius
0398 |     if r<=19.8: return nearest_certified_clear(self.p,c,r)
0399 |     d=unit(sub(c,self.p)); v=(-d[1],d[0])
0400 |     b=self.cfg.offset*r
0401 |     candidates=[add(c,mul(v,b)),add(c,mul(v,-b))] if b>0 else [c]
0402 |     # Whole feasible set must remain in guaranteed signal range.
0403 |     valid=[q for q in candidates if maxdist(q,t.poly)<=999.0]
0404 |     if not valid: return c # first positive set always has radius below 751m
0405 |     if next_site is None:
0406 |         return min(valid,key=lambda q:dist(self.p,q))
0407 |     return min(valid,key=lambda q:dist(self.p,q)+0.15*dist(q,next_site))
0408 |
0409 | def _serve(self,i:int,next_site:Point|None):
0410 |     t=self.tracks[i]; steps=0
0411 |     while t.status=='FOUND':
0412 |         if steps>=18: raise RuntimeError('local tracking bound exceeded')
0413 |         if t.radius<=19.8:
0414 |             q=nearest_certified_clear(self.p,t.center,t.radius)
0415 |             self._clear(i,q); break
0416 |         if self.cfg.speculative_r>0 and t.radius<=self.cfg.speculative_r:
0417 |             # Research-only risk-taking; frozen policy clears by certificate.
0418 |             if self._clear(i,t.center,certified=False): break
0419 |         q=self._tracking_point(t,next_site)
0420 |         oldr=t.radius
0421 |         result=self._observe(i,q); steps+=1

```

```

0422 |     if result=='no_signal':
0423 |         # This is an invariant failure, not a cue for random exploration.
0424 |         raise RuntimeError('lost signal at a guaranteed reception point')
0425 |     if result=='direction' and t.radius>0.9*oldr+1e-3:
0426 |         raise RuntimeError('local contraction invariant failed')
0427 |     self.max_local_steps=max(self.max_local_steps,steps)
0428 |     if self.cfg.opportunistic_clear:
0429 |         self._sweep_found(self.p)
0430 |     self._certify_and_prune()
0431 |     if self.cfg.incidental and not self.coverage_done and self.cleared<16:
0432 |         # Research-only extra full scans, disabled in the frozen policy.
0433 |         if self._worth_extra_scan():
0434 |             self._scan(self.p)
0435 |
0436 | def _worth_extra_scan(self) -> bool:
0437 |     """Local research helper, reachable only with incidental=True."""
0438 |     if self.cfg.scan_policy=='distance':
0439 |         return min((dist(self.p,s) for s in self.scans),default=math.inf)>=self.cfg.scan_spacing
0440 |     if not self.pending:return False
0441 |     hypothetical=list(self.pending); k=0
0442 |     while k<len(hypothetical):
0443 |         rest=hypothetical[:k]+hypothetical[k+1:]
0444 |         if self._coverage(self.scans+[self.p]+rest):hypothetical.pop(k)
0445 |         else:k+=1
0446 |     if len(hypothetical)==len(self.pending):return False
0447 |     def length(sites):
0448 |         return sum(dist(a,b) for a,b in zip([self.p]+sites,sites))
0449 |     movement_saved=(length(self.pending)-length(hypothetical))/5.
0450 |     q=sum(t.status=='UNKNOWN' for t in self.tracks.values())
0451 |     # Compare an explicit path-saving proxy with an upper bound on scan cost.
0452 |     return movement_saved>6.*q+5.
0453 |
0454 | def _choose(self,next_site:Point|None) -> int|None:
0455 |     found=[i for i,t in self.tracks.items() if t.status=='FOUND']
0456 |     if not found: return None
0457 |     if next_site is None or self.cfg.mode=='sequential':
0458 |         return min(found,key=lambda i:dist(self.p,self.tracks[i].center)+self.tracks[i].radius)
0459 |     if self.cfg.mode=='deferred': return None
0460 |     values=[]
0461 |     for i in found:
0462 |         t=self.tracks[i]; c,r=t.center,t.radius
0463 |         detour=dist(self.p,c)+dist(c,next_site)-dist(self.p,next_site)
0464 |         # A service excursion may end away from c. Penalize uncertainty in meters.
0465 |         score=detour+1.0*r
0466 |         values.append((score,i))
0467 |     score,i=min(values)
0468 |     return i if score<=self.cfg.insert_m else None
0469 |
0470 | def run(self) -> dict:
0471 |     self._scan((0.,0.)); self._make_plan()
0472 |     loops=0
0473 |     while True:
0474 |         loops+=1
0475 |         if loops>1000 or self.actions>5000: raise RuntimeError('progress limit exceeded')
0476 |         if self.cleared==16:
0477 |             # Total cardinality upper bound itself is a valid completion certificate.
0478 |             break
0479 |         found=[i for i,t in self.tracks.items() if t.status=='FOUND']
0480 |         unknown=[i for i,t in self.tracks.items() if t.status=='UNKNOWN']
0481 |         if not found and not unknown: break
0482 |         next_site=self.pending[0] if self.pending else None
0483 |         i=self._choose(next_site)
0484 |         if i is not None:
0485 |             self._serve(i,next_site)
0486 |         elif next_site is not None:
0487 |             self.pending.pop(0); self._scan(next_site)
0488 |         elif unknown:
0489 |             # Planned union was validated. Never mark channels absent just from counts.
0490 |             self._certify_and_prune()
0491 |             if not self.coverage_done: raise RuntimeError('coverage certificate missing')
0492 |         else: break
0493 |     return dict(cleared=self.cleared,virtual_time_s=self.virtual_time,
0494 |               average_time_s=self.virtual_time/self.cleared if self.cleared else None,
0495 |               actions=self.actions,scan_sites=len(self.scans),coverage_certified=self.search_reason=='disk_union',search_certificate=self.search_reason,
0496 |               certificate_checks=self.certificate_checks,max_vertices=self.max_vertices,
0497 |               max_local_steps=self.max_local_steps,
0498 |               channel_status={str(i):t.status for i,t in self.tracks.items()})
0499 |
0500 |

```

```

0501 | def run_case(api, audit=None):
0502 |     return Strategy(api,Config(),audit=audit).run()

```

source/Q3/q3_official_solver.py (第 503 行起; 前缀见本地版)

```

0503 |
0504 |
0505 | import argparse,datetime,http.client,json,os,socket,sys,time,unicodedata,uuid
0506 | from dataclasses import asdict
0507 | from pathlib import Path
0508 | from urllib.parse import urlsplit
0509 |
0510 | class ProtocolError(RuntimeError):pass
0511 | class BudgetExceeded(RuntimeError):pass
0512 | class ResultUnknown(RuntimeError):pass
0513 |
0514 | class OfficialClient:
0515 |     def __init__(self,base_url,robot_id,log_path,timeout=5.0,retries=6):
0516 |         parsed=urlsplit(base_url)
0517 |         if parsed.scheme!='http' or not parsed.hostname or parsed.path not in ('','/') or parsed.query or parsed.fragment:
0518 |             raise ValueError('接口地址格式错误')
0519 |         if parsed.hostname not in ('127.0.0.1','localhost','::1'):
0520 |             raise ValueError('官方接口只能使用本机回环地址')
0521 |         if not robot_id or len(robot_id.encode('utf-8'))>64 or any(unicodedata.category(c) in ('Cc','Cf') for c in robot_id):
0522 |             raise ValueError('参赛队号格式不合法')
0523 |         self.host=parsed.hostname;self.port=parsed.port or 80;self.robot_id=robot_id
0524 |         self.timeout=timeout;self.retries=retries;self.conn=None
0525 |         self.prefix=uuid.uuid4().hex[:12];self.serial=0;self.deadline=None
0526 |         self.virtual_time=0.0;self.max_virtual=360000.0;self.entered=False;self.exited=False;self.retry_count=0
0527 |         self.position=(0.0,0.0);self.current_channel=1;self.pending=None
0528 |         self.exit_attempted=False;self.exit_response=None
0529 |         self.log_failed=False;self.log_error=None
0530 |         p=Path(log_path);p.parent.mkdir(parents=True,exist_ok=True);self.log=p.open('x',encoding='utf-8')
0531 |         self._log({'kind':'metadata','program':'Q3','robot_id':'<TEAM_ID>','utc':datetime.datetime.now(datetime.timezone.utc).isoformat()})
0532 |     def _log(self,obj):
0533 |         try:
0534 |             self.log.write(json.dumps(obj,ensure_ascii=False,allow_nan=False,separators=(',',':'))+'\n');self.log.flush()
0535 |         except Exception as exc:
0536 |             self.log_failed=True;self.log_error=repr(exc)
0537 |             print(f'日志写入失败: {exc}',file=sys.stderr,flush=True)
0538 |     def _disconnect(self):
0539 |         if self.conn:
0540 |             try:self.conn.close()
0541 |             except OSError:pass
0542 |         self.conn=None
0543 |     def _connection(self,timeout=None):
0544 |         if self.conn is None:
0545 |             c=http.client.HTTPConnection(self.host,self.port,timeout=timeout or self.timeout);c.connect()
0546 |             if c.sock.c.sock.setsockopt(socket.IPPROTO_TCP,socket.TCP_NODELAY,1)
0547 |                 self.conn=c
0548 |             if timeout is not None:self.conn.timeout=timeout
0549 |             if self.conn.sock and timeout is not None:self.conn.sock.settimeout(timeout)
0550 |             return self.conn
0551 |     def _safe_payload(self,payload):
0552 |         out=dict(payload);out['robot_id']='<TEAM_ID>';return out
0553 |     @staticmethod
0554 |     def _finite(value):
0555 |         if isinstance(value,bool) or not isinstance(value,(int,float)):return False
0556 |         try:return math.isfinite(float(value))
0557 |         except (OverflowError,TypeError,ValueError):return False
0558 |     def _cutoff(self,path):
0559 |         if self.deadline is None:return None
0560 |         return self.deadline-(0.25 if path=='/exit' else 8.0)
0561 |     def _attempt_timeout(self,path):
0562 |         cutoff=self._cutoff(path)
0563 |         if cutoff is None:return self.timeout
0564 |         remaining=cutoff-time.monotonic()
0565 |         if remaining<=0:raise BudgetExceeded('现实运行时间不足, 停止发送新动作')
0566 |         return max(0.05,min(self.timeout,remaining))
0567 |     def _validate_success(self,path,status,obj):
0568 |         if not isinstance(obj,dict):raise ResultUnknown('响应JSON顶层不是对象')
0569 |         if not isinstance(obj.get('accepted'),bool):raise ResultUnknown('accepted字段异常')
0570 |         for key in ('real_timestamp_ms','virtual_time_s'):
0571 |             if not self._finite(obj.get(key)) or float(obj[key])<0:raise ResultUnknown(f'{key}字段异常')
0572 |         if status!=200:
0573 |             if obj['accepted'] is False:raise ProtocolError(f'HTTP {status}: 请求已明确拒绝')

```

```

0574 |     raise ResultUnknown(f'HTTP {status}与accepted=true矛盾')
0575 | if obj['accepted'] is not True:raise ProtocolError('请求未被接受')
0576 | if float(obj['virtual_time_s'])+1e-5<self.virtual_time:raise ResultUnknown('virtual_time_s倒退')
0577 | if path=='/enter':
0578 |     for key in ('max_virtual_duration_s','max_real_duration_s','remaining_real_duration_s'):
0579 |         if not self._finite(obj.get(key)) or float(obj[key])<0:raise ResultUnknown(f'{key}字段异常')
0580 |     rem=float(obj['remaining_real_duration_s'])
0581 |     if rem>1200 or not rem.is_integer():raise ResultUnknown('remaining_real_duration_s异常')
0582 | elif path=='/measure':
0583 |     kind=obj.get('measure_result')
0584 |     if kind not in ('no_signal','near','direction'):raise ResultUnknown('measure_result异常')
0585 |     if kind=='direction' and (not self._finite(obj.get('svd_deg')) or not 0<=float(obj['svd_deg'])<360):raise ResultUnknown('svd_deg异常')
0586 | elif path=='/clear' and obj.get('clear_result') not in ('success','no_target_in_range'):
0587 |     raise ResultUnknown('clear_result异常')
0588 | elif path=='/exit' and obj.get('exit_reason')!='user_exit':raise ResultUnknown('exit_reason异常')
0589 | def _request(self,path,payload,body=None):
0590 |     body=body or json.dumps(payload,ensure_ascii=False,separators=(',',':'),allow_nan=False).encode('utf-8')
0591 |     if self.pending is None:
0592 |         self.pending={'path':path,'payload':dict(payload),'body':body}
0593 |         self._log({'kind':'request','path':path,'payload':self._safe_payload(payload)})
0594 |     last=None;unknown=False
0595 |     for attempt in range(self.retries+1):
0596 |         started=time.monotonic()
0597 |         try:
0598 |             timeout=self._attempt_timeout(path);c=self._connection(timeout)
0599 |             c.request('POST',path,body,{'Content-Type':'application/json'; charset=utf-8'})
0600 |             r=c.getresponse();status=r.status;raw=r.read()
0601 |             try:obj=json.loads(raw.decode('utf-8'),parse_constant=lambda x:(_ for _ in []).throw(ValueError(x)))
0602 |             except (ValueError,UnicodeDecodeError) as exc:
0603 |                 self._log({'kind':'invalid_response','path':path,'request_id':payload['request_id'],'attempt':attempt,'http_status':status,'raw':raw[:1000].d
↵ encode('utf-8','replace'))
0604 |                 raise ResultUnknown('响应不是有效JSON') from exc
0605 |                 self._log({'kind':'response','path':path,'request_id':payload['request_id'],'attempt':attempt,'elapsed_s':time.monotonic()-started,'http_stat
↵ us':status,'body':obj})
0606 |                 if status in (429,500,502,503,504):
0607 |                     common_ok=(isinstance(obj,dict) and obj.get('accepted') is False
0608 |                         and self._finite(obj.get('real_timestamp_ms'))
0609 |                         and self._finite(obj.get('virtual_time_s')))
0610 |                     if not common_ok:raise ResultUnknown(f'HTTP {status}响应状态不可信')
0611 |                     raise OSError(f'HTTP {status}')
0612 |                 try:self._validate_success(path,status,obj)
0613 |                 except ProtocolError:
0614 |                     self.pending=None
0615 |                     raise
0616 |                 self.virtual_time=float(obj['virtual_time_s']);self.pending=None;return obj
0617 |             except BudgetExceeded:
0618 |                 if not unknown:
0619 |                     self.pending=None
0620 |                     self._log({'kind':'aborted_before_send','path':path,'request_id':payload['request_id'],'attempt':attempt})
0621 |                     raise
0622 |             except ProtocolError:raise
0623 |             except (OSError,TimeoutError,http.client.HTTPException,ResultUnknown) as exc:
0624 |                 last=exc;unknown=unknown or isinstance(exc,ResultUnknown) or not (isinstance(exc,OSError) and str(exc).startswith('HTTP '))
0625 |                 self._disconnect();self._log({'kind':'retry','path':path,'request_id':payload['request_id'],'attempt':attempt,'elapsed_s':time.monotonic()-start
↵ ed,'error':repr(exc)})
0626 |                 if attempt==self.retries:break
0627 |                 self.retry_count+=1
0628 |                 delay=min(0.1*2**attempt,1.5);cutoff=self._cutoff(path)
0629 |                 if cutoff is not None:delay=min(delay,max(0.0,cutoff-time.monotonic()))
0630 |                 if delay<=0:break
0631 |                 time.sleep(delay)
0632 |             if unknown:raise ResultUnknown(f'动作结果未知, 只能保留原request_id: {last}')
0633 |             self.pending=None;raise ConnectionError(f'同一请求重试后仍无有效响应: {last}')
0634 | def action(self,path,position=None,channel=None):
0635 |     if path not in ('/enter','/measure','/clear','/exit'):raise ValueError('接口路径不合法')
0636 |     if self.pending is not None:raise ResultUnknown('上一动作结果未知, 禁止生成新的request_id')
0637 |     if path in ('/measure','/clear') and (position is None or channel is None):raise ValueError('动作缺少位置或频道')
0638 |     if path in ('/enter','/exit') and (position is not None or channel is not None):raise ValueError('enter/exit不应包含位置或频道')
0639 |     clean_position=None
0640 |     if position is not None:
0641 |         if len(position)!=2 or not all(self._finite(v) and abs(float(v))<=2e6 for v in position):raise ValueError('坐标不合法')
0642 |         clean_position=(float(position[0]),float(position[1]))
0643 |     if channel is not None and (isinstance(channel,bool) or not isinstance(channel,int) or not 1<=channel<=20):raise ValueError('频道不合法')
0644 |     if path in ('/measure','/clear'):
0645 |         upper=dist(self.position,clean_position)/5.0+5.0
0646 |         if path=='/measure' and channel!=self.current_channel:upper+=1.0
0647 |         if self.virtual_time+upper>self.max_virtual-0.001:raise BudgetExceeded('下一动作将超过虚拟时间上限')
0648 |     self._attempt_timeout(path)

```

```

0649 | self.serial+=1
0650 | payload={'arena_id':'default','robot_id':self.robot_id,'request_id':f'{self.prefix}-{self.serial}'}
0651 | if clean_position is not None:
0652 |     payload['position']={'x':clean_position[0],'y':clean_position[1]}
0653 | if channel is not None:
0654 |     payload['channel']=channel
0655 | out=self._request(path,payload)
0656 | if path in ('/measure','/clear'):self.position=clean_position
0657 | if path=='/measure':self.current_channel=channel
0658 | return out
0659 | def wait_until_open(self,wait_s=300.0):
0660 |     end=time.monotonic()+wait_s
0661 |     while time.monotonic()<end:
0662 |         try:
0663 |             with socket.create_connection((self.host,self.port),timeout=.5):return
0664 |         except OSError:time.sleep(.25)
0665 |     raise TimeoutError('等待官方模拟器接口超时')
0666 | def enter(self):
0667 |     started=time.monotonic();out=self.action('/enter');rem=out.get('remaining_real_duration_s')
0668 |     self.deadline=started+float(rem);self.max_virtual=float(out['max_virtual_duration_s']);self.entered=True;return out
0669 | def measure(self,p,ch):return self.action('/measure',p,ch)
0670 | def clear(self,p,ch):return self.action('/clear',p,ch)
0671 | def exit(self):
0672 |     if self.exited:return self.exit_response
0673 |     self.exit_attempted=True
0674 |     out=self.action('/exit');self.exited=True;self.exit_response=out;return out
0675 | def can_exit(self):return self.entered and not self.exited and not self.exit_attempted and self.pending is None and (self.deadline is None or
↪ time.monotonic()<self.deadline-0.25)
0676 | def close(self):
0677 |     self._disconnect()
0678 |     try:self.log.close()
0679 |     except OSError as exc:
0680 |         self.log_failed=True;self.log_error=repr(exc)
0681 |         print(f'日志关闭失败: {exc}',file=sys.stderr,flush=True)
0682 |
0683 | def main():
0684 |     ap=argparse.ArgumentParser(description='问题三正式测试程序')
0685 |     ap.add_argument('--robot-id',default=None,help='参赛队号; 不写入代码')
0686 |     ap.add_argument('--url',default='http://127.0.0.1:2026')
0687 |     ap.add_argument('--wait',type=float,default=300.0)
0688 |     args=ap.parse_args()
0689 |     robot_id=args.robot_id or os.environ.get('CUMCM_ROBOT_ID') or input('请输入参赛队号 (仅本次运行使用): ').strip()
0690 |     stamp=datetime.datetime.now().strftime('%Y%m%d_%H%M%S%f')
0691 |     log=f'runs/q3_{stamp}.json';api=OfficialClient(args.url,robot_id,log)
0692 |     ok=False;started=time.perf_counter();summary=None
0693 |     try:
0694 |         print('等待问题3测试接口开放.....',flush=True);api.wait_until_open(args.wait);api.enter();started=time.perf_counter()
0695 |         print('已进入测试, 开始自动搜索定位与清除.',flush=True)
0696 |         summary=run_case(api);api.exit()
0697 |         summary.update(program_runtime_s=time.perf_counter()-started,retries=api.retry_count)
0698 |         Path(log).with_suffix('.summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
0699 |         api._log({'kind':'summary','strategy_complete':True,'exit_confirmed':api.exited,**summary});ok=True
0700 |         if api.log_failed:raise OSError(f'运行已完成, 但结构化日志不完整: {api.log_error}')
0701 |         average='N/A' if summary['average_time_s'] is None else f'{summary["average_time_s"]:.3f}'
0702 |         print(f'完成: 清除 {summary["cleared"]} 个频道, 虚拟时间 {summary["virtual_time_s"]:.3f} s, 平均 {average} s/源")
0703 |         print(f'本地运行记录: {log}')
0704 |     except (Exception,KeyboardInterrupt) as exc:
0705 |         ok=False
0706 |         api._log({'kind':'fatal','error':repr(exc),'complete':False});print(f'运行未完成: {exc}',file=sys.stderr,flush=True)
0707 |         if api.can_exit():
0708 |             try:api.exit()
0709 |             except Exception as exit_exc:api._log({'kind':'exit_failure','error':repr(exit_exc),'pending':api.pending and
↪ {'path':api.pending['path'],'request_id':api.pending['payload']}['request_id']})
0710 |         finally:api.close()
0711 |     return 0 if ok and not api.log_failed else 1
0712 | if __name__=='__main__':raise SystemExit(main())

```

source/Q3/q3_local_simulator.py

```

0001 | #!/usr/bin/env python3
0002 | from __future__ import annotations
0003 | from dataclasses import dataclass
0004 | import argparse,datetime,json,math,random,time
0005 | from pathlib import Path
0006 | import q3_local_solver as solver
0007 |

```

```

0008 | @dataclass(frozen=True)
0009 | class Source:
0010 |     channel:int;position:tuple[float,float];radius:float;phase:float
0011 |
0012 | class Simulator:
0013 |     def __init__(self,sources,seed):
0014 |         self.sources=[s.channel:s for s in sources];self.cleared=set();self.p=(0.0,0.0);self.channel=1
0015 |         self.time=0.0;self.distance=0.0;self.measurements=0;self.switches=0;self.failed_clears=0;self.seed=seed
0016 |     def _move(self,p):
0017 |         d=math.dist(self.p,p);self.distance+=d;self.time+=d/5.0;self.p=p
0018 |     def _error(self,s,p):
0019 |         return max(-1.0,min(1.0,0.62*math.sin(p[0]/211+p[1]/173+s.phase)+0.38*math.cos(p[1]/359-p[0]/283+s.phase*1.37)))
0020 |     def measure(self,p,ch):
0021 |         self._move(p);self.time+=5;self.measurements+=1
0022 |         if ch!=self.channel:self.time+=1;self.switches+=1
0023 |         self.channel=ch;s=self.sources.get(ch);out={'accepted':True,'virtual_time_s':self.time}
0024 |         if s is None or ch in self.cleared or math.dist(p,s.position)>s.radius:out['measure_result']='no_signal'
0025 |         elif math.dist(p,s.position)<=5:out['measure_result']='near'
0026 |         else:
0027 |             a=math.degrees(math.atan2(s.position[1]-p[1],s.position[0]-p[0]))
0028 |             out.update(measure_result='direction',svd_deg=round((a+self._error(s,p))%360,2))
0029 |         return out
0030 |     def clear(self,p,ch):
0031 |         self._move(p);s=self.sources.get(ch);ok=s is not None and ch not in self.cleared and math.dist(p,s.position)<=20
0032 |         if ok:self.cleared.add(ch);self.time+=5
0033 |         else:self.time+=3;self.failed_clears+=1
0034 |         return {'accepted':True,'virtual_time_s':self.time,'clear_result': 'success' if ok else 'no_target_in_range'}
0035 |
0036 |     def make_case(seed):
0037 |         r=random.Random(seed);n=r.randint(10,16);chs=r.sample(range(1,21),n);out=[]
0038 |         for ch in chs:
0039 |             a=r.random()*math.tau;rr=1800*math.sqrt(r.random());p=(rr*math.cos(a),rr*math.sin(a))
0040 |             out.append(Source(ch,p,r.uniform(1000,1500),r.random()*math.tau))
0041 |         return out
0042 |
0043 |     def _average(cases):
0044 |         if not cases:return None
0045 |         n=len(cases);sources=sum(c['source_count'] for c in cases);cleared=sum(c['cleared'] for c in cases)
0046 |         per_source=[c['average_time_s'] for c in cases if c['average_time_s'] is not None]
0047 |         return {
0048 |             'case_count':n,
0049 |             'source_count_per_case':sum(c['source_count'] for c in cases)/n,
0050 |             'cleared_per_case':sum(c['cleared'] for c in cases)/n,
0051 |             'virtual_time_s_per_case':sum(c['virtual_time_s'] for c in cases)/n,
0052 |             'case_equal_mean_time_s_per_source':sum(per_source)/len(per_source) if per_source else None,
0053 |             'pooled_time_s_per_cleared_source':sum(c['virtual_time_s'] for c in cases)/cleared if cleared else None,
0054 |             'distance_m_per_case':sum(c['distance_m'] for c in cases)/n,
0055 |             'movement_time_s_per_case':sum(c['distance_m'] for c in cases)/(5*n),
0056 |             'measurement_time_s_per_case':5*sum(c['measurements'] for c in cases)/n,
0057 |             'switch_time_s_per_case':sum(c['switches'] for c in cases)/n,
0058 |             'clear_action_time_s_per_case':(5*cleared+3*sum(c['failed_clears'] for c in cases))/n,
0059 |             'measurements_per_case':sum(c['measurements'] for c in cases)/n,
0060 |             'switches_per_case':sum(c['switches'] for c in cases)/n,
0061 |             'failed_clears_per_case':sum(c['failed_clears'] for c in cases)/n,
0062 |         }
0063 |
0064 |     def _print_average(title,avg):
0065 |         if not avg:return
0066 |         print(title)
0067 |         print(f' 平均每局源数: {avg["source_count_per_case"]:.2f}')
0068 |         print(f' 平均每局清除数: {avg["cleared_per_case"]:.2f}')
0069 |         print(f' 平均每局总耗时: {avg["virtual_time_s_per_case"]:.2f} s')
0070 |         if avg['case_equal_mean_time_s_per_source'] is not None:print(f' 逐局等权平均每源耗时: {avg["case_equal_mean_time_s_per_source"]:.2f} s')
0071 |         if avg['pooled_time_s_per_cleared_source'] is not None:print(f' 合并源数加权每源耗时: {avg["pooled_time_s_per_cleared_source"]:.2f} s')
0072 |         print(f' 平均每局移动距离: {avg["distance_m_per_case"]:.2f} m')
0073 |         print(f' 时间分解 (移动/检测/换台/清除) : {avg["movement_time_s_per_case"]:.2f} / {avg["measurement_time_s_per_case"]:.2f} /
↪ {avg["switch_time_s_per_case"]:.2f} / {avg["clear_action_time_s_per_case"]:.2f} s')
0074 |         print(f' 平均每局测向次数: {avg["measurements_per_case"]:.2f}')
0075 |         print(f' 平均每局切频道次数: {avg["switches_per_case"]:.2f}')
0076 |         print(f' 平均每局失败清除次数: {avg["failed_clears_per_case"]:.3f}')
0077 |
0078 |     def main():
0079 |         ap=argparse.ArgumentParser(description='问题三本地连续随机测试')
0080 |         ap.add_argument('--loops',type=int,default=None);ap.add_argument('--seed',type=int,default=None)
0081 |         ap.add_argument('--output',type=Path,default=None);args=ap.parse_args()
0082 |         loops=args.loops if args.loops is not None else int(input('请输入测试局数: ').strip()) or 1000'
0083 |         base=args.seed if args.seed is not None else random.SystemRandom().randrange(1,2**63)
0084 |         rng=random.Random(base);cases=[];start=time.perf_counter();success=0
0085 |         for i in range(loops):

```

```

0086 | seed=rng.randrange(1,2**63);src=make_case(seed);sim=Simulator(src,seed)
0087 | try:
0088 |     res=solver.run_case(sim);all_clear=len(sim.cleared)==len(src);err=None
0089 | except Exception as exc:
0090 |     res={};all_clear=False;err=repr(exc)
0091 | success+=int(all_clear)
0092 | cases.append({'index':i+1,'seed':seed,'source_count':len(src),'all_cleared':all_clear,
0093 |             'cleared':len(sim.cleared),'virtual_time_s':sim.time,'average_time_s':sim.time/len(sim.cleared) if sim.cleared else None,
0094 |             'distance_m':sim.distance,'measurements':sim.measurements,'switches':sim.switches,'failed_clears':sim.failed_clears,
0095 |             'solver':res,'error':err})
0096 | if (i+1)%max(1,min(100,loops//20 or 1))==0:print(f'进度: {i+1}/{loops}, 全清除 {success}/{i+1}',flush=True)
0097 |
0098 | good=[c for c in cases if c['all_cleared']]
0099 | avg_all=_average(cases);avg_good=_average(good)
0100 | summary={
0101 |     'problem':3,'loops':loops,'base_seed':base,
0102 |     'full_clear_cases':success,'full_clear_rate':success/loops if loops else 0,
0103 |     'average_all_cases':avg_all,
0104 |     'average_successful_cases':avg_good,
0105 |     'mean_virtual_time_s':avg_good['virtual_time_s_per_case'] if avg_good else None,
0106 |     'mean_average_time_s':avg_good['case_equal_mean_time_s_per_source'] if avg_good else None,
0107 |     'pooled_average_time_s':avg_good['pooled_time_s_per_cleared_source'] if avg_good else None,
0108 |     'runtime_s':time.perf_counter()-start,'cases':cases
0109 | }
0110 | out=args.output or Path('results')/f'q3_local_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
0111 | out.parent.mkdir(parents=True,exist_ok=True);out.write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
0112 |
0113 | print()
0114 | print(f'完成: {success}/{loops} 局全清除, 全清除率 {summary["full_clear_rate"]:.2%}')
0115 | _print_average('【全部测试局平均】',avg_all)
0116 | if success != loops:_print_average('【全清除成功局平均】',avg_good)
0117 | print(f'本机实际运行耗时: {summary["runtime_s"]:.2f} s')
0118 | print(f'结果已保存: {out}')
0119 |
0120 | if __name__=='__main__':main()

```

source/Q3/run_local.bat

```

0001 | @echo off
0002 | setlocal
0003 | chcp 65001 >nul
0004 | pushd "%~dp0" || exit /b 1
0005 | python q3_local_simulator.py %*
0006 | set "RC=%ERRORLEVEL%"
0007 | popd
0008 | pause
0009 | exit /b %RC%

```

source/Q3/run_official.bat

```

0001 | @echo off
0002 | setlocal
0003 | chcp 65001 >nul
0004 | pushd "%~dp0" || exit /b 1
0005 | python q3_official_solver.py %*
0006 | set "RC=%ERRORLEVEL%"
0007 | popd
0008 | pause
0009 | exit /b %RC%

```

source/Q4/q4_local_solver.py

```

0001 | #!/usr/bin/env python3
0002 | from __future__ import annotations
0003 | from dataclasses import dataclass,field
0004 | import math,random
0005 |
0006 | Point=tuple[float,float]
0007 | TAU=2*math.pi
0008 | EPS=1e-7
0009 | DELTA=math.radians(1.0051) # 1° physical bound + 0.005° output rounding + 0.0001° guard

```

```

0010 | CLEAR_CERT=19.8
0011 | LAMBDA=0.5
0012 | ETA=0.05
0013 | REUSE_MIN_BASELINE=25.0
0014 |
0015 | def add(a,b):return a[0]+b[0],a[1]+b[1]
0016 | def sub(a,b):return a[0]-b[0],a[1]-b[1]
0017 | def mul(a,s):return a[0]*s,a[1]*s
0018 | def dot(a,b):return a[0]*b[0]+a[1]*b[1]
0019 | def cross(a,b):return a[0]*b[1]-a[1]*b[0]
0020 | def norm(a):return math.hypot(*a)
0021 | def dist(a,b):return math.hypot(a[0]-b[0],a[1]-b[1])
0022 | def maxdist(p,poly):return max((dist(p,v) for v in poly),default=0.0)
0023 |
0024 | def clip(poly,n,b):
0025 |     if not poly:return []
0026 |     b+=EPS*max(1.0,norm(n));out=[];a=poly[-1];fa=dot(n,a)-b
0027 |     for c in poly:
0028 |         fc=dot(n,c)-b
0029 |         if (fa<=0)!= (fc<=0):
0030 |             t=fa/(fa-fc);out.append(add(a,mul(sub(c,a),t)))
0031 |         if fc<=0:out.append(c)
0032 |         a,fa=c,fc
0033 |     clean=[]
0034 |     for p in out:
0035 |         if not clean or dist(p,clean[-1])>1e-9:clean.append(p)
0036 |     if len(clean)>1 and dist(clean[0],clean[-1])<1e-9:clean.pop()
0037 |     return clean
0038 |
0039 | def arena_polygon(sides=64):
0040 |     r=1800/math.cos(math.pi/sides)
0041 |     return [(r*math.cos(TAU*k/sides+math.pi/sides),r*math.sin(TAU*k/sides+math.pi/sides)) for k in range(sides)]
0042 |
0043 | def direction_update(poly,p,theta_deg,U):
0044 |     a=math.radians(theta_deg);lo=(math.cos(a-DELTA),math.sin(a-DELTA));hi=(math.cos(a+DELTA),math.sin(a+DELTA))
0045 |     for n in ((lo[1],-lo[0]),(-hi[1],hi[0])):poly=clip(poly,n,dot(n,p))
0046 |     u=(math.cos(a),math.sin(a));poly=clip(poly,u,dot(u,p)+U)
0047 |     return poly
0048 |
0049 | def _diameter_circle(a,b):
0050 |     c=mul(add(a,b),.5);return c,dist(a,b)/2
0051 |
0052 | def _circum(a,b,c):
0053 |     u=sub(b,a);v=sub(c,a);d=2*cross(u,v)
0054 |     if abs(d)<1e-18:return None
0055 |     uu=dot(u,u);vv=dot(v,v);o=add(a,((v[1]*uu-u[1]*vv)/d,(u[0]*vv-v[0]*uu)/d))
0056 |     return o,dist(o,a)
0057 |
0058 | def mec(points):
0059 |     if not points:raise RuntimeError('可行位置集合为空')
0060 |     pts=list(points);random.Random(271828).shuffle(pts);circle=None
0061 |     for i,p in enumerate(pts):
0062 |         if circle and dist(circle[0],p)<=circle[1]+1e-9:continue
0063 |         circle=(p,0.0)
0064 |         for j,q in enumerate(pts[i:]):
0065 |             if dist(circle[0],q)<=circle[1]+1e-9:continue
0066 |             circle=_diameter_circle(p,q);pq=sub(q,p);left=right=None
0067 |             for r in pts[j:]:
0068 |                 if dist(circle[0],r)<=circle[1]+1e-9:continue
0069 |                 cc=_circum(p,q,r)
0070 |                 if cc is None:continue
0071 |                 side=cross(pq,sub(r,p));v=cross(pq,sub(cc[0],p))
0072 |                 if side>0 and (left is None or v>left[0]):left=v,cc
0073 |                 if side<0 and (right is None or v<right[0]):right=v,cc
0074 |                 options=[x[1] for x in (left,right) if x is not None]
0075 |                 if options:circle=min(options,key=lambda z:z[1])
0076 |     c=circle[0];return c,maxdist(c,points)+1e-6
0077 |
0078 | def nearest_clear(p,c,r):
0079 |     slack=CLEAR_CERT-r
0080 |     if slack<0:raise ValueError('尚未达到清除证书')
0081 |     d=dist(p,c)
0082 |     if d<=slack:return p
0083 |     return add(c,mul(sub(p,c),slack/d))
0084 |
0085 | def coverage_route():
0086 |     inner=[(995*math.cos(k*math.pi/4),995*math.sin(k*math.pi/4)) for k in range(8)]
0087 |     outer=[(1838*math.cos(j*math.pi/8),1838*math.sin(j*math.pi/8)) for j in range(16)]
0088 |     order=[14,15]+list(range(14))

```

```

0089 | return [(0.0,0.0)]+inner+[outer[j] for j in order]
0090 |
0091 | @dataclass
0092 | class Track:
0093 |     status:str='UNKNOWN'
0094 |     poly:list[Point]=field(default_factory=arena_polygon)
0095 |     center:Point=(0.0,0.0)
0096 |     radius:float=math.inf
0097 |     anchor:Point|None=None
0098 |     last:Point|None=None
0099 |     bearing_deg:float=0.0
0100 |     U:float=1500.0
0101 |     measurements:int=0
0102 |
0103 | class Strategy:
0104 |     def __init__(self,api):
0105 |         self.api=api;self.tracks=[i:Track() for i in range(1,21)];self.p=(0.0,0.0);self.channel=1
0106 |         self.virtual_time=0.0;self.actions=0;self.cleared=0;self.route=coverage_route();self.coverage_index=0
0107 |         self.coverage_done=False;self.max_pair_rounds=0
0108 |     def _update_mec(self,t):t.center,t.radius=mec(t.poly)
0109 |     def _positive(self,i,p,bearing):
0110 |         t=self.tracks[i];U=min(1500.0,maxdist(p,t.poly)+EPS);new=direction_update(t.poly,p,bearing,U)
0111 |         if not new:raise RuntimeError('频道{i}的位置集合为空')
0112 |         t.status='FOUND';t.poly=new;t.anchor=p;t.bearing_deg=float(bearing);t.U=min(U,maxdist(p,new)+EPS);t.measurements+=1;self._update_mec(t)
0113 |     def _clear(self,i,p,certified=True):
0114 |         r=self.api.clear(p,i)
0115 |         if not r.get('accepted'):raise RuntimeError('清除请求被拒绝')
0116 |         self.p=p;self.virtual_time=float(r['virtual_time_s']);self.actions+=1
0117 |         if r.get('clear_result')=='success':self.tracks[i].status='CLEARED';self.cleared+=1;return True
0118 |         if r.get('clear_result')!='no_target_in_range':raise RuntimeError('未知清除结果')
0119 |         if certified:raise RuntimeError('频道{i}认证清除失败')
0120 |         return False
0121 |     def _observe(self,i,p):
0122 |         r=self.api.measure(p,i)
0123 |         if not r.get('accepted'):raise RuntimeError('检测请求被拒绝')
0124 |         self.p=p;self.channel=i;self.virtual_time=float(r['virtual_time_s']);self.actions+=1;self.tracks[i].last=p
0125 |         kind=r.get('measure_result')
0126 |         if kind=='direction':self._positive(i,p,float(r['svd_deg']))
0127 |         elif kind=='near':self.tracks[i].status='FOUND';self._clear(i,p,certified=True)
0128 |         elif kind!='no_signal':raise RuntimeError('未知检测结果')
0129 |         return kind
0130 |     def _known_count(self):return sum(t.status in ('FOUND','CLEARED') for t in self.tracks.values())
0131 |     def _finish_unknown_if_possible(self):
0132 |         if self._known_count()>=16:
0133 |             for t in self.tracks.values():
0134 |                 if t.status=='UNKNOWN':t.status='ABSENT'
0135 |             self.coverage_done=True
0136 |         elif self.coverage_index>=len(self.route):
0137 |             for t in self.tracks.values():
0138 |                 if t.status=='UNKNOWN':t.status='ABSENT'
0139 |             self.coverage_done=True
0140 |     def _reuse(self,limit=2):
0141 |         cand=[]
0142 |         for i,t in self.tracks.items():
0143 |             if t.status=='FOUND' and t.radius>CLEAR_CERT and maxdist(self.p,t.poly)<=999.0:
0144 |                 if t.last is None or dist(self.p,t.last)<REUSE_MIN_BASELINE:continue
0145 |                 cand.append((t.radius,i))
0146 |         for _i in sorted(cand,reverse=True)[:limit]:
0147 |             self._observe(i,self.p)
0148 |     def _scan_site(self,p):
0149 |         unknown=[i for i,t in self.tracks.items() if t.status=='UNKNOWN']
0150 |         unknown.sort(key=lambda i:(i!=self.channel,i))
0151 |         for i in unknown:
0152 |             self._observe(i,p)
0153 |             if self._known_count()>=16:break
0154 |         self._reuse(2);self.coverage_index+=1;self._finish_unknown_if_possible()
0155 |     def _dual_negative(self,t,anchor,u,U):
0156 |         t.poly=clip(t.poly,u,dot(u,anchor)+LAMBDA*U)
0157 |         if not t.poly:raise RuntimeError('双无信号裁剪后集合为空')
0158 |         t.U=min(LAMBDA/math.cos(DELTA)*U,maxdist(anchor,t.poly)+EPS);self._update_mec(t)
0159 |     def _pair_round(self,i):
0160 |         t=self.tracks[i]
0161 |         if t.anchor is None:raise RuntimeError('缺少正锚点')
0162 |         U=t.U;a=math.radians(t.bearing_deg);u=(math.cos(a),math.sin(a));v=(-u[1],u[0])
0163 |         q1=add(t.anchor,add(mul(u,LAMBDA*U),mul(v,ETA*U)));q2=add(t.anchor,add(mul(u,LAMBDA*U),mul(v,-ETA*U)))
0164 |         if dist(self.p,q2)<dist(self.p,q1):q1,q2=q2,q1
0165 |         r1=self._observe(i,q1)
0166 |         if self.tracks[i].status!='FOUND' or r1!='no_signal':return
0167 |         r2=self._observe(i,q2)

```

```

0168 | if self.tracks[i].status!='FOUND' or r2!='no_signal':return
0169 | self._dual_negative(t,t.anchor,u,U)
0170 | def _service(self,i):
0171 |     rounds=0
0172 |     while self.tracks[i].status=='FOUND':
0173 |         t=self.tracks[i]
0174 |         if t.radius<=CLEAR_CERT:
0175 |             self._clear(i,nearest_clear(self.p,t.center,t.radius));break
0176 |         if rounds>=9:raise RuntimeError(f'频道{i}成对探测超过上限')
0177 |         self._pair_round(i);rounds+=1
0178 |         self.max_pair_rounds=max(self.max_pair_rounds,rounds);self._reuse(2);self._finish_unknown_if_possible()
0179 | def _choose_insert(self,next_site):
0180 |     rows=[]
0181 |     for i,t in self.tracks.items():
0182 |         if t.status!='FOUND':continue
0183 |         if next_site is None:score=dist(self.p,t.center)+t.radius
0184 |         else:score=dist(self.p,t.center)+dist(t.center,next_site)-dist(self.p,next_site)+t.radius
0185 |         rows.append((score,i))
0186 |         if not rows:return None
0187 |         score,i=min(rows);return i if next_site is None or score<=600.0 else None
0188 | def run(self):
0189 |     if self.coverage_index==0:self._scan_site(self.route[0])
0190 |     loops=0
0191 |     while loops<2000:
0192 |         loops+=1
0193 |         self._finish_unknown_if_possible()
0194 |         found=[i for i,t in self.tracks.items() if t.status=='FOUND']
0195 |         unknown=[i for i,t in self.tracks.items() if t.status=='UNKNOWN']
0196 |         if not found and not unknown:break
0197 |         next_site=self.route[self.coverage_index] if self.coverage_index<len(self.route) and not self.coverage_done else None
0198 |         i=self._choose_insert(next_site)
0199 |         if i is not None:self._service(i);continue
0200 |         if next_site is not None:self._scan_site(next_site);continue
0201 |         if found:self._service(min(found,key=lambda j:dist(self.p,self.tracks[j].center)+self.tracks[j].radius));continue
0202 |         self._finish_unknown_if_possible()
0203 |         else:raise RuntimeError('策略循环超过上限')
0204 |         return {'cleared':self.cleared,'virtual_time_s':self.virtual_time,'average_time_s':self.virtual_time/self.cleared if self.cleared else None,
0205 |
↪ 'actions':self.actions,'coverage_sites_visited':self.coverage_index,'coverage_done':self.coverage_done,'max_pair_rounds':self.max_pair_rounds,
0206 |         'channel_status':{str(i):t.status for i,t in self.tracks.items()}}
0207 |
0208 | def run_case(api):return Strategy(api).run()

```

source/Q4/q4_official_solver.py（第 209 行起；前缀见本地版）

```

0209 |
0210 |
0211 | import argparse,datetime,http.client,json,os,socket,sys,time,unicodedata,uuid
0212 | from pathlib import Path
0213 | from urllib.parse import urlsplit
0214 |
0215 | class ProtocolError(RuntimeError):pass
0216 | class BudgetExceeded(RuntimeError):pass
0217 | class ResultUnknown(RuntimeError):pass
0218 |
0219 | class OfficialClient:
0220 |     def __init__(self,base_url,robot_id,log_path,timeout=5.0,retries=6):
0221 |         parsed=urlsplit(base_url)
0222 |         if parsed.scheme!='http' or not parsed.hostname or parsed.path not in ('','/') or parsed.query or parsed.fragment:raise
↪ ValueError('接口地址格式错误')
0223 |         if parsed.hostname not in ('127.0.0.1','localhost','::1'):raise ValueError('官方接口只能使用本机回环地址')
0224 |         if not robot_id or len(robot_id.encode('utf-8'))>64 or any(unicodedata.category(c) in ('Cc','Cf') for c in robot_id):raise
↪ ValueError('参赛队号格式不合法')
0225 |         self.host=parsed.hostname;self.port=parsed.port or 80;self.robot_id=robot_id;self.timeout=timeout;self.retries=retries;self.conn=None
0226 |         self.prefix=uuid.uuid4().hex[:12];self.serial=0;self.deadline=None;self.virtual_time=0.0;self.max_virtual=360000.0
0227 |         self.entered=False;self.exited=False;self.retry_count=0;self.position=(0.0,0.0);self.current_channel=1;self.pending=None
0228 |         self.exit_attempted=False;self.exit_response=None
0229 |         self.log_failed=False;self.log_error=None
0230 |         p=Path(log_path);p.parent.mkdir(parents=True,exist_ok=True);self.log=p.open('x',encoding='utf-8')
0231 |         self._log({'kind':'metadata','program':'Q4','robot_id':'<TEAM_ID>','utc':datetime.datetime.now(datetime.timezone.utc).isoformat()})
0232 |     def _log(self,obj):
0233 |         try:self.log.write(json.dumps(obj,ensure_ascii=False,allow_nan=False,separators=(',',':'))+'\n');self.log.flush()
0234 |         except Exception as exc:
0235 |             self.log_failed=True;self.log_error=repr(exc);print(f'日志写入失败: {exc}',file=sys.stderr,flush=True)
0236 |     def _disconnect(self):
0237 |         if self.conn:

```

```

0238 |     try:self.conn.close()
0239 |     except OSError:pass
0240 |     self.conn=None
0241 | def _connection(self,timeout=None):
0242 |     if self.conn is None:
0243 |         c=http.client.HTTPConnection(self.host,self.port,timeout=timeout or self.timeout);c.connect()
0244 |         if c.sock.c.sock.setsockopt(socket.IPPROTO_TCP,socket.TCP_NODELAY,1)
0245 |             self.conn=c
0246 |         if timeout is not None:self.conn.timeout=timeout
0247 |         if self.conn.sock and timeout is not None:self.conn.sock.settimeout(timeout)
0248 |         return self.conn
0249 | @staticmethod
0250 | def _finite(value):
0251 |     if isinstance(value,bool) or not isinstance(value,(int,float)):return False
0252 |     try:return math.isfinite(float(value))
0253 |     except (OverflowError,TypeError,ValueError):return False
0254 | def _cutoff(self,path):
0255 |     if self.deadline is None:return None
0256 |     return self.deadline-(0.25 if path=='/exit' else 8.0)
0257 | def _attempt_timeout(self,path):
0258 |     cutoff=self._cutoff(path)
0259 |     if cutoff is None:return self.timeout
0260 |     remaining=cutoff-time.monotonic()
0261 |     if remaining<=0:raise BudgetExceeded('现实运行时间不足, 停止发送新动作')
0262 |     return max(0.05,min(self.timeout,remaining))
0263 | def _validate_success(self,path,status,obj):
0264 |     if not isinstance(obj,dict):raise ResultUnknown('响应JSON顶层不是对象')
0265 |     if not isinstance(obj.get('accepted'),bool):raise ResultUnknown('accepted字段异常')
0266 |     for key in ('real_timestamp_ms','virtual_time_s'):
0267 |         if not self._finite(obj.get(key)) or float(obj[key])<0:raise ResultUnknown(f'{key}字段异常')
0268 |     if status!=200:
0269 |         if obj['accepted'] is False:raise ProtocolError(f'HTTP {status}: 请求已明确拒绝')
0270 |         raise ResultUnknown(f'HTTP {status}与accepted=true矛盾')
0271 |     if obj['accepted'] is not True:raise ProtocolError('请求未被接受')
0272 |     if float(obj['virtual_time_s'])+1e-5<self.virtual_time:raise ResultUnknown('virtual_time_s倒退')
0273 |     if path=='/enter':
0274 |         for key in ('max_virtual_duration_s','max_real_duration_s','remaining_real_duration_s'):
0275 |             if not self._finite(obj.get(key)) or float(obj[key])<0:raise ResultUnknown(f'{key}字段异常')
0276 |         rem=float(obj['remaining_real_duration_s'])
0277 |         if rem>1200 or not rem.is_integer():raise ResultUnknown('remaining_real_duration_s异常')
0278 |     elif path=='/measure':
0279 |         kind=obj.get('measure_result')
0280 |         if kind not in ('no_signal','near','direction'):raise ResultUnknown('measure_result异常')
0281 |         if kind=='direction' and (not self._finite(obj.get('svd_deg')) or not 0<=float(obj['svd_deg'])<360):raise ResultUnknown('svd_deg异常')
0282 |         elif path=='/clear' and obj.get('clear_result') not in ('success','no_target_in_range'):raise ResultUnknown('clear_result异常')
0283 |         elif path=='/exit' and obj.get('exit_reason')!='user_exit':raise ResultUnknown('exit_reason异常')
0284 | def _request(self,path,payload,body=None):
0285 |     body=body or json.dumps(payload,ensure_ascii=False,separators=(',',':'),allow_nan=False).encode('utf-8')
0286 |     if self.pending is None:
0287 |         self.pending={'path':path,'payload':dict(payload),'body':body}
0288 |         safe=dict(payload);safe['robot_id']='<TEAM_ID>';self._log({'kind':'request','path':path,'payload':safe})
0289 |         last=None;unknown=False
0290 |         for attempt in range(self.retries+1):
0291 |             started=time.monotonic()
0292 |             try:
0293 |                 timeout=self._attempt_timeout(path);c=self._connection(timeout);c.request('POST',path,body,{ 'Content-Type': 'application/json';
↪ charset=utf-8'})
0294 |                 r=c.getresponse();status=r.status;raw=r.read()
0295 |                 try:obj=json.loads(raw.decode('utf-8'),parse_constant=lambda x:(_ for _ in []).throw(ValueError(x)))
0296 |                 except (ValueError,UnicodeDecodeError) as exc:
0297 |                     self._log({'kind':'invalid_response','path':path,'request_id':payload['request_id'],'attempt':attempt,'http_status':status,'raw':raw[:1000].d
↪ ecode('utf-8','replace'))
0298 |                     raise ResultUnknown('响应不是有效JSON') from exc
0299 |                     self._log({'kind':'response','path':path,'request_id':payload['request_id'],'attempt':attempt,'elapsed_s':time.monotonic()-started,'http_stat
↪ us':status,'body':obj})
0300 |                     if status in (429,500,502,503,504):
0301 |                         common_ok=(isinstance(obj,dict) and obj.get('accepted') is False
0302 |                             and self._finite(obj.get('real_timestamp_ms'))
0303 |                             and self._finite(obj.get('virtual_time_s'))))
0304 |                         if not common_ok:raise ResultUnknown(f'HTTP {status}响应状态不可信')
0305 |                         raise OSError(f'HTTP {status}')
0306 |                     try:self._validate_success(path,status,obj)
0307 |                     except ProtocolError:self.pending=None;raise
0308 |                     self.virtual_time=float(obj['virtual_time_s']);self.pending=None;return obj
0309 |             except BudgetExceeded:
0310 |                 if not unknown:
0311 |                     self.pending=None;self._log({'kind':'aborted_before_send','path':path,'request_id':payload['request_id'],'attempt':attempt})
0312 |                 raise

```

```

0313 | except ProtocolError:raise
0314 | except (OSError,TimeoutError,http.client.HTTPException,ResultUnknown) as exc:
0315 |     last=exc;unknown=unknown or isinstance(exc,ResultUnknown) or not (isinstance(exc,OSError) and str(exc).startswith('HTTP '))
0316 |     self._disconnect();self._log({'kind':'retry','path':path,'request_id':payload['request_id'],'attempt':attempt,'elapsed_s':time.monotonic()-start
↵ ed,'error':repr(exc)})
0317 |         if attempt==self.retries:break
0318 |         self.retry_count+=1;delay=min(.1*2**attempt,1.5);cutoff=self._cutoff(path)
0319 |         if cutoff is not None:delay=min(delay,max(0.0,cutoff-time.monotonic()))
0320 |         if delay<=0:break
0321 |         time.sleep(delay)
0322 |         if unknown:raise ResultUnknown(f'动作结果未知，只能保留原request_id: {last}')
0323 |         self.pending=None;raise ConnectionError(f'同一请求重试后仍无有效响应: {last}')
0324 | def action(self,path,position=None,channel=None):
0325 |     if path not in ('/enter','/measure','/clear','/exit'):raise ValueError('接口路径不合法')
0326 |     if self.pending is not None:raise ResultUnknown('上一动作结果未知，禁止生成新的request_id')
0327 |     if path in ('/measure','/clear') and (position is None or channel is None):raise ValueError('动作缺少位置或频道')
0328 |     if path in ('/enter','/exit') and (position is not None or channel is not None):raise ValueError('enter/exit不应包含位置或频道')
0329 |     clean_position=None
0330 |     if position is not None:
0331 |         if len(position)!=2 or not all(self._finite(v) and abs(float(v))<=2e6 for v in position):raise ValueError('坐标不合法')
0332 |         clean_position=(float(position[0]),float(position[1]))
0333 |     if channel is not None and (isinstance(channel,bool) or not isinstance(channel,int) or not 1<=channel<=20):raise ValueError('频道不合法')
0334 |     if path in ('/measure','/clear'):
0335 |         upper=dist(self.position,clean_position)/5.0+5.0
0336 |         if path=='/measure' and channel!=self.current_channel:upper+=1.0
0337 |         if self.virtual_time+upper>self.max_virtual-0.001:raise BudgetExceeded('下一动作将超过虚拟时间上限')
0338 |     self._attempt_timeout(path);self.serial+=1;payload={'arena_id':'default','robot_id':self.robot_id,'request_id':f'{self.prefix}-{self.serial}'}
0339 |     if clean_position is not None:payload['position']={'x':clean_position[0],'y':clean_position[1]}
0340 |     if channel is not None:payload['channel']=channel
0341 |     out=self._request(path,payload)
0342 |     if path in ('/measure','/clear'):self.position=clean_position
0343 |     if path=='/measure':self.current_channel=channel
0344 |     return out
0345 | def wait_until_open(self,wait_s=300.0):
0346 |     end=time.monotonic()+wait_s
0347 |     while time.monotonic()<end:
0348 |         try:
0349 |             with socket.create_connection((self.host,self.port),timeout=.5):return
0350 |         except OSError:time.sleep(.25)
0351 |     raise TimeoutError('等待官方模拟器接口超时')
0352 | def enter(self):
0353 |     started=time.monotonic();out=self.action('/enter');rem=out.get('remaining_real_duration_s')
0354 |     if not isinstance(rem,(int,float)) or not 0<=float(rem)<=1200:raise ProtocolError('remaining_real_duration_s 异常')
0355 |     self.deadline=started+float(rem);self.max_virtual=float(out['max_virtual_duration_s']);self.entered=True;return out
0356 | def measure(self,p,ch):return self.action('/measure',p,ch)
0357 | def clear(self,p,ch):return self.action('/clear',p,ch)
0358 | def exit(self):
0359 |     if self.exited:return self.exit_response
0360 |     self.exit_attempted=True;out=self.action('/exit');self.exited=True;self.exit_response=out;return out
0361 | def can_exit(self):return self.entered and not self.exited and not self.exit_attempted and self.pending is None and (self.deadline is None or
↵ time.monotonic()<self.deadline-0.25)
0362 | def close(self):
0363 |     self._disconnect()
0364 |     try:self.log.close()
0365 |     except OSError as exc:
0366 |         self.log_failed=True;self.log_error=repr(exc);print(f'日志关闭失败: {exc}',file=sys.stderr,flush=True)
0367 |
0368 | def main():
0369 |     ap=argparse.ArgumentParser(description='问题四正式测试程序')
0370 |     ap.add_argument('--robot-id',default=None,help='参赛队号；不写入代码')
0371 |     ap.add_argument('--url',default='http://127.0.0.1:2026');ap.add_argument('--wait',type=float,default=300.0);args=ap.parse_args()
0372 |     robot_id=args.robot_id or os.environ.get('CUMCM_ROBOT_ID') or
↵ input('请输入参赛队号（仅本次运行使用）：').strip();stamp=datetime.datetime.now().strftime('%Y%m%d_%H%M%S_%f')
0373 |     log=f'runs/q4_{stamp}.json';api=OfficialClient(args.url,robot_id,log);ok=False;started=time.perf_counter()
0374 |     try:
0375 |         print('等待问题4测试接口开放.....',flush=True);api.wait_until_open(args.wait);api.enter();started=time.perf_counter()
0376 |         print('已进入测试，开始自动搜索定位与清除。',flush=True);summary=run_case(api);api.exit()
0377 |         summary.update(program_runtime_s=time.perf_counter()-started,retries=api.retry_count)
0378 |         Path(log).with_suffix('.summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8');api._log({'kind':'summar
↵ y','strategy_complete':True,'exit_confirmed':api.exited,**summary});ok=True
0379 |         if api.log_failed:raise OSError(f'运行已完成，但结构化日志不完整: {api.log_error}')
0380 |         average='N/A' if summary['average_time_s'] is None else f'{summary["average_time_s"]:.3f}'
0381 |         print(f'完成：清除 {summary["cleared"]} 个频道，虚拟时间 {summary["virtual_time_s"]:.3f} s，平均 {average} s/源")
0382 |         print(f'本地运行记录: {log}')
0383 |     except (Exception,KeyboardInterrupt) as exc:
0384 |         ok=False
0385 |         api._log({'kind':'fatal','error':repr(exc),'complete':False});print(f'运行未完成: {exc}',file=sys.stderr,flush=True)
0386 |     if api.can_exit():

```

```

0387 |     try:api.exit()
0388 |     except Exception as exit_exc:api._log({'kind':'exit_failure','error':repr(exit_exc),'pending':api.pending and
↪ {'path':api.pending['path'],'request_id':api.pending['payload']}['request_id']})
0389 |     finally:api.close()
0390 |     return 0 if ok and not api.log_failed else 1
0391 | if __name__=='__main__':raise SystemExit(main())

```

source/Q4/q4_local_simulator.py

```

0001 | #!/usr/bin/env python3
0002 | from __future__ import annotations
0003 | from dataclasses import dataclass
0004 | import argparse,datetime,json,math,random,time
0005 | from pathlib import Path
0006 | import q4_local_solver as solver
0007 |
0008 | @dataclass(frozen=True)
0009 | class Source:
0010 |     channel:int;position:tuple[float,float];radius:float;directional:bool;heading:float;phase:float
0011 |
0012 | class Simulator:
0013 |     def __init__(self,sources):
0014 |         self.sources={s.channel:s for s in sources};self.cleared=set();self.p=(0.0,0.0);self.channel=1;self.time=0.0
0015 |         self.distance=0.0;self.measurements=0;self.switches=0;self.failed_clears=0
0016 |     def _move(self,p):
0017 |         d=math.dist(self.p,p);self.distance+=d;self.time+=d/5;self.p=p
0018 |     def _visible(self,s,p):
0019 |         if not s.directional:return True
0020 |         d=(math.cos(s.heading),math.sin(s.heading));return (p[0]-s.position[0])*d[0]+(p[1]-s.position[1])*d[1]>=-1e-10
0021 |     def _error(self,s,p):
0022 |         return max(-1.0,min(1.0,.61*math.sin(p[0]/209+p[1]/181+s.phase)+.39*math.cos(p[1]/347-p[0]/277+s.phase*1.29)))
0023 |     def measure(self,p,ch):
0024 |         self._move(p);self.time+=5;self.measurements+=1
0025 |         if ch!=self.channel:self.time+=1;self.switches+=1
0026 |         self.channel=ch;s=self.sources.get(ch);out={'accepted':True,'virtual_time_s':self.time}
0027 |         if s is None or ch in self.cleared or math.dist(p,s.position)>s.radius or not self._visible(s,p):out['measure_result']='no_signal'
0028 |         elif math.dist(p,s.position)<=5:out['measure_result']='near'
0029 |         else:
0030 |
↪ a=math.degrees(math.atan2(s.position[1]-p[1],s.position[0]-p[0]));out.update(measure_result='direction',svd_deg=round((a+self._error(s,p))%360,2))
0031 |         return out
0032 |     def clear(self,p,ch):
0033 |         self._move(p);s=self.sources.get(ch);ok=s is not None and ch not in self.cleared and math.dist(p,s.position)<=20
0034 |         if ok:self.cleared.add(ch);self.time+=5
0035 |         else:self.time+=3;self.failed_clears+=1
0036 |         return {'accepted':True,'virtual_time_s':self.time,'clear_result':'success' if ok else 'no_target_in_range'}
0037 |
0038 | def make_case(seed):
0039 |     r=random.Random(seed);n=r.randint(10,16);chs=r.sample(range(1,21),n);nd=r.randint(1,n-1);dirs=set(r.sample(chs,nd));out=[]
0040 |     for ch in chs:
0041 |         a=r.random()*math.tau;rr=1800*math.sqrt(r.random());p=(rr*math.cos(a),rr*math.sin(a))
0042 |         out.append(Source(ch,p,r.uniform(1000,1500),ch in dirs,r.random()*math.tau,r.random()*math.tau))
0043 |     return out
0044 |
0045 | def _average(cases):
0046 |     if not cases:return None
0047 |     n=len(cases);sources=sum(c['source_count'] for c in cases);cleared=sum(c['cleared'] for c in cases)
0048 |     per_source=[c['average_time_s'] for c in cases if c['average_time_s'] is not None]
0049 |     return {
0050 |         'case_count':n,
0051 |         'source_count_per_case':sum(c['source_count'] for c in cases)/n,
0052 |         'directional_count_per_case':sum(c['directional_count'] for c in cases)/n,
0053 |         'cleared_per_case':sum(c['cleared'] for c in cases)/n,
0054 |         'virtual_time_s_per_case':sum(c['virtual_time_s'] for c in cases)/n,
0055 |         'case_equal_mean_time_s_per_source':sum(per_source)/len(per_source) if per_source else None,
0056 |         'pooled_time_s_per_cleared_source':sum(c['virtual_time_s'] for c in cases)/cleared if cleared else None,
0057 |         'distance_m_per_case':sum(c['distance_m'] for c in cases)/n,
0058 |         'movement_time_s_per_case':sum(c['distance_m'] for c in cases)/(5*n),
0059 |         'measurement_time_s_per_case':5*sum(c['measurements'] for c in cases)/n,
0060 |         'switch_time_s_per_case':sum(c['switches'] for c in cases)/n,
0061 |         'clear_action_time_s_per_case':(5*cleared+3*sum(c['failed_clears'] for c in cases))/n,
0062 |         'measurements_per_case':sum(c['measurements'] for c in cases)/n,
0063 |         'switches_per_case':sum(c['switches'] for c in cases)/n,
0064 |         'failed_clears_per_case':sum(c['failed_clears'] for c in cases)/n,
0065 |     }
0066 |

```

```

0067 | def _print_average(title,avg):
0068 |     if not avg: return
0069 |     print(title)
0070 |     print(f' 平均每局源数: {avg["source_count_per_case"]:.2f}')
0071 |     print(f' 平均每局定向源数: {avg["directional_count_per_case"]:.2f}')
0072 |     print(f' 平均每局清除数: {avg["cleared_per_case"]:.2f}')
0073 |     print(f' 平均每局总耗时: {avg["virtual_time_s_per_case"]:.2f} s')
0074 |     if avg['case_equal_mean_time_s_per_source'] is not None: print(f' 逐局等权平均每源耗时: {avg["case_equal_mean_time_s_per_source"]:.2f} s')
0075 |     if avg['pooled_time_s_per_cleared_source'] is not None: print(f' 合并源数加权每源耗时: {avg["pooled_time_s_per_cleared_source"]:.2f} s')
0076 |     print(f' 平均每局移动距离: {avg["distance_m_per_case"]:.2f} m')
0077 |     print(f' 时间分解 (移动/检测/换台/清除) : {avg["movement_time_s_per_case"]:.2f} / {avg["measurement_time_s_per_case"]:.2f} /
↪ {avg["switch_time_s_per_case"]:.2f} / {avg["clear_action_time_s_per_case"]:.2f} s')
0078 |     print(f' 平均每局测向次数: {avg["measurements_per_case"]:.2f}')
0079 |     print(f' 平均每局切频道次数: {avg["switches_per_case"]:.2f}')
0080 |     print(f' 平均每局失败清除次数: {avg["failed_clears_per_case"]:.3f}')
0081 |
0082 | def main():
0083 |     ap=argparse.ArgumentParser(description='问题四本地连续随机测试')
0084 |     ap.add_argument('--loops',type=int,default=None);ap.add_argument('--seed',type=int,default=None);ap.add_argument('--output',type=Path,defa
↪ ult=None);args=ap.parse_args()
0085 |     loops=args.loops if args.loops is not None else int(input('请输入测试局数: ').strip()) or 1000)
0086 |     base=args.seed if args.seed is not None else
↪ random.SystemRandom().randrange(1,2**63);rng=random.Random(base);cases=[];okn=0;start=time.perf_counter()
0087 |     for k in range(loops):
0088 |         seed=rng.randrange(1,2**63);src=make_case(seed);sim=Simulator(src)
0089 |         try:res=solver.run_case(sim);ok=len(sim.cleared)==len(src);err=None
0090 |         except Exception as exc:res={};ok=False;err=repr(exc)
0091 |         okn+=int(ok);cases.append({'index':k+1,'seed':seed,'source_count':len(src),'directional_count':sum(s.directional for s in src),
0092 |             'all_cleared':ok,'cleared':len(sim.cleared),'virtual_time_s':sim.time,'average_time_s':sim.time/len(sim.cleared) if sim.cleared else None,
0093 |             'distance_m':sim.distance,'measurements':sim.measurements,'switches':sim.switches,'failed_clears':sim.failed_clears,'solver':res,'error':err})
0094 |         if (k+1)%max(1,min(50,loops//20 or 1))==0:print(f'进度: {k+1}/{loops}, 全清除 {okn}/{k+1}',flush=True)
0095 |
0096 |     good=[c for c in cases if c['all_cleared']]
0097 |     avg_all=_average(cases);avg_good=_average(good)
0098 |     summary={
0099 |         'problem':4,'loops':loops,'base_seed':base,
0100 |         'full_clear_cases':okn,'full_clear_rate':okn/loops if loops else 0,
0101 |         'average_all_cases':avg_all,
0102 |         'average_successful_cases':avg_good,
0103 |         'mean_virtual_time_s':avg_good['virtual_time_s_per_case'] if avg_good else None,
0104 |         'mean_average_time_s':avg_good['case_equal_mean_time_s_per_source'] if avg_good else None,
0105 |         'pooled_average_time_s':avg_good['pooled_time_s_per_cleared_source'] if avg_good else None,
0106 |         'runtime_s':time.perf_counter()-start,'cases':cases
0107 |     }
0108 |     out=args.output or
↪ Path('results')/f'q4_local_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json";out.parent.mkdir(parents=True,exist_ok=True)
0109 |     out.write_text(json.dumps(summary,ensure_ascii=False,indent=2,encoding='utf-8')
0110 |
0111 |     print()
0112 |     print(f'完成: {okn}/{loops} 局全清除, 全清除率 {summary["full_clear_rate"]:.2%}')
0113 |     _print_average('【全部测试局平均】',avg_all)
0114 |     if okn != loops: _print_average('【全清除成功局平均】',avg_good)
0115 |     print(f'本机实际运行耗时: {summary["runtime_s"]:.2f} s')
0116 |     print(f'结果已保存: {out}')
0117 |
0118 | if __name__=='__main__':main()

```

source/Q4/run_local.bat

```

0001 | @echo off
0002 | setlocal
0003 | chcp 65001 >nul
0004 | pushd "%~dp0" || exit /b 1
0005 | python q4_local_simulator.py %*
0006 | set "RC=%ERRORLEVEL%"
0007 | popd
0008 | pause
0009 | exit /b %RC%

```

source/Q4/run_official.bat

```

0001 | @echo off
0002 | setlocal
0003 | chcp 65001 >nul

```

```
0004 | pushd "%~dp0" || exit /b 1
0005 | python q4_official_solver.py %*
0006 | set "RC=%ERRORLEVEL%"
0007 | popd
0008 | pause
0009 | exit /b %RC%
```

source/requirements.txt

```
0001 | # Python >= 3.10
0002 | # No third-party packages are required; the programs use the standard library only.
```